

Python 3 — Biblioteca Estándar

Tomo 3 de 6

Contenido de este tomo:

1. platform— Access to underlying platform's identifying data
2. errno— Standard errno system symbols
3. ctypes— A foreign function library for Python
4. Command-line interface libraries
5. argparse— Parser for command-line options, arguments and subcommands
6. optparse— Parser for command line options
7. getpass— Portable password input
8. fileinput— Iterate over lines from multiple input streams
9. curses— Terminal handling for character-cell displays
10. curses.ascii— Utilities for ASCII characters
11. curses.panel— A panel stack extension for curses
12. cmd— Support for line-oriented command interpreters
13. Concurrent Execution
14. threading— Thread-based parallelism
15. multiprocessing— Process-based parallelism
16. multiprocessing.shared_memory— Shared memory for direct access across processes
17. Theconcurrentpackage
18. concurrent.futures— Launching parallel tasks
19. concurrent.interpreters— Multiple interpreters in the same process
20. subprocess— Subprocess management
21. sched— Event scheduler
22. queue— A synchronized queue class
23. contextvars— Context Variables
24. _thread— Low-level threading API
25. Networking and Interprocess Communication

- 26. `asyncio`— Asynchronous I/O
- 27. `socket`— Low-level networking interface
- 28. `ssl`— TLS/SSL wrapper for socket objects
- 29. `select`— Waiting for I/O completion
- 30. `selectors`— High-level I/O multiplexing
- 31. `signal`— Set handlers for asynchronous events
- 32. `mmap`— Memory-mapped file support
- 33. Internet Data Handling
- 34. `email`— An email and MIME handling package
- 35. `json`— JSON encoder and decoder
- 36. `mailbox`— Manipulate mailboxes in various formats
- 37. `mimetypes`— Map filenames to MIME types
- 38. `base64`— Base16, Base32, Base64, Base85 Data Encodings
- 39. `binascii`— Convert between binary and ASCII
- 40. `quopri`— Encode and decode MIME quoted-printable data
- 41. Structured Markup Processing Tools
- 42. `html`— HyperText Markup Language support
- 43. `html.parser`— Simple HTML and XHTML parser
- 44. `html.entities`— Definitions of HTML general entities
- 45. XML Processing Modules

platform— Access to underlying platform's identifying data

Source code: `Lib/platform.py`

■ Nota: NoteSpecific platforms listed alphabetically, with Linux included in the Unix section.

Cross platform

platform.architecture(executable=sys.executable,bits="",linkage="")

Queries the given executable (defaults to the Python interpreter binary) for various architecture information. Returns a tuple(bits,linkage) which contain information about the bit architecture and the linkage format used for the executable. Both values are returned as strings. Values that cannot be determined are returned as given by the parameter presets. If bits is given as "", the `sizeof(pointer)` (or `sizeof(long)` on Python version < 1.5.2) is used as indicator for the supported pointer size. The func

platform.machine()

Returns the machine type, e.g. 'AMD64'. An empty string is returned if the value cannot be determined. The output is platform-dependent and may differ in casing and naming conventions.

platform.node()

Returns the computer's network name (may not be fully qualified!). An empty string is returned if the value cannot be determined.

platform.platform(aliased=False,terse=False)

Returns a single string identifying the underlying platform with as much useful information as possible. The output is intended to be human readable rather than machine parseable. It may look different on different platforms and this is intended. If `aliased` is true, the function will use aliases for various platforms that report system names which differ from their common names, for example SunOS will be reported as Solaris. The `system_alias()` function is used to implement this. Setting `terse` to true caus

platform.processor()

Returns the (real) processor name, e.g. 'amd64'. An empty string is returned if the value cannot be determined. Note that many platforms do not provide this information or simply return the same value as `machine()`. NetBSD does this.

platform.python_build()

Returns a tuple(buildno,builddate) stating the Python build number and date as strings.

platform.python_compiler()

Returns a string identifying the compiler used for compiling Python.

platform.python_branch()

Returns a string identifying the Python implementation SCM branch.

platform.python_implementation()

Returns a string identifying the Python implementation. Possible return values are: 'CPython', 'IronPython', 'Jython', 'PyPy'.

platform.python_revision()

Returns a string identifying the Python implementation SCM revision.

platform.python_version()

Returns the Python version as string'major.minor.patchlevel'.Note that unlike the `sys.version`, the returned value will always include the patchlevel (it defaults to 0).

platform.python_version_tuple()

Returns the Python version as tuple(major,minor,patchlevel)of strings.Note that unlike the `sys.version`, the returned value will always include the patchlevel (it defaults to'0').

platform.release()

Returns the system's release, e.g.'2.2.0'or'NT'. An empty string is returned if the value cannot be determined.

platform.system()

Returns the system/OS name, such as'Linux','Darwin','Java','Windows'. An empty string is returned if the value cannot be determined.On iOS and Android, this returns the user-facing OS name (i.e,'iOS','iPadOS'or'Android'). To obtain the kernel name ('Darwin'or'Linux'), use`os.uname()`.

platform.system_alias(system,release,version)

Returns(system,release,version)aliased to common marketing names used for some systems. It also does some reordering of the information in some cases where it would otherwise cause confusion.

platform.version()

Returns the system's release version, e.g.'#3ondegas'. An empty string is returned if the value cannot be determined.On iOS and Android, this is the user-facing OS version. To obtain the Darwin or Linux kernel version, use`os.uname()`.

platform.uname()

Fairly portable uname interface. Returns anamedtuple()containing six attributes:system,node,release,version,machine, andprocessor.processoris resolved late, on demand.Note: the first two attribute names differ from the names presented by`os.uname()`, where they are namedsysnameandnodename.Entries which cannot be determined are set to".Changed in version 3.3:Result changed from a tuple to anamedtuple().Changed in version 3.9:processoris resolved late instead of immediately.

platform.invalidate_caches()

Clear out the internal cache of information, such as the`uname()`. This is typically useful when the `platform'snode()`is changed by an external process and one needs to retrieve the updated value.Added in version 3.14.

Java platform

platform.java_ver(release="",vendor="",vminfo=("",),osinfo=("",))

Version interface for Jython.Returns a tuple(release,vendor,vminfo,osinfo)withvminfobeing a tuple(vm_name,vm_release,vm_vendor)andosinfobeing a tuple(os_name,os_version,os_arch). Values which cannot be determined are set to the defaults given as parameters (which all default to").Deprecated since version 3.13, will be removed in version 3.15:It was largely untested, had a confusing API, and was only useful for Jython support.

Windows platform

platform.win32_ver(release="",version="",csd="",ptype="")

Get additional version information from the Windows Registry and return a tuple(release,version,csd,ptype)referring to OS release, version number, CSD level (service pack) and OS type (multi/single processor). Values which cannot be determined are set to the defaults given as parameters (which all default to an empty string).As a hint:ptypeis'UniprocessorFree'on single processor NT machines and'MultiprocessorFree'on multi processor machines. The'Free'refers to the OS version being free of debugg

platform.win32_edition()

Returns a string representing the current Windows edition, orNoneif the value cannot be determined. Possible values include but are not limited to'Enterprise','IoTUAP','ServerStandard', and'nanoserver'.Added in version 3.8.

platform.win32_is_iot()

ReturnTrueif the Windows edition returned bywin32_edition()is recognized as an IoT edition.Added in version 3.8.

macOS platform

platform.mac_ver(release="",versioninfo=("", ""),machine="")

Get macOS version information and return it as tuple(release,versioninfo,machine)withversioninfobeing a tuple(version,dev_stage,non_release_version).Entries which cannot be determined are set to''. All tuple entries are strings.

iOS platform

platform.ios_ver(system="",release="",model="",is_simulator=False)

Get iOS version information and return it as anamedtuple()with the following attributes:systemis the OS name; either'iOS'or'iPadOS'.releaseis the iOS version number as a string (e.g.,'17.2').modelis the device model identifier; this will be a string like'iPhone13,2'for a physical device, or'iPhone'on a simulator.is_simulatoris a boolean describing if the app is running on a simulator or a physical device.Entries which cannot be determined are set to the defaults given as parameters.

Unix platforms

platform.libc_ver(executable=sys.executable,lib="",version="",chunksize=16384)

Tries to determine the libc version against which the file executable (defaults to the Python interpreter) is linked. Returns a tuple of strings(lib,version)which default to the given parameters in case the lookup fails.Note that this function has intimate knowledge of how different libc versions add symbols to the executable is probably only usable for executables compiled usinggcc.The file is read and scanned in chunks ofchunksizebytes.

Linux platforms

platform.freedesktop_os_release()

Get operating system identification fromos-releasefile and return it as a dict. Theos-releasefile is afreedesktop.org standardand is available in most Linux distributions. A noticeable exception is Android and

Android-based distributions. Raises `OSError` subclass when neither `/etc/os-release` nor `/usr/lib/os-release` can be read. On success, the function returns a dictionary where keys and values are strings. Values have their special characters like `"` and `$` unquoted. The fields `NAME`, `ID`, and `PRETTY_NAME` are all

Android platform

`platform.android_ver(release="", api_level=0, manufacturer="", model="", device="", is_emulator=False)`

Get Android device information. Returns a `namedtuple()` with the following attributes. Values which cannot be determined are set to the defaults given as parameters. `release`- Android version, as a string (e.g. "14"). `api_level`- API level of the running device, as an integer (e.g. 34 for Android 14). To get the API level which Python was built against, see `sys.getandroidapi_level()`. `manufacturer`- Manufacturer name. `model`- Model name- typically the marketing name or model number. `device`- Device name- typically the

Command-line usage

`platform` can also be invoked directly using the `-m` switch of the interpreter:

```
python -m platform [--terse] [--nonaliased] [{nonaliased,terse} ...]
```

The following options are accepted:

--terse

Print terse information about the platform. This is equivalent to calling `platform.platform()` with the `terse` argument set to `True`.

--nonaliased

Print platform information without system/OS name aliasing. This is equivalent to calling `platform.platform()` with the `aliased` argument set to `True`.

You can also pass one or more positional arguments (`terse`, `nonaliased`) to explicitly control the output format. These behave similarly to their corresponding options.

errno— Standard errno system symbols

This module makes available standard `errno` system symbols. The value of each symbol is the corresponding integer value. The names and descriptions are borrowed from `linux/include/errno.h`, which should be all-inclusive.

errno.errorcode

Dictionary providing a mapping from the `errno` value to the string name in the underlying system. For instance, `errno.errorcode[errno.EPERM]` maps to `'EPERM'`.

To translate a numeric error code to an error message, use `os.strerror()`.

Of the following list, symbols that are not used on the current platform are not defined by the module. The specific list of defined symbols is available as `errno.errorcode.keys()`. Symbols available can include:

errno.EPERM

Operation not permitted. This error is mapped to the exception `PermissionError`.

errno.ENOENT

No such file or directory. This error is mapped to the exception `FileNotFoundError`.

errno.ESRCH

No such process. This error is mapped to the exception `ProcessLookupError`.

errno.EINTR

Interrupted system call. This error is mapped to the exception `InterruptedError`.

errno.EIO

I/O error

errno.ENXIO

No such device or address

errno.E2BIG

Arg list too long

errno.ENOEXEC

Exec format error

errno.EBADF

Bad file number

errno.ECHILD

No child processes. This error is mapped to the exception `ChildProcessError`.

errno.EAGAIN

Try again. This error is mapped to the exception `BlockingIOError`.

errno.ENOMEM

Out of memory

errno.EACCES

Permission denied. This error is mapped to the exception `PermissionError`.

errno.EFAULT

Bad address

errno.ENOTBLK

Block device required

errno.EBUSY

Device or resource busy

errno.EEXIST

File exists. This error is mapped to the `exceptionFileExistsError`.

errno.EXDEV

Cross-device link

errno.ENODEV

No such device

errno.ENOTDIR

Not a directory. This error is mapped to the `exceptionNotADirectoryError`.

errno.EISDIR

Is a directory. This error is mapped to the `exceptionIsADirectoryError`.

errno.EINVAL

Invalid argument

errno.ENFILE

File table overflow

errno.EMFILE

Too many open files

errno.ENOTTY

Not a typewriter

errno.ETXTBSY

Text file busy

errno.EFBIG

File too large

errno.ENOSPC

No space left on device

errno.ESPIPE

Illegal seek

errno.EROFS

Read-only file system

errno.EMLINK

Too many links

errno.EPIPE

Broken pipe. This error is mapped to the exceptionBrokenPipeError.

errno.EDOM

Math argument out of domain of func

errno.ERANGE

Math result not representable

errno.EDEADLK

Resource deadlock would occur

errno.ENAMETOOLONG

File name too long

errno.ENOLCK

No record locks available

errno.ENOSYS

Function not implemented

errno.ENOTEMPTY

Directory not empty

errno.ELOOP

Too many symbolic links encountered

errno.EWOULDBLOCK

Operation would block. This error is mapped to the exceptionBlockingIOError.

errno.ENOMSG

No message of desired type

errno.EIDRM

Identifier removed

errno.ECHRNG

Channel number out of range

errno.EL2NSYNC

Level 2 not synchronized

errno.EL3HLT

Level 3 halted

errno.EL3RST

Level 3 reset

errno.ELNRNG

Link number out of range

errno.EUNATCH

Protocol driver not attached

errno.ENOCSI

No CSI structure available

errno.EL2HLT

Level 2 halted

errno.EBADE

Invalid exchange

errno.EBADR

Invalid request descriptor

errno.EXFULL

Exchange full

errno.ENOANO

No anode

errno.EBADRQC

Invalid request code

errno.EBADSLT

Invalid slot

errno.EDEADLOCK

File locking deadlock error

errno.EBFONT

Bad font file format

errno.ENOSTR

Device not a stream

errno.ENODATA

No data available

errno.ETIME

Timer expired

errno.ENOSR

Out of streams resources

errno.ENONET

Machine is not on the network

errno.ENOPKG

Package not installed

errno.EREMOTE

Object is remote

errno.ENOLINK

Link has been severed

errno.EADV

Advertise error

errno.ESRMNT

Srmount error

errno.ECOMM

Communication error on send

errno.EPROTO

Protocol error

errno.EMULTIHOP

Multihop attempted

errno.EDOTDOT

RFS specific error

errno.EBADMSG

Not a data message

errno.EOVERFLOW

Value too large for defined data type

errno.ENOTUNIQ

Name not unique on network

errno.EBADFD

File descriptor in bad state

errno.EREMCHG

Remote address changed

errno.ELIBACC

Can not access a needed shared library

errno.ELIBBAD

Accessing a corrupted shared library

errno.ELIBSCN

.lib section in a.out corrupted

errno.ELIBMAX

Attempting to link in too many shared libraries

errno.ELIBEXEC

Cannot exec a shared library directly

errno.EILSEQ

Illegal byte sequence

errno.ERESTART

Interrupted system call should be restarted

errno.ESTRPIPE

Streams pipe error

errno.EUSERS

Too many users

errno.ENOTSOCK

Socket operation on non-socket

errno.EDESTADDRREQ

Destination address required

errno.EMSGSIZE

Message too long

errno.EPROTOTYPE

Protocol wrong type for socket

errno.ENOPROTOOPT

Protocol not available

errno.EPROTONOSUPPORT

Protocol not supported

errno.ESOCKTNOSUPPORT

Socket type not supported

errno.EOPNOTSUPP

Operation not supported on transport endpoint

errno.ENOTSUP

Operation not supportedAdded in version 3.2.

errno.EPFNOSUPPORT

Protocol family not supported

errno.EAFNOSUPPORT

Address family not supported by protocol

errno.EADDRINUSE

Address already in use

errno.EADDRNOTAVAIL

Cannot assign requested address

errno.ENETDOWN

Network is down

errno.ENETUNREACH

Network is unreachable

errno.ENETRESET

Network dropped connection because of reset

errno.ECONNABORTED

Software caused connection abort. This error is mapped to the exceptionConnectionAbortedError.

errno.ECONNRESET

Connection reset by peer. This error is mapped to the exceptionConnectionResetError.

errno.ENOBUFS

No buffer space available

errno.EISCONN

Transport endpoint is already connected

errno.ENOTCONN

Transport endpoint is not connected

errno.ESHUTDOWN

Cannot send after transport endpoint shutdown. This error is mapped to the exceptionBrokenPipeError.

errno.ETOOMANYREFS

Too many references: cannot splice

errno.ETIMEDOUT

Connection timed out. This error is mapped to the exceptionTimeoutError.

errno.ECONNREFUSED

Connection refused. This error is mapped to the exceptionConnectionRefusedError.

errno.EHOSTDOWN

Host is down

errno.EHOSTUNREACH

No route to host

errno.EHWPOISON

Memory page has hardware error. Added in version 3.14.

errno.EALREADY

Operation already in progress. This error is mapped to the exceptionBlockingIOError.

errno.EINPROGRESS

Operation now in progress. This error is mapped to the exceptionBlockingIOError.

errno.ESTALE

Stale NFS file handle

errno.EUCLEAN

Structure needs cleaning

errno.ENOTNAM

Not a XENIX named type file

errno.ENAVAIL

No XENIX semaphores available

errno.EISNAM

Is a named type file

errno.EREMOTEIO

Remote I/O error

errno.EDQUOT

Quota exceeded

errno.EQFULL

Interface output queue is fullAdded in version 3.11.

errno.ENOMEDIUM

No medium found

errno.EMEDIUMTYPE

Wrong medium type

errno.ENOKEY

Required key not available

errno.EKEYEXPIRED

Key has expired

errno.EKEYREVOKED

Key has been revoked

errno.EKEYREJECTED

Key was rejected by service

errno.ERFKILL

Operation not possible due to RF-kill

errno.ELOCKUNMAPPED

Locked lock was unmapped

errno.ENOTACTIVE

Facility is not active

errno.EAUTH

Authentication errorAdded in version 3.2.

errno.EBADARCH

Bad CPU type in executableAdded in version 3.2.

errno.EBADEXEC

Bad executable (or shared library)Added in version 3.2.

errno.EBADMACHO

Malformed Mach-o fileAdded in version 3.2.

errno.EDEVERR

Device errorAdded in version 3.2.

errno.EFTYPE

Inappropriate file type or formatAdded in version 3.2.

errno.ENEEDAUTH

Need authenticatorAdded in version 3.2.

errno.ENOATTR

Attribute not foundAdded in version 3.2.

errno.ENOPOLICY

Policy not foundAdded in version 3.2.

errno.EPROCLIM

Too many processesAdded in version 3.2.

errno.EPROCUNAVAIL

Bad procedure for programAdded in version 3.2.

errno.EPROGMISMATCH

Program version wrongAdded in version 3.2.

errno.EPROGUNAVAIL

RPC prog. not availAdded in version 3.2.

errno.EPWROFF

Device power is offAdded in version 3.2.

errno.EBADRPC

RPC struct is badAdded in version 3.2.

errno.ERPCMISMATCH

RPC version wrongAdded in version 3.2.

errno.ESHLIBVERS

Shared library version mismatchAdded in version 3.2.

errno.ENOTCAPABLE

Capabilities insufficient. This error is mapped to the exceptionPermissionError.Availability: WASI, FreeBSDAdded in version 3.11.1.

errno.ECANCELED

Operation canceledAdded in version 3.2.

errno.EOWNERDEAD

Owner diedAdded in version 3.2.

errno.ENOTRECOVERABLE

State not recoverableAdded in version 3.2.

ctypes— A foreign function library for Python

Source code: `Lib/ctypes`

`ctypes` is a foreign function library for Python. It provides C compatible data types, and allows calling functions in DLLs or shared libraries. It can be used to wrap these libraries in pure Python.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

ctypes tutorial

Note: Some code samples reference the `ctypes.c_inttype`. On platforms where `sizeof(long) == sizeof(int)` it is an alias to `_long`. So, you should not be confused if `c_long` is printed if you would expect `c_int`— they are actually the same type.

Loading dynamic link libraries

`ctypes` exports the `cdll`, and on Windows `windll` and `oledll` objects, for loading dynamic link libraries.

You load libraries by accessing them as attributes of these objects. `cdll` loads libraries which export functions using the standard C `cdecl` calling convention, while `windll` libraries call functions using the `stdcall` calling convention. `oledll` also uses the `stdcall` calling convention, and assumes the functions return a `Windows HRESULT` error code. The error code is used to automatically raise an `OSError` exception when the function call fails.

Changed in version 3.3: Windows errors used to raise `WindowsError`, which is now an alias of `OSError`.

Here are some examples for Windows. Note that `msvcrt` is the MS standard C library containing most standard C functions, and uses the `cdecl` calling convention:

```
>>> from ctypes import *
>>> print(windll.kernel32)
<WinDLL 'kernel32', handle ... at ...>
>>> print(cdll.msvcrt)
<CDLL 'msvcrt', handle ... at ...>
>>> libc = cdll.msvcrt
>>>
```

Windows appends the usual `.dll` file suffix automatically.

■ **Note:** Accessing the standard C library through `cdll.msvcrt` will use an outdated version of the library that may be incompatible with the one being used by Python. Where possible, use native Python functionality, or else import and use the `msvcrt` module.

Other systems require the filename including the extension to load a library, so attribute access can not be used to load libraries. Either the `LoadLibrary()` method of the `dll` loaders should be used, or you should load the library by creating an instance of `CDLL` by calling the constructor.

For example, on Linux:

```
>>> cdll.LoadLibrary("libc.so.6")
<CDLL 'libc.so.6', handle ... at ...>
>>> libc = CDLL("libc.so.6")
>>> libc
<CDLL 'libc.so.6', handle ... at ...>
>>>
```

On macOS:


```
>>> cdll.LoadLibrary("libc.dylib")
<CDLL 'libc.dylib', handle ... at ...>
>>> libc = CDLL("libc.dylib")
>>> libc
<CDLL 'libc.dylib', handle ... at ...>
```

Accessing functions from loaded dlls

Functions are accessed as attributes of dll objects:

```
>>> libc.printf
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.GetModuleHandleA)
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.MyOwnFunction)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "ctypes.py", line 239, in __getattr__
      func = _StdcallFuncPtr(name, self)
AttributeError: function 'MyOwnFunction' not found
>>>
```

Note that win32 system dlls like `kernel32` and `user32` often export ANSI as well as UNICODE versions of a function. The UNICODE version is exported with `W` appended to the name, while the ANSI version is exported with `A` appended to the name. The `win32GetModuleHandle` function, which returns a module handle for a given module name, has the following C prototype, and a macro is used to expose one of them as `GetModuleHandle` depending on whether UNICODE is defined or not:

```
/* ANSI version */
HMODULE GetModuleHandleA(LPCSTR lpModuleName);
/* UNICODE version */
HMODULE GetModuleHandleW(LPCWSTR lpModuleName);
```

`windll` does not try to select one of them by magic, you must access the version you need by specifying `GetModuleHandleA` or `GetModuleHandleW` explicitly, and then call it with bytes or string objects respectively.

Sometimes, dlls export functions with names which aren't valid Python identifiers, like `"??2@YAPAXI@Z"`. In this case you have to use `getattr()` to retrieve the function:

```
>>> getattr(cdll.msvcrt, "??2@YAPAXI@Z")
<_FuncPtr object at 0x...>
>>>
```

On Windows, some dlls export functions not by name but by ordinal. These functions can be accessed by indexing the dll object with the ordinal number:

```
>>> cdll.kernel32[1]
<_FuncPtr object at 0x...>
>>> cdll.kernel32[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "ctypes.py", line 310, in __getitem__
      func = _StdcallFuncPtr(name, self)
AttributeError: function ordinal 0 not found
>>>
```

Calling functions

You can call these functions like any other Python callable. This example uses the `rand()` function, which takes no arguments and returns a pseudo-random integer:

```
>>> print(libc.rand())
1804289383
```

On Windows, you can call the `GetModuleHandleA()` function, which returns a win32 module handle (passing `None` as a single argument to call it with a `NULL` pointer):

```
>>> print(hex(windll.kernel32.GetModuleHandleA(None)))
0x1d000000
>>>
```

`ValueError` is raised when you call an stdcall function with the cdecl calling convention, or vice versa:

```
>>> cdll.kernel32.GetModuleHandleA(None)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with not enough arguments (4 bytes missing)
>>>
```

```
>>> windll.msvcrt.printf(b"spam")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with too many arguments (4 bytes in excess)
>>>
```

To find out the correct calling convention you have to look into the C header file or the documentation for the function you want to call.

On Windows, `ctypes` uses win32 structured exception handling to prevent crashes from general protection faults when functions are called with invalid argument values:

```
>>> windll.kernel32.GetModuleHandleA(32)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OSError: exception: access violation reading 0x00000020
>>>
```

There are, however, enough ways to crash Python with `ctypes`, so you should be careful anyway. The `faulthandler` module can be helpful in debugging crashes (e.g. from segmentation faults produced by erroneous C library calls).

`None`, integers, bytes objects and (unicode) strings are the only native Python objects that can directly be used as parameters in these function calls. `None` is passed as a `CNULL` pointer, bytes objects and strings are passed as pointer to the memory block that contains their data (`char*` or `wchar_t*`). Python integers are passed as the platform's default `Cint` type, their value is masked to fit into the C type.

Before we move on calling functions with other parameter types, we have to learn more about `ctypes` data types.

Fundamental data types

`ctypes` defines a number of primitive C compatible data types:

```
ctypes type | C type | Python type | _type_ | c_bool | _Bool | bool | '?' | c_char | char |
1-character | bytes | 'c' | c_wchar | wchar_t | 1-character | str | 'u' | c_byte | char | int | '
b' | c_ubyte | unsigned | char | int | 'B' | c_short | short | int | 'h' | c_ushort | unsigned |
short | int | 'H' | c_int | int | int | 'i' | * | c_int8 | int8_t | int | * | c_int16 | int16_t
| int | * | c_int32 | int32_t | int | * | c_int64 | int64_t | int | * | c_uint | unsigned | int
| int | 'I' | * | c_uint8 | uint8_t | int | * | c_uint16 | uint16_t | int | * | c_uint32 |
uint32_t | int | * | c_uint64 | uint64_t | int | * | c_long | long | int | 'l' | c_ulong |
unsigned | long | int | 'L' | c_longlong | long | long | int | 'q' | * | c_ulonglong | unsigned
| long | long | int | 'Q' | * | c_size_t | s
```

Additionally, if IEC 60559 compatible complex arithmetic (Annex G) is supported in both C and `libffi`, the following complex types are available:

```
ctypes type | C type | Python type | _type_ | c_float_complex | float | complex | complex | 'F'
| c_double_complex | double | complex | complex | 'D' | c_longdouble_complex | long | double |
complex | complex | 'G'
```

All these types can be created by calling them with an optional initializer of the correct type and value:

```
>>> c_int()
c_long(0)
>>> c_wchar_p("Hello, World")
c_wchar_p(140018365411392)
>>> c_ushort(-3)
c_ushort(65533)
>>>
```

The constructors for numeric types will convert input using `__bool__()`, `__index__()` (for int), `__float__()` or `__complex__()`. This means `c_bool` accepts any object with a truth value:

```
>>> empty_list = []
>>> c_bool(empty_list)
c_bool(False)
```

Since these types are mutable, their value can also be changed afterwards:

```
>>> i = c_int(42)
>>> print(i)
c_long(42)
>>> print(i.value)
42
>>> i.value = -99
>>> print(i.value)
-99
>>>
```

Assigning a new value to instances of the pointer types `c_char_p`, `c_wchar_p`, and `c_void_p` changes the memory location they point to, not the contents of the memory block (of course not, because Python string objects are immutable):

```
>>> s = "Hello, World"
>>> c_s = c_wchar_p(s)
>>> print(c_s)
c_wchar_p(139966785747344)
>>> print(c_s.value)
Hello World
>>> c_s.value = "Hi, there"
>>> print(c_s)                # the memory location has changed
c_wchar_p(139966783348904)
>>> print(c_s.value)
Hi, there
>>> print(s)                  # first object is unchanged
Hello, World
>>>
```

You should be careful, however, not to pass them to functions expecting pointers to mutable memory. If you need mutable memory blocks, `ctypes` has a `create_string_buffer()` function which creates these in various ways. The current memory block contents can be accessed (or changed) with the `raw` property; if you want to access it as NUL terminated string, use the `value` property:

```
>>> from ctypes import *
>>> p = create_string_buffer(3)                # create a 3 byte buffer, initialized to NUL bytes
>>> print(sizeof(p), repr(p.raw))
3 b'\x00\x00\x00'
>>> p = create_string_buffer(b"Hello")         # create a buffer containing a NUL terminated string
>>> print(sizeof(p), repr(p.raw))
6 b'Hello\x00'
>>> print(repr(p.value))
b'Hello'
>>> p = create_string_buffer(b"Hello", 10)    # create a 10 byte buffer
>>> print(sizeof(p), repr(p.raw))
```

```

10 b'Hello\x00\x00\x00\x00\x00'
>>> p.value = b'Hi'
>>> print(sizeof(p), repr(p.raw))
10 b'Hi\x00lo\x00\x00\x00\x00\x00'
>>>

```

The `create_string_buffer()` function replaces the `oldc_buffer()` function (which is still available as an alias). To create a mutable memory block containing unicode characters of the C `typewchar_t`, use the `create_unicode_buffer()` function.

Calling functions, continued

Note that `printf` prints to the real standard output channel, `not to sys.stdout`, so these examples will only work at the console prompt, not from within `IDLE` or `PythonWin`:

```

>>> printf = libc.printf
>>> printf(b"Hello, %s\n", b"World!")
Hello, World!
14
>>> printf(b"Hello, %S\n", "World!")
Hello, World!
14
>>> printf(b"%d bottles of beer\n", 42)
42 bottles of beer
19
>>> printf(b"%f bottles of beer\n", 42.5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: Don't know how to convert parameter 2
>>>

```

As has been mentioned before, all Python types except integers, strings, and bytes objects have to be wrapped in their corresponding `ctype` type, so that they can be converted to the required C data type:

```

>>> printf(b"An int %d, a double %f\n", 1234, c_double(3.14))
An int 1234, a double 3.140000
31
>>>

```

Calling variadic functions

On a lot of platforms calling variadic functions through `ctypes` is exactly the same as calling functions with a fixed number of parameters. On some platforms, and in particular ARM64 for Apple Platforms, the calling convention for variadic functions is different than that for regular functions.

On those platforms it is required to specify the `argtypes` attribute for the regular, non-variadic, function arguments:

```
libc.printf.argtypes = [ctypes.c_char_p]
```

Because specifying the attribute does not inhibit portability it is advised to always specify `argtypes` for all variadic functions.

Calling functions with your own custom data types

You can also customize `ctypes` argument conversion to allow instances of your own classes be used as function arguments. `ctypes` looks for an `_as_parameter_` attribute and uses this as the function argument. The attribute must be an integer, string, bytes, `ctypes` instance, or an object with an `_as_parameter_` attribute:

```

>>> class Bottles:
...     def __init__(self, number):
...         self._as_parameter_ = number
...
>>> bottles = Bottles(42)
>>> printf(b"%d bottles of beer\n", bottles)

```

```

42 bottles of beer
19
>>>

```

If you don't want to store the instance's data in the `_as_parameter_instance` variable, you could define a property which makes the attribute available on request.

Specifying the required argument types (function prototypes)

It is possible to specify the required argument types of functions exported from DLLs by setting the `argtypes` attribute.

`argtypes` must be a sequence of C data types (the `printf()` function is probably not a good example here, because it takes a variable number and different types of parameters depending on the format string, on the other hand this is quite handy to experiment with this feature):

```

>>> printf.argtypes = [c_char_p, c_char_p, c_int, c_double]
>>> printf(b"String '%s', Int %d, Double %f\n", b"Hi", 10, 2.2)
String 'Hi', Int 10, Double 2.200000
37
>>>

```

Specifying a format protects against incompatible argument types (just as a prototype for a C function), and tries to convert the arguments to valid types:

```

>>> printf(b"%d %d %d", 1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: 'int' object cannot be interpreted as ctypes.c_char_p
>>> printf(b"%s %d %f\n", b"X", 2, 3)
X 2 3.000000
13
>>>

```

If you have defined your own classes which you pass to function calls, you have to implement a `from_param()` class method for them to be able to use them in the `argtypes` sequence. The `from_param()` class method receives the Python object passed to the function call, it should do a typecheck or whatever is needed to make sure this object is acceptable, and then return the object itself, its `_as_parameter_attribute`, or whatever you want to pass as the C function argument in this case. Again, the result should be an integer, string, bytes, a ctypes instance, or an object with an `_as_parameter_attribute`.

Return types

By default functions are assumed to return the `Cint` type. Other return types can be specified by setting the `restype` attribute of the function object.

The C prototype of `time()` is `time_t time(t*)`. Because `time_t` might be of a different type than the default return type `int`, you should specify the `restype` attribute:

```

>>> libc.time.restype = c_time_t

```

The argument types can be specified using `argtypes`:

```

>>> libc.time.argtypes = (POINTER(c_time_t),)

```

To call the function with a NULL pointer as first argument, use `None`:

```

>>> print(libc.time(None))
1150640792

```

Here is a more advanced example, it uses the `strchr()` function, which expects a string pointer and a char, and returns a pointer to a string:

```

>>> strchr = libc.strchr

```

```
>>> strchr(b"abcdef", ord("d"))
8059983
>>> strchr.restype = c_char_p    # c_char_p is a pointer to a string
>>> strchr(b"abcdef", ord("d"))
b'def'
>>> print(strchr(b"abcdef", ord("x")))
None
>>>
```

If you want to avoid the `ord("x")` calls above, you can set the `argtypes` attribute, and the second argument will be converted from a single character Python bytes object into a C char:

```
>>> strchr.restype = c_char_p
>>> strchr.argtypes = [c_char_p, c_char]
>>> strchr(b"abcdef", b"d")
b'def'
>>> strchr(b"abcdef", b"def")
Traceback (most recent call last):
  ctypes.ArgumentError: argument 2: TypeError: one character bytes, bytearray or integer expected
>>> print(strchr(b"abcdef", b"x"))
None
>>> strchr(b"abcdef", b"d")
b'def'
>>>
```

You can also use a callable Python object (a function or a class for example) as the `restype` attribute, if the foreign function returns an integer. The callable will be called with the integer the C function returns, and the result of this call will be used as the result of your function call. This is useful to check for error return values and automatically raise an exception:

```
>>> GetModuleHandle = windll.kernel32.GetModuleHandleA
>>> def ValidHandle(value):
...     if value == 0:
...         raise WinError()
...     return value
...
>>>
>>> GetModuleHandle.restype = ValidHandle
>>> GetModuleHandle(None)
486539264
>>> GetModuleHandle("something silly")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in ValidHandle
OSError: [Errno 126] The specified module could not be found.
>>>
```

`WinError` is a function which will call `WindowsFormatMessage()` api to get the string representation of an error code, and returns an exception. `WinError` takes an optional error code parameter, if no one is used, it calls `GetLastError()` to retrieve it.

Please note that a much more powerful error checking mechanism is available through the `errcheck` attribute; see the reference manual for details.

Passing pointers (or: passing parameters by reference)

Sometimes a C api function expects a pointer to a data type as parameter, probably to write into the corresponding location, or if the data is too large to be passed by value. This is also known as passing parameters by reference.

`ctypes` exports the `byref()` function which is used to pass parameters by reference. The same effect can be achieved with the `pointer()` function, although `pointer()` does a lot more work since it constructs a real pointer object, so it is faster to use `byref()` if you don't need the pointer object in Python itself:

```

>>> i = c_int()
>>> f = c_float()
>>> s = create_string_buffer(b'\000' * 32)
>>> print(i.value, f.value, repr(s.value))
0 0.0 b''
>>> libc sscanf(b"1 3.14 Hello", b"%d %f %s",
...             byref(i), byref(f), s)
3
>>> print(i.value, f.value, repr(s.value))
1 3.1400001049 b'Hello'
>>>

```

Structures and unions

Structures and unions must derive from the `Structure` and `Union` base classes which are defined in the `ctypes` module. Each subclass must define a `_fields_` attribute. `_fields_` must be a list of 2-tuples, containing a field name and a field type.

The field type must be a ctypes type like `c_int`, or any other derived ctypes type: `structure`, `union`, `array`, `pointer`.

Here is a simple example of a `POINT` structure, which contains two integers named `x` and `y`, and also shows how to initialize a structure in the constructor:

```

>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = [("x", c_int),
...                 ("y", c_int)]
...
>>> point = POINT(10, 20)
>>> print(point.x, point.y)
10 20
>>> point = POINT(y=5)
>>> print(point.x, point.y)
0 5
>>> POINT(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: too many initializers
>>>

```

You can, however, build much more complicated structures. A structure can itself contain other structures by using a structure as a field type.

Here is a `RECT` structure which contains two `POINT`s named `upperleft` and `lowerright`:

```

>>> class RECT(Structure):
...     _fields_ = [("upperleft", POINT),
...                 ("lowerright", POINT)]
...
>>> rc = RECT(point)
>>> print(rc.upperleft.x, rc.upperleft.y)
0 5
>>> print(rc.lowerright.x, rc.lowerright.y)
0 0
>>>

```

Nested structures can also be initialized in the constructor in several ways:

```

>>> r = RECT(POINT(1, 2), POINT(3, 4))
>>> r = RECT((1, 2), (3, 4))

```

Field descriptors can be retrieved from the class, they are useful for debugging because they can provide useful information. See `CField`:

```

>>> POINT.x
<ctypes.CField 'x' type=c_int, ofs=0, size=4>

```

```
>>> POINT.y
<ctypes.CField 'y' type=c_int, ofs=4, size=4>
>>>
```

■■ Advertencia: Warningctypesdoes not support passing unions or structures with bit-fields to functions by value. While this may work on 32-bit x86, it's not guaranteed by the library to work in the general case. Unions and structures with bit-fields should always be passed to functions by pointer.

Structure/union layout, alignment and byte order

By default, Structure and Union fields are laid out in the same way the C compiler does it. It is possible to override this behavior entirely by specifying a `_layout_class` attribute in the subclass definition; see the attribute documentation for details.

It is possible to specify the maximum alignment for the fields and/or for the structure itself by setting the class attributes `_pack_` and/or `_align_`, respectively. See the attribute documentation for details.

ctypesuses the native byte order for Structures and Unions. To build structures with non-native byte order, you can use one of the `BigEndianStructure`, `LittleEndianStructure`, `BigEndianUnion`, and `LittleEndianUnion` base classes. These classes cannot contain pointer fields.

Bit fields in structures and unions

It is possible to create structures and unions containing bit fields. Bit fields are only possible for integer fields, the bit width is specified as the third item in the `_fields_` tuples:

```
>>> class Int(Structure):
...     _fields_ = [("first_16", c_int, 16),
...                 ("second_16", c_int, 16)]
...
>>> print(Int.first_16)
<ctypes.CField 'first_16' type=c_int, ofs=0, bit_size=16, bit_offset=0>
>>> print(Int.second_16)
<ctypes.CField 'second_16' type=c_int, ofs=0, bit_size=16, bit_offset=16>
```

It is important to note that bit field allocation and layout in memory are not defined as a C standard; their implementation is compiler-specific. By default, Python will attempt to match the behavior of a “native” compiler for the current platform. See the `_layout_` attribute for details on the default behavior and how to change it.

Arrays

Arrays are sequences, containing a fixed number of instances of the same type.

The recommended way to create array types is by multiplying a data type with a positive integer:

```
TenPointsArrayType = POINT * 10
```

Here is an example of a somewhat artificial data type, a structure containing 4 POINTs among other stuff:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
>>> class MyStruct(Structure):
...     _fields_ = [("a", c_int),
...                 ("b", c_float),
...                 ("point_array", POINT * 4)]
>>>
>>> print(len(MyStruct().point_array))
4
>>>
```

Instances are created in the usual way, by calling the class:

```
arr = TenPointsArrayType()
```



```
for pt in arr:
    print(pt.x, pt.y)
```

The above code print a series of 00lines, because the array contents is initialized to zeros.

Initializers of the correct type can also be specified:

```
>>> from ctypes import *
>>> TenIntegers = c_int * 10
>>> ii = TenIntegers(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
>>> print(ii)
<c_long_Array_10 object at 0x...>
>>> for i in ii: print(i, end=" ")
...
1 2 3 4 5 6 7 8 9 10
>>>
```

Pointers

Pointer instances are created by calling the `pointer()` function on a ctypes type:

```
>>> from ctypes import *
>>> i = c_int(42)
>>> pi = pointer(i)
>>>
```

Pointer instances have a `contents` attribute which returns the object to which the pointer points, the object above:

```
>>> pi.contents
c_long(42)
>>>
```

Note that ctypes does not have OOR (original object return), it constructs a new, equivalent object each time you retrieve an attribute:

```
>>> pi.contents is i
False
>>> pi.contents is pi.contents
False
>>>
```

Assigning another `c_int` instance to the pointer's `contents` attribute would cause the pointer to point to the memory location where this is stored:

```
>>> i = c_int(99)
>>> pi.contents = i
>>> pi.contents
c_long(99)
>>>
```

Pointer instances can also be indexed with integers:

```
>>> pi[0]
99
>>>
```

Assigning to an integer index changes the pointed to value:

```
>>> print(i)
c_long(99)
>>> pi[0] = 22
>>> print(i)
c_long(22)
>>>
```

It is also possible to use indexes different from 0, but you must know what you're doing, just as in C: You can access or change arbitrary memory locations. Generally you only use this feature if you receive a pointer from a

C function, and you know that the pointer actually points to an array instead of a single item.

Behind the scenes, the `pointer()` function does more than simply create pointer instances, it has to create pointer types first. This is done with the `POINTER()` function, which accepts any ctypes type, and returns a new type:

```
>>> PI = POINTER(c_int)
>>> PI
<class 'ctypes.LP_c_long'>
>>> PI(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: expected c_long instead of int
>>> PI(c_int(42))
<ctypes.LP_c_long object at 0x...>
>>>
```

Calling the pointer type without an argument creates a `NULL` pointer. `NULL` pointers have a `False` boolean value:

```
>>> null_ptr = POINTER(c_int)()
>>> print(bool(null_ptr))
False
>>>
```

ctypes checks for `NULL` when dereferencing pointers (but dereferencing invalid non-`NULL` pointers would crash Python):

```
>>> null_ptr[0]
Traceback (most recent call last):
  ....
ValueError: NULL pointer access
>>>
```

```
>>> null_ptr[0] = 1234
Traceback (most recent call last):
  ....
ValueError: NULL pointer access
>>>
```

Thread safety without the GIL

From Python 3.13 onward, the GIL can be disabled on the free-threaded build. In ctypes, reads and writes to a single object concurrently is safe, but not across multiple objects:

```
>>> number = c_int(42)
>>> pointer_a = pointer(number)
>>> pointer_b = pointer(number)
```

In the above, it's only safe for one object to read and write to the address at once if the GIL is disabled. So, `pointer_a` can be shared and written to across multiple threads, but only if `pointer_b` is not also attempting to do the same. If this is an issue, consider using `threading.Lock` to synchronize access to memory:

```
>>> import threading
>>> lock = threading.Lock()
>>> # Thread 1
>>> with lock:
...     pointer_a.contents = 24
>>> # Thread 2
>>> with lock:
...     pointer_b.contents = 42
```

Type conversions

Usually, ctypes does strict type checking. This means, if you have `POINTER(c_int)` in the `argtypes` list of a function or as the type of a member field in a structure definition, only instances of exactly the same type are accepted.

There are some exceptions to this rule, where ctypes accepts other objects. For example, you can pass compatible array instances instead of pointer types. So, for `POINTER(c_int)`, ctypes accepts an array of `c_int`:

```
>>> class Bar(Structure):
...     _fields_ = [("count", c_int), ("values", POINTER(c_int))]
...
>>> bar = Bar()
>>> bar.values = (c_int * 3)(1, 2, 3)
>>> bar.count = 3
>>> for i in range(bar.count):
...     print(bar.values[i])
...
1
2
3
>>>
```

In addition, if a function argument is explicitly declared to be a pointer type (such as `POINTER(c_int)`) in `argtypes`, an object of the pointed type (`c_int` in this case) can be passed to the function. ctypes will apply the `requiredbyref()` conversion in this case automatically.

To set a `POINTER` type field to `NULL`, you can assign `None`:

```
>>> bar.values = None
>>>
```

Sometimes you have instances of incompatible types. In C, you can cast one type into another type. ctypes provides a `cast()` function which can be used in the same way. The `Bar` structure defined above accepts `POINTER(c_int)` pointers or `c_int` arrays for its `values` field, but not instances of other types:

```
>>> bar.values = (c_byte * 4)()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: incompatible types, c_byte_Array_4 instance instead of LP_c_long instance
>>>
```

For these cases, the `cast()` function is handy.

The `cast()` function can be used to cast a ctypes instance into a pointer to a different ctypes data type. `cast()` takes two parameters, a ctypes object that is or can be converted to a pointer of some kind, and a ctypes pointer type. It returns an instance of the second argument, which references the same memory block as the first argument:

```
>>> a = (c_byte * 4)()
>>> cast(a, POINTER(c_int))
<ctypes.LP_c_long object at ...>
>>>
```

So, `cast()` can be used to assign to the `values` field of `Bar` the structure:

```
>>> bar = Bar()
>>> bar.values = cast((c_byte * 4)(), POINTER(c_int))
>>> print(bar.values[0])
0
>>>
```

Incomplete Types

Incomplete Types are structures, unions or arrays whose members are not yet specified. In C, they are specified by forward declarations, which are defined later:

```
struct cell; /* forward declaration */

struct cell {
    char *name;
    struct cell *next;
```

```
};
```

The straightforward translation into ctypes code would be this, but it does not work:

```
>>> class cell(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("next", POINTER(cell))]
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in cell
NameError: name 'cell' is not defined
>>>
```

because the newclasscell is not available in the class statement itself. In ctypes, we can define the cell class and set the `_fields_` attribute later, after the class statement:

```
>>> from ctypes import *
>>> class cell(Structure):
...     pass
...
>>> cell._fields_ = [("name", c_char_p),
...                  ("next", POINTER(cell))]
>>>
```

Let's try it. We create two instances of cell, and let them point to each other, and finally follow the pointer chain a few times:

```
>>> c1 = cell()
>>> c1.name = b"foo"
>>> c2 = cell()
>>> c2.name = b"bar"
>>> c1.next = pointer(c2)
>>> c2.next = pointer(c1)
>>> p = c1
>>> for i in range(8):
...     print(p.name, end=" ")
...     p = p.next[0]
...
foo bar foo bar foo bar foo bar
>>>
```

Callback functions

ctypes allows creating C callable function pointers from Python callables. These are sometimes called callback functions.

First, you must create a class for the callback function. The class knows the calling convention, the return type, and the number and types of arguments this function will receive.

The `CFUNCTYPE()` factory function creates types for callback functions using the cdecl calling convention. On Windows, the `WINFUNCTYPE()` factory function creates types for callback functions using the stdcall calling convention.

Both of these factory functions are called with the result type as first argument, and the callback functions expected argument types as the remaining arguments.

I will present an example here which uses the standard C library's `qsort()` function, that is used to sort items with the help of a callback function. `qsort()` will be used to sort an array of integers:

```
>>> IntArray5 = c_int * 5
>>> ia = IntArray5(5, 1, 7, 33, 99)
>>> qsort = libc.qsort
>>> qsort.restype = None
>>>
```

qsort() must be called with a pointer to the data to sort, the number of items in the data array, the size of one item, and a pointer to the comparison function, the callback. The callback will then be called with two pointers to items, and it must return a negative integer if the first item is smaller than the second, a zero if they are equal, and a positive integer otherwise.

So our callback function receives pointers to integers, and must return an integer. First we create the type for the callback function:

```
>>> CMPFUNC = CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
>>>
```

To get started, here is a simple callback that shows the values it gets passed:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return 0
...
>>> cmp_func = CMPFUNC(py_cmp_func)
>>>
```

The result:

```
>>> qsort(ia, len(ia), sizeof(c_int), cmp_func)
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 5 7
py_cmp_func 1 7
>>>
```

Now we can actually compare the two items and return a useful result:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>>
>>> qsort(ia, len(ia), sizeof(c_int), CMPFUNC(py_cmp_func))
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

As we can easily check, our array is sorted now:

```
>>> for i in ia: print(i, end=" ")
...
1 5 7 33 99
>>>
```

The function factories can be used as decorator factories, so we may as well write:

```
>>> @CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
... def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>> qsort(ia, len(ia), sizeof(c_int), py_cmp_func)
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

■ Nota: NoteMake sure you keep references toCFUNCTYPE()objects as long as they are used from C code.ctypesdoesn't, and if you don't, they may be garbage collected, crashing your program when a callback is made.Also, note that if the callback function is called in a thread created outside of Python's control (e.g. by the foreign code that calls the callback), ctypes creates a new dummy Python thread on every invocation. This behavior is correct for most purposes, but it means that values stored withthreading.localwillnotsurvive across different callbacks, even when those calls are made from the same C

Accessing values exported from dlls

Some shared libraries not only export functions, they also export variables. An example in the Python library itself is thePy_Version, Python runtime version number encoded in a single constant integer.

ctypescan access values like this with theirin_dll()class methods of the type.pythonapiis a predefined symbol giving access to the Python C api:

```
>>> version = ctypes.c_int.in_dll(ctypes.pythonapi, "Py_Version")
>>> print(hex(version.value))
0x30c00a0
```

An extended example which also demonstrates the use of pointers accesses thePyImport_FrozenModulespointer exported by Python.

Quoting the docs for that value:

This pointer is initialized to point to an array of_frozenrecords, terminated by one whose members are allNULLor zero. When a frozen module is imported, it is searched in this table. Third-party code could play tricks with this to provide a dynamically created collection of frozen modules.

So manipulating this pointer could even prove useful. To restrict the example size, we show only how this table can be read withctypes:

```
>>> from ctypes import *
>>>
>>> class struct_frozen(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("code", POINTER(c_ubyte)),
...                 ("size", c_int),
...                 ("get_code", POINTER(c_ubyte)), # Function pointer
...                 ]
...
>>>
```

We have defined the_frozendata type, so we can get the pointer to the table:

```
>>> FrozenTable = POINTER(struct_frozen)
>>> table = FrozenTable.in_dll(pythonapi, "_PyImport_FrozenBootstrap")
>>>
```

Since

```
>>> for item in table:
...     if item.name is None:
...         break
...     print(item.name.decode("ascii"), item.size)
...
_frozen_importlib 31764
_frozen_importlib_external 41499
zipimport 12345
>>>
```

The fact that standard Python has a frozen module and a frozen package (indicated by the negativesize member) is not well known, it is only used for testing. Try it out withimport__hello__for example.

Surprises

There are some edges in ctypes where you might expect something other than what actually happens.

Consider the following example:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
>>> class RECT(Structure):
...     _fields_ = ("a", POINT), ("b", POINT)
...
>>> p1 = POINT(1, 2)
>>> p2 = POINT(3, 4)
>>> rc = RECT(p1, p2)
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
1 2 3 4
>>> # now swap the two points
>>> rc.a, rc.b = rc.b, rc.a
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
3 4 3 4
>>>
```

Hm. We certainly expected the last statement to print 3 4 1 2. What happened? Here are the steps of the `rc.a, rc.b = rc.b, rc.a` line above:

```
>>> temp0, temp1 = rc.b, rc.a
>>> rc.a = temp0
>>> rc.b = temp1
>>>
```

Note that `temp0` and `temp1` are objects still using the internal buffer of the `rc` object above. So executing `rc.a = temp0` copies the buffer contents of `temp0` into `rc`'s buffer. This, in turn, changes the contents of `temp1`. So, the last assignment `rc.b = temp1`, doesn't have the expected effect.

Keep in mind that retrieving sub-objects from Structure, Unions, and Arrays doesn't copy the sub-object, instead it retrieves a wrapper object accessing the root-object's underlying buffer.

Another example that may behave differently from what one would expect is this:

```
>>> s = c_char_p()
>>> s.value = b"abc def ghi"
>>> s.value
b'abc def ghi'
>>> s.value is s.value
False
>>>
```

■ **Nota:** Note Objects instantiated from `c_char_p` can only have their value set to bytes or integers.

Why is it printing `False`? ctypes instances are objects containing a memory block plus some descriptors accessing the contents of the memory. Storing a Python object in the memory block does not store the object itself, instead the contents of the object is stored. Accessing the contents again constructs a new Python object each time!

Variable-sized data types

ctypes provides some support for variable-sized arrays and structures.

The `resize()` function can be used to resize the memory buffer of an existing ctypes object. The function takes the object as first argument, and the requested size in bytes as the second argument. The memory block cannot be made smaller than the natural memory block specified by the object's type, a `ValueError` is raised if this is tried:

```
>>> short_array = (c_short * 4)()
```

```
>>> print(sizeof(short_array))
8
>>> resize(short_array, 4)
Traceback (most recent call last):
...
ValueError: minimum size is 8
>>> resize(short_array, 32)
>>> sizeof(short_array)
32
>>> sizeof(type(short_array))
8
>>>
```

This is nice and fine, but how would one access the additional elements contained in this array? Since the type still only knows about 4 elements, we get errors accessing other elements:

```
>>> short_array[:]
[0, 0, 0, 0]
>>> short_array[7]
Traceback (most recent call last):
...
IndexError: invalid index
>>>
```

Another way to use variable-sized data types with `ctypes` is to use the dynamic nature of Python, and (re-)define the data type after the required size is already known, on a case by case basis.

ctypes reference

Finding shared libraries

When programming in a compiled language, shared libraries are accessed when compiling/linking a program, and when the program is run.

The purpose of the `find_library()` function is to locate a library in a way similar to what the compiler or runtime loader does (on platforms with several versions of a shared library the most recent should be loaded), while the `ctypes` library loaders act like when a program is run, and call the runtime loader directly.

The `ctypes.util` module provides a function which can help to determine the library to load.

ctypes.util.find_library(name)

Try to find a library and return a pathname. `name` is the library name without any prefix like `lib`, suffix like `.so`, `.dylib` or version number (this is the form used for the `posix` linker option `-l`). If no library can be found, returns `None`.

The exact functionality is system dependent.

On Linux, `find_library()` tries to run external programs (`/sbin/ldconfig`, `gcc`, `objdump` and `ld`) to find the library file. It returns the filename of the library file.

Note that if the output of these programs does not correspond to the dynamic linker used by Python, the result of this function may be misleading.

Changed in version 3.6: On Linux, the value of the environment variable `LD_LIBRARY_PATH` is used when searching for libraries, if a library cannot be found by any other means.

Here are some examples:

```
>>> from ctypes.util import find_library
>>> find_library("m")
'libm.so.6'
```



```
>>> find_library("c")
'libc.so.6'
>>> find_library("bz2")
'libbz2.so.1.0'
>>>
```

On macOS and Android, `find_library()` uses the system's standard naming schemes and paths to locate the library, and returns a full pathname if successful:

```
>>> from ctypes.util import find_library
>>> find_library("c")
'/usr/lib/libc.dylib'
>>> find_library("m")
'/usr/lib/libm.dylib'
>>> find_library("bz2")
'/usr/lib/libbz2.dylib'
>>> find_library("AGL")
'/System/Library/Frameworks/AGL.framework/AGL'
>>>
```

On Windows, `find_library()` searches along the system search path, and returns the full pathname, but since there is no predefined naming scheme a call like `find_library("c")` will fail and return `None`.

If wrapping a shared library with `ctypes`, it may be better to determine the shared library name at development time, and hardcode that into the wrapper module instead of using `find_library()` to locate the library at runtime.

Listing loaded shared libraries

When writing code that relies on code loaded from shared libraries, it can be useful to know which shared libraries have already been loaded into the current process.

The `ctypes.util` module provides the `dlllist()` function, which calls the different APIs provided by the various platforms to help determine which shared libraries have already been loaded into the current process.

The exact output of this function will be system dependent. On most platforms, the first entry of this list represents the current process itself, which may be an empty string. For example, on glibc-based Linux, the return may look like:

```
>>> from ctypes.util import dlllist
>>> dlllist()
['', 'linux-vdso.so.1', '/lib/x86_64-linux-gnu/libm.so.6', '/lib/x86_64-linux-gnu/libc.so.6', ... ]
```

Loading shared libraries

There are several ways to load shared libraries into the Python process. One way is to instantiate `CDLL` or one of its subclasses:

`class ctypes.CDLL(name, mode=DEFAULT_MODE, handle=None, use_errno=False, use_last_error=False, winmode=None)`

`_handle`

`_name`

Represents a loaded shared library. Functions in this library use the standard C calling convention, and are assumed to return `int`. The Python global interpreter lock is released before calling any function exported by these libraries, and reacquired afterwards. For different function behavior, use a subclass: `OleDLL`, `WinDLL`, or `PyDLL`. If you have an existing handle to an already loaded shared library, it can be passed as the `handle` argument to wrap the opened library in a new `CDLL` object. In this case, `name` is

The system handle used to access the library.

The name of the library passed in the constructor.

classctypes.OleDLL

SeeCDLL, the superclass, for common information. Functions in this library use the stdcall calling convention, and are assumed to return the windows specific HRESULT code. HRESULT values contain information specifying whether the function call failed or succeeded, together with additional error code. If the return value signals a failure, an OSError is automatically raised. Availability: Windows Changed in version 3.3: WindowsError used to be raised, which is now an alias of OSError.

classctypes.WinDLL

SeeCDLL, the superclass, for common information. Functions in these libraries use the stdcall calling convention, and are assumed to return int by default. Availability: Windows

classctypes.PyDLL

SeeCDLL, the superclass, for common information. When functions in this library are called, the Python GIL is not released during the function call, and after the function execution the Python error flag is checked. If the error flag is set, a Python exception is raised. Thus, this is only useful to call Python C API functions directly.

ctypes.RTLD_GLOBAL

Flag to use as mode parameter. On platforms where this flag is not available, it is defined as the integer zero.

ctypes.RTLD_LOCAL

Flag to use as mode parameter. On platforms where this is not available, it is the same as RTLD_GLOBAL.

ctypes.DEFAULT_MODE

The default mode which is used to load shared libraries. On OSX 10.3, this is RTLD_GLOBAL, otherwise it is the same as RTLD_LOCAL.

Shared libraries can also be loaded by using one of the prefabricated objects, which are instances of the LibraryLoader class, either by calling the LoadLibrary() method, or by retrieving the library as attribute of the loader instance.

classctypes.LibraryLoader(dlltype)

LoadLibrary(name)

Class which loads shared libraries. dlltype should be one of the CDLL, PyDLL, WinDLL, or OleDLL types. __getattr__() has special behavior: It allows loading a shared library by accessing it as attribute of a library loader instance. The result is cached, so repeated attribute accesses return the same library each time. LoadLibrary(name) Load a shared library into the process and return it. This method always returns a new instance of the library.

Load a shared library into the process and return it. This method always returns a new instance of the library.

These prefabricated library loaders are available:

ctypes.cdll

Creates CDLL instances.

ctypes.windll

Creates WinDLL instances. Availability: Windows

ctypes.ole32

Creates OleDLL instances. Availability: Windows

ctypes.pydll

Creates PyDLL instances.

For accessing the C Python api directly, a ready-to-use Python shared library object is available:

ctypes.pythonapi

An instance of `PyDLL` that exposes Python C API functions as attributes. Note that all these functions are assumed to return `Cint`, which is of course not always the truth, so you have to assign the correct `restype` attribute to use these functions.

Loading a library through any of these objects raises an `auditing event` `ctypes.dlopenwith` string argument `name`, the name used to load the library.

Accessing a function on a loaded library raises an `auditing event` `ctypes.dlsymwith` arguments `library` (the library object) and `name` (the symbol's name as a string or integer).

In cases when only the library handle is available rather than the object, accessing a function raises an `auditing event` `ctypes.dlsym/handlewith` arguments `handle` (the raw library handle) and `name`.

Foreign functions

As explained in the previous section, foreign functions can be accessed as attributes of loaded shared libraries. The function objects created in this way by default accept any number of arguments, accept any `ctypes` data instances as arguments, and return the default result type specified by the library loader.

They are instances of a private local class `_FuncPtr` (not exposed in `ctypes`) which inherits from the private `_CFuncPtr` class:

```
>>> import ctypes
>>> lib = ctypes.CDLL(None)
>>> issubclass(lib._FuncPtr, ctypes._CFuncPtr)
True
>>> lib._FuncPtr is ctypes._CFuncPtr
False
```

class ctypes._CFuncPtr

restype

argtypes

errcheck

callable(result, func, arguments)

Base class for C callable foreign functions. Instances of foreign functions are also C compatible data types; they represent C function pointers. This behavior can be customized by assigning to special attributes of the foreign function object. `restype` Assign a `ctypes` type to specify the result type of the foreign function.

`UseNoneforvoid`, a function not returning anything. It is possible to assign a callable Python object that is not a `ctypes` type, in this case the function is assumed to return a `Ci`

Assign a `ctypes` type to specify the result type of the foreign function. `UseNoneforvoid`, a function not returning anything. It is possible to assign a callable Python object that is not a `ctypes` type, in this case the function is assumed to return a `Cint`, and the callable will be called with this integer, allowing further processing or error checking. Using this is deprecated, for more flexible post processing or error checking use a `ctypes` data type `asrestype` and assign a callable to the `errcheck`

Assign a tuple of `ctypes` types to specify the argument types that the function accepts. Functions using the `stdcall` calling convention can only be called with the same number of arguments as the length of this tuple; functions using the C calling convention accept additional, unspecified arguments as well. When a foreign function is called, each actual argument is passed to the `from_param()` class method of the items in the `argtypes` tuple, this method allows adapting the actual argument to an object that

Assign a Python function or another callable to this attribute. The callable will be called with three or more arguments: `callable(result, func, arguments)` `result` is what the foreign function returns, as specified by `therestypeattribute`. `func` is the foreign function object itself, this allows reusing the same callable object to check or post process the results of several functions. `arguments` is a tuple containing the parameters originally passed to the function call, this allows specializing the behavior

`result` is what the foreign function returns, as specified by `therestypeattribute`. `func` is the foreign function object itself, this allows reusing the same callable object to check or post process the results of several functions. `arguments` is a tuple containing the parameters originally passed to the function call, this allows specializing the behavior on the arguments used.

On Windows, when a foreign function call raises a system exception (for example, due to an access violation), it will be captured and replaced with a suitable Python exception. Further, an auditing `eventtypes.set_exception` with `argumentcode` will be raised, allowing an audit hook to replace the exception with its own.

Some ways to invoke foreign function calls as well as some of the functions in this module may raise an auditing `eventtypes.call_function` with `argumentsfunctionpointerandarguments`.

Function prototypes

Foreign functions can also be created by instantiating function prototypes. Function prototypes are similar to function prototypes in C; they describe a function (return type, argument types, calling convention) without defining an implementation. The factory functions must be called with the desired result type and the argument types of the function, and can be used as decorator factories, and as such, be applied to functions through the `@wrappersyntax`. See `Callback functions` for examples.

ctypes.CFUNCTYPE(restype,*argtypes,use_errno=False,use_last_error=False)

The returned function prototype creates functions that use the standard C calling convention. The function will release the GIL during the call. If `use_errno` is set to true, the `ctypes` private copy of the `systemerrno` variable is exchanged with the `realerrno` value before and after the call; `use_last_error` does the same for the Windows error code.

ctypes.WINFUNCTYPE(restype,*argtypes,use_errno=False,use_last_error=False)

The returned function prototype creates functions that use the `stdcall` calling convention. The function will release the GIL during the call. `use_errno` and `use_last_error` have the same meaning as above. Availability: Windows

ctypes.PYFUNCTYPE(restype,*argtypes)

The returned function prototype creates functions that use the Python calling convention. The function will not release the GIL during the call.

Function prototypes created by these factory functions can be instantiated in different ways, depending on the type and number of the parameters in the call:

prototype(address)

Returns a foreign function at the specified address which must be an integer.

prototype(callable)

Create a C callable function (a callback function) from a Python callable.

prototype(func_spec[,paramflags])

Returns a foreign function exported by a shared library. `func_spec` must be a 2-tuple (`name_or_ordinal`, `library`). The first item is the name of the exported function as string, or the ordinal of the exported function as small integer. The second item is the shared library instance.

prototype(vtbl_index,name[,paramflags[,iid]])

Returns a foreign function that will call a COM method. `vtbl_index` is the index into the virtual function table, a small non-negative integer. `name` is name of the COM method. `iid` is an optional pointer to the interface identifier which is used in extended error reporting. If `iid` is not specified, an `OSError` is raised if the COM method call fails. If `iid` is specified, a `COMError` is raised instead. COM methods use a special calling convention: They require a pointer to the COM interface as first argument, in addition

The optional `paramflags` parameter creates foreign function wrappers with much more functionality than the features described above.

`paramflags` must be a tuple of the same length as `argtypes`.

Each item in this tuple contains further information about a parameter, it must be a tuple containing one, two, or three items.

The first item is an integer containing a combination of direction flags for the parameter:

1

2

4

Specifies an input parameter to the function.

Output parameter. The foreign function fills in a value.

Input parameter which defaults to the integer zero.

The optional second item is the parameter name as string. If this is specified, the foreign function can be called with named parameters.

The optional third item is the default value for this parameter.

The following example demonstrates how to wrap the `WindowsMessageBoxW` function so that it supports default parameters and named arguments. The C declaration from the windows header file is this:

```
WINUSERAPI int WINAPI
MessageBoxW(
    HWND hwnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT uType);
```

Here is the wrapping with `ctypes`:

```
>>> from ctypes import c_int, WINFUNCTYPE, windll
>>> from ctypes.wintypes import HWND, LPCWSTR, UINT
>>> prototype = WINFUNCTYPE(c_int, HWND, LPCWSTR, LPCWSTR, UINT)
>>> paramflags = (1, "hwnd", 0), (1, "text", "Hi"), (1, "caption", "Hello from ctypes"), (1, "flags", 0)
>>> MessageBox = prototype(("MessageBoxW", windll.user32), paramflags)
```

The `MessageBox` foreign function can now be called in these ways:

```
>>> MessageBox()
>>> MessageBox(text="Spam, spam, spam")
>>> MessageBox(flags=2, text="foo bar")
```

A second example demonstrates output parameters. The `win32GetWindowRect` function retrieves the dimensions of a specified window by copying them into `RECT` structure that the caller has to supply. Here is the C declaration:

```
WINUSERAPI BOOL WINAPI
GetWindowRect(
    HWND hwnd,
```

```
LPRECT lpRect);
```

Here is the wrapping withctypes:

```
>>> from ctypes import POINTER, WINFUNCTYPE, windll, WinError
>>> from ctypes.wintypes import BOOL, HWND, RECT
>>> prototype = WINFUNCTYPE(BOOL, HWND, POINTER(RECT))
>>> paramflags = (1, "hwnd"), (2, "lprect")
>>> GetWindowRect = prototype("GetWindowRect", windll.user32, paramflags)
>>>
```

Functions with output parameters will automatically return the output parameter value if there is a single one, or a tuple containing the output parameter values when there are more than one, so the GetWindowRect function now returns a RECT instance, when called.

Output parameters can be combined with theerrcheckprotocol to do further output processing and error checking. The win32GetWindowRectapi function returns aBOOLto signal success or failure, so this function could do the error checking, and raises an exception when the api call failed:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
...     return args
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

If theerrcheckfunction returns the argument tuple it receives unchanged,ctypescontinues the normal processing it does on the output parameters. If you want to return a tuple of window coordinates instead of aRECTinstance, you can retrieve the fields in the function and return them instead, the normal processing will no longer take place:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
...     rc = args[1]
...     return rc.left, rc.top, rc.bottom, rc.right
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

Utility functions

ctypes.addressof(obj)

Returns the address of the memory buffer as integer.objmust be an instance of a ctypes type.Raises anauditing eventctypes.addressofwith argumentobj.

ctypes.alignment(obj_or_type)

Returns the alignment requirements of a ctypes type.obj_or_typemust be a ctypes type or instance.

ctypes.byref(obj[,offset])

Returns a light-weight pointer toobj, which must be an instance of a ctypes type.offsetdefaults to zero, and must be an integer that will be added to the internal pointer value.byref(obj,offset)corresponds to this C code:(((char*)&obj)+offset)The returned object can only be used as a foreign function call parameter. It behaves similar topointer(obj), but the construction is a lot faster.

ctypes.CopyComPointer(src,dst)

Copies a COM pointer fromsrctodstand returns the Windows specificHRESULTvalue.Ifsrcis notNULL, itsAddRefmethod is called, incrementing the reference count.In contrast, the reference count ofdstwill not be decremented before assigning the new value. UnlessdstisNULL, the caller is responsible for decrementing the

reference count by calling its `Release` method when necessary. Availability: Windows Added in version 3.14.

`ctypes.cast(obj, type)`

This function is similar to the cast operator in C. It returns a new instance of `type` which points to the same memory block as `obj`. `type` must be a pointer type, and `obj` must be an object that can be interpreted as a pointer.

`ctypes.create_string_buffer(init, size=None)`

`ctypes.create_string_buffer(size)`

This function creates a mutable character buffer. The returned object is a `ctypes` array of `c_char`. If `size` is given (and not `None`), it must be an int. It specifies the size of the returned array. If the `init` argument is given, it must be bytes. It is used to initialize the array items. Bytes not initialized this way are set to zero (NUL). If `size` is not given (or if it is `None`), the buffer is made one element larger than `init`, effectively adding a NUL terminator. If both arguments are given, `size` must not be less

`ctypes.create_unicode_buffer(init, size=None)`

`ctypes.create_unicode_buffer(size)`

This function creates a mutable unicode character buffer. The returned object is a `ctypes` array of `c_wchar`. The function takes the same arguments as `create_string_buffer()` except `init` must be a string and `size` counts `c_wchar`. Raises an `audit` event `ctypes.create_unicode_buffer` with arguments `init, size`.

`ctypes.DllCanUnloadNow()`

This function is a hook which allows implementing in-process COM servers with `ctypes`. It is called from the `DllCanUnloadNow` function that the `_ctypes` extension dll exports. Availability: Windows

`ctypes.DllGetClassObject()`

This function is a hook which allows implementing in-process COM servers with `ctypes`. It is called from the `DllGetClassObject` function that the `_ctypes` extension dll exports. Availability: Windows

`ctypes.util.find_library(name)`

Try to find a library and return a pathname. `name` is the library name without any prefix like `lib`, suffix like `.so`, `.dylib` or version number (this is the form used for the `posix` linker option `-l`). If no library can be found, returns `None`. The exact functionality is system dependent. See [Finding shared libraries](#) for complete documentation.

`ctypes.util.find_msvcr()`

Returns the filename of the VC runtime library used by Python, and by the extension modules. If the name of the library cannot be determined, `None` is returned. If you need to free memory, for example, allocated by an extension module with a call to `free(void*)`, it is important that you use the function in the same library that allocated the memory. Availability: Windows

`ctypes.util.dllist()`

Try to provide a list of paths of the shared libraries loaded into the current process. These paths are not normalized or processed in any way. The function can raise `OSError` if the underlying platform APIs fail. The exact functionality is system dependent. On most platforms, the first element of the list represents the current executable file. It may be an empty string. Availability: Windows, macOS, iOS, glibc, BSD libc, musl Added in version 3.14.

`ctypes.FormatError([code])`

Returns a textual description of the error code `code`. If no error code is specified, the last error code is used by calling the Windows API function `GetLastError()`. Availability: Windows

`ctypes.GetLastError()`

Returns the last error code set by Windows in the calling thread. This function calls the `WindowsGetLastError()` function directly, it does not return the ctypes-private copy of the error code. Availability: Windows

ctypes.get_errno()

Returns the current value of the ctypes-private copy of the `systemerrno` variable in the calling thread. Raises an `auditing event` `ctypes.get_errno` with no arguments.

ctypes.get_last_error()

Returns the current value of the ctypes-private copy of the `systemLastError` variable in the calling thread. Availability: Windows Raises an `auditing event` `ctypes.get_last_error` with no arguments.

ctypes.memmove(dst,src,count)

Same as the standard C `memmove` library function: copies `count` bytes from `src` to `dst`. `dst` and `src` must be integers or ctypes instances that can be converted to pointers.

ctypes.memset(dst,c,count)

Same as the standard C `memset` library function: fills the memory block at address `dst` with `count` bytes of value `c`. `dst` must be an integer specifying an address, or a ctypes instance.

ctypes.POINTER(type,/)

Create or return a ctypes pointer type. Pointer types are cached and reused internally, so calling this function repeatedly is cheap. `type` must be a ctypes type. CPython implementation detail: The resulting pointer type is cached in the `__pointer_type__` attribute of `type`. It is possible to set this attribute before the first call to `POINTER` in order to set a custom pointer type. However, doing this is discouraged: manually creating a suitable pointer type is difficult without relying on implementation details.

ctypes.pointer(obj,/)

Create a new pointer instance, pointing to `obj`. The returned object is of the type `POINTER(type(obj))`. Note: If you just want to pass a pointer to an object to a foreign function call, you should use `byref(obj)` which is much faster.

ctypes.resize(obj,size)

This function resizes the internal memory buffer of `obj`, which must be an instance of a ctypes type. It is not possible to make the buffer smaller than the native size of the object's type, as given by `sizeof(type(obj))`, but it is possible to enlarge the buffer.

ctypes.set_errno(value)

Set the current value of the ctypes-private copy of the `systemerrno` variable in the calling thread to `value` and return the previous value. Raises an `auditing event` `ctypes.set_errno` with argument `errno`.

ctypes.set_last_error(value)

Sets the current value of the ctypes-private copy of the `systemLastError` variable in the calling thread to `value` and return the previous value. Availability: Windows Raises an `auditing event` `ctypes.set_last_error` with argument `error`.

ctypes.sizeof(obj_or_type)

Returns the size in bytes of a ctypes type or instance memory buffer. Does the same as the C `sizeof` operator.

ctypes.string_at(ptr,size=-1)

Return the byte string at void `*ptr`. If `size` is specified, it is used as size, otherwise the string is assumed to be zero-terminated. Raises an `auditing event` `ctypes.string_at` with arguments `sptr`, `size`.

ctypes.WinError(code=None,descr=None)

Creates an instance of `OSError`. If `code` is not specified, `GetLastError()` is called to determine the error code. If `descr` is not specified, `FormatError()` is called to get a textual description of the error. Availability: Windows Changed in version 3.3: An instance of `WindowsError` used to be created, which is now an alias of `OSError`.

ctypes.wstring_at(ptr,size=-1)

Return the wide-character string at void `*ptr`. If `size` is specified, it is used as the number of characters of the string, otherwise the string is assumed to be zero-terminated. Raises an auditing event `ctypes.wstring_at` with arguments `ptr,size`.

ctypes.memoryview_at(ptr,size,readonly=False)

Return a `memoryview` object of length `size` that references memory starting at void `*ptr`. If `readonly` is true, the returned `memoryview` object can not be used to modify the underlying memory. (Changes made by other means will still be reflected in the returned object.) This function is similar to `string_at()` with the key difference of not making a copy of the specified memory. It is a semantically equivalent (but more efficient) alternative to `memoryview((c_byte*size).from_address(ptr))`. (While `from_address()` only

Data types

class ctypes._CData

from_buffer(source[,offset])

from_buffer_copy(source[,offset])

from_address(address)

from_param(obj)

in_dll(library,name)

__pointer_type__

__b_base__

__b_needsfree__

__objects

This non-public class is the common base class of all ctypes data types. Among other things, all ctypes type instances contain a memory block that hold C compatible data; the address of the memory block is returned by the `addressof()` helper function. Another instance variable is exposed as `__objects`; this contains other Python objects that need to be kept alive in case the memory block contains pointers. Common methods of ctypes data types, these are all class methods (to be exact, they are methods o

This method returns a ctypes instance that shares the buffer of the `source` object. The `source` object must support the writeable buffer interface. The optional `offset` parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a `ValueError` is raised. Raises an auditing event `ctypes.cdata/buffer` with arguments `pointer,size,offset`.

This method creates a ctypes instance, copying the buffer from the `source` object buffer which must be readable. The optional `offset` parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a `ValueError` is raised. Raises an auditing event `ctypes.cdata/buffer` with arguments `pointer,size,offset`.

This method returns a ctypes type instance using the memory specified by `address` which must be an integer. This method, and others that indirectly call this method, raises an auditing event `ctypes.cdata` with argument `address`.

This method adapts `obj` to a ctypes type. It is called with the actual object used in a foreign function call when the type is present in the foreign function's argument tuple; it must return an object that can be used as a function call parameter. All ctypes data types have a default implementation of this classmethod that normally returns `obj` if that is an instance of the type. Some types accept other objects as well.

This method returns a ctypes type instance exported by a shared library. `name` is the name of the symbol that exports the data, `library` is the loaded shared library.

The pointer type that was created by calling `POINTER()` for corresponding ctypes data type. If a pointer type was not yet created, the attribute is missing. Added in version 3.14.

Sometimes ctypes data instances do not own the memory block they contain, instead they share part of the memory block of a base object. The `_b_base_read-only` member is the root ctypes object that owns the memory block.

This read-only variable is true when the ctypes data instance has allocated the memory block itself, false otherwise.

This member is either `None` or a dictionary containing Python objects that need to be kept alive so that the memory block contents is kept valid. This object is only exposed for debugging; never modify the contents of this dictionary.

Fundamental data types

class ctypes._SimpleCData

value

type

This non-public class is the base class of all fundamental ctypes data types. It is mentioned here because it contains the common attributes of the fundamental ctypes data types. `_SimpleCData` is a subclass of `_CData`, so it inherits their methods and attributes. ctypes data types that are not and do not contain pointers can now be pickled. Instances have a single attribute: `value`. This attribute contains the actual value of the instance. For integer and pointer types, it is an integer, for character typ

This attribute contains the actual value of the instance. For integer and pointer types, it is an integer, for character types, it is a single character bytes object or string, for character pointer types it is a Python bytes object or string. When the `value` attribute is retrieved from a ctypes instance, usually a new object is returned each time. ctypes does not implement original object return, always a new object is constructed. The same is true for all other ctypes object instances.

Class attribute that contains an internal type code, as a single-character string. See **Fundamental data types** for a summary. Types marked * in the summary may be (or always are) aliases of a different `_SimpleCData` subclass, and will not necessarily use the listed type code. For example, if the platform's `long`, `longlong` and `time_t` types are the same, then `c_long`, `c_longlong` and `c_time_t` refer to a single class, `c_long`, whose `_type_code` is 'l'. The 'L' code will be unused. See also `The array and struct modules`, as we

Fundamental data types, when returned as foreign function call results, or, for example, by retrieving structure field members or array items, are transparently converted to native Python types. In other words, if a foreign function has a restype of `c_char_p`, you will always receive a Python bytes object, not a `c_char_p` instance.

Subclasses of fundamental data types do not inherit this behavior. So, if a foreign function's restype is a subclass of `c_void_p`, you will receive an instance of this subclass from the function call. Of course, you can get the value of the pointer by accessing the `value` attribute.

These are the fundamental ctypes data types:

class ctypes.c_byte

Represents the Csignedchardatatype, and interprets the value as small integer. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_char

Represents the Cchardatatype, and interprets the value as a single character. The constructor accepts an optional string initializer, the length of the string must be exactly one character.

classctypes.c_char_p

Represents the Cchar*datatype when it points to a zero-terminated string. For a general character pointer that may also point to binary data, POINTER(c_char) must be used. The constructor accepts an integer address, or a bytes object.

classctypes.c_double

Represents the Cdoubledatatype. The constructor accepts an optional float initializer.

classctypes.c_longdouble

Represents the Clongdoubledatatype. The constructor accepts an optional float initializer. On platforms where sizeof(longdouble)==sizeof(double) it is an alias to c_double.

classctypes.c_float

Represents the Cfloatdatatype. The constructor accepts an optional float initializer.

classctypes.c_double_complex

Represents the Cdoublecomplexdatatype, if available. The constructor accepts an optional complex initializer. Added in version 3.14.

classctypes.c_float_complex

Represents the Cfloatcomplexdatatype, if available. The constructor accepts an optional complex initializer. Added in version 3.14.

classctypes.c_longdouble_complex

Represents the Clongdoublecomplexdatatype, if available. The constructor accepts an optional complex initializer. Added in version 3.14.

classctypes.c_int

Represents the Csignedintdatatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where sizeof(int)==sizeof(long) it is an alias to c_long.

classctypes.c_int8

Represents the C 8-bitsignedintdatatype. It is an alias for c_byte.

classctypes.c_int16

Represents the C 16-bitsignedintdatatype. Usually an alias for c_short.

classctypes.c_int32

Represents the C 32-bitsignedintdatatype. Usually an alias for c_int.

classctypes.c_int64

Represents the C 64-bitsignedintdatatype. Usually an alias for c_longlong.

classctypes.c_long

Represents the Csignedlongdatatype. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_longlong

Represents the Csignedlonglongdatatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms wheresizeof(longlong)==sizeof(long)it is an alias to c_long.

classctypes.c_short

Represents the Csignedshortdatatype. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_size_t

Represents the Csize_tdatatype. Usually an alias for another unsigned integer type.

classctypes.c_ssize_t

Represents thePy_ssize_tdatatype. This is a signed version ofsize_t; that is, the POSIXssize_ttype. Usually an alias for another integer type.Added in version 3.2.

classctypes.c_time_t

Represents the Ctime_tdatatype. Usually an alias for another integer type.Added in version 3.12.

classctypes.c_ubyte

Represents the Cunsignedchardatatype, it interprets the value as small integer. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_uint

Represents the Cunsignedintdatatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms wheresizeof(int)==sizeof(long)it is an alias for c_ulong.

classctypes.c_uint8

Represents the C 8-bitunsignedintdatatype. It is an alias for c_ubyte.

classctypes.c_uint16

Represents the C 16-bitunsignedintdatatype. Usually an alias for c_ushort.

classctypes.c_uint32

Represents the C 32-bitunsignedintdatatype. Usually an alias for c_uint.

classctypes.c_uint64

Represents the C 64-bitunsignedintdatatype. Usually an alias for c_ulonglong.

classctypes.c_ulong

Represents the Cunsignedlongdatatype. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_ulonglong

Represents the Cunsignedlonglongdatatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms wheresizeof(longlong)==sizeof(long)it is an alias to c_long.

classctypes.c_ushort

Represents the Cunsignedshortdatatype. The constructor accepts an optional integer initializer; no overflow checking is done.

classctypes.c_void_p

Represents the Cvoid*type. The value is represented as integer. The constructor accepts an optional integer initializer.

classctypes.c_wchar

Represents the Cwchar_tdatatype, and interprets the value as a single character unicode string. The constructor accepts an optional string initializer, the length of the string must be exactly one character.

classctypes.c_wchar_p

Represents the Cwchar_t*datatype, which must be a pointer to a zero-terminated wide character string. The constructor accepts an integer address, or a string.

classctypes.c_bool

Represent the Cbooldatatype (more accurately, _Boolfrom C99). Its value can beTrueorFalse, and the constructor accepts any object that has a truth value.

classctypes.HRESULT

Represents aHRESULTvalue, which contains success or error information for a function or method call.Availability: Windows

classctypes.py_object

Represents the CPyObject*datatype. Calling this without an argument creates aNULLPyObject*pointer.Changed in version 3.14:py_objectis now ageneric type.

Thectypes.wintypesmodule provides quite some other Windows specific data types, for exampleHWND,WPARAM,VARIANT_BOOLorDWORD. Some useful structures likeMSGorRECTare also defined.

Structured data types

classctypes.Union(*args,kw)**

Abstract base class for unions in native byte order.Unions share common attributes and behavior with structures; seeStructureddocumentation for details.

classctypes.BigEndianUnion(*args,kw)**

Abstract base class for unions inbig endianbyte order.Added in version 3.11.

classctypes.LittleEndianUnion(*args,kw)**

Abstract base class for unions inlittle endianbyte order.Added in version 3.11.

classctypes.BigEndianStructure(*args,kw)**

Abstract base class for structures inbig endianbyte order.

classctypes.LittleEndianStructure(*args,kw)**

Abstract base class for structures inlittle endianbyte order.

Structures and unions with non-native byte order cannot contain pointer type fields, or any other data types containing pointer type fields.

classctypes.Structure(*args,kw)**

fields

pack

align

layout

anonymous

Abstract base class for structures innativebyte order. Concrete structure and union types must be created by subclassing one of these types, and at least define a `_fields_class` variable. `ctypes` will create descriptors which allow reading and writing the fields by direct attribute accesses. These are the `_fields_A` sequence defining the structure fields. The items must be 2-tuples or 3-tuples. The first item is the name of the field, the second item specifies the type of the field; it can be any `ctyp`

A sequence defining the structure fields. The items must be 2-tuples or 3-tuples. The first item is the name of the field, the second item specifies the type of the field; it can be any `ctypes` data type. For integer type fields like `int`, a third optional item can be given. It must be a small positive integer defining the bit width of the field. Field names must be unique within one structure or union. This is not checked, only one field can be accessed when names are repeated. It is possible t

An optional small integer that allows overriding the alignment of structure fields in the instance. This is only implemented for the MSVC-compatible memory layout (see `_layout_`). Setting `_pack_` to 0 is the same as not setting it at all. Otherwise, the value must be a positive power of two. The effect is equivalent to `#pragma pack(N)` in C, except `ctypes` may allow larger than what the compiler accepts. `_pack_` must already be defined when `_fields_` is assigned, otherwise it will have no effect. Deprecated since ve

An optional small integer that allows increasing the alignment of the structure when being packed or unpacked to/from memory. The value must not be negative. The effect is equivalent to `__attribute__((aligned(N)))` on GCC or `#pragma align(N)` on MSVC, except `ctypes` may allow values that the compiler would reject. `_align_` can only increase a structure's alignment requirements. Setting it to 0 or 1 has no effect. Using values that are not powers of two is discouraged and may lead to surprising behavior. `_align_mu`

An optional string naming the struct/union layout. It can currently be set to: "ms": the layout used by the Microsoft compiler (MSVC). On GCC and Clang, this layout can be selected with `__attribute__((ms_struct))`. "gcc-sysv": the layout used by GCC with the System V or "SysV-like" data model, as used on Linux and macOS. With this layout, `_pack_` must be unset or zero. If not set explicitly, `ctypes` will use a default that matches the platform conventions. This default may change in future Python releases

An optional sequence that lists the names of unnamed (anonymous) fields. `_anonymous_` must be already defined when `_fields_` is assigned, otherwise it will have no effect. The fields listed in this variable must be structure or union type fields. `ctypes` will create descriptors in the structure type that allows accessing the nested fields directly, without the need to create the structure or union field. Here is an example type (Windows):
`class U(Union): _fields_=[("lptdesc", POINTER(TYPEDESC)), ("lpadesc", POI`

`class ctypes.CField(*args,**kw)`

name

type

offset

byte_offset

byte_size

size

is_bitfield

bit_offset

bit_size

is_anonymous

Descriptor for fields of a `Structure` and `Union`. For example: `>>> class Color(Structure): ... _fields_ = (('red', c_uint8), ... ('green', c_uint8), ... ('blue', c_uint8), ... ('intense', c_bool, 1), ... ('blinking', c_bool, 1), ...) ... >>> Color.red < ctypes.CField 'red' type=c_ubyte, ofs=0, size=1 >>>> Color.green.type < class 'ctypes.c_ubyte' >>>> Color.blue.byte_offset 2 >>>> Color.intense < ctypes.CField 'intense' type=c_bool, ofs=3, bit_size=1, bit_offset=0 >>>> Color.blinking.bit_offset 1` All attributes are read-only. `CField` objects a

Name of the field, as a string.

Type of the field, as a `ctypes` class.

Offset of the field, in bytes. For bitfields, this is the offset of the underlying byte-aligned storage unit; see `bit_offset`.

Size of the field, in bytes. For bitfields, this is the size of the underlying storage unit. Typically, it has the same size as the bitfield's type.

For non-bitfields, equivalent to `byte_size`. For bitfields, this contains a backwards-compatible bit-packed value that combines `bit_size` and `bit_offset`. Prefer using the explicit attributes instead.

True if this is a bitfield.

The location of a bitfield within its storage unit, that is, within `byte_size` bytes of memory starting at `byte_offset`. To get the field's value, read the storage unit as an integer, shift left by `bit_offset` and take the `bit_size` least significant bits. For non-bitfields, `bit_offset` is zero and `bit_size` is equal to `byte_size * 8`.

True if this field is anonymous, that is, it contains nested sub-fields that should be merged into a containing structure or union.

Arrays and pointers

`ctypes.Array(*args)`

`_length_`

`_type_`

Abstract base class for arrays. The recommended way to create concrete array types is by multiplying any `ctypes` data type with a non-negative integer. Alternatively, you can subclass this type and define `_length_` and `_type_` class variables. Array elements can be read and written using standard subscript and slice accesses; for slice reads, the resulting object is not itself an `Array`. `_length_` A positive integer specifying the number of elements in the array. Out-of-range subscripts result in an `IndexError`.

A positive integer specifying the number of elements in the array. Out-of-range subscripts result in an `IndexError`. Will be returned by `len()`.

Specifies the type of each element in the array.

`ctypes.ARRAY(type, length)`

Create an array. Equivalent to `type * length`, where `type` is a `ctypes` data type and `length` an integer. This function is soft deprecated in favor of multiplication. There are no plans to remove it.

`ctypes._Pointer`

`_type_`

contents

Private, abstract base class for pointers. Concrete pointer types are created by calling `POINTER()` with the type that will be pointed to; this is done automatically by `pointer()`. If a pointer points to an array, its elements can be read and written using standard subscript and slice accesses. Pointer objects have no size, `solen()` will raise `TypeError`. Negative subscripts will read from the memory before the pointer (as in C), and out-of-range

subscripts will probably crash with an access violation (if

Specifies the type pointed to.

Returns the object to which the pointer points. Assigning to this attribute changes the pointer to point to the assigned object.

Exceptions

exceptiontypes.ArgumentError

This exception is raised when a foreign function call cannot convert one of the passed arguments.

exceptiontypes.COMError(hresult,text,details)

hresult

text

details

This exception is raised when a COM method call failed.
hresultThe integer value representing the error code.
textThe error message.
detailsThe 5-tuple(descr,source,helpfile,helpcontext,progid).
descris the textual description.
sourceis the language-dependentProgIDfor the class or application that raised the error.
helpfileis the path of the help file.
helpcontextis the help context identifier.
progidis theProgIDof the interface that defined the error.
Availability: WindowsAdded in version 3.14.

The integer value representing the error code.

The error message.

The 5-tuple(descr,source,helpfile,helpcontext,progid).
descris the textual description.
sourceis the language-dependentProgIDfor the class or application that raised the error.
helpfileis the path of the help file.
helpcontextis the help context identifier.
progidis theProgIDof the interface that defined the error.

Command-line interface libraries

The modules described in this chapter assist with implementing command line and terminal interfaces for applications.

Here's an overview:

- `argparse`— Parser for command-line options, arguments and subcommands
- `optparse`— Parser for command line options
- `getpass`— Portable password input
- `fileinput`— Iterate over lines from multiple input streams
- `curses`— Terminal handling for character-cell displays
- `curses.textpad`— Text input widget for curses programs
- `curses.ascii`— Utilities for ASCII characters
- `curses.panel`— A panel stack extension for curses
- `cmd`— Support for line-oriented command interpreters

argparse— Parser for command-line options, arguments and subcommands

Added in version 3.2.

Source code: [Lib/argparse.py](#)

■ Nota: NoteWhileargparseis the default recommended standard library module for implementing basic command line applications, authors with more exacting requirements for exactly how their command line applications behave may find it doesn't provide the necessary level of control. Refer toChoosing an argument parsing libraryfor alternatives to consider whenargparsedoesn't support behaviors that the application requires (such as entirely disabling support for interspersed options and positional arguments, or accepting option parameter values that start with-even when they correspond to another defined o

Tutorial

This page contains the API reference information. For a more gentle introduction to Python command-line parsing, have a look at theargparse tutorial.

Theargparsemodule makes it easy to write user-friendly command-line interfaces. The program defines what arguments it requires, andargparsewill figure out how to parse those out ofsys.argv. Theargparsemodule also automatically generates help and usage messages. The module will also issue errors when users give the program invalid arguments.

Theargparsemodule's support for command-line interfaces is built around an instance ofargparse.ArgumentParser. It is a container for argument specifications and has options that apply to the parser as whole:

```
parser = argparse.ArgumentParser(
    prog='ProgramName',
    description='What the program does',
    epilog='Text at the bottom of help')
```

TheArgumentParser.add_argument()method attaches individual argument specifications to the parser. It supports positional arguments, options that accept values, and on/off flags:

```
parser.add_argument('filename')           # positional argument
parser.add_argument('-c', '--count')      # option that takes a value
parser.add_argument('-v', '--verbose',
                    action='store_true')  # on/off flag
```

TheArgumentParser.parse_args()method runs the parser and places the extracted data in aargparse.Namespaceobject:

```
args = parser.parse_args()
print(args.filename, args.count, args.verbose)
```

■ Nota: NoteIf you're looking for a guide about how to upgradeoptparsecode toargparse, seeUpgrading Optparse Code.

ArgumentParser objects

classargparse.ArgumentParser(prog=None,usage=None,description=None,epilog=None,parents=[],formatter_class=argparse.HelpFormatter,prefix_chars='-',fromfile_prefix_chars=None,argument_default=None,conflict_handler='error',add_help=True,allow_abbrev=True,exit_on_error=True,*,suggest_on_error=False,colors)

Create a newArgumentParserobject. All parameters should be passed as keyword arguments. Each parameter has its own more detailed description below, but in short they are:prog- The name of the program

(default: generated from the `__main__` module attributes and `sys.argv[0]`)
usage- The string describing the program usage (default: generated from arguments added to parser)
description- Text to display before the argument help (by default, no text)
epilog- Text to display after the argument help (by default)

The following sections describe how each of these are used.

prog

By default, `ArgumentParser` calculates the name of the program to display in help messages depending on the way the Python interpreter was run:

- The `basename` of `sys.argv[0]` if a file was passed as argument.
- The Python interpreter name followed by `sys.argv[0]` if a directory or a zipfile was passed as argument.
- The Python interpreter name followed by `-m` followed by the module or package name if the `-m` option was used.

This default is almost always desirable because it will make the help messages match the string that was used to invoke the program on the command line. However, to change this default behavior, another value can be supplied using the `prog=` argument to `ArgumentParser`:

```
>>> parser = argparse.ArgumentParser(prog='myprogram')
>>> parser.print_help()
usage: myprogram [-h]
```

options:

```
-h, --help  show this help message and exit
```

Note that the program name, whether determined from `sys.argv[0]`, from the `__main__` module attributes or from the `prog=` argument, is available to help messages using the `%(prog)s` format specifier.

```
>>> parser = argparse.ArgumentParser(prog='myprogram')
>>> parser.add_argument('--foo', help='foo of the %(prog)s program')
>>> parser.print_help()
usage: myprogram [-h] [--foo F00]
```

options:

```
-h, --help  show this help message and exit
--foo F00   foo of the myprogram program
```

Changed in version 3.14: The default `prog` value now reflects how `__main__` was actually executed, rather than always being `os.path.basename(sys.argv[0])`.

usage

By default, `ArgumentParser` calculates the usage message from the arguments it contains. The default message can be overridden with the `usage=` keyword argument:

```
>>> parser = argparse.ArgumentParser(prog='PROG', usage='%(prog)s [options]')
>>> parser.add_argument('--foo', nargs='?', help='foo help')
>>> parser.add_argument('bar', nargs='+', help='bar help')
>>> parser.print_help()
usage: PROG [options]
```

positional arguments:

```
bar          bar help
```

options:

```
-h, --help  show this help message and exit
--foo [F00] foo help
```

The `%(prog)s` format specifier is available to fill in the program name in your usage messages.

When a custom usage message is specified for the main parser, you may also want to consider passing theprogargument toadd_subparsers()or theprogand theusagearguments toadd_parser(), to ensure consistent command prefixes and usage information across subparsers.

description

Most calls to theArgumentParserconstructor will use thedescription=keyword argument. This argument gives a brief description of what the program does and how it works. In help messages, the description is displayed between the command-line usage string and the help messages for the various arguments.

By default, the description will be line-wrapped so that it fits within the given space. To change this behavior, see theformatter_classargument.

epilog

Some programs like to display additional description of the program after the description of the arguments. Such text can be specified using theepilog=argument toArgumentParser:

```
>>> parser = argparse.ArgumentParser(
...     description='A foo that bars',
...     epilog="And that's how you'd foo a bar")
>>> parser.print_help()
usage: argparse.py [-h]
```

A foo that bars

options:

-h, --help show this help message and exit

And that's how you'd foo a bar

As with thedescriptionargument, theepilog=text is by default line-wrapped, but this behavior can be adjusted with theformatter_classargument toArgumentParser.

parents

Sometimes, several parsers share a common set of arguments. Rather than repeating the definitions of these arguments, a single parser with all the shared arguments and passed toparents=argument toArgumentParsercan be used. Theparents=argument takes a list ofArgumentParserobjects, collects all the positional and optional actions from them, and adds these actions to theArgumentParserobject being constructed:

```
>>> parent_parser = argparse.ArgumentParser(add_help=False)
>>> parent_parser.add_argument('--parent', type=int)

>>> foo_parser = argparse.ArgumentParser(parents=[parent_parser])
>>> foo_parser.add_argument('foo')
>>> foo_parser.parse_args(['--parent', '2', 'XXX'])
Namespace(foo='XXX', parent=2)

>>> bar_parser = argparse.ArgumentParser(parents=[parent_parser])
>>> bar_parser.add_argument('--bar')
>>> bar_parser.parse_args(['--bar', 'YYY'])
Namespace(bar='YYY', parent=None)
```

Note that most parent parsers will specifyadd_help=False. Otherwise, theArgumentParserwill see two-h/--helpoptions (one in the parent and one in the child) and raise an error.

■ **Nota:** NoteYou must fully initialize the parsers before passing them viaparents=. If you change the parent parsers after the child parser, those changes will not be reflected in the child.

formatter_class

ArgumentParser objects allow the help formatting to be customized by specifying an alternate formatting class. Currently, there are four such classes:

class argparse.RawDescriptionHelpFormatter

class argparse.RawTextHelpFormatter

class argparse.ArgumentDefaultsHelpFormatter

class argparse.MetavarTypeHelpFormatter

RawDescriptionHelpFormatter and RawTextHelpFormatter give more control over how textual descriptions are displayed. By default, ArgumentParser objects line-wrap the description and epilog texts in command-line help messages:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     description='''this description
...         was indented weird
...         but that is okay''',
...     epilog='''
...         likewise for this epilog whose whitespace will
...         be cleaned up and whose words will be wrapped
...         across a couple lines''')
>>> parser.print_help()
usage: PROG [-h]
```

this description was indented weird but that is okay

options:

-h, --help show this help message and exit

likewise for this epilog whose whitespace will be cleaned up and whose words
will be wrapped across a couple lines

Passing RawDescriptionHelpFormatter as formatter_class= indicates that description and epilog are already correctly formatted and should not be line-wrapped:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.RawDescriptionHelpFormatter,
...     description=textwrap.dedent('''\
...         Please do not mess up this text!
...         -----
...         I have indented it
...         exactly the way
...         I want it
...     '''))
>>> parser.print_help()
usage: PROG [-h]
```

Please do not mess up this text!

I have indented it
exactly the way
I want it

options:

-h, --help show this help message and exit

RawTextHelpFormatter maintains whitespace for all sorts of help text, including argument descriptions. However, multiple newlines are replaced with one. If you wish to preserve multiple blank lines, add spaces between the newlines.

ArgumentDefaultsHelpFormatter automatically adds information about default values to each of the argument help messages:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
>>> parser.add_argument('--foo', type=int, default=42, help='F00!')
>>> parser.add_argument('bar', nargs='*', default=[1, 2, 3], help='BAR!')
>>> parser.print_help()
usage: PROG [-h] [--foo F00] [bar ...]

positional arguments:
  bar                BAR! (default: [1, 2, 3])

options:
  -h, --help  show this help message and exit
  --foo F00   F00! (default: 42)
```

MetavarTypeHelpFormatter uses the name of the type argument for each argument as the display name for its values (rather than using the dest as the regular formatter does):

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.MetavarTypeHelpFormatter)
>>> parser.add_argument('--foo', type=int)
>>> parser.add_argument('bar', type=float)
>>> parser.print_help()
usage: PROG [-h] [--foo int] float

positional arguments:
  float

options:
  -h, --help  show this help message and exit
  --foo int
```

prefix_chars

Most command-line options will use - as the prefix, e.g. -f/--foo. Parsers that need to support different or additional prefix characters, e.g. for options like +f or /foo, may specify them using the prefix_chars= argument to the ArgumentParser constructor:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='+-/')
>>> parser.add_argument('+f')
>>> parser.add_argument('++bar')
>>> parser.parse_args('+f X ++bar Y'.split())
Namespace(bar='Y', f='X')
```

The prefix_chars= argument defaults to '-'. Supplying a set of characters that does not include - will cause -f/--foo options to be disallowed.

fromfile_prefix_chars

Sometimes, when dealing with a particularly long argument list, it may make sense to keep the list of arguments in a file rather than typing it out at the command line. If the fromfile_prefix_chars= argument is given to the ArgumentParser constructor, then arguments that start with any of the specified characters will be treated as files, and will be replaced by the arguments they contain. For example:

```
>>> with open('args.txt', 'w', encoding=sys.getfilesystemencoding()) as fp:
...     fp.write('-f\nbar')
...
>>> parser = argparse.ArgumentParser(fromfile_prefix_chars='@')
>>> parser.add_argument('-f')
>>> parser.parse_args(['-f', 'foo', '@args.txt'])
```

```
Namespace(f='bar')
```

Arguments read from a file must be one per line by default (but see also `convert_arg_line_to_args()`) and are treated as if they were in the same place as the original file referencing argument on the command line. So in the example above, the expression `['-f','foo','@args.txt']` is considered equivalent to the expression `['-f','foo','-f','bar']`.

■ **Nota:** Note Empty lines are treated as empty strings (`''`), which are allowed as values but not as arguments. Empty lines that are read as arguments will result in an “unrecognized arguments” error.

`ArgumentParser` uses filesystem encoding and error handler to read the file containing arguments.

The `fromfile_prefix_chars` argument defaults to `None`, meaning that arguments will never be treated as file references.

Changed in version 3.12: `ArgumentParser` changed encoding and errors to read arguments files from default (e.g. `locale.getpreferredencoding(False)` and `"strict"`) to the filesystem encoding and error handler. Arguments file should be encoded in UTF-8 instead of ANSI Codepage on Windows.

argument_default

Generally, argument defaults are specified either by passing a default to `add_argument()` or by calling these `set_defaults()` methods with a specific set of name-value pairs. Sometimes however, it may be useful to specify a single parser-wide default for arguments. This can be accomplished by passing the `argument_default=keyword` argument to `ArgumentParser`. For example, to globally suppress attribute creation on `parse_args()` calls, we supply `argument_default=SUPPRESS`:

```
>>> parser = argparse.ArgumentParser(argument_default=argparse.SUPPRESS)
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar', nargs='?')
>>> parser.parse_args(['--foo', '1', 'BAR'])
Namespace(bar='BAR', foo='1')
>>> parser.parse_args([])
Namespace()
```

allow_abbrev

Normally, when you pass an argument list to the `parse_args()` method of an `ArgumentParser`, it recognizes abbreviations of long options.

This feature can be disabled by setting `allow_abbrev` to `False`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', allow_abbrev=False)
>>> parser.add_argument('--foobar', action='store_true')
>>> parser.add_argument('--foonley', action='store_false')
>>> parser.parse_args(['--foon'])
usage: PROG [-h] [--foobar] [--foonley]
PROG: error: unrecognized arguments: --foon
```

Added in version 3.5.

conflict_handler

`ArgumentParser` objects do not allow two actions with the same option string. By default, `ArgumentParser` objects raise an exception if an attempt is made to create an argument with an option string that is already in use:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-f', '--foo', help='old foo help')
>>> parser.add_argument('--foo', help='new foo help')
Traceback (most recent call last):
...
ArgumentError: argument --foo: conflicting option string(s): --foo
```

Sometimes (e.g. when using parents) it may be useful to simply override any older arguments with the same option string. To get this behavior, the value 'resolve' can be supplied to the `conflict_handler=argument` of `ArgumentParser`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', conflict_handler='resolve')
>>> parser.add_argument('-f', '--foo', help='old foo help')
>>> parser.add_argument('--foo', help='new foo help')
>>> parser.print_help()
usage: PROG [-h] [-f F00] [--foo F00]
```

```
options:
  -h, --help  show this help message and exit
  -f F00      old foo help
  --foo F00   new foo help
```

Note that `ArgumentParser` objects only remove an action if all of its option strings are overridden. So, in the example above, the old `-f/--foo` action is retained as the `-f` action, because only the `--foo` option string was overridden.

`add_help`

By default, `ArgumentParser` objects add an option which simply displays the parser's help message. If `-h` or `--help` is supplied at the command line, the `ArgumentParser` help will be printed.

Occasionally, it may be useful to disable the addition of this help option. This can be achieved by passing `False` as the `add_help=` argument to `ArgumentParser`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False)
>>> parser.add_argument('--foo', help='foo help')
>>> parser.print_help()
usage: PROG [--foo F00]
```

```
options:
  --foo F00  foo help
```

The help option is typically `-h/--help`. The exception to this is if the `prefix_chars=` is specified and does not include `-`, in which case `-h` and `--help` are not valid options. In this case, the first character in `prefix_chars` is used to prefix the help options:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='+/')
>>> parser.print_help()
usage: PROG [+h]
```

```
options:
  +h, ++help  show this help message and exit
```

`exit_on_error`

Normally, when you pass an invalid argument list to the `parse_args()` method of an `ArgumentParser`, it will print a message to `sys.stderr` and exit with a status code of 2.

If the user would like to catch errors manually, the feature can be enabled by setting `exit_on_error` to `False`:

```
>>> parser = argparse.ArgumentParser(exit_on_error=False)
>>> parser.add_argument('--integers', type=int)
... _StoreAction(option_strings=['--integers'], dest='integers', nargs=None, const=None, default=None, type=<class
>>> try:
...     parser.parse_args('--integers a'.split())
... except argparse.ArgumentError:
...     print('Catching an argumentError')
...
Catching an argumentError
```

Added in version 3.9.

suggest_on_error

By default, when a user passes an invalid argument choice or subparser name, `ArgumentParser` will exit with error info and list the permissible argument choices (if specified) or subparser names as part of the error message.

If the user would like to enable suggestions for mistyped argument choices and subparser names, the feature can be enabled by setting `suggest_on_error` to `True`. Note that this only applies for arguments when the choices specified are strings:

```
>>> parser = argparse.ArgumentParser(suggest_on_error=True)
>>> parser.add_argument('--action', choices=['debug', 'dryrun'])
>>> parser.parse_args(['--action', 'debugg'])
usage: tester.py [-h] [--action {debug,dryrun}]
tester.py: error: argument --action: invalid choice: 'debugg', maybe you meant 'debug'? (choose from debug, dry
```

If you're writing code that needs to be compatible with older Python versions and want to opportunistically use `suggest_on_error` when it's available, you can set it as an attribute after initializing the parser instead of using the keyword argument:

```
>>> parser = argparse.ArgumentParser(description='Process some integers.')
>>> parser.suggest_on_error = True
```

Added in version 3.14.

color

By default, the help message is printed in color using ANSI escape sequences. If you want plain text help messages, you can disable this in your local environment, or in the argument parser itself by setting `color` to `False`:

```
>>> parser = argparse.ArgumentParser(description='Process some integers.',
...                                color=False)
>>> parser.add_argument('--action', choices=['sum', 'max'])
>>> parser.add_argument('integers', metavar='N', type=int, nargs='+',
...                    help='an integer for the accumulator')
>>> parser.parse_args(['--help'])
```

Note that when `color=True`, colored output depends on both environment variables and terminal capabilities. However, if `color=False`, colored output is always disabled, even if environment variables like `FORCE_COLOR` are set.

■ **Note:** NoteError messages will include color codes when redirecting stderr to a file. To avoid this, set the `NO_COLOR` or `PYTHON_COLORS` environment variable (for example, `NO_COLOR=1 python script.py 2>errors.txt`).

Added in version 3.14.

The add_argument() method

`ArgumentParser.add_argument(name or flags..., *, action, nargs, const, default, type, choices, required, help, metavar, dest, deprecated)`

Define how a single command-line argument should be parsed. Each parameter has its own more detailed description below, but in short they are: `name or flags`- Either a name or a list of option strings, e.g. 'foo' or '-f', '--foo'. `action`- The basic type of action to be taken when this argument is encountered at the command line. `nargs`- The number of command-line arguments that should be consumed. `const`- A constant value required by some `action` and `nargs` selections. `default`- The value produced if the argument

The following sections describe how each of these are used.

name or flags

The `add_argument()` method must know whether an optional argument, like `-f` or `--foo`, or a positional argument, like a list of filenames, is expected. The first arguments passed to `add_argument()` must therefore be either a series of flags, or a simple argument name.

For example, an optional argument could be created like:

```
>>> parser.add_argument('-f', '--foo')
```

while a positional argument could be created like:

```
>>> parser.add_argument('bar')
```

When `parse_args()` is called, optional arguments will be identified by the prefix, and the remaining arguments will be assumed to be positional:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-f', '--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args(['BAR'])
Namespace(bar='BAR', foo=None)
>>> parser.parse_args(['BAR', '--foo', 'F00'])
Namespace(bar='BAR', foo='F00')
>>> parser.parse_args(['--foo', 'F00'])
usage: PROG [-h] [-f F00] bar
PROG: error: the following arguments are required: bar
```

By default, `argparse` automatically handles the internal naming and display names of arguments, simplifying the process without requiring additional configuration. As such, you do not need to specify the destination metavar parameters. For optional arguments, the `dest` parameter defaults to the argument name, with underscores replacing hyphens. The `metavar` parameter defaults to the upper-cased name. For example:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo-bar')
>>> parser.parse_args(['--foo-bar', 'F00-BAR'])
Namespace(foo_bar='F00-BAR')
>>> parser.print_help()
usage: [-h] [--foo-bar F00-BAR]
```

optional arguments:

```
-h, --help show this help message and exit
--foo-bar F00-BAR
```

action

`ArgumentParser` objects associate command-line arguments with actions. These actions can do just about anything with the command-line arguments associated with them, though most actions simply add an attribute to the object returned by `parse_args()`. The `action` keyword argument specifies how the command-line arguments should be handled. The supplied actions are:

- `'store'`- This just stores the argument's value. This is the default action.
- `'store_const'`- This stores the value specified by the `const` keyword argument; note that the `const` keyword argument defaults to `None`. The `'store_const'` action is most commonly used with optional arguments that specify some sort of flag. For example:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_const', const=42)
>>> parser.parse_args(['--foo'])
Namespace(foo=42)
```
- `'store_true'` and `'store_false'`- These are special cases of `'store_const'` that respectively store the values `True` and `False` with default values of `False` and `True`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_true')
>>> parser.add_argument('--bar', action='store_false')
>>> parser.add_argument('--baz', action='store_false')
>>> parser.parse_args(['--foo --bar'].split())
Namespace(foo=True, bar=False, baz=False)
```
- `'append'`- This appends each argument value to a list. It is useful for allowing an option to be specified multiple times. If the default value is a non-empty list, the parsed value will start with the default list's elements and any

values from the command line will be appended after those default values. Example usage:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='append')
```

- 'append_const'- This appends the value specified by the `const` keyword argument to a list; note that the `const` keyword argument defaults to `None`. The 'append_const' action is typically useful when multiple arguments need to store constants to the same list. For example:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--str', dest='types', action='append_const', const=str)
>>> parser.add_argument('--i
```

- 'extend'- This appends each item from a multi-value argument to a list. The 'extend' action is typically used with the `nargs` keyword argument value '+' or '*'. Note that when `nargs` is `None` (the default) or '?', each character of the argument string will be appended to the list. Example usage:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument("--foo", action="extend", nargs="+", type=str)
>>> parser.parse_ar
```

- 'count'- This counts the number of times an argument occurs. For example, this is useful for increasing verbosity levels:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--verbose', '-v', action='count', default=0)
>>> parser.parse_args(['-vvv'])
Namespace(verbose=3)
Note, the default will be None unless explicitly set to 0.
```
- 'help'- This prints a complete help message for all the options in the current parser and then exits. By default a help action is automatically added to the parser. See `ArgumentParser` for details of how the output is created.
- 'version'- This expects a `version=keyword` argument in the `add_argument()` call, and prints version information and exits when invoked:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--version', action='version', version='%s2.0')
>>> parser.parse_args(['--version'])
PROG 2.0
```

You may also specify an arbitrary action by passing an `Action` subclass (e.g. `BooleanOptionalAction`) or other object that implements the same interface. Only actions that consume command-line arguments (e.g. 'store', 'append', 'extend', or custom actions with non-zero `nargs`) can be used with positional arguments.

The recommended way to create a custom action is to extend `Action`, overriding the `__call__()` method and optionally the `__init__()` and `format_usage()` methods. You can also register custom actions using the `register()` method and reference them by their registered name.

An example of a custom action:

```
>>> class FooAction(argparse.Action):
...     def __init__(self, option_strings, dest, nargs=None, **kwargs):
...         if nargs is not None:
...             raise ValueError("nargs not allowed")
...         super().__init__(option_strings, dest, **kwargs)
...     def __call__(self, parser, namespace, values, option_string=None):
...         print('%r %r %r' % (namespace, values, option_string))
...         setattr(namespace, self.dest, values)
...
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=FooAction)
>>> parser.add_argument('bar', action=FooAction)
>>> args = parser.parse_args('1 --foo 2'.split())
Namespace(bar=None, foo=None) '1' None
Namespace(bar='1', foo=None) '2' '--foo'
>>> args
Namespace(bar='1', foo='2')
```

For more details, see `Action`.

nargs

`ArgumentParser` objects usually associate a single command-line argument with a single action to be taken. The `nargs` keyword argument associates a different number of command-line arguments with a single action. See also `Specifying ambiguous arguments`. The supported values are:

- `N` (an integer). `N` arguments from the command line will be gathered together into a list. For example: `>>> parser = argparse.ArgumentParser()>>> parser.add_argument('--foo', nargs=2)>>> parser.add_argument('bar', nargs=1)>>> parser.parse_args('c --foo a b'.split()) Namespace(bar='c', foo=['a', 'b'])` Note that `nargs=1` produces a list of one item. This is different from the default, in which the item is produced

- `'?'`. One argument will be consumed from the command line if possible, and produced as a single item. If no command-line argument is present, the value from `default` will be produced. Note that for optional arguments, there is an additional case - the option string is present but not followed by a command-line argument. In this case the value from `const` will be produced. Some examples to illustrate

- `'*'`. All command-line arguments present are gathered into a list. Note that it generally doesn't make much sense to have more than one positional argument with `nargs='*'`, but multiple optional arguments with `nargs='*'` is possible. For example: `>>> parser = argparse.ArgumentParser()>>> parser.add_argument('--foo', nargs='*')>>> parser.add_argument('--bar', nargs='*')>>> parser.add_argument('baz', nargs='*')>`

- `'+'`. Just like `'*'`, all command-line arguments present are gathered into a list. Additionally, an error message will be generated if there wasn't at least one command-line argument present. For example: `>>> parser = argparse.ArgumentParser(prog='PROG')>>> parser.add_argument('foo', nargs='+')>>> parser.parse_args(['a', 'b']) Namespace(foo=['a', 'b'])>>> parser.parse_args([]) usage: PROG [-h] foo [foo ...] PR`

If the `nargs` keyword argument is not provided, the number of arguments consumed is determined by the `action`. Generally this means a single command-line argument will be consumed and a single item (not a list) will be produced. Actions that do not consume command-line arguments (e.g. `'store_const'`) set `nargs=0`.

const

The `const` argument of `add_argument()` is used to hold constant values that are not read from the command line but are required for the various `ArgumentParser` actions. The two most common uses of it are:

- When `add_argument()` is called with `action='store_const'` or `action='append_const'`. These actions add the `const` value to one of the attributes of the object returned by `parse_args()`. See the `action` description for examples. If `const` is not provided to `add_argument()`, it will receive a default value of `None`.
- When `add_argument()` is called with option strings (like `--foo`) and `nargs='?'`. This creates an optional argument that can be followed by zero or one command-line arguments. When parsing the command line, if the option string is encountered with no command-line argument following it, the value from `const` will be used. See the `nargs` description for examples.

Changed in version 3.11: `const=None` by default, including when `action='append_const'` or `action='store_const'`.

default

All optional arguments and some positional arguments may be omitted at the command line. The `default` keyword argument of `add_argument()`, whose value defaults to `None`, specifies what value should be used if the command-line argument is not present. For optional arguments, the `default` value is used when the option string was not present at the command line:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args(['--foo', '2'])
Namespace(foo='2')
>>> parser.parse_args([])
Namespace(foo=42)
```

If the target namespace already has an attribute set, the `action` default will not overwrite it:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args([], namespace=argparse.Namespace(foo=101))
Namespace(foo=101)
```

If the default value is a string, the parser parses the value as if it were a command-line argument. In particular, the parser applies any type conversion argument, if provided, before setting the attribute on the Namespace return value. Otherwise, the parser uses the value as is:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--length', default='10', type=int)
>>> parser.add_argument('--width', default=10.5, type=int)
>>> parser.parse_args()
Namespace(length=10, width=10.5)
```

For positional arguments with nargs equal to '?' or '*', the default value is used when no command-line argument was present:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('foo', nargs='?', default=42)
>>> parser.parse_args(['a'])
Namespace(foo='a')
>>> parser.parse_args([])
Namespace(foo=42)
```

For required arguments, the default value is ignored. For example, this applies to positional arguments with nargs values other than '?' or '*', or optional arguments marked as required=True.

Providing default=argparse.SUPPRESS causes no attribute to be added if the command-line argument was not present:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=argparse.SUPPRESS)
>>> parser.parse_args([])
Namespace()
>>> parser.parse_args(['--foo', '1'])
Namespace(foo='1')
```

type

By default, the parser reads command-line arguments in as simple strings. However, quite often the command-line string should instead be interpreted as another type, such as a float or int. The type keyword for add_argument() allows any necessary type-checking and type conversions to be performed.

If the type keyword is used with the default keyword, the type converter is only applied if the default is a string.

The argument to type can be a callable that accepts a single string or the name of a registered type (see register()). If the function raises ArgumentError, TypeError, or ValueError, the exception is caught and a nicely formatted error message is displayed. Other exception types are not handled.

Common built-in types and functions can be used as type converters:

```
import argparse
import pathlib

parser = argparse.ArgumentParser()
parser.add_argument('count', type=int)
parser.add_argument('distance', type=float)
parser.add_argument('street', type=ascii)
parser.add_argument('code_point', type=ord)
parser.add_argument('datapath', type=pathlib.Path)
```

User defined functions can be used as well:

```
>>> def hyphenated(string):
...     return '-'.join([word[:4] for word in string.casefold().split()])
...
>>> parser = argparse.ArgumentParser()
>>> _ = parser.add_argument('short_title', type=hyphenated)
>>> parser.parse_args(['The Tale of Two Cities'])
```

```
Namespace(short_title='the-tale-of-two-citi')
```

The `bool()` function is not recommended as a type converter. All it does is convert empty strings to `False` and non-empty strings to `True`. This is usually not what is desired.

In general, the `type` keyword is a convenience that should only be used for simple conversions that can only raise one of the three supported exceptions. Anything with more interesting error-handling or resource management should be done downstream after the arguments are parsed.

For example, JSON or YAML conversions have complex error cases that require better reporting than can be given by the `type` keyword. A `JSONDecodeError` would not be well formatted and a `FileNotFoundError` exception would not be handled at all.

Even `FileType` has its limitations for use with the `type` keyword. If one argument uses `FileType` and then a subsequent argument fails, an error is reported but the file is not automatically closed. In this case, it would be better to wait until after the parser has run and then use the `with`-statement to manage the files.

For type checkers that simply check against a fixed set of values, consider using the `choices` keyword instead.

choices

Some command-line arguments should be selected from a restricted set of values. These can be handled by passing a sequence object as the `choices` keyword argument to `add_argument()`. When the command line is parsed, argument values will be checked, and an error message will be displayed if the argument was not one of the acceptable values:

```
>>> parser = argparse.ArgumentParser(prog='game.py')
>>> parser.add_argument('move', choices=['rock', 'paper', 'scissors'])
>>> parser.parse_args(['rock'])
Namespace(move='rock')
>>> parser.parse_args(['fire'])
usage: game.py [-h] {rock,paper,scissors}
game.py: error: argument move: invalid choice: 'fire' (choose from 'rock',
'paper', 'scissors')
```

Any sequence can be passed as the `choices` value, `list` objects, `tuple` objects, and custom sequences are all supported.

Use of `enum.Enum` is not recommended because it is difficult to control its appearance in usage, help, and error messages.

Note that `choices` are checked after any type conversions have been performed, so objects in `choices` should match the types specified. This can make `choices` appear unfamiliar in usage, help, or error messages.

To keep `choices` user-friendly, consider a custom type wrapper that converts and formats values, or omit `type` and handle conversion in your application code.

Formatted choices override the default `metavar` which is normally derived from `dest`. This is usually what you want because the user never sees the `dest` parameter. If this display isn't desirable (perhaps because there are many choices), just specify an explicit `metavar`.

required

In general, the `argparse` module assumes that flags like `-f` and `--bar` indicate optional arguments, which can always be omitted at the command line. To make an option required, `True` can be specified for the `required=` keyword argument to `add_argument()`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', required=True)
>>> parser.parse_args(['--foo', 'BAR'])
Namespace(foo='BAR')
>>> parser.parse_args([])
```

```
usage: [-h] --foo F00
: error: the following arguments are required: --foo
```

As the example shows, if an option is marked as required, `parse_args()` will report an error if that option is not present at the command line.

■ **Nota:** `Required` options are generally considered bad form because users expect options to be optional, and thus they should be avoided when possible.

help

The `help` value is a string containing a brief description of the argument. When a user requests help (usually by using `--help` at the command line), these help descriptions will be displayed with each argument.

The help strings can include various format specifiers to avoid repetition of things like the program name or the argument default. The available specifiers include the program name, `%(prog)s` and most keyword arguments to `add_argument()`, e.g. `%(default)s`, `%(type)s`, etc.:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('bar', nargs='?', type=int, default=42,
...                     help='the bar to %(prog)s (default: %(default)s)')
>>> parser.print_help()
usage: frobble [-h] [bar]
```

```
positional arguments:
  bar      the bar to frobble (default: 42)
```

```
options:
  -h, --help  show this help message and exit
```

As the help string supports %-formatting, if you want a literal % to appear in the help string, you must escape it as `%%`.

`argparse` supports silencing the help entry for certain options, by setting the `help` value to `argparse.SUPPRESS`:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('--foo', help=argparse.SUPPRESS)
>>> parser.print_help()
usage: frobble [-h]
```

```
options:
  -h, --help  show this help message and exit
```

metavar

When `ArgumentParser` generates help messages, it needs some way to refer to each expected argument. By default, `ArgumentParser` objects use the `dest` value as the “name” of each object. By default, for positional argument actions, the `dest` value is used directly, and for optional argument actions, the `dest` value is uppercased. So, a single positional argument with `dest='bar'` will be referred to as `bar`. A single optional argument `--foo` that should be followed by a single command-line argument will be referred to as `FOO`. An example:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
usage: [-h] [--foo F00] bar
```

```
positional arguments:
  bar
```

```
options:
  -h, --help  show this help message and exit
```

```
--foo F00
```

An alternative name can be specified with `metavar`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', metavar='YYY')
>>> parser.add_argument('bar', metavar='XXX')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
usage: [-h] [--foo YYY] XXX
```

positional arguments:

XXX

options:

-h, --help show this help message and exit
--foo YYY

Note that `metavar` only changes the displayed name - the name of the attribute on the `parse_args()` object is still determined by the `dest` value.

Different values of `nargs` may cause the `metavar` to be used multiple times. Providing a tuple to `metavar` specifies a different display for each of the arguments:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', nargs=2)
>>> parser.add_argument('--foo', nargs=2, metavar=('bar', 'baz'))
>>> parser.print_help()
usage: PROG [-h] [-x X X] [--foo bar baz]
```

options:

-h, --help show this help message and exit
-x X X
--foo bar baz

dest

Most `ArgumentParser` actions add some value as an attribute of the object returned by `parse_args()`. The name of this attribute is determined by the `dest` keyword argument of `add_argument()`. For positional argument actions, `dest` is normally supplied as the first argument to `add_argument()`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('bar')
>>> parser.parse_args(['XXX'])
Namespace(bar='XXX')
```

For optional argument actions, the value of `dest` is normally inferred from the option strings. `ArgumentParser` generates the value of `dest` by taking the first long option string and stripping away the initial `--` string. If no long option strings were supplied, `dest` will be derived from the first short option string by stripping the initial character. Any internal characters will be converted to `_` characters to make sure the string is a valid attribute name. The examples below illustrate this behavior:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('-f', '--foo-bar', '--foo')
>>> parser.add_argument('-x', '-y')
>>> parser.parse_args('-f 1 -x 2'.split())
Namespace(foo_bar='1', x='2')
>>> parser.parse_args('--foo 1 -y 2'.split())
Namespace(foo_bar='1', x='2')
```

`dest` allows a custom attribute name to be provided:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', dest='bar')
```



```
>>> parser.parse_args('--foo XXX'.split())
Namespace(bar='XXX')
```

deprecated

During a project's lifetime, some arguments may need to be removed from the command line. Before removing them, you should inform your users that the arguments are deprecated and will be removed.

The `deprecated` keyword argument of `add_argument()`, which defaults to `False`, specifies if the argument is deprecated and will be removed in the future. For arguments, if `deprecated` is `True`, then a warning will be printed to `sys.stderr` when the argument is used:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='snake.py')
>>> parser.add_argument('--legs', default=0, type=int, deprecated=True)
>>> parser.parse_args([])
Namespace(legs=0)
>>> parser.parse_args(['--legs', '4'])
snake.py: warning: option '--legs' is deprecated
Namespace(legs=4)
```

Added in version 3.13.

Action classes

Action classes implement the Action API, a callable which returns a callable which processes arguments from the command-line. Any object which follows this API may be passed as the `action` parameter to `add_argument()`.

`class argparse.Action(option_strings, dest, nargs=None, const=None, default=None, type=None, choices=None, required=False, help=None, metavar=None)`

`__call__(parser, namespace, values, option_string=None)`

`format_usage()`

Action objects are used by an `ArgumentParser` to represent the information needed to parse a single argument from one or more strings from the command line. The `Action` class must accept the two positional arguments plus any keyword arguments passed to `ArgumentParser.add_argument()` except for the `action` itself. Instances of `Action` (or return value of any callable to the `action` parameter) should have attributes `dest`, `option_strings`, `default`, `type`, `required`, `help`, etc. defined. The easiest way to ensure these attribute

Action instances should be callable, so subclasses must override the `__call__()` method, which should accept four parameters: `parser`- The `ArgumentParser` object which contains this action. `namespace`- The `Namespace` object that will be returned by `parse_args()`. `Most actions add an attribute to this object using setattr(). values- The associated command-line arguments, with any type conversions applied. Type conversions are specified with the type keyword argument to add_argument(). option_string- The option string`

Action subclasses can define a `format_usage()` method that takes no argument and return a string which will be used when printing the usage of the program. If such method is not provided, a sensible default will be used.

`class argparse.BooleanOptionalAction`

A subclass of `Action` for handling boolean flags with positive and negative options. Adding a single argument such as `--foo` automatically creates both `--foo` and `--no-foo` options, storing `True` and `False` respectively: `>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=argparse.BooleanOptionalAction)
>>> parser.parse_args(['--no-foo'])
Namespace(foo=False)` Added in version 3.9.

The parse_args() method

`ArgumentParser.parse_args(args=None, namespace=None)`

Convert argument strings to objects and assign them as attributes of the namespace. Return the populated namespace. Previous calls to `add_argument()` determine exactly what objects are created and how they are assigned. See the documentation for `add_argument()` for details.

`args`- List of strings to parse. The default is taken from `sys.argv`.

`namespace`- An object to take the attributes. The default is a new empty `Namespace` object.

Option value syntax

The `parse_args()` method supports several ways of specifying the value of an option (if it takes one). In the simplest case, the option and its value are passed as two separate arguments:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('--foo')
>>> parser.parse_args(['-x', 'X'])
Namespace(foo=None, x='X')
>>> parser.parse_args(['--foo', 'F00'])
Namespace(foo='F00', x=None)
```

For long options (options with names longer than a single character), the option and value can also be passed as a single command-line argument, using `=` to separate them:

```
>>> parser.parse_args(['--foo=F00'])
Namespace(foo='F00', x=None)
```

For short options (options only one character long), the option and its value can be concatenated:

```
>>> parser.parse_args(['-xX'])
Namespace(foo=None, x='X')
```

Several short options can be joined together, using only a single prefix, as long as only the last option (or none of them) requires a value:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', action='store_true')
>>> parser.add_argument('-y', action='store_true')
>>> parser.add_argument('-z')
>>> parser.parse_args(['-xyzZ'])
Namespace(x=True, y=True, z='Z')
```

Invalid arguments

While parsing the command line, `parse_args()` checks for a variety of errors, including ambiguous options, invalid types, invalid options, wrong number of positional arguments, etc. When it encounters such an error, it exits and prints the error along with a usage message:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo', type=int)
>>> parser.add_argument('bar', nargs='?')

>>> # invalid type
>>> parser.parse_args(['--foo', 'spam'])
usage: PROG [-h] [--foo F00] [bar]
PROG: error: argument --foo: invalid int value: 'spam'

>>> # invalid option
>>> parser.parse_args(['--bar'])
usage: PROG [-h] [--foo F00] [bar]
PROG: error: no such option: --bar

>>> # wrong number of arguments
>>> parser.parse_args(['spam', 'badger'])
usage: PROG [-h] [--foo F00] [bar]
PROG: error: extra arguments found: badger
```

Arguments containing-

The `parse_args()` method attempts to give errors whenever the user has clearly made a mistake, but some situations are inherently ambiguous. For example, the command-line argument `-1` could either be an attempt to specify an option or an attempt to provide a positional argument. The `parse_args()` method is cautious here: positional arguments may only begin with `-` if they look like negative numbers and there are no options in the parser that look like negative numbers:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('foo', nargs='?')

>>> # no negative number options, so -1 is a positional argument
>>> parser.parse_args(['-x', '-1'])
Namespace(foo=None, x='-1')

>>> # no negative number options, so -1 and -5 are positional arguments
>>> parser.parse_args(['-x', '-1', '-5'])
Namespace(foo='-5', x='-1')

>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-1', dest='one')
>>> parser.add_argument('foo', nargs='?')

>>> # negative number options present, so -1 is an option
>>> parser.parse_args(['-1', 'X'])
Namespace(foo=None, one='X')

>>> # negative number options present, so -2 is an option
>>> parser.parse_args(['-2'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: no such option: -2

>>> # negative number options present, so both -1s are options
>>> parser.parse_args(['-1', '-1'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: argument -1: expected one argument
```

If you have positional arguments that must begin with `-` and don't look like negative numbers, you can insert the pseudo-argument `--` which tells `parse_args()` that everything after that is a positional argument:

```
>>> parser.parse_args(['--', '-f'])
Namespace(foo='-f', one=None)
```

See also the `argparse` howto on ambiguous arguments for more details.

Argument abbreviations (prefix matching)

The `parse_args()` method by default allows long options to be abbreviated to a prefix, if the abbreviation is unambiguous (the prefix matches a unique option):

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-bacon')
>>> parser.add_argument('-badger')
>>> parser.parse_args(['-bac MMM'].split())
Namespace(bacon='MMM', badger=None)
>>> parser.parse_args(['-bad WOOD'].split())
Namespace(bacon=None, badger='WOOD')
>>> parser.parse_args(['-ba BA'].split())
usage: PROG [-h] [-bacon BACON] [-badger BADGER]
PROG: error: ambiguous option: -ba could match -badger, -bacon
```

An error is produced for arguments that could produce more than one options. This feature can be disabled by setting `allow_abbrev` to `False`.

Beyondsys.argv

Sometimes it may be useful to have an `ArgumentParser` parse arguments other than those of `sys.argv`. This can be accomplished by passing a list of strings to `parse_args()`. This is useful for testing at the interactive prompt:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument(
...     'integers', metavar='int', type=int, choices=range(10),
...     nargs='+', help='an integer in the range 0..9')
>>> parser.add_argument(
...     '--sum', dest='accumulate', action='store_const', const=sum,
...     default=max, help='sum the integers (default: find the max)')
>>> parser.parse_args(['1', '2', '3', '4'])
Namespace(accumulate=<built-in function max>, integers=[1, 2, 3, 4])
>>> parser.parse_args(['1', '2', '3', '4', '--sum'])
Namespace(accumulate=<built-in function sum>, integers=[1, 2, 3, 4])
```

The Namespace object

class argparse.Namespace

Simple class used by default by `parse_args()` to create an object holding attributes and return it. This class is deliberately simple, just an object subclass with a readable string representation. If you prefer to have dict-like view of the attributes, you can use the standard Python idiom, `vars()`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> args = parser.parse_args(['--foo', 'BAR'])
>>> vars(args)
{'foo': 'BAR'}
```

It may also be useful to have an `ArgumentParser` assign attributes to an al

Other utilities

Subcommands

`ArgumentParser.add_subparsers(*[,title][,description][,prog][,parser_class][,action][,dest][,required][,help][,metavar])`

Many programs split up their functionality into a number of subcommands, for example, the `svn` program can invoke subcommands like `svn checkout`, `svn update`, and `svn commit`. Splitting up functionality this way can be a particularly good idea when a program performs several different functions which require different kinds of command-line arguments. `ArgumentParser` supports the creation of such subcommands with the `add_subparsers()` method. The `add_subparsers()` method is normally called with no arguments and ret

FileType objects

`class argparse.FileType(mode='r', bufsize=-1, encoding=None, errors=None)`

The `FileType` factory creates objects that can be passed to the `type` argument of `ArgumentParser.add_argument()`. Arguments that have `FileType` objects as their type will open command-line arguments as files with the requested modes, buffer sizes, encodings and error handling (see the `open()` function for more details):

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--raw', type=argparse.FileType('wb', 0))
>>> parser.add_argument('out', type=argparse.FileType('w', encoding='UTF-8'))
>>> parser.parse_arg
```

Argument groups

`ArgumentParser.add_argument_group(title=None, description=None, *[,argument_default][,conflict_handler])`

By default, `ArgumentParser` groups command-line arguments into “positional arguments” and “options” when displaying help messages. When there is a better conceptual grouping of arguments than this default one, appropriate groups can be created using the `add_argument_group()` method:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False)
>>> group = parser.add_argument_group('group')
>>> group.add_argument('--fo
```

```
o',help='foo help')>>>group.add_argument('bar',help='bar help')>>>parser.print_help()usage:
```

Mutual exclusion

ArgumentParser.add_mutually_exclusive_group(required=False)

Create a mutually exclusive group. `argparse` will make sure that only one of the arguments in the mutually exclusive group was present on the command line:

```
>>>parser=argparse.ArgumentParser(prog='PROG')>>>group=parser.add_mutually_exclusive_group()>>>group.add_argument('--foo',action='store_true')>>>group.add_argument('--bar',action='store_false')>>>parser.parse_args(['--foo'])Namespace(bar=True,foo=True)>>>parser.parse_args(['--bar'])Namespace(bar=False,foo=False)>>>parser.parse_args(['--foo','--bar'])Namespace(bar=False,foo=True)
```

Parser defaults

ArgumentParser.set_defaults(kwargs)**

Most of the time, the attributes of the object returned by `parse_args()` will be fully determined by inspecting the command-line arguments and the argument actions. `set_defaults()` allows some additional attributes that are determined without any inspection of the command line to be added:

```
>>>parser=argparse.ArgumentParser()>>>parser.add_argument('foo',type=int)>>>parser.set_defaults(bar=42,baz='badger')>>>parser.parse_args(['736'])Namespace(bar=42,baz='badger',foo=736)
```

Note that defaults can be set a

ArgumentParser.get_default(dest)

Get the default value for a namespace attribute, as set by either `add_argument()` or by `set_defaults()`:

```
>>>parser=argparse.ArgumentParser()>>>parser.add_argument('--foo',default='badger')>>>parser.get_default('foo')'badger'
```

Printing help

In most typical applications, `parse_args()` will take care of formatting and printing any usage or error messages. However, several formatting methods are available:

ArgumentParser.print_usage(file=None)

Print a brief description of how the `ArgumentParser` should be invoked on the command line. If `file is None`, `sys.stdout` is assumed.

ArgumentParser.print_help(file=None)

Print a help message, including the program usage and information about the arguments registered with the `ArgumentParser`. If `file is None`, `sys.stdout` is assumed.

There are also variants of these methods that simply return a string instead of printing it:

ArgumentParser.format_usage()

Return a string containing a brief description of how the `ArgumentParser` should be invoked on the command line.

ArgumentParser.format_help()

Return a string containing a help message, including the program usage and information about the arguments registered with the `ArgumentParser`.

Partial parsing

ArgumentParser.parse_known_args(args=None,namespace=None)

Sometimes a script only needs to handle a specific set of command-line arguments, leaving any unrecognized arguments for another script or program. In these cases, the `parse_known_args()` method can be useful. This method works similarly to `parse_args()`, but it does not raise an error for extra, unrecognized arguments. Instead,

it parses the known arguments and returns a two item tuple that contains the populated namespace and the list of any unrecognized arguments.>>>parser=argparse.ArgumentParser()

■■ Advertencia: WarningPrefix matchingrules apply toparse_known_args(). The parser may consume an option even if it's just a prefix of one of its known options, instead of leaving it in the remaining arguments list.

Customizing file parsing

ArgumentParser.convert_arg_line_to_args(arg_line)

Arguments that are read from a file (see thefromfile_prefix_charskeyword argument to theArgumentParserconstructor) are read one argument per line.convert_arg_line_to_args()can be overridden for fancier reading.This method takes a single argumentarg_linewhich is a string read from the argument file. It returns a list of arguments parsed from this string. The method is called once per line read from the argument file, in order.A useful override of this method is one that treats each space-separat

Exiting methods

ArgumentParser.exit(status=0,message=None)

This method terminates the program, exiting with the specifiedstatusand, if given, it prints amessagetosys.stderrbefore that. The user can override this method to handle these steps differently:classErrorCatchingArgumentParser(argparse.ArgumentParser):defexit(self,status=0,message=None):ifstatus:raiseException(f'Exiting because of an error:{message}')exit(status)

ArgumentParser.error(message)

This method prints a usage message, including themessage, tosys.stderrand terminates the program with a status code of 2.

Intermixed parsing

ArgumentParser.parse_intermixed_args(args=None,namespace=None)

ArgumentParser.parse_known_intermixed_args(args=None,namespace=None)

A number of Unix commands allow the user to intermix optional arguments with positional arguments. Theparse_intermixed_args()andparse_known_intermixed_args()methods support this parsing style.These parsers do not support all theargparsefeatures, and will raise exceptions if unsupported features are used. In particular, subparsers, and mutually exclusive groups that include both optionals and positionals are not supported.The following example shows the difference betweenparse_known_args()andpa

Registering custom types or actions

ArgumentParser.register(registry_name,value,object)

Sometimes it's desirable to use a custom string in error messages to provide more user-friendly output. In these cases,register()can be used to register custom actions or types with a parser and allow you to reference the type by their registered name instead of their callable name.Theregister()method accepts three arguments - aregistry_name, specifying the internal registry where the object will be stored (e.g.,action,type),value, which is the key under which the object will be registered, and

Exceptions

exceptionargparse.ArgumentError

An error from creating or using an argument (optional or positional).The string value of this exception is the message, augmented with information about the argument that caused it.

exceptionargparse.ArgumentTypeError

Raised when something goes wrong converting a command line string to a type.

Guides and Tutorials

- [Argparse Tutorial](#)
- [Migratingoptparsecode toargparse](#)

optparse— Parser for command line options

Source code: [Lib/optparse.py](#)

Choosing an argument parsing library

The standard library includes three argument parsing libraries:

- `getopt`: a module that closely mirrors the procedural `CgetoptAPI`. Included in the standard library since before the initial Python 1.0 release.
- `optparse`: a declarative replacement for `getopt` that provides equivalent functionality without requiring each application to implement its own procedural option parsing logic. Included in the standard library since the Python 2.3 release.
- `argparse`: a more opinionated alternative to `optparse` that provides more functionality by default, at the expense of reduced application flexibility in controlling exactly how arguments are processed. Included in the standard library since the Python 2.7 and Python 3.2 releases.

In the absence of more specific argument parsing design constraints, `argparse` is the recommended choice for implementing command line applications, as it offers the highest level of baseline functionality with the least application level code.

`getopt` is retained almost entirely for backwards compatibility reasons. However, it also serves a niche use case as a tool for prototyping and testing command line argument handling in `getopt`-based C applications.

`optparse` should be considered as an alternative to `argparse` in the following cases:

- an application is already using `optparse` and doesn't want to risk the subtle behavioural changes that may arise when migrating to `argparse`
- the application requires additional control over the way options and positional parameters are interleaved on the command line (including the ability to disable the interleaving feature completely)
- the application requires additional control over the incremental parsing of command line elements (while `argparse` does support this, the exact way it works in practice is undesirable for some use cases)
- the application requires additional control over the handling of options which accept parameter values that may start with `-` (such as delegated options to be passed to invoked subprocesses)
- the application requires some other command line parameter processing behavior which `argparse` does not support, but which can be implemented in terms of the lower level interface offered by `optparse`

These considerations also mean that `optparse` is likely to provide a better foundation for library authors writing third party command line argument processing libraries.

As a concrete example, consider the following two command line argument parsing configurations, the first using `optparse`, and the second using `argparse`:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)

import argparse
```



```

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', '--output')
    parser.add_argument('-v', dest='verbose', action='store_true')
    parser.add_argument('rest', nargs='*')
    args = parser.parse_args()
    process(args.rest, output=args.output, verbose=args.verbose)

```

The most obvious difference is that in the `optparse` version, the non-option arguments are processed separately by the application after the option processing is complete. In the `argparse` version, positional arguments are declared and processed in the same way as the named options.

However, the `argparse` version will also handle some parameter combination differently from the way the `optparse` version would handle them. For example (amongst other differences):

- supplying `-o -v` gives `output="-v"` and `verbose=False` when using `optparse`, but a usage error with `argparse` (complaining that no value has been supplied for `-o/--output`, since `-v` is interpreted as meaning the verbosity flag)
- similarly, supplying `-o --` gives `output="--"` and `args=()` when using `optparse`, but a usage error with `argparse` (also complaining that no value has been supplied for `-o/--output`, since `--` is interpreted as terminating the option processing and treating all remaining values as positional arguments)
- supplying `-o=foo` gives `output="=foo"` when using `optparse`, but `output="foo"` with `argparse` (since `=` is special cased as an alternative separator for option parameter values)

Whether these differing behaviors in the `argparse` version are considered desirable or a problem will depend on the specific command line application use case.

See also

`click` is a third party argument processing library (originally based on `optparse`), which allows command line applications to be developed as a set of decorated command implementation functions.

Other third party libraries, such as `typerormsgspec-click`, allow command line interfaces to be specified in ways that more effectively integrate with static checking of Python type annotations.

Introduction

`optparse` is a more convenient, flexible, and powerful library for parsing command-line options than the `minimalist getopt` module. `optparse` uses a more declarative style of command-line parsing: you create an instance of `OptionParser`, populate it with options, and parse the command line. `optparse` allows users to specify options in the conventional GNU/POSIX syntax, and additionally generates usage and help messages for you.

Here's an example of using `optparse` in a simple script:

```

from optparse import OptionParser
...
parser = OptionParser()
parser.add_option("-f", "--file", dest="filename",
                  help="write report to FILE", metavar="FILE")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose", default=True,
                  help="don't print status messages to stdout")

```

```
(options, args) = parser.parse_args()
```

With these few lines of code, users of your script can now do the “usual thing” on the command-line, for example:

```
<yourscript> --file=outfile -q
```

As it parses the command line, `optparse` sets attributes of the `options` object returned by `parse_args()` based on user-supplied command-line values. When `parse_args()` returns from parsing this command line, `options.filename` will be "outfile" and `options.verbose` will be `False`. `optparse` supports both long and short options, allows short options to be merged together, and allows options to be associated with their arguments in a variety of ways. Thus, the following command lines are all equivalent to the above example:

```
<yourscript> -f outfile --quiet
<yourscript> --quiet --file outfile
<yourscript> -q -foutfile
<yourscript> -qfoutfile
```

Additionally, users can run one of the following

```
<yourscript> -h
<yourscript> --help
```

and `optparse` will print out a brief summary of your script's options:

```
Usage: <yourscript> [options]
```

Options:

-h, --help	show this help message and exit
-f FILE, --file=FILE	write report to FILE
-q, --quiet	don't print status messages to stdout

where the value of `yourscript` is determined at runtime (normally from `sys.argv[0]`).

Background

`optparse` was explicitly designed to encourage the creation of programs with straightforward command-line interfaces that follow the conventions established by the `getopt()` family of functions available to C developers. To that end, it supports only the most common command-line syntax and semantics conventionally used under Unix. If you are unfamiliar with these conventions, reading this section will allow you to acquaint yourself with them.

Terminology

argument

option

option argument

positional argument

required option

a string entered on the command-line, and passed by the shell to `exec()` or `execv()`. In Python, arguments are elements of `sys.argv[1:]` (`sys.argv[0]` is the name of the program being executed). Unix shells also use the term "word". It is occasionally desirable to substitute an argument list other than `sys.argv[1:]`, so you should read "argument" as "an element of `sys.argv[1:]`, or of some other list provided as a substitute for `sys.argv[1:]`".

an argument used to supply extra information to guide or customize the execution of a program. There are many different syntaxes for options; the traditional Unix syntax is a hyphen ("-") followed by a single letter, e.g. -x or -F. Also, traditional Unix syntax allows multiple options to be merged into a single argument, e.g. -x-F is equivalent to -xF. The GNU project introduced -- followed by a series of hyphen-separated words, e.g. --file or --dry-run. These are the only two option syntaxes provided

an argument that follows an option, is closely associated with that option, and is consumed from the argument list when that option is. With `optparse`, option arguments may either be in a separate argument from their option: `-f foo --file foo` or included in the same argument: `-ffoo --file=foo` Typically, a given option either takes an

argument or it doesn't. Lots of people want an "optional option arguments" feature, meaning that some options will take an argument if they see it, and won't if they don't

something leftover in the argument list after options have been parsed, i.e. after options and their arguments have been parsed and removed from the argument list.

an option that must be supplied on the command-line; note that the phrase "required option" is self-contradictory in English. `optparse` doesn't prevent you from implementing required options, but doesn't give you much help at it either.

For example, consider this hypothetical command-line:

```
prog -v --report report.txt foo bar
```

`-v` and `--report` are both options. Assuming that `--report` takes one argument, `report.txt` is an option argument. `foo` and `bar` are positional arguments.

What are options for?

Options are used to provide extra information to tune or customize the execution of a program. In case it wasn't clear, options are usually optional. A program should be able to run just fine with no options whatsoever. (Pick a random program from the Unix or GNU toolsets. Can it run without any options at all and still make sense? The main exceptions are `find`, `tar`, and `dd`—all of which are mutant oddballs that have been rightly criticized for their non-standard syntax and confusing interfaces.)

Lots of people want their programs to have "required options". Think about it. If it's required, then it's not optional! If there is a piece of information that your program absolutely requires in order to run successfully, that's what positional arguments are for.

As an example of good command-line interface design, consider the humble `cp` utility, for copying files. It doesn't make much sense to try to copy files without supplying a destination and at least one source. Hence, `cp` fails if you run it with no arguments. However, it has a flexible, useful syntax that does not require any options at all:

```
cp SOURCE DEST
cp SOURCE ... DEST-DIR
```

You can get pretty far with just that. Most `cp` implementations provide a bunch of options to tweak exactly how the files are copied: you can preserve mode and modification time, avoid following symlinks, ask before clobbering existing files, etc. But none of this distracts from the core mission of `cp`, which is to copy either one file to another, or several files to another directory.

What are positional arguments for?

Positional arguments are for those pieces of information that your program absolutely, positively requires to run.

A good user interface should have as few absolute requirements as possible. If your program requires 17 distinct pieces of information in order to run successfully, it doesn't much matter how you get that information from the user—most people will give up and walk away before they successfully run the program. This applies whether the user interface is a command-line, a configuration file, or a GUI: if you make that many demands on your users, most of them will simply give up.

In short, try to minimize the amount of information that users are absolutely required to supply—use sensible defaults whenever possible. Of course, you also want to make your programs reasonably flexible. That's what options are for. Again, it doesn't matter if they are entries in a config file, widgets in the "Preferences" dialog of a GUI, or command-line options—the more options you implement, the more flexible your program is, and the more complicated its implementation becomes. Too much flexibility has drawbacks as well, of course; too many options can overwhelm users and make your code much harder to maintain.

Tutorial

While `optparse` is quite flexible and powerful, it's also straightforward to use in most cases. This section covers the code patterns that are common to any `optparse`-based program.

First, you need to import the `OptionParser` class; then, early in the main program, create an `OptionParser` instance:

```
from optparse import OptionParser
...
parser = OptionParser()
```

Then you can start defining options. The basic syntax is:

```
parser.add_option(opt_str, ...,
                  attr=value, ...)
```

Each option has one or more option strings, such as `-f` or `--file`, and several option attributes that tell `optparse` what to expect and what to do when it encounters that option on the command line.

Typically, each option will have one short option string and one long option string, e.g.:

```
parser.add_option("-f", "--file", ...)
```

You're free to define as many short option strings and as many long option strings as you like (including zero), as long as there is at least one option string overall.

The option strings passed to `OptionParser.add_option()` are effectively labels for the option defined by that call. For brevity, we will frequently refer to encountering an option on the command line; in reality, `optparse` encounters option strings and looks up options from them.

Once all of your options are defined, instruct `optparse` to parse your program's command line:

```
(options, args) = parser.parse_args()
```

(If you like, you can pass a custom argument list to `parse_args()`, but that's rarely necessary: by default it uses `sys.argv[1:]`.)

`parse_args()` returns two values:

- `options`, an object containing values for all of your options—e.g. if `--file` takes a single string argument, then `options.file` will be the filename supplied by the user, or `None` if the user did not supply that option
- `args`, the list of positional arguments leftover after parsing options

This tutorial section only covers the four most important option attributes: `action`, `type`, `dest` (destination), and `help`. Of these, `action` is the most fundamental.

Understanding option actions

Actions tell `optparse` what to do when it encounters an option on the command line. There is a fixed set of actions hard-coded into `optparse`; adding new actions is an advanced topic covered in section [Extending optparse](#). Most actions tell `optparse` to store a value in some variable—for example, take a string from the command line and store it in an attribute of `options`.

If you don't specify an option action, `optparse` defaults to `store`.

The store action

The most common option action is `store`, which tells `optparse` to take the next argument (or the remainder of the current argument), ensure that it is of the correct type, and store it to your chosen destination.

For example:

```
parser.add_option("-f", "--file",
```

```
action="store", type="string", dest="filename")
```

Now let's make up a fake command line and ask `optparse` to parse it:

```
args = ["-f", "foo.txt"]
(options, args) = parser.parse_args(args)
```

When `optparse` sees the option string `-f`, it consumes the next argument, `foo.txt`, and stores it in `options.filename`. So, after this call to `parse_args()`, `options.filename` is `"foo.txt"`.

Some other option types supported by `optparse` are `int` and `float`. Here's an option that expects an integer argument:

```
parser.add_option("-n", type="int", dest="num")
```

Note that this option has no long option string, which is perfectly acceptable. Also, there's no explicit action, since the default is `store`.

Let's parse another fake command-line. This time, we'll jam the option argument right up against the option: since `-n42` (one argument) is equivalent to `-n 42` (two arguments), the code

```
(options, args) = parser.parse_args(["-n42"])
print(options.num)
```

will print `42`.

If you don't specify a type, `optparse` assumes `string`. Combined with the fact that the default action is `store`, that means our first example can be a lot shorter:

```
parser.add_option("-f", "--file", dest="filename")
```

If you don't supply a destination, `optparse` figures out a sensible default from the option strings: if the first long option string is `--foo-bar`, then the default destination is `foo_bar`. If there are no long option strings, `optparse` looks at the first short option string: the default destination for `-f` is `f`.

`optparse` also includes the built-in `complex` type. Adding types is covered in section [Extending optparse](#).

Handling boolean (flag) options

Flag options—set a variable to `true` or `false` when a particular option is seen—are quite common. `optparse` supports them with two separate actions, `store_true` and `store_false`. For example, you might have a `verbose` flag that is turned on with `-v` and off with `-q`:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose")
```

Here we have two different options with the same destination, which is perfectly OK. (It just means you have to be a bit careful when setting default values—see below.)

When `optparse` encounters `-v` on the command line, it sets `options.verbose` to `True`; when it encounters `-q`, `options.verbose` is set to `False`.

Other actions

Some other actions supported by `optparse` are:

"store_const"

"append"

"count"

"callback"

store a constant value, pre-set via `Option.const`

append this option's argument to a list

increment a counter by one

call a specified function

These are covered in section [Reference Guide](#), and section [Option Callbacks](#).

Default values

All of the above examples involve setting some variable (the “destination”) when certain command-line options are seen. What happens if those options are never seen? Since we didn't supply any defaults, they are all set to `None`. This is usually fine, but sometimes you want more control. `optparse` lets you supply a default value for each destination, which is assigned before the command line is parsed.

First, consider the verbose/quiet example. If we want `optparse` to set `verbose` to `True` unless `-q` is seen, then we can do this:

```
parser.add_option("-v", action="store_true", dest="verbose", default=True)
parser.add_option("-q", action="store_false", dest="verbose")
```

Since default values apply to the destination rather than to any particular option, and these two options happen to have the same destination, this is exactly equivalent:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose", default=True)
```

Consider this:

```
parser.add_option("-v", action="store_true", dest="verbose", default=False)
parser.add_option("-q", action="store_false", dest="verbose", default=True)
```

Again, the default value for `verbose` will be `True`: the last default value supplied for any particular destination is the one that counts.

A clearer way to specify default values is the `set_defaults()` method of `OptionParser`, which you can call at any time before calling `parse_args()`:

```
parser.set_defaults(verbose=True)
parser.add_option(...)
(options, args) = parser.parse_args()
```

As before, the last value specified for a given option destination is the one that counts. For clarity, try to use one method or the other of setting default values, not both.

Generating help

`optparse`'s ability to generate help and usage text automatically is useful for creating user-friendly command-line interfaces. All you have to do is supply a `help` value for each option, and optionally a short usage message for your whole program. Here's an `OptionParser` populated with user-friendly (documented) options:

```
usage = "usage: %prog [options] arg1 arg2"
parser = OptionParser(usage=usage)
parser.add_option("-v", "--verbose",
                  action="store_true", dest="verbose", default=True,
                  help="make lots of noise [default]")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose",
                  help="be vewwy quiet (I'm hunting wabbits)")
parser.add_option("-f", "--filename",
                  metavar="FILE", help="write output to FILE")
parser.add_option("-m", "--mode",
                  default="intermediate",
                  help="interaction mode: novice, intermediate, "
                  "or expert [default: %default]")
```

If `optparse` encounters either `--help` on the command-line, or if you just call `parser.print_help()`, it prints the following to standard output:

```
Usage: <yourscript> [options] arg1 arg2
```

Options:

```
-h, --help            show this help message and exit
-v, --verbose          make lots of noise [default]
-q, --quiet            be vewwy quiet (I'm hunting wabbits)
-f FILE, --filename=FILE write output to FILE
-m MODE, --mode=MODE   interaction mode: novice, intermediate, or
                        expert [default: intermediate]
```

(If the help output is triggered by a help option, `optparse` exits after printing the help text.)

There's a lot going on here to help `optparse` generate the best possible help message:

- the script defines its own usage message: `usage="usage: %prog [options] arg1 arg2"` `optparse` expands `%prog` in the usage string to the name of the current program, i.e. `os.path.basename(sys.argv[0])`. The expanded string is then printed before the detailed option help. If you don't supply a usage string, `optparse` uses a bland but sensible default: `"Usage:%prog[options]"`, which is fine if your script doesn't
- every option defines a help string, and doesn't worry about line-wrapping—`optparse` takes care of wrapping lines and making the help output look good.
- options that take a value indicate this fact in their automatically generated help message, e.g. for the "mode" option: `-mMODE, --mode=MODE` Here, "MODE" is called the meta-variable: it stands for the argument that the user is expected to supply to `-m/--mode`. By default, `optparse` converts the destination variable name to uppercase and uses that for the meta-variable. Sometimes, that's not what you want
- options that have a default value can include `%default` in the help string—`optparse` will replace it with `str()` of the option's default value. If an option has no default value (or the default value is `None`), `%default` expands to `none`.

Grouping Options

When dealing with many options, it is convenient to group these options for better help output.

An `OptionParser` can contain several option groups, each of which can contain several options.

An option group is obtained using the class `OptionGroup`:

`class optparse.OptionGroup(parser, title, description=None)`

where `parser` is the `OptionParser` instance the group will be inserted in to `title` is the group title `description`, optional, is a long description of the group

`OptionGroup` inherits from `OptionContainer` (like `OptionParser`) and so the `add_option()` method can be used to add an option to the group.

Once all the options are declared, using the `OptionParser` method `add_option_group()` the group is added to the previously defined parser.

Continuing with the parser defined in the previous section, adding an `OptionGroup` to a parser is easy:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk. "
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.")
parser.add_option_group(group)
```

This would result in the following help output:

```
Usage: <yourscript> [options] arg1 arg2
```

Options:

```
-h, --help          show this help message and exit
-v, --verbose       make lots of noise [default]
-q, --quiet         be vewwy quiet (I'm hunting wabbits)
-f FILE, --filename=FILE
                    write output to FILE
-m MODE, --mode=MODE interaction mode: novice, intermediate, or
                    expert [default: intermediate]
```

Dangerous Options:

Caution: use these options at your own risk. It is believed that some of them bite.

-g Group option.

A bit more complete example might involve using more than one group: still extending the previous example:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk. "
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.")
parser.add_option_group(group)

group = OptionGroup(parser, "Debug Options")
group.add_option("-d", "--debug", action="store_true",
                help="Print debug information")
group.add_option("-s", "--sql", action="store_true",
                help="Print all SQL statements executed")
group.add_option("-e", action="store_true", help="Print every action done")
parser.add_option_group(group)
```

that results in the following output:

Usage: <yourscript> [options] arg1 arg2

Options:

```
-h, --help          show this help message and exit
-v, --verbose       make lots of noise [default]
-q, --quiet         be vewwy quiet (I'm hunting wabbits)
-f FILE, --filename=FILE
                    write output to FILE
-m MODE, --mode=MODE interaction mode: novice, intermediate, or expert
                    [default: intermediate]
```

Dangerous Options:

Caution: use these options at your own risk. It is believed that some of them bite.

-g Group option.

Debug Options:

```
-d, --debug         Print debug information
-s, --sql           Print all SQL statements executed
-e                 Print every action done
```

Another interesting method, in particular when working programmatically with option groups is:

OptionParser.get_option_group(opt_str)

Return the `OptionGroup` to which the short or long option string `opt_str` (e.g. `'-o'` or `'--option'`) belongs. If there's no such `OptionGroup`, return `None`.

Printing a version string

Similar to the brief usage string, `optparse` can also print a version string for your program. You have to supply the string as the `version` argument to `OptionParser`:

```
parser = OptionParser(usage="%prog [-f] [-q]", version="%prog 1.0")
```

`%prog` is expanded just like it is in `usage`. Apart from that, `version` can contain anything you like. When you supply it, `optparse` automatically adds a `--version` option to your parser. If it encounters this option on the command line, it expands your `version` string (by replacing `%prog`), prints it to `stdout`, and exits.

For example, if your script is called `/usr/bin/foo`:

```
$ /usr/bin/foo --version
foo 1.0
```

The following two methods can be used to print and get the `version` string:

`OptionParser.print_version(file=None)`

Print the version message for the current program (`self.version`) to `file` (default `stdout`). As with `print_usage()`, any occurrence of `%prog` in `self.version` is replaced with the name of the current program. Does nothing if `self.version` is empty or undefined.

`OptionParser.get_version()`

Same as `print_version()` but returns the version string instead of printing it.

How `optparse` handles errors

There are two broad classes of errors that `optparse` has to worry about: programmer errors and user errors. Programmer errors are usually erroneous calls to `OptionParser.add_option()`, e.g. invalid option strings, unknown option attributes, missing option attributes, etc. These are dealt with in the usual way: raise an exception (either `optparse.OptionError` or `TypeError`) and let the program crash.

Handling user errors is much more important, since they are guaranteed to happen no matter how stable your code is. `optparse` can automatically detect some user errors, such as bad option arguments (passing `-n4x` where `-n` takes an integer argument), missing arguments (`-n` at the end of the command line, where `-n` takes an argument of any type). Also, you can call `OptionParser.error()` to signal an application-defined error condition:

```
(options, args) = parser.parse_args()
...
if options.a and options.b:
    parser.error("options -a and -b are mutually exclusive")
```

In either case, `optparse` handles the error the same way: it prints the program's usage message and an error message to standard error and exits with error status 2.

Consider the first example above, where the user passes `4x` to an option that takes an integer:

```
$ /usr/bin/foo -n 4x
Usage: foo [options]
```

```
foo: error: option -n: invalid integer value: '4x'
```

Or, where the user fails to pass a value at all:

```
$ /usr/bin/foo -n
Usage: foo [options]
```

```
foo: error: -n option requires an argument
```

`optparse`-generated error messages take care always to mention the option involved in the error; be sure to do the same when calling `OptionParser.error()` from your application code.

If `optparse`'s default error-handling behaviour does not suit your needs, you'll need to subclass `OptionParser` and override `itsexit()` and/or `error()` methods.

Putting it all together

Here's what `optparse`-based scripts usually look like:

```
from optparse import OptionParser
...
def main():
    usage = "usage: %prog [options] arg"
    parser = OptionParser(usage)
    parser.add_option("-f", "--file", dest="filename",
                      help="read data from FILENAME")
    parser.add_option("-v", "--verbose",
                      action="store_true", dest="verbose")
    parser.add_option("-q", "--quiet",
                      action="store_false", dest="verbose")
    ...
    (options, args) = parser.parse_args()
    if len(args) != 1:
        parser.error("incorrect number of arguments")
    if options.verbose:
        print("reading %s..." % options.filename)
    ...

if __name__ == "__main__":
    main()
```

Reference Guide

Creating the parser

The first step in using `optparse` is to create an `OptionParser` instance.

`class optparse.OptionParser(...)`

`usage(default: "%prog [options]")`

`option_list(default: [])`

`option_class(default: optparse.Option)`

`version(default: None)`

`conflict_handler(default: "error")`

`description(default: None)`

`formatter(default: a new IndentedHelpFormatter)`

`add_help_option(default: True)`

`prog`

`epilog(default: None)`

The `OptionParser` constructor has no required arguments, but a number of optional keyword arguments. You should always pass them as keyword arguments, i.e. do not rely on the order in which the arguments are declared. `usage(default: "%prog [options]")` The usage summary to print when your program is run incorrectly or with a help option. When `optparse` prints the usage string, it expands `%prog` to `os.path.basename(sys.argv[0])` (or `toprogi` if you passed that keyword argument). To suppress a usage message, pas

The usage summary to print when your program is run incorrectly or with a help option. When `optparse` prints the usage string, it expands `%prog` to `os.path.basename(sys.argv[0])` (or `toprog` if you passed that keyword argument). To suppress a usage message, pass the special value `optparse.SUPPRESS_USAGE`.

A list of `Option` objects to populate the parser with. The options in `option_list` are added after any options in `standard_option_list` (a class attribute that may be set by `OptionParser` subclasses), but before any version or help options. Deprecated; use `add_option()` after creating the parser instead.

Class to use when adding options to the parser in `add_option()`.

A version string to print when the user supplies a version option. If you supply a true value for `version`, `optparse` automatically adds a version option with the single option string `--version`. The substring `%prog` is expanded the same as for `usage`.

Specifies what to do when options with conflicting option strings are added to the parser; see section [Conflicts between options](#).

A paragraph of text giving a brief overview of your program. `optparse` reformats this paragraph to fit the current terminal width and prints it when the user requests help (after `usage`, but before the list of options).

An instance of `optparse.HelpFormatter` that will be used for printing help text. `optparse` provides two concrete classes for this purpose: `IndentedHelpFormatter` and `TitledHelpFormatter`.

If true, `optparse` will add a help option (with option strings `-h` and `--help`) to the parser.

The string to use when expanding `%prog` in `usage` and `version` instead of `os.path.basename(sys.argv[0])`.

A paragraph of help text to print after the option help.

Populating the parser

There are several ways to populate the parser with options. The preferred way is by using `OptionParser.add_option()`, as shown in section [Tutorial.add_option\(\)](#) can be called in one of two ways:

- pass it an `Option` instance (as returned by `make_option()`)
- pass it any combination of positional and keyword arguments that are acceptable to `make_option()` (i.e., to the `Option` constructor), and it will create the `Option` instance for you

The other alternative is to pass a list of pre-constructed `Option` instances to the `OptionParser` constructor, as in:

```
option_list = [
    make_option("-f", "--filename",
                action="store", type="string", dest="filename"),
    make_option("-q", "--quiet",
                action="store_false", dest="verbose"),
]
parser = OptionParser(option_list=option_list)
```

(`make_option()` is a factory function for creating `Option` instances; currently it is an alias for the `Option` constructor. A future version of `optparse` may split `Option` into several classes, and `make_option()` will pick the right class to instantiate. Do not instantiate `Option` directly.)

Defining options

Each `Option` instance represents a set of synonymous command-line option strings, e.g. `-f` and `--file`. You can specify any number of short or long option strings, but you must specify at least one overall option string.

The canonical way to create an `Option` instance is with the `add_option()` method of `OptionParser`.

`OptionParser.add_option(option)`

`OptionParser.add_option(*opt_str, attr=value, ...)`

"store"

"store_const"

"store_true"

"store_false"

"append"

"append_const"

"count"

"callback"

"help"

To define an option with only a short option string: `parser.add_option("-f", attr=value,...)` And to define an option with only a long option string: `parser.add_option("--foo", attr=value,...)` The keyword arguments define attributes of the new `Option` object. The most important option attribute is `action`, and it largely determines which other attributes are relevant or required. If you pass irrelevant option attributes, or fail to pass required ones, `optparse` raises an `OptionError` exception explaining your

store this option's argument (default)

store a constant value, pre-set via `Option.const`

store `True`

store `False`

append this option's argument to a list

append a constant value to a list, pre-set via `Option.const`

increment a counter by one

call a specified function

print a usage message including all options and the documentation for them

As you can see, most actions involve storing or updating a value somewhere. `optparse` always creates a special object for this, conventionally called `options`, which is an instance of `optparse.Values`.

class `optparse.Values`

An object holding parsed argument names and values as attributes. Normally created by calling `parser.parse_args()` when calling `OptionParser.parse_args()`, and can be overridden by a custom subclass passed to the `values` argument of `OptionParser.parse_args()` (as described in [Parsing arguments](#)).

Option arguments (and various other values) are stored as attributes of this object, according to the `dest` (destination) option attribute.

For example, when you call

```
parser.parse_args()
```

one of the first things `optparse` does is create the `options` object:

```
options = Values()
```

If one of the options in this parser is defined with

```
parser.add_option("-f", "--file", action="store", type="string", dest="filename")
```

and the command-line being parsed includes any of the following:

```
-ffoo
-f foo
--file=foo
--file foo
```

then `optparse`, on seeing this option, will do the equivalent of

```
options.filename = "foo"
```

The `type` and `dest` option attributes are almost as important as `action`, but `action` is the only one that makes sense for all options.

Option attributes

class `optparse.Option`

A single command line argument, with various attributes passed by keyword to the constructor. Normally created with `OptionParser.add_option()` rather than directly, and can be overridden by a custom class via the `option_class` argument to `OptionParser`.

The following option attributes may be passed as keyword arguments to `OptionParser.add_option()`. If you pass an option attribute that is not relevant to a particular option, or fail to pass a required option attribute, `optparse` raises `OptionError`.

Option.action

(default: "store") Determines `optparse`'s behaviour when this option is seen on the command line; the available options are documented [here](#).

Option.type

(default: "string") The argument type expected by this option (e.g., "string" or "int"); the available option types are documented [here](#).

Option.dest

(default: derived from option strings) If the option's action implies writing or modifying a value somewhere, this tells `optparse` where to write it: `dest` names an attribute of the `options` object that `optparse` builds as it parses the command line.

Option.default

The value to use for this option's destination if the option is not seen on the command line. See also `OptionParser.set_defaults()`.

Option.nargs

(default: 1) How many arguments of type `type` should be consumed when this option is seen. If > 1 , `optparse` will store a tuple of values to `dest`.

Option.const

For actions that store a constant value, the constant value to store.

Option.choices

For options of type "choice", the list of strings the user may choose from.

Option.callback

For options with action "callback", the callable to call when this option is seen. See section [Option Callbacks](#) for detail on the arguments passed to the callable.

Option.callback_args

Option.callback_kwargs

Additional positional and keyword arguments to pass to callback after the four standard callback arguments.

Option.help

Help text to print for this option when listing all available options after the user supplies a help option (such as --help). If no help text is supplied, the option will be listed without help text. To hide this option, use the special value `optparse.SUPPRESS_HELP`.

Option.metavar

(default: derived from option strings) Stand-in for the option argument(s) to use when printing help text. See section [Tutorial](#) for an example.

Standard option actions

The various option actions all have slightly different requirements and effects. Most actions have several relevant option attributes which you may specify to guide `optparse`'s behaviour; a few have required attributes, which you must specify for any option using that action.

- `"store"` [relevant: type, dest, nargs, choices] The option must be followed by an argument, which is converted to a value according to `type` and stored in `dest`. If `nargs > 1`, multiple arguments will be consumed from the command line; all will be converted according to `type` and stored to `dest` as a tuple. See the [Standard option types](#) section. If `choices` is supplied (a list or tuple of strings), the type defaults to `"ch"`.
- `"store_const"` [required: const; relevant: dest] The value `const` is stored in `dest`. Example: `parser.add_option("-q", "--quiet", action="store_const", const=0, dest="verbose")` `parser.add_option("-v", "--verbose", action="store_const", const=1, dest="verbose")` `parser.add_option("--noisy", action="store_const", const=2, dest="verbose")` If `--noisy` is seen, `optparse` will set `options.verbose=2`.
- `"store_true"` [relevant: dest] A special case of `"store_const"` that stores `True` to `dest`.
- `"store_false"` [relevant: dest] Like `"store_true"`, but stores `False`. Example: `parser.add_option("--clobber", action="store_true", dest="clobber")` `parser.add_option("--no-clobber", action="store_false", dest="clobber")`.
- `"append"` [relevant: type, dest, nargs, choices] The option must be followed by an argument, which is appended to the list in `dest`. If no default value for `dest` is supplied, an empty list is automatically created when `optparse` first encounters this option on the command-line. If `nargs > 1`, multiple arguments are consumed, and a tuple of length `nargs` is appended to `dest`. The defaults for `type` and `dest` are the same as
- `"append_const"` [required: const; relevant: dest] Like `"store_const"`, but the value `const` is appended to `dest`; as with `"append"`, `dest` defaults to `None`, and an empty list is automatically created the first time the option is encountered.
- `"count"` [relevant: dest] Increment the integer stored at `dest`. If no default value is supplied, `dest` is set to zero before being incremented the first time. Example: `parser.add_option("-v", action="count", dest="verbosity")` The first time `-v` is seen on the command line, `optparse` does the equivalent of: `options.verbosity=0` `options.verbosity+=1` Every subsequent occurrence of `-v` results in `options.verbosity+=1`.
- `"callback"` [required: callback; relevant: type, nargs, callback_args, callback_kwargs] Call the function specified by `callback`, which is called as `func(option, opt_str, value, parser, *args, **kwargs)` See section [Option Callbacks](#) for more detail.
- `"help"` Prints a complete help message for all the options in the current option parser. The help message is constructed from the `usage` string passed to `OptionParser`'s constructor and the `help` string passed to every option. If no `help` string is supplied for an option, it will still be listed in the help message. To omit an option entirely, use the special value `optparse.SUPPRESS_HELP`. `optparse` automatically

- "version" Prints the version number supplied to the OptionParser to stdout and exits. The version number is actually formatted and printed by the `print_version()` method of OptionParser. Generally only relevant if the `version` argument is supplied to the OptionParser constructor. As with `help` options, you will rarely create `version` options, since `optparse` automatically adds them when needed.

Standard option types

`optparse` has five built-in option types: "string", "int", "choice", "float" and "complex". If you need to add new option types, see section [Extending optparse](#).

Arguments to string options are not checked or converted in any way: the text on the command line is stored in the destination (or passed to the callback) as-is.

Integer arguments (type "int") are parsed as follows:

- if the number starts with 0x, it is parsed as a hexadecimal number
- if the number starts with 0, it is parsed as an octal number
- if the number starts with 0b, it is parsed as a binary number
- otherwise, the number is parsed as a decimal number

The conversion is done by calling `int()` with the appropriate base (2, 8, 10, or 16). If this fails, so will `optparse`, although with a more useful error message.

"float" and "complex" option arguments are converted directly with `float()` and `complex()`, with similar error-handling.

"choice" options are a subtype of "string" options. The `choices` option attribute (a sequence of strings) defines the set of allowed option arguments. `optparse.check_choice()` compares user-supplied option arguments against this master list and raises `OptionValueError` if an invalid string is given.

Parsing arguments

The whole point of creating and populating an OptionParser is to call its `parse_args()` method.

OptionParser.parse_args(args=None, values=None)

args

values

options

args

Parse the command-line options found in `args`. The input parameters are `args` the list of arguments to process (default: `sys.argv[1:]`) `values` a `Values` object to store option arguments in (default: a new instance of `Values`) – if you give an existing object, the option defaults will not be initialized on it and the return value is a pair (`options`, `args`) where `options` is the same object that was passed in as `values`, or the `optparse.Values` instance created by `optparse` `args` the leftover positional arguments after all options have

the list of arguments to process (default: `sys.argv[1:]`)

a `Values` object to store option arguments in (default: a new instance of `Values`) – if you give an existing object, the option defaults will not be initialized on it

the same object that was passed in as `values`, or the `optparse.Values` instance created by `optparse`

the leftover positional arguments after all options have been processed

The most common usage is to supply neither keyword argument. If you supply `values`, it will be modified with repeated `setattr()` calls (roughly one for every option argument stored to an option destination) and returned by `parse_args()`.

If `parse_args()` encounters any errors in the argument list, it calls the `OptionParser.error()` method with an appropriate end-user error message. This ultimately terminates your process with an exit status of 2 (the traditional Unix exit status for command-line errors).

Querying and manipulating your option parser

The default behavior of the option parser can be customized slightly, and you can also poke around your option parser and see what's there. `OptionParser` provides several methods to help you out:

`OptionParser.disable_interspersed_args()`

Set parsing to stop on the first non-option. For example, if `a` and `b` are both simple options that take no arguments, `optparse` normally accepts this syntax: `prog -a arg1 -b arg2` and treats it as equivalent to `prog -a -b arg1 arg2`. To disable this feature, call `disable_interspersed_args()`. This restores traditional Unix syntax, where option parsing stops with the first non-option argument. Use this if you have a command processor which runs another command which has options of its own and you want to make sure the

`OptionParser.enable_interspersed_args()`

Set parsing to not stop on the first non-option, allowing interspersing switches with command arguments. This is the default behavior.

`OptionParser.get_option(opt_str)`

Returns the `Option` instance with the option string `opt_str`, or `None` if no options have that option string.

`OptionParser.has_option(opt_str)`

Return `True` if the `OptionParser` has an option with option string `opt_str` (e.g., `-q` or `--verbose`).

`OptionParser.remove_option(opt_str)`

If the `OptionParser` has an option corresponding to `opt_str`, that option is removed. If that option provided any other option strings, all of those option strings become invalid. If `opt_str` does not occur in any option belonging to this `OptionParser`, raises `ValueError`.

Conflicts between options

If you're not careful, it's easy to define options with conflicting option strings:

```
parser.add_option("-n", "--dry-run", ...)
...
parser.add_option("-n", "--noisy", ...)
```

(This is particularly true if you've defined your own `OptionParser` subclass with some standard options.)

Every time you add an option, `optparse` checks for conflicts with existing options. If it finds any, it invokes the current conflict-handling mechanism. You can set the conflict-handling mechanism either in the constructor:

```
parser = OptionParser(..., conflict_handler=handler)
```

or with a separate call:

```
parser.set_conflict_handler(handler)
```

The available conflict handlers are:

"error"(default)

"resolve"

assume option conflicts are a programming error and raise `OptionConflictError`

resolve option conflicts intelligently (see below)

As an example, let's define an `OptionParser` that resolves conflicts intelligently and add conflicting options to it:


```

parser = OptionParser(conflict_handler="resolve")
parser.add_option("-n", "--dry-run", ..., help="do no harm")
parser.add_option("-n", "--noisy", ..., help="be noisy")

```

At this point, `optparse` detects that a previously added option is already using the `-n` option string. Since `conflict_handler` is "resolve", it resolves the situation by removing `-n` from the earlier option's list of option strings. Now `--dry-run` is the only way for the user to activate that option. If the user asks for help, the help message will reflect that:

```

Options:
  --dry-run      do no harm
  ...
  -n, --noisy    be noisy

```

It's possible to whittle away the option strings for a previously added option until there are none left, and the user has no way of invoking that option from the command-line. In that case, `optparse` removes that option completely, so it doesn't show up in help text or anywhere else. Carrying on with our existing `OptionParser`:

```

parser.add_option("--dry-run", ..., help="new dry-run option")

```

At this point, the original `-n/--dry-run` option is no longer accessible, so `optparse` removes it, leaving this help text:

```

Options:
  ...
  -n, --noisy    be noisy
  --dry-run      new dry-run option

```

Cleanup

`OptionParser` instances have several cyclic references. This should not be a problem for Python's garbage collector, but you may wish to break the cyclic references explicitly by calling `destroy()` on your `OptionParser` once you are done with it. This is particularly useful in long-running applications where large object graphs are reachable from your `OptionParser`.

Other methods

`OptionParser` supports several other public methods:

OptionParser.set_usage(usage)

Set the usage string according to the rules described above for the `usage` constructor keyword argument. Passing `None` sets the default usage string; use `optparse.SUPPRESS_USAGE` to suppress a usage message.

OptionParser.print_usage(file=None)

Print the usage message for the current program (`self.usage`) to `file` (default `stdout`). Any occurrence of the string `%prog` in `self.usage` is replaced with the name of the current program. Does nothing if `self.usage` is empty or not defined.

OptionParser.get_usage()

Same as `print_usage()` but returns the usage string instead of printing it.

OptionParser.set_defaults(dest=value,...)

Set default values for several option destinations at once. Using `set_defaults()` is the preferred way to set default values for options, since multiple options can share the same destination. For example, if several "mode" options all set the same destination, any one of them can set the default, and the last one wins:

```

parser.add_option("--advanced", action="store_const", dest="mode", const="advanced", default="novice")
# overridden below
parser.add_option("--novice", action="store_const", dest="mode", co

```

Option Callbacks

When `optparse`'s built-in actions and types aren't quite enough for your needs, you have two choices: extend `optparse` or define a callback option. Extending `optparse` is more general, but overkill for a lot of simple cases. Quite often a simple callback is all you need.

There are two steps to defining a callback option:

- define the option itself using the "callback" action
- write the callback; this is a function (or method) that takes at least four arguments, as described below

Defining a callback option

As always, the easiest way to define a callback option is by using the `OptionParser.add_option()` method. Apart from action, the only option attribute you must specify is `callback`, the function to call:

```
parser.add_option("-c", action="callback", callback=my_callback)
```

`callback` is a function (or other callable object), so you must have already defined `my_callback()` when you create this callback option. In this simple case, `optparse` doesn't even know if it takes any arguments, which usually means that the option takes no arguments—the mere presence of `-c` on the command-line is all it needs to know. In some circumstances, though, you might want your callback to consume an arbitrary number of command-line arguments. This is where writing callbacks gets tricky; it's covered later in this section.

`optparse` always passes four particular arguments to your callback, and it will only pass additional arguments if you specify them via `callback_args` and `callback_kwargs`. Thus, the minimal callback function signature is:

```
def my_callback(option, opt, value, parser):
```

The four arguments to a callback are described below.

There are several other option attributes that you can supply when you define a callback option:

type

nargs

callback_args

callback_kwargs

`type` has its usual meaning: as with the "store" or "append" actions, it instructs `optparse` to consume one argument and convert it to `type`. Rather than storing the converted value(s) anywhere, though, `optparse` passes it to your callback function.

`nargs` also has its usual meaning: if it is supplied and `> 1`, `optparse` will consume `nargs` arguments, each of which must be convertible to `type`. It then passes a tuple of converted values to your callback.

`callback_args` is a tuple of extra positional arguments to pass to the callback

`callback_kwargs` is a dictionary of extra keyword arguments to pass to the callback

How callbacks are called

All callbacks are called as follows:

```
func(option, opt_str, value, parser, *args, **kwargs)
```

where

option

opt_str

value

parser

parser.largs

parser.rargs

parser.values

args

kwargs

is the Option instance that's calling the callback

is the option string seen on the command-line that's triggering the callback. (If an abbreviated long option was used, `opt_str` will be the full, canonical option string—e.g. if the user puts `--fooon` on the command-line as an abbreviation for `--foobar`, then `opt_str` will be `--foobar`.)

is the argument to this option seen on the command-line. `optparse` will only expect an argument if `type` is set; the type of `value` will be the type implied by the option's type. If `type` for this option is `None` (no argument expected), then `value` will be `None`. If `nargs > 1`, `value` will be a tuple of values of the appropriate type.

is the `OptionParser` instance driving the whole thing, mainly useful because you can access some other interesting data through its instance attributes: `parser.largs` the current list of leftover arguments, ie. arguments that have been consumed but are neither options nor option arguments. Feel free to modify `parser.largs`, e.g. by adding more arguments to it. (This list will become `args`, the second return value of `parse_args()`.) `parser.rargs` the current list of remaining arguments, ie. with `opt_str` and `val`

the current list of leftover arguments, ie. arguments that have been consumed but are neither options nor option arguments. Feel free to modify `parser.largs`, e.g. by adding more arguments to it. (This list will become `args`, the second return value of `parse_args()`.)

the current list of remaining arguments, ie. with `opt_str` and `value` (if applicable) removed, and only the arguments following them still there. Feel free to modify `parser.rargs`, e.g. by consuming more arguments.

the object where option values are by default stored (an instance of `optparse.OptionValues`). This lets callbacks use the same mechanism as the rest of `optparse` for storing option values; you don't need to mess around with globals or closures. You can also access or modify the value(s) of any options already encountered on the command-line.

is a tuple of arbitrary positional arguments supplied via the `callback_args` option attribute.

is a dictionary of arbitrary keyword arguments supplied via `callback_kwargs`.

Raising errors in a callback

The callback function should raise `OptionValueError` if there are any problems with the option or its argument(s). `optparse` catches this and terminates the program, printing the error message you supply to `stderr`. Your message should be clear, concise, accurate, and mention the option at fault. Otherwise, the user will have a hard time figuring out what they did wrong.

Callback example 1: trivial callback

Here's an example of a callback option that takes no arguments, and simply records that the option was seen:

```
def record_foo_seen(option, opt_str, value, parser):
    parser.values.saw_foo = True

parser.add_option("--foo", action="callback", callback=record_foo_seen)
```

Of course, you could do that with the `"store_true"` action.

Callback example 2: check option order

Here's a slightly more interesting example: record the fact that `-a` is seen, but blow up if it comes after `-b` in the command-line.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use -a after -b")
    parser.values.a = 1
...
parser.add_option("-a", action="callback", callback=check_order)
parser.add_option("-b", action="store_true", dest="b")
```

Callback example 3: check option order (generalized)

If you want to reuse this callback for several similar options (set a flag, but blow up if `-b` has already been seen), it needs a bit of work: the error message and the flag that it sets must be generalized.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use %s after -b" % opt_str)
    setattr(parser.values, option.dest, 1)
...
parser.add_option("-a", action="callback", callback=check_order, dest='a')
parser.add_option("-b", action="store_true", dest="b")
parser.add_option("-c", action="callback", callback=check_order, dest='c')
```

Callback example 4: check arbitrary condition

Of course, you could put any condition in there—you're not limited to checking the values of already-defined options. For example, if you have options that should not be called when the moon is full, all you have to do is this:

```
def check_moon(option, opt_str, value, parser):
    if is_moon_full():
        raise OptionValueError("%s option invalid when moon is full"
                                % opt_str)
    setattr(parser.values, option.dest, 1)
...
parser.add_option("--foo",
                  action="callback", callback=check_moon, dest="foo")
```

(The definition of `is_moon_full()` is left as an exercise for the reader.)

Callback example 5: fixed arguments

Things get slightly more interesting when you define callback options that take a fixed number of arguments. Specifying that a callback option takes arguments is similar to defining a `store` or `append` option: if you define `type`, then the option takes one argument that must be convertible to that type; if you further define `nargs`, then the option takes `nargs` arguments.

Here's an example that just emulates the standard `store` action:

```
def store_value(option, opt_str, value, parser):
    setattr(parser.values, option.dest, value)
...
parser.add_option("--foo",
                  action="callback", callback=store_value,
                  type="int", nargs=3, dest="foo")
```

Note that `optparser` takes care of consuming 3 arguments and converting them to integers for you; all you have to do is store them. (Or whatever; obviously you don't need a callback for this example.)

Callback example 6: variable arguments

Things get hairy when you want an option to take a variable number of arguments. For this case, you must write a callback, as `optparse` doesn't provide any built-in capabilities for it. And you have to deal with certain intricacies of conventional Unix command-line parsing that `optparse` normally handles for you. In particular, callbacks should implement the conventional rules for bare--and-arguments:

- either--or-can be option arguments
- bare--(if not the argument to some option): halt command-line processing and discard the--
- bare-(if not the argument to some option): halt command-line processing but keep the-(append it to `parser.largs`)

If you want an option that takes a variable number of arguments, there are several subtle, tricky issues to worry about. The exact implementation you choose will be based on which trade-offs you're willing to make for your application (which is why `optparse` doesn't support this sort of thing directly).

Nevertheless, here's a stab at a callback for an option with variable arguments:

```
def vararg_callback(option, opt_str, value, parser):
    assert value is None
    value = []

    def floatable(str):
        try:
            float(str)
            return True
        except ValueError:
            return False

    for arg in parser.rargs:
        # stop on --foo like options
        if arg[:2] == "--" and len(arg) > 2:
            break
        # stop on -a, but not on -3 or -3.0
        if arg[:1] == "-" and len(arg) > 1 and not floatable(arg):
            break
        value.append(arg)

    del parser.rargs[:len(value)]
    setattr(parser.values, option.dest, value)

...
parser.add_option("-c", "--callback", dest="vararg_attr",
                  action="callback", callback=vararg_callback)
```

Extending `optparse`

Since the two major controlling factors in how `optparse` interprets command-line options are the action and type of each option, the most likely direction of extension is to add new actions and new types.

Adding new types

To add new types, you need to define your own subclass of `optparse`'s `Option` class. This class has a couple of attributes that define `optparse`'s types: `TYPES` and `TYPE_CHECKER`.

Option.TYPES

A tuple of type names; in your subclass, simply define a new tuple `TYPE` that builds on the standard one.

Option.TYPE_CHECKER

A dictionary mapping type names to type-checking functions. A type-checking function has the following signature: `def check_mytype(option, opt, value)` where `option` is an `Option` instance, `opt` is an option string (e.g., `-f`), and `value` is the string from the command line that must be checked and converted to your desired type. `check_mytype()` should return an object of the hypothetical type `mytype`. The value returned by a type-checking function will wind up in the `OptionValues` instance returned by `OptionParser.pars`

Here's a silly example that demonstrates adding a "complex" option type to parse Python-style complex numbers on the command line. (This is even sillier than it used to be, because `optparse` 1.3 added built-in support for complex numbers, but never mind.)

First, the necessary imports:

```
from copy import copy
from optparse import Option, OptionValueError
```

You need to define your type-checker first, since it's referred to later (in the `TYPE_CHECKER` class attribute of your `Option` subclass):

```
def check_complex(option, opt, value):
    try:
        return complex(value)
    except ValueError:
        raise OptionValueError(
            "option %s: invalid complex value: %r" % (opt, value))
```

Finally, the `Option` subclass:

```
class MyOption (Option):
    TYPES = Option.TYPES + ("complex",)
    TYPE_CHECKER = copy(Option.TYPE_CHECKER)
    TYPE_CHECKER["complex"] = check_complex
```

(If we didn't make a `copy()` of `Option.TYPE_CHECKER`, we would end up modifying the `TYPE_CHECKER` attribute of `optparse`'s `Option` class. This being Python, nothing stops you from doing that except good manners and common sense.)

That's it! Now you can write a script that uses the new option type just like any other `optparse`-based script, except you have to instruct your `OptionParser` to use `MyOption` instead of `Option`:

```
parser = OptionParser(option_class=MyOption)
parser.add_option("-c", type="complex")
```

Alternately, you can build your own option list and pass it to `OptionParser`; if you don't use `add_option()` in the above way, you don't need to tell `OptionParser` which option class to use:

```
option_list = [MyOption("-c", action="store", type="complex", dest="c")]
parser = OptionParser(option_list=option_list)
```

Adding new actions

Adding new actions is a bit trickier, because you have to understand that `optparse` has a couple of classifications for actions:

“store” actions

“typed” actions

actions that result in `optparse` storing a value to an attribute of the current `OptionValues` instance; these options require a `dest` attribute to be supplied to the `Option` constructor.

actions that take a value from the command line and expect it to be of a certain type; or rather, a string that can be converted to a certain type. These options require a `type` attribute to the `Option` constructor.

These are overlapping sets: some default “store” actions are “store”, “store_const”, “append”, and “count”, while the default “typed” actions are “store”, “append”, and “callback”.

When you add an action, you need to categorize it by listing it in at least one of the following class attributes of `Option` (all are lists of strings):

Option.ACTIONS

All actions must be listed in `ACTIONS`.

Option.STORE_ACTIONS

“store” actions are additionally listed here.

Option.TYPED_ACTIONS

“typed” actions are additionally listed here.

Option.ALWAYS_TYPED_ACTIONS

Actions that always take a type (i.e. whose options always take a value) are additionally listed here. The only effect of this is that `optparse` assigns the default type, “string”, to options with no explicit type whose action is listed in `ALWAYS_TYPED_ACTIONS`.

In order to actually implement your new action, you must override `Option.take_action()` method and add a case that recognizes your action.

For example, let’s add an “extend” action. This is similar to the standard “append” action, but instead of taking a single value from the command-line and appending it to an existing list, “extend” will take multiple values in a single comma-delimited string, and extend an existing list with them. That is, if --names is an “extend” option of type “string”, the command line

```
--names=foo,bar --names blah --names ding,dong
```

would result in a list

```
["foo", "bar", "blah", "ding", "dong"]
```

Again we define a subclass of `Option`:

```
class MyOption(Option):

    ACTIONS = Option.ACTIONS + ("extend",)
    STORE_ACTIONS = Option.STORE_ACTIONS + ("extend",)
    TYPED_ACTIONS = Option.TYPED_ACTIONS + ("extend",)
    ALWAYS_TYPED_ACTIONS = Option.ALWAYS_TYPED_ACTIONS + ("extend",)

    def take_action(self, action, dest, opt, value, values, parser):
        if action == "extend":
            lvalue = value.split(",")
            values.ensure_value(dest, []).extend(lvalue)
        else:
            Option.take_action(
                self, action, dest, opt, value, values, parser)
```

Features of note:

- “extend” both expects a value on the command-line and stores that value somewhere, so it goes in both `STORE_ACTIONS` and `TYPED_ACTIONS`.
- to ensure that `optparse` assigns the default type of “string” to “extend” actions, we put the “extend” action in `ALWAYS_TYPED_ACTIONS` as well.
- `MyOption.take_action()` implements just this one new action, and passes control back to `Option.take_action()` for the standard `optparse` actions.

- `values` is an instance of the `optparse.Values` class, which provides the very useful `ensure_value()` method. `ensure_value()` is essentially `getattr()` with a safety valve; it is called as `values.ensure_value(attr, value)`. If the `attr` attribute of `values` doesn't exist or is `None`, then `ensure_value()` first sets it to `value`, and then returns `value`. This is very handy for actions like "extend", "append", and "count", all

Exceptions

`exceptionoptparse.OptionError`

Raised if an `Option` instance is created with invalid or inconsistent arguments.

`exceptionoptparse.OptionConflictError`

Raised if conflicting options are added to an `OptionParser`.

`exceptionoptparse.OptionValueError`

Raised if an invalid option value is encountered on the command line.

`exceptionoptparse.BadOptionError`

Raised if an invalid option is passed on the command line.

`exceptionoptparse.AmbiguousOptionError`

Raised if an ambiguous option is passed on the command line.

getpass— Portable password input

Source code: `Lib/getpass.py`

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

The `getpass` module provides two functions:

`getpass.getpass(prompt='Password:', stream=None, *, echo_char=None)`

Prompt the user for a password without echoing. The user is prompted using the string `prompt`, which defaults to `'Password: '`. On Unix, the prompt is written to the file-like object `stream` using the `replace` error handler if needed. `stream` defaults to the controlling terminal (`/dev/tty`) or if that is unavailable to `sys.stderr` (this argument is ignored on Windows). The `echo_char` argument controls how user input is displayed while typing. If `echo_char` is `None` (default), input remains hidden. Otherwise, `echo_char` mu

`exception getpass.GetPassWarning`

A `UserWarning` subclass issued when password input may be echoed.

`getpass.getuser()`

Return the “login name” of the user. This function checks the environment variables `LOGNAME`, `USER`, `LNAME` and `USERNAME`, in order, and returns the value of the first one which is set to a non-empty string. If none are set, the login name from the password database is returned on systems which support the `pwd` module, otherwise, an `OSError` is raised. In general, this function should be preferred over `os.getlogin()`. Changed in version 3.13: Previously, various exceptions beyond just `OSError` were raised.

fileinput— Iterate over lines from multiple input streams

Source code: `Lib/fileinput.py`

This module implements a helper class and functions to quickly write a loop over standard input or a list of files. If you just want to read or write one file see `open()`.

The typical use is:

```
import fileinput
for line in fileinput.input(encoding="utf-8"):
    process(line)
```

This iterates over the lines of all files listed in `sys.argv[1:]`, defaulting to `sys.stdin` if the list is empty. If a filename is '-', it is also replaced by `sys.stdin` and the optional arguments `mode` and `openhook` are ignored. To specify an alternative list of filenames, pass it as the first argument to `input()`. A single file name is also allowed.

All files are opened in text mode by default, but you can override this by specifying the `mode` parameter in the call to `input()` or `FileInput`. If an I/O error occurs during opening or reading a file, `IOError` is raised.

Changed in version 3.3: `IOError` used to be raised; it is now an alias of `OSError`.

If `sys.stdin` is used more than once, the second and further use will return no lines, except perhaps for interactive use, or if it has been explicitly reset (e.g. `sys.stdin.seek(0)`).

Empty files are opened and immediately closed; the only time their presence in the list of filenames is noticeable at all is when the last file opened is empty.

Lines are returned with any newlines intact, which means that the last line in a file may not have one.

You can control how files are opened by providing an opening hook via the `openhook` parameter to `fileinput.input()` or `FileInput`. The hook must be a function that takes two arguments, `filename` and `mode`, and returns an accordingly opened file-like object. If `encoding` and/or `errors` are specified, they will be passed to the hook as additional keyword arguments. This module provides a `hook_compressed()` to support compressed files.

The following function is the primary interface of this module:

`fileinput.input(files=None, inplace=False, backup='', *, mode='r', openhook=None, encoding=None, errors=None)`

Create an instance of the `FileInput` class. The instance will be used as global state for the functions of this module, and is also returned to use during iteration. The parameters to this function will be passed along to the constructor of the `FileInput` class. The `FileInput` instance can be used as a context manager in the `with` statement. In this example, `input` is closed after the `with` statement is exited, even if an exception occurs: `with fileinput.input(files=('spam.txt', 'eggs.txt'), encoding="utf-8") as f: fo`

The following functions use the global state created by `fileinput.input()`; if there is no active state, `RuntimeError` is raised.

`fileinput.filename()`

Return the name of the file currently being read. Before the first line has been read, returns `None`.

`fileinput.fileeno()`

Return the integer “file descriptor” for the current file. When no file is opened (before the first line and between files), returns -1.

`fileinput.lineno()`

Return the cumulative line number of the line that has just been read. Before the first line has been read, returns 0. After the last line of the last file has been read, returns the line number of that line.

fileinput.filelineno()

Return the line number in the current file. Before the first line has been read, returns 0. After the last line of the last file has been read, returns the line number of that line within the file.

fileinput.isfirstline()

Return True if the line just read is the first line of its file, otherwise return False.

fileinput.isstdin()

Return True if the last line was read from sys.stdin, otherwise return False.

fileinput.nextfile()

Close the current file so that the next iteration will read the first line from the next file (if any); lines not read from the file will not count towards the cumulative line count. The filename is not changed until after the first line of the next file has been read. Before the first line has been read, this function has no effect; it cannot be used to skip the first file. After the last line of the last file has been read, this function has no effect.

fileinput.close()

Close the sequence.

The class which implements the sequence behavior provided by the module is available for subclassing as well:

class fileinput.FileInput(files=None, inplace=False, backup="", *, mode='r', openhook=None, encoding=None, errors=None)

Class FileInput is the implementation; its methods filename(), lineno(), filelineno(), isfirstline(), isstdin(), nextfile() and close() correspond to the functions of the same name in the module. In addition it is iterable and has a readline() method which returns the next input line. The sequence must be accessed in strictly sequential order; random access and readline() cannot be mixed. With mode you can specify which file mode will be passed to open(). It must be one of 'r' and 'rb'. The openhook, when given,

Optional in-place filtering: if the keyword argument inplace=True is passed to fileinput.input() or to the FileInput constructor, the file is moved to a backup file and standard output is directed to the input file (if a file of the same name as the backup file already exists, it will be replaced silently). This makes it possible to write a filter that rewrites its input file in place. If the backup parameter is given (typically as backup='.<someextension>'), it specifies the extension for the backup file, and the backup file remains around; by default, the extension is '.bak' and it is deleted when the output file is closed. In-place filtering is disabled when standard input is read.

The two following opening hooks are provided by this module:

fileinput.hook_compressed(filename, mode, *, encoding=None, errors=None)

Transparently opens files compressed with gzip and bzip2 (recognized by the extensions '.gz' and '.bz2') using the gzip and bz2 modules. If the filename extension is not '.gz' or '.bz2', the file is opened normally (ie, using open()) without any decompression. The encoding and errors values are passed to io.TextIOWrapper for compressed files and open for normal files. Usage

example: fi = fileinput.FileInput(openhook=fileinput.hook_compressed, encoding="utf-8")
Changed in version 3.10: The keyword-only parameter encoding

fileinput.hook_encoded(encoding, errors=None)

Returns a hook which opens each file with open(), using the given encoding and errorsto read the file. Usage
example: fi = fileinput.FileInput(openhook=fileinput.hook_encoded("utf-8", "surrogateescape"))
Changed in version 3.6: Added the optional errors parameter. Deprecated since version 3.10: This function is deprecated

since `fileinput.input()` and `FileInput` now have `encoding` and `errors` parameters.

curses— Terminal handling for character-cell displays

Source code: [Lib/curses](#)

The `curses` module provides an interface to the `curses` library, the de-facto standard for portable advanced terminal handling.

While `curses` is most widely used in the Unix environment, versions are available for Windows, DOS, and possibly other systems as well. This extension module is designed to match the API of `ncurses`, an open-source `curses` library hosted on Linux and the BSD variants of Unix.

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

Availability: Unix.

■ Nota: Note Whenever the documentation mentions a character it can be specified as an integer, a one-character Unicode string or a one-byte byte string. Whenever the documentation mentions a character string it can be specified as a Unicode string or a byte string.

See also

Module `curses.ascii`

Module `curses.panel`

Module `curses.textpad`

Curses Programming with Python

Utilities for working with ASCII characters, regardless of your locale settings.

A panel stack extension that adds depth to `curses` windows.

Editable text widget for `curses` supporting Emacs-like bindings.

Tutorial material on using `curses` with Python, by Andrew Kuchling and Eric Raymond.

Functions

The module `curses` defines the following exception:

exception `curses.error`

Exception raised when a `curses` library function returns an error.

■ Nota: Note Whenever x or y arguments to a function or a method are optional, they default to the current cursor location. Whenever attr is optional, it defaults to `A_NORMAL`.

The module `curses` defines the following functions:

`curses.assume_default_colors(fg,bg,)`

Allow use of default values for colors on terminals supporting this feature. Use this to support transparency in your application. Assign terminal default foreground/background colors to color number-1.

So `init_pair(x,COLOR_RED,-1)` will initialize pair x as red on default background

and `init_pair(x,-1,COLOR_BLUE)` will initialize pair x as default foreground on blue. Change the definition of the

`color_pair0to(fg,bg)`. Added in version 3.14.

`curses.baudrate()`

Return the output speed of the terminal in bits per second. On software terminal emulators it will have a fixed high value. Included for historical reasons; in former times, it was used to write output loops for time delays and occasionally to change interfaces depending on the line speed.

`curses.beep()`

Emit a short attention sound.

`curses.can_change_color()`

Return `True` or `False`, depending on whether the programmer can change the colors displayed by the terminal.

`curses.cbreak()`

Enter cbreak mode. In cbreak mode (sometimes called “rare” mode) normal tty line buffering is turned off and characters are available to be read one by one. However, unlike raw mode, special characters (interrupt, quit, suspend, and flow control) retain their effects on the tty driver and calling program. Calling `firstraw()` then `cbreak()` leaves the terminal in cbreak mode.

`curses.color_content(color_number)`

Return the intensity of the red, green, and blue (RGB) components in the `color_color_number`, which must be between 0 and `COLORS-1`. Return a 3-tuple, containing the R,G,B values for the given color, which will be between 0 (no component) and 1000 (maximum amount of component).

`curses.color_pair(pair_number)`

Return the attribute value for displaying text in the specified color pair. Only the first 256 color pairs are supported. This attribute value can be combined with `A_STANDOUT`, `A_REVERSE`, and the other `A_*` attributes. `pair_number()` is the counterpart to this function.

`curses.curs_set(visibility)`

Set the cursor state. `visibility` can be set to 0, 1, or 2, for invisible, normal, or very visible. If the terminal supports the visibility requested, return the previous cursor state; otherwise raise an exception. On many terminals, the “visible” mode is an underline cursor and the “very visible” mode is a block cursor.

`curses.def_prog_mode()`

Save the current terminal mode as the “program” mode, the mode when the running program is using curses. (Its counterpart is the “shell” mode, for when the program is not in curses.) Subsequent calls to `reset_prog_mode()` will restore this mode.

`curses.def_shell_mode()`

Save the current terminal mode as the “shell” mode, the mode when the running program is not using curses. (Its counterpart is the “program” mode, when the program is using curses capabilities.) Subsequent calls to `reset_shell_mode()` will restore this mode.

`curses.delay_output(ms)`

Insert an `ms` millisecond pause in output.

`curses.doupdate()`

Update the physical screen. The curses library keeps two data structures, one representing the current physical screen contents and a virtual screen representing the desired next state. The `doupdate()` ground updates the physical screen to match the virtual screen. The virtual screen may be updated by an `outrefresh()` call after write operations such as `addstr()` have been performed on a window. The normal `refresh()` call is

simply `noutrefresh()` followed by `doupdate()`; if you have to update multiple windows,

`curses.echo()`

Enter echo mode. In echo mode, each character input is echoed to the screen as it is entered.

`curses.endwin()`

De-initialize the library, and return terminal to normal status.

`curses.erasechar()`

Return the user's current erase character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

`curses.filter()`

The `filter()` routine, if used, must be called before `initscr()` is called. The effect is that, during those calls, `LINES` is set to 1; the capabilities `clear`, `cup`, `cud`, `cud1`, `cuu1`, `cuu`, `vpa` are disabled; and the `home` string is set to the value of `cr`. The effect is that the cursor is confined to the current line, and so are screen updates. This may be used for enabling character-at-a-time line editing without touching the rest of the screen.

`curses.flash()`

Flash the screen. That is, change it to reverse-video and then change it back in a short interval. Some people prefer such as 'visible bell' to the audible attention signal produced by `beep()`.

`curses.flushinp()`

Flush all input buffers. This throws away any typeahead that has been typed by the user and has not yet been processed by the program.

`curses.getmouse()`

After `getch()` returns `KEY_MOUSE` to signal a mouse event, this method should be called to retrieve the queued mouse event, represented as a 5-tuple `(id,x,y,z,bstate)`. `id` is an ID value used to distinguish multiple devices, and `x,y,z` are the event's coordinates. (`z` is currently unused.) `bstate` is an integer value whose bits will be set to indicate the type of event, and will be the bitwise OR of one or more of the following constants, where `n` is the button number from 1 to 5: `BUTTONn_PRESSED`, `BUTTONn_RELEASED`, `BU`

`curses.getsyx()`

Return the current coordinates of the virtual screen cursor as a tuple `(y,x)`. If `leaveok` is currently `True`, then return `(-1,-1)`.

`curses.getwin(file)`

Read window related data stored in the file by an earlier `window.putwin()` call. The routine then creates and initializes a new window using that data, returning the new window object.

`curses.has_colors()`

Return `True` if the terminal can display colors; otherwise, return `False`.

`curses.has_extended_color_support()`

Return `True` if the module supports extended colors; otherwise, return `False`. Extended color support allows more than 256 color pairs for terminals that support more than 16 colors (e.g. `xterm-256color`). Extended color support requires ncurses version 6.1 or later. Added in version 3.10.

`curses.has_ic()`

Return `True` if the terminal has insert- and delete-character capabilities. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.has_il()

Return `True` if the terminal has insert- and delete-line capabilities, or can simulate them using scrolling regions. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.has_key(ch)

Take a key value `ch`, and return `True` if the current terminal type recognizes a key with that value.

curses.halfdelay(tenths)

Used for half-delay mode, which is similar to `cbreak` mode in that characters typed by the user are immediately available to the program. However, after blocking for `tenths` tenths of seconds, raise an exception if nothing has been typed. The value `of tenths` must be a number between 1 and 255. Use `nocbreak()` to leave half-delay mode.

curses.init_color(color_number,r,g,b)

Change the definition of a color, taking the number of the color to be changed followed by three RGB values (for the amounts of red, green, and blue components). The value of `color_number` must be between 0 and `COLORS-1`. Each of `r,g,b`, must be a value between 0 and 1000. When `init_color()` is used, all occurrences of that color on the screen immediately change to the new definition. This function is a no-op on most terminals; it is active only if `can_change_color()` returns `True`.

curses.init_pair(pair_number,fg,bg)

Change the definition of a color-pair. It takes three arguments: the number of the color-pair to be changed, the foreground color number, and the background color number. The value of `pair_number` must be between 1 and `COLOR_PAIRS-1` (the 0 color pair can only be changed by use of `use_default_colors()` and `assume_default_colors()`). The value of `fg` and `bg` arguments must be between 0 and `COLORS-1`, or, after calling `use_default_colors()` or `assume_default_colors()`, -1. If the color-pair was previously initialized, the screen is

curses.initscr()

Initialize the library. Return a window object which represents the whole screen. Note if there is an error opening the terminal, the underlying curses library may cause the interpreter to exit.

curses.is_term_resized(nlines,ncols)

Return `True` if `resize_term()` would modify the window structure, `False` otherwise.

curses.isendwin()

Return `True` if `endwin()` has been called (that is, the curses library has been deinitialized).

curses.keyname(k)

Return the name of the key numbered `k` as a bytes object. The name of a key generating printable ASCII character is the key's character. The name of a control-key combination is a two-byte bytes object consisting of a caret (`b'^'`) followed by the corresponding printable ASCII character. The name of an alt-key combination (128–255) is a bytes object consisting of the prefix `b'M-'` followed by the name of the corresponding ASCII character.

curses.killchar()

Return the user's current line kill character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

curses.longname()

Return a bytes object containing the terminfo long name field describing the current terminal. The maximum length of a verbose description is 128 characters. It is defined only after the call to `initscr()`.

curses.meta(flag)

If `flag` is `True`, allow 8-bit characters to be input. If `flag` is `False`, allow only 7-bit chars.

curses.mouseinterval(interval)

Set the maximum time in milliseconds that can elapse between press and release events in order for them to be recognized as a click, and return the previous interval value. The default value is 200 milliseconds, or one fifth of a second.

curses.mousemask(mousemask)

Set the mouse events to be reported, and return a tuple(`availmask`,`oldmask`).`availmask` indicates which of the specified mouse events can be reported; on complete failure it returns 0. `oldmask` is the previous value of the given window's mouse event mask. If this function is never called, no mouse events are ever reported.

curses.napms(ms)

Sleep for `ms` milliseconds.

curses.newpad(nlines,ncols)

Create and return a pointer to a new pad data structure with the given number of lines and columns. Return a pad as a window object. A pad is like a window, except that it is not restricted by the screen size, and is not necessarily associated with a particular part of the screen. Pads can be used when a large window is needed, and only a part of the window will be on the screen at one time. Automatic refreshes of pads (such as from scrolling or echoing of input) do not occur. Therefore `refresh()` and

curses.newwin(nlines,ncols)

curses.newwin(nlines,ncols,begin_y,begin_x)

Return a new window, whose left-upper corner is at `(begin_y,begin_x)`, and whose height/width is `nlines/ncols`. By default, the window will extend from the specified position to the lower right corner of the screen.

curses.nl()

Enter newline mode. This mode translates the return key into newline on input, and translates newline into return and line-feed on output. Newline mode is initially on.

curses.nocbreak()

Leave `cbreak` mode. Return to normal "cooked" mode with line buffering.

curses.noecho()

Leave echo mode. Echoing of input characters is turned off.

curses.nonl()

Leave newline mode. Disable translation of return into newline on input, and disable low-level translation of newline into newline/return on output (but this does not change the behavior of `addch('\n')`, which always does the equivalent of return and line feed on the virtual screen). With translation off, curses can sometimes speed up vertical motion a little; also, it will be able to detect the return key on input.

curses.noqiflush()

When the `noqiflush()` routine is used, normal flush of input and output queues associated with the `INTR`, `QUIT` and `SUSP` characters will not be done. You may want to call `noqiflush()` in a signal handler if you want output to continue as though the interrupt had not occurred, after the handler exits.

curses.noraw()

Leave raw mode. Return to normal “cooked” mode with line buffering.

curses.pair_content(pair_number)

Return a tuple(fg,bg)containing the colors for the requested color pair. The value ofpair_numbermust be between0andCOLOR_PAIRS-1.

curses.pair_number(attr)

Return the number of the color-pair set by the attribute valueattr.color_pair()is the counterpart to this function.

curses.putp(str)

Equivalent totputs(str,1,putchar); emit the value of a specified terminfo capability for the current terminal. Note that the output ofputp()always goes to standard output.

curses.qiflush([flag])

IfflagisFalse, the effect is the same as callingnoqiflush(). IfflagisTrue, or no argument is provided, the queues will be flushed when these control characters are read.

curses.raw()

Enter raw mode. In raw mode, normal line buffering and processing of interrupt, quit, suspend, and flow control keys are turned off; characters are presented to curses input functions one by one.

curses.reset_prog_mode()

Restore the terminal to “program” mode, as previously saved bydef_prog_mode().

curses.reset_shell_mode()

Restore the terminal to “shell” mode, as previously saved bydef_shell_mode().

curses.resetty()

Restore the state of the terminal modes to what it was at the last call tosavetty().

curses.resize_term(nlines,ncols)

Backend function used byresizeterm(), performing most of the work; when resizing the windows,resize_term()blank-fills the areas that are extended. The calling application should fill in these areas with appropriate data. Theresize_term()function attempts to resize all windows. However, due to the calling convention of pads, it is not possible to resize these without additional interaction with the application.

curses.resizeterm(nlines,ncols)

Resize the standard and current windows to the specified dimensions, and adjusts other bookkeeping data used by the curses library that record the window dimensions (in particular the SIGWINCH handler).

curses.savetty()

Save the current state of the terminal modes in a buffer, usable byresetty().

curses.get_escdelay()

Retrieves the value set byset_escdelay().Added in version 3.9.

curses.set_escdelay(ms)

Sets the number of milliseconds to wait after reading an escape character, to distinguish between an individual escape character entered on the keyboard from escape sequences sent by cursor and function keys.Added in version 3.9.

curses.get_tabsize()

Retrieves the value set byset_tabsize().Added in version 3.9.

curses.set_tabsize(size)

Sets the number of columns used by the curses library when converting a tab character to spaces as it adds the tab to a window. Added in version 3.9.

curses.setsyx(y,x)

Set the virtual screen cursor to y,x. If y and x are both -1, then leave ok set to True.

curses.setupterm(term=None,fd=-1)

Initialize the terminal. term is a string giving the terminal name, or None; if omitted or None, the value of the TERM environment variable will be used. fd is the file descriptor to which any initialization sequences will be sent; if not supplied or -1, the file descriptor for sys.stdout will be used.

curses.start_color()

Must be called if the programmer wants to use colors, and before any other color manipulation routine is called. It is good practice to call this routine right after initscr(). start_color() initializes eight basic colors (black, red, green, yellow, blue, magenta, cyan, and white), and two global variables in the curses module, COLORS and COLOR_PAIRS, containing the maximum number of colors and color-pairs the terminal can support. It also restores the colors on the terminal to the values they had when

curses.termattrs()

Return a logical OR of all video attributes supported by the terminal. This information is useful when a curses program needs complete control over the appearance of the screen.

curses.termname()

Return the value of the environment variable TERM, as a bytes object, truncated to 14 characters.

curses.tigetflag(capname)

Return the value of the Boolean capability corresponding to the terminfo capability name capname as an integer. Return the value -1 if capname is not a Boolean capability, or 0 if it is canceled or absent from the terminal description.

curses.tigetnum(capname)

Return the value of the numeric capability corresponding to the terminfo capability name capname as an integer. Return the value -2 if capname is not a numeric capability, or -1 if it is canceled or absent from the terminal description.

curses.tigetstr(capname)

Return the value of the string capability corresponding to the terminfo capability name capname as a bytes object. Return None if capname is not a terminfo "string capability", or is canceled or absent from the terminal description.

curses.tparm(str,...)

Instantiate the bytes object str with the supplied parameters, where str should be a parameterized string obtained from the terminfo database. E.g. tparm(tigetstr("cup"),5,3) could result in b'\033[6;4H', the exact result depending on terminal type.

curses.typeahead(fd)

Specify that the file descriptor fd be used for typeahead checking. If fd is -1, then no typeahead checking is done. The curses library does "line-breakout optimization" by looking for typeahead periodically while updating the screen. If input is found, and it is coming from a tty, the current update is postponed until refresh or doupdate is called again, allowing faster response to commands typed in advance. This function allows specifying a different file descriptor for typeahead checking.

curses.unctrl(ch)

Return a bytes object which is a printable representation of the character `ch`. Control characters are represented as a caret followed by the character, for example `asb'^C'`. Printing characters are left as they are.

curses.ungetch(ch)

Push `ch` so the next `getch()` will return it. Note Only one `ch` can be pushed before `getch()` is called.

curses.update_lines_cols()

Update the `LINES` and `COLS` module variables. Useful for detecting manual screen resize. Added in version 3.5.

curses.unget_wch(ch)

Push `ch` so the next `get_wch()` will return it. Note Only one `ch` can be pushed before `get_wch()` is called. Added in version 3.3.

curses.ungetmouse(id,x,y,z,bstate)

Push a `KEY_MOUSE` event onto the input queue, associating the given state data with it.

curses.use_env(flag)

If used, this function should be called before `initscr()` or `newterm` are called. When `flag` is `False`, the values of lines and columns specified in the terminfo database will be used, even if environment variables `LINES` and `COLUMNS` (used by default) are set, or if `curses` is running in a window (in which case default behavior would be to use the window size if `LINES` and `COLUMNS` are not set).

curses.use_default_colors()

Equivalent to `assume_default_colors(-1,-1)`.

curses.wrapper(func,/, *args, **kwargs)

Initialize `curses` and call another callable object, `func`, which should be the rest of your `curses`-using application. If the application raises an exception, this function will restore the terminal to a sane state before re-raising the exception and generating a traceback. The callable object `func` is then passed the main window `'stdscr'` as its first argument, followed by any other arguments passed to `wrapper()`. Before calling `func`, `wrapper()` turns on `cbreak` mode, turns off `echo`, enables the terminal

Window Objects

class curses.window

Window objects, as returned by `initscr()` and `newwin()` above, have the following methods and attributes:

window.addch(ch[,attr])

window.addch(y,x,ch[,attr])

Paint character `ch` at `(y,x)` with attributes `attr`, overwriting any character previously painted at that location. By default, the character position and attributes are the current settings for the window object. Note Writing outside the window, subwindow, or pad raises a `curses.error`. Attempting to write to the lower right corner of a window, subwindow, or pad will cause an exception to be raised after the character is printed.

window.addnstr(str,n[,attr])

window.addnstr(y,x,str,n[,attr])

Paint at most `n` characters of the character string `str` at `(y,x)` with attributes `attr`, overwriting anything previously on the display.

window.addstr(str[,attr])

window.addstr(y,x,str[,attr])

Paint the character string `strat(y,x)` with attributes `attr`, overwriting anything previously on the display. Note Writing outside the window, subwindow, or pad raises `curses.error`. Attempting to write to the lower right corner of a window, subwindow, or pad will cause an exception to be raised after the string is printed. A bug in `ncurses`, the backend for this Python module, can cause `SegFaults` when resizing windows. This is fixed in `ncurses-6.1-20190511`. If you are stuck with an earlier `ncurses`, you ca

window.attroff(attr)

Remove attribute `attr` from the “background” set applied to all writes to the current window.

window.attron(attr)

Add attribute `attr` to the “background” set applied to all writes to the current window.

window.attrset(attr)

Set the “background” set of attributes to `attr`. This set is initially 0 (no attributes).

window.bkgd(ch[,attr])

Set the background property of the window to the character `ch`, with attributes `attr`. The change is then applied to every character position in that window: The attribute of every character in the window is changed to the new background attribute. Wherever the former background character appears, it is changed to the new background character.

window.bkgdset(ch[,attr])

Set the window’s background. A window’s background consists of a character and any combination of attributes. The attribute part of the background is combined (OR’ed) with all non-blank characters that are written into the window. Both the character and attribute parts of the background are combined with the blank characters. The background becomes a property of the character and moves with the character through any scrolling and insert/delete line/character operations.

window.border([ls[,rs[,ts[,bs[,tl[,tr[,bl[,br]]]]]]]])

Draw a border around the edges of the window. Each parameter specifies the character to use for a specific part of the border; see the table below for more details. Note A 0 value for any parameter will cause the default character to be used for that parameter. Keyword parameters cannot be used. The defaults are listed in this table:

Parameter	Description	Default value
<code>ls</code>	Left side	<code>ACS_VLINE</code>
<code>rs</code>	Right side	<code>ACS_VLINE</code>
<code>ts</code>	Top	<code>ACS_HLINE</code>
<code>bs</code>	Bottom	<code>ACS_HLINE</code>
<code>tl</code>	Upper-left corner	<code>ACS_ULCORNER</code>
<code>tr</code>	Upper-right corner	<code>ACS_URCORNE</code>

window.box([vertch,horch])

Similar to `border()`, but both `ls` and `rs` are `vertch` and both `ts` and `bs` are `horch`. The default corner characters are always used by this function.

window.chgat(attr)

window.chgat(num,attr)

window.chgat(y,x,attr)

window.chgat(y,x,num,attr)

Set the attributes of `num` characters at the current cursor position, or at position `(y,x)` if supplied. If `num` is not given or is -1, the attribute will be set on all the characters to the end of the line. This function moves cursor to position `(y,x)` if supplied. The changed line will be touched using the `touchline()` method so that the contents will be redisplayed by the next window refresh.

window.clear()

Like `erase()`, but also cause the whole window to be repainted upon next call to `refresh()`.

window.clearok(flag)

If `flag` is `True`, the next call to `refresh()` will clear the window completely.

window.clrtoebol()

Erase from cursor to the end of the window: all lines below the cursor are deleted, and then the equivalent of `clrtoebol()` is performed.

window.clrtoeol()

Erase from cursor to the end of the line.

window.cursyncup()

Update the current cursor position of all the ancestors of the window to reflect the current cursor position of the window.

window.delch([y,x])

Delete any character at `(y,x)`.

window.deleteln()

Delete the line under the cursor. All following lines are moved up by one line.

window.derwin(begin_y,begin_x)**window.derwin(nlines,ncols,begin_y,begin_x)**

An abbreviation for “derive window”, `derwin()` is the same as calling `subwin()`, except that `begin_y` and `begin_x` are relative to the origin of the window, rather than relative to the entire screen. Return a window object for the derived window.

window.echochar(ch[,attr])

Add character `ch` with attribute `attr`, and immediately call `refresh()` on the window.

window.enclose(y,x)

Test whether the given pair of screen-relative character-cell coordinates are enclosed by the given window, returning `True` or `False`. It is useful for determining what subset of the screen windows enclose the location of a mouse event. Changed in version 3.10: Previously it returned 1 or 0 instead of `True` or `False`.

window.encoding

Encoding used to encode method arguments (Unicode strings and characters). The encoding attribute is inherited from the parent window when a subwindow is created, for example with `window.subwin()`. By default, current locale encoding is used (see `locale.getencoding()`). Added in version 3.3.

window.erase()

Clear the window.

window.getbegyx()

Return a tuple `(y,x)` of coordinates of upper-left corner.

window.getbkgd()

Return the given window’s current background character/attribute pair.

window.getch([y,x])

Get a character. Note that the integer returned does not have to be in ASCII range: function keys, keypad keys and so on are represented by numbers higher than 255. In no-delay mode, return -1 if there is no input, otherwise wait until a key is pressed.

window.get_wch([y,x])

Get a wide character. Return a character for most keys, or an integer for function keys, keypad keys, and other special keys. In no-delay mode, raise an exception if there is no input. Added in version 3.3.

window.getkey([y,x])

Get a character, returning a string instead of an integer, as `getch()` does. Function keys, keypad keys and other special keys return a multibyte string containing the key name. In no-delay mode, raise an exception if there is no input.

window.getmaxyx()

Return a tuple (y,x) of the height and width of the window.

window.getparyx()

Return the beginning coordinates of this window relative to its parent window as a tuple (y,x). Return (-1,-1) if this window has no parent.

window.getstr()

window.getstr(n)

window.getstr(y,x)

window.getstr(y,x,n)

Read a bytes object from the user, with primitive line editing capacity. The maximum value for n is 2047. Changed in version 3.14: The maximum value for n was increased from 1023 to 2047.

window.getyx()

Return a tuple (y,x) of current cursor position relative to the window's upper-left corner.

window.hline(ch,n)

window.hline(y,x,ch,n)

Display a horizontal line starting at (y,x) with length n consisting of the character ch.

window.idcok(flag)

If `flag` is `False`, `curses` no longer considers using the hardware insert/delete character feature of the terminal; if `flag` is `True`, use of character insertion and deletion is enabled. When `curses` is first initialized, use of character insert/delete is enabled by default.

window.idlok(flag)

If `flag` is `True`, `curses` will try and use hardware line editing facilities. Otherwise, line insertion/deletion are disabled.

window.immedok(flag)

If `flag` is `True`, any change in the window image automatically causes the window to be refreshed; you no longer have to call `refresh()` yourself. However, it may degrade performance considerably, due to repeated calls to `wrefresh`. This option is disabled by default.

window.inch([y,x])

Return the character at the given position in the window. The bottom 8 bits are the character proper, and upper bits are the attributes.

window.insch(ch[,attr])

window.insch(y,x,ch[,attr])

Paint character `ch` at `(y,x)` with attributes `attr`, moving the line from position `x` right by one character.

window.insdel(nlines)

Insert `nlines` lines into the specified window above the current line. Then `nlines` bottom lines are lost. For negative `nlines`, delete `nlines` lines starting with the one under the cursor, and move the remaining lines up. The bottom `nlines` lines are cleared. The current cursor position remains the same.

window.insertln()

Insert a blank line under the cursor. All following lines are moved down by one line.

window.insnstr(str,n[,attr])

window.insnstr(y,x,str,n[,attr])

Insert a character string (as many characters as will fit on the line) before the character under the cursor, up to `n` characters. If `n` is zero or negative, the entire string is inserted. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to `y,x`, if specified).

window.insstr(str[,attr])

window.insstr(y,x,str[,attr])

Insert a character string (as many characters as will fit on the line) before the character under the cursor. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to `y,x`, if specified).

window.instr([n])

window.instr(y,x[,n])

Return a bytes object of characters, extracted from the window starting at the current cursor position, or at `y,x` if specified. Attributes are stripped from the characters. If `n` is specified, `instr()` returns a string at most `n` characters long (exclusive of the trailing NUL). The maximum value for `n` is 2047. Changed in version 3.14: The maximum value for `n` was increased from 1023 to 2047.

window.is_linetouched(line)

Return `True` if the specified line was modified since the last call to `refresh()`; otherwise return `False`. Raise a `curses.error` exception if `line` is not valid for the given window.

window.is_wintouched()

Return `True` if the specified window was modified since the last call to `refresh()`; otherwise return `False`.

window.keypad(flag)

If `flag` is `True`, escape sequences generated by some keys (keypad, function keys) will be interpreted by `curses`. If `flag` is `False`, escape sequences will be left as is in the input stream.

window.leaveok(flag)

If `flag` is `True`, cursor is left where it is on update, instead of being at "cursor position." This reduces cursor movement where possible. If possible the cursor will be made invisible. If `flag` is `False`, cursor will always be at "cursor position" after an update.

window.move(new_y,new_x)

Move cursor to `(new_y,new_x)`.

window.mvderwin(y,x)

Move the window inside its parent window. The screen-relative parameters of the window are not changed. This routine is used to display different parts of the parent window at the same physical position on the screen.

window.mvwin(new_y,new_x)

Move the window so its upper-left corner is at(new_y,new_x).

window.nodelay(flag)

If flag is True, getch() will be non-blocking.

window.notimeout(flag)

If flag is True, escape sequences will not be timed out. If flag is False, after a few milliseconds, an escape sequence will not be interpreted, and will be left in the input stream as is.

window.noutrefresh()

Mark for refresh but wait. This function updates the data structure representing the desired state of the window, but does not force an update of the physical screen. To accomplish that, call doupdate().

window.overlay(destwin[,sminrow,smincol,dminrow,dmincol,dmaxrow,dmaxcol])

Overlay the window on top of destwin. The windows need not be the same size, only the overlapping region is copied. This copy is non-destructive, which means that the current background character does not overwrite the old contents of destwin. To get fine-grained control over the copied region, the second form of overlay() can be used. sminrow and smincol are the upper-left coordinates of the source window, and the other variables mark a rectangle in the destination window.

window.overwrite(destwin[,sminrow,smincol,dminrow,dmincol,dmaxrow,dmaxcol])

Overwrite the window on top of destwin. The windows need not be the same size, in which case only the overlapping region is copied. This copy is destructive, which means that the current background character overwrites the old contents of destwin. To get fine-grained control over the copied region, the second form of overwrite() can be used. sminrow and smincol are the upper-left coordinates of the source window, the other variables mark a rectangle in the destination window.

window.putwin(file)

Write all data associated with the window into the provided file object. This information can be later retrieved using the getwin() function.

window.redrawln(beg,num)

Indicate that the num screen lines, starting at line beg, are corrupted and should be completely redrawn on the next refresh() call.

window.redrawwin()

Touch the entire window, causing it to be completely redrawn on the next refresh() call.

window.refresh([pminrow,pmincol,sminrow,smincol,smaxrow,smaxcol])

Update the display immediately (sync actual screen with previous drawing/deleting methods). The 6 optional arguments can only be specified when the window is a pad created with newpad(). The additional parameters are needed to indicate what part of the pad and screen are involved. pminrow and pmincol specify the upper left-hand corner of the rectangle to be displayed in the pad. sminrow, smincol, smaxrow, and smaxcol specify the edges of the rectangle to be displayed on the screen. The lower right-hand c

window.resize(nlines,ncols)

Reallocate storage for a curses window to adjust its dimensions to the specified values. If either dimension is larger than the current values, the window's data is filled with blanks that have the current background rendition (as set by `bkgroundset()`) merged into them.

window.scroll([lines=1])

Scroll the screen or scrolling region upward by `lines` lines.

window.scrollok(flag)

Control what happens when the cursor of a window is moved off the edge of the window or scrolling region, either as a result of a newline action on the bottom line, or typing the last character of the last line. If `flag` is `False`, the cursor is left on the bottom line. If `flag` is `True`, the window is scrolled up one line. Note that in order to get the physical scrolling effect on the terminal, it is also necessary to call `idlok()`.

window.setscrreg(top,bottom)

Set the scrolling region from `linetop` to `linebottom`. All scrolling actions will take place in this region.

window.standend()

Turn off the standout attribute. On some terminals this has the side effect of turning off all attributes.

window.standout()

Turn on attribute `A_STANDOUT`.

window.subpad(begin_y,begin_x)

window.subpad(nlines,ncols,begin_y,begin_x)

Return a sub-window, whose upper-left corner is at `(begin_y,begin_x)`, and whose width/height is `ncols/nlines`.

window.subwin(begin_y,begin_x)

window.subwin(nlines,ncols,begin_y,begin_x)

Return a sub-window, whose upper-left corner is at `(begin_y,begin_x)`, and whose width/height is `ncols/nlines`. By default, the sub-window will extend from the specified position to the lower right corner of the window.

window.syncdown()

Touch each location in the window that has been touched in any of its ancestor windows. This routine is called by `refresh()`, so it should almost never be necessary to call it manually.

window.syncok(flag)

If `flag` is `True`, then `syncup()` is called automatically whenever there is a change in the window.

window.syncup()

Touch all locations in ancestors of the window that have been changed in the window.

window.timeout(delay)

Set blocking or non-blocking read behavior for the window. If `delay` is negative, blocking read is used (which will wait indefinitely for input). If `delay` is zero, then non-blocking read is used, and `getch()` will return -1 if no input is waiting. If `delay` is positive, then `getch()` will block for `delay` milliseconds, and return -1 if there is still no input at the end of that time.

window.touchline(start,count[,changed])

Pretend `count` lines have been changed, starting with `linestart`. If `changed` is supplied, it specifies whether the affected lines are marked as having been changed (`changed=True`) or unchanged (`changed=False`).

window.touchwin()

Pretend the whole window has been changed, for purposes of drawing optimizations.

window.untouchwin()

Mark all lines in the window as unchanged since the last call to `refresh()`.

window.vline(ch,n[,attr])

window.vline(y,x,ch,n[,attr])

Display a vertical line starting at `(y,x)` with length `n` consisting of the character `ch` with attributes `attr`.

Constants

The `curses` module defines the following data members:

curses.ERR

Some curses routines that return an integer, such as `getch()`, return `ERR` upon failure.

curses.OK

Some curses routines that return an integer, such as `napms()`, return `OK` upon success.

curses.version

A bytes object representing the current version of the module.

curses.ncurses_version

A named tuple containing the three components of the ncurses library version: `major`, `minor`, and `patch`. All values are integers. The components can also be accessed by name, so `curses.ncurses_version[0]` is equivalent to `curses.ncurses_version.major` and so on. Availability: if the ncurses library is used. Added in version 3.8.

curses.COLORS

The maximum number of colors the terminal can support. It is defined only after the call to `start_color()`.

curses.COLOR_PAIRS

The maximum number of color pairs the terminal can support. It is defined only after the call to `start_color()`.

curses.COLS

The width of the screen, i.e., the number of columns. It is defined only after the call to `initscr()`. Updated by `update_lines_cols()`, `resize_term()` and `resize_term()`.

curses.LINES

The height of the screen, i.e., the number of lines. It is defined only after the call to `initscr()`. Updated by `update_lines_cols()`, `resize_term()` and `resize_term()`.

Some constants are available to specify character cell attributes. The exact constants available are system dependent.

Attribute	Meaning	curses.	A_ALTCHARSET	Alternate character set mode	curses.	A_BLINK	Blink mode	curses.	A_BOLD	Bold mode	curses.	A_DIM	Dim mode	curses.	A_INVIS	Invisible or blank mode	curses.	A_ITALIC	Italic mode	curses.	A_NORMAL	Normal attribute	curses.	A_PROTECT	Protected mode	curses.	A_REVERSE	Reverse background and foreground colors	curses.	A_STANDOUT	Standout mode	curses.	A_UNDERLINE	Underline mode	curses.	A_HORIZONTAL	Horizontal highlight	curses.	A_LEFT	Left highlight	curses.	A_LOW	Low highlight	curses.	A_RIGHT	Right highlight	curses.	A_TOP	Top highlight	curses.	A_VERTICAL	Vertical highlight
-----------	---------	---------	--------------	------------------------------	---------	---------	------------	---------	--------	-----------	---------	-------	----------	---------	---------	-------------------------	---------	----------	-------------	---------	----------	------------------	---------	-----------	----------------	---------	-----------	--	---------	------------	---------------	---------	-------------	----------------	---------	--------------	----------------------	---------	--------	----------------	---------	-------	---------------	---------	---------	-----------------	---------	-------	---------------	---------	------------	--------------------

Added in version 3.7:`A_ITALIC` was added.

Several constants are available to extract corresponding attributes returned by some methods.

Bit-mask	Meaning
<code>curses.A_ATTRIBUTES</code>	Bit-mask to extract attributes
<code>curses.A_CHARTEXT</code>	Bit-mask to extract a character
<code>curses.A_COLOR</code>	Bit-mask to extract color-pair field information

Keys are referred to by integer constants with names starting with `KEY_`. The exact keycaps available are system dependent.

Key constant	Key
<code>curses.KEY_MIN</code>	Minimum key value
<code>curses.KEY_BREAK</code>	Break key (unreliable)
<code>curses.KEY_DOWN</code>	Down-arrow
<code>curses.KEY_UP</code>	Up-arrow
<code>curses.KEY_LEFT</code>	Left-arrow
<code>curses.KEY_RIGHT</code>	Right-arrow
<code>curses.KEY_HOME</code>	Home key (upward+left arrow)
<code>curses.KEY_BACKSPACE</code>	Backspace (unreliable)
<code>curses.KEY_F0</code>	Function keys. Up to 64 function keys are supported.
<code>curses.KEY_Fn</code>	Value of function key <code>n</code>
<code>curses.KEY_DL</code>	Delete line
<code>curses.KEY_IL</code>	Insert line
<code>curses.KEY_DC</code>	Delete character
<code>curses.KEY_IC</code>	Insert char or enter insert mode
<code>curses.KEY_EIC</code>	Exit insert char mode
<code>curses.KEY_CLEAR</code>	Clear screen
<code>curses.KEY_EOS</code>	Clear to end of screen
<code>curses.KEY_EOL</code>	Clear to end of line

On VT100s and their software emulations, such as X terminal emulators, there are normally at least four function keys (`KEY_F1`, `KEY_F2`, `KEY_F3`, `KEY_F4`) available, and the arrow keys mapped to `KEY_UP`, `KEY_DOWN`, `KEY_LEFT` and `KEY_RIGHT` in the obvious way. If your machine has a PC keyboard, it is safe to expect arrow keys and twelve function keys (older PC keyboards may have only ten function keys); also, the following keypad mappings are standard:

Keycap	Constant
Insert	<code>KEY_IC</code>
Delete	<code>KEY_DC</code>
Home	<code>KEY_HOME</code>
End	<code>KEY_END</code>
Page Up	<code>KEY_PPAGE</code>
Page Down	<code>KEY_NPAGE</code>

The following table lists characters from the alternate character set. These are inherited from the VT100 terminal, and will generally be available on software emulations such as X terminals. When there is no graphic available, `curses` falls back on a crude printable ASCII approximation.

■ **Nota:** Note These are available only after `initscr()` has been called.

ACS code	Meaning
<code>curses.ACS_BBSS</code>	alternate name for upper right corner
<code>curses.ACS_BLOCK</code>	solid square block
<code>curses.ACS_BOARD</code>	board of squares
<code>curses.ACS_BSBS</code>	alternate name for horizontal line
<code>curses.ACS_BSSB</code>	alternate name for upper left corner
<code>curses.ACS_BSSS</code>	alternate name for top tee
<code>curses.ACS_BTEE</code>	bottom tee
<code>curses.ACS_BULLET</code>	bullet
<code>curses.ACS_CKBOARD</code>	checker board (stipple)
<code>curses.ACS_DARROW</code>	arrow pointing down
<code>curses.ACS_DEGREE</code>	degree symbol
<code>curses.ACS_DIAMOND</code>	diamond
<code>curses.ACS_GEQUAL</code>	greater-than-or-equal-to
<code>curses.ACS_HLINE</code>	horizontal line
<code>curses.ACS_LANTERN</code>	lantern symbol
<code>curses.ACS_LARROW</code>	left arrow
<code>curses.ACS_LEQUAL</code>	less-than-or-equal-to
<code>curses.ACS_LLCO</code>	

The following table lists mouse button constants used by `getmouse()`:

Mouse button constant	Meaning
<code>curses.BUTTONn_PRESSED</code>	Mouse button <code>n</code> pressed
<code>curses.BUTTONn_RELEASED</code>	Mouse button <code>n</code> released
<code>curses.BUTTONn_CLICKED</code>	Mouse button <code>n</code> clicked
<code>curses.BUTTONn_DOUBLE_CLICKED</code>	Mouse button <code>n</code> double clicked
<code>curses.BUTTONn_TRIPLE_CLICKED</code>	Mouse button <code>n</code> triple clicked
<code>curses.BUTTON_SHIFT</code>	Shift was down during button state change
<code>curses.BUTTON_CTRL</code>	Control was down during button state change
<code>curses.BUTTON_ALT</code>	Control was down during button state change

Changed in version 3.10: The `BUTTON5_*` constants are now exposed if they are provided by the underlying `curses` library.

The following table lists the predefined colors:

Constant	Color
<code>curses.COLOR_BLACK</code>	Black
<code>curses.COLOR_BLUE</code>	Blue
<code>curses.COLOR_CYAN</code>	Cyan (light greenish blue)
<code>curses.COLOR_GREEN</code>	Green
<code>curses.COLOR_MAGENTA</code>	Magenta (purplish red)
<code>curses.COLOR_RED</code>	Red
<code>curses.COLOR_WHITE</code>	White
<code>curses.COLOR_YELLOW</code>	Yellow

curses.textpad— Text input widget for curses programs

The `curses.textpad` module provides a `Textbox` class that handles elementary text editing in a curses window, supporting a set of keybindings resembling those of Emacs (thus, also of Netscape Navigator, BBedit 6.x, FrameMaker, and many other programs). The module also provides a rectangle-drawing function useful for framing text boxes or for other purposes.

The module `curses.textpad` defines the following function:

`curses.textpad.rectangle(win,uly,ulx,lry,lrx)`

Draw a rectangle. The first argument must be a window object; the remaining arguments are coordinates relative to that window. The second and third arguments are the y and x coordinates of the upper left hand corner of the rectangle to be drawn; the fourth and fifth arguments are the y and x coordinates of the lower right hand corner. The rectangle will be drawn using VT100/IBM PC forms characters on terminals that make this possible (including xterm and most other software terminal emulators)

Textbox objects

You can instantiate a `Textbox` object as follows:

`class curses.textpad.Textbox(win)`

`edit([validator])`

`do_command(ch)`

`gather()`

`stripspaces`

Return a textbox widget object. The `win` argument should be a `curses.window` object in which the textbox is to be contained. The edit cursor of the textbox is initially located at the upper left hand corner of the containing window, with coordinates (0,0). The instance's `stripspaces` flag is initially on. `Textbox` objects have the following methods: `edit([validator])` This is the entry point you will normally use. It accepts editing keystrokes until one of the termination keystrokes is entered. If `validator` is

This is the entry point you will normally use. It accepts editing keystrokes until one of the termination keystrokes is entered. If `validator` is supplied, it must be a function. It will be called for each keystroke entered with the keystroke as a parameter; command dispatch is done on the result. This method returns the window contents as a string; whether blanks in the window are included is affected by the `stripspaces` attribute.

Process a single command keystroke. Here are the supported special keystrokes: `KeystrokeActionControl-AG` to left edge of window. `Control-B` Cursor left, wrapping to previous line if appropriate. `Control-D` Delete character under cursor. `Control-E` Go to right edge (`stripspaces` off) or end of line (`stripspaces` on). `Control-F` Cursor right, wrapping to next line when appropriate. `Control-G` Terminate, returning the window contents. `Control-H` Delete character backward. `Control-J` Terminate if the window is 1 line, or

Return the window contents as a string; whether blanks in the window are included is affected by the `stripspaces` member.

This attribute is a flag which controls the interpretation of blanks in the window. When it is on, trailing blanks on each line are ignored; any cursor motion that would land the cursor on a trailing blank goes to the end of that line instead, and trailing blanks are stripped when the window contents are gathered.

curses.ascii— Utilities for ASCII characters

Source code: `Lib/curses/ascii.py`

The `curses.ascii` module supplies name constants for ASCII characters and functions to test membership in various ASCII character classes. The constants supplied are names for control characters as follows:

```
Name | Meaning | curses.ascii. | NUL | curses.ascii. | SOH | Start of heading, console interrupt
| curses.ascii. | STX | Start of text | curses.ascii. | ETX | End of text | curses.ascii. | EOT
| End of transmission | curses.ascii. | ENQ | Enquiry, goes with | ACK | flow control |
curses.ascii. | ACK | Acknowledgement | curses.ascii. | BEL | Bell | curses.ascii. | BS |
Backspace | curses.ascii. | TAB | Tab | curses.ascii. | HT | Alias for | TAB | : "Horizontal
tab" | curses.ascii. | LF | Line feed | curses.ascii. | NL | Alias for | LF | : "New line" |
curses.ascii. | VT | Vertical tab | curses.ascii. | FF | Form feed | curses.ascii. | CR |
Carriage return | curses.ascii. | SO | Shift-out, begin alternate character set | curses.ascii.
| SI | Shift-in, resume default character set | curses.as
```

Note that many of these have little practical significance in modern usage. The mnemonics derive from teleprinter conventions that predate digital computers.

The module supplies the following functions, patterned on those in the standard C library:

`curses.ascii.isalnum(c)`

Checks for an ASCII alphanumeric character; it is equivalent to `isalpha(c)` or `isdigit(c)`.

`curses.ascii.isalpha(c)`

Checks for an ASCII alphabetic character; it is equivalent to `isupper(c)` or `islower(c)`.

`curses.ascii.isascii(c)`

Checks for a character value that fits in the 7-bit ASCII set.

`curses.ascii.isblank(c)`

Checks for an ASCII whitespace character; space or horizontal tab.

`curses.ascii.iscntrl(c)`

Checks for an ASCII control character (in the range 0x00 to 0x1f or 0x7f).

`curses.ascii.isdigit(c)`

Checks for an ASCII decimal digit, '0' through '9'. This is equivalent to `cinstring.digits`.

`curses.ascii.isgraph(c)`

Checks for ASCII any printable character except space.

`curses.ascii.islower(c)`

Checks for an ASCII lower-case character.

`curses.ascii.isprint(c)`

Checks for any ASCII printable character including space.

`curses.ascii.ispunct(c)`

Checks for any printable ASCII character which is not a space or an alphanumeric character.

`curses.ascii.isspace(c)`

Checks for ASCII white-space characters; space, line feed, carriage return, form feed, horizontal tab, vertical tab.

curses.ascii.isupper(c)

Checks for an ASCII uppercase letter.

curses.ascii.isxdigit(c)

Checks for an ASCII hexadecimal digit. This is equivalent to `cinstring.hexdigits`.

curses.ascii.isctrl(c)

Checks for an ASCII control character (ordinal values 0 to 31).

curses.ascii.ismeta(c)

Checks for a non-ASCII character (ordinal values 0x80 and above).

These functions accept either integers or single-character strings; when the argument is a string, it is first converted using the built-in function `ord()`.

Note that all these functions check ordinal bit values derived from the character of the string you pass in; they do not actually know anything about the host machine's character encoding.

The following two functions take either a single-character string or integer byte value; they return a value of the same type.

curses.ascii.ascii(c)

Return the ASCII value corresponding to the low 7 bits of `c`.

curses.ascii.ctrl(c)

Return the control character corresponding to the given character (the character bit value is bitwise-anded with 0x1f).

curses.ascii.alt(c)

Return the 8-bit character corresponding to the given ASCII character (the character bit value is bitwise-ored with 0x80).

The following function takes either a single-character string or integer value; it returns a string.

curses.ascii.unctrl(c)

Return a string representation of the ASCII character `c`. If `c` is printable, this string is the character itself. If the character is a control character (0x00–0x1f) the string consists of a caret (^) followed by the corresponding uppercase letter. If the character is an ASCII delete (0x7f) the string is '^?'. If the character has its meta bit (0x80) set, the meta bit is stripped, the preceding rules applied, and '!' prepended to the result.

curses.ascii.controlnames

A 33-element string array that contains the ASCII mnemonics for the thirty-two ASCII control characters from 0 (NUL) to 0x1f (US), in order, plus the mnemonic `SP` for the space character.

curses.panel— A panel stack extension for curses

Panels are windows with the added feature of depth, so they can be stacked on top of each other, and only the visible portions of each window will be displayed. Panels can be added, moved up or down in the stack, and removed.

Functions

The module `curses.panel` defines the following functions:

`curses.panel.bottom_panel()`

Returns the bottom panel in the panel stack.

`curses.panel.new_panel(win)`

Returns a panel object, associating it with the given window `win`. Be aware that you need to keep the returned panel object referenced explicitly. If you don't, the panel object is garbage collected and removed from the panel stack.

`curses.panel.top_panel()`

Returns the top panel in the panel stack.

`curses.panel.update_panels()`

Updates the virtual screen after changes in the panel stack. This does not call `curses.doupdate()`, so you'll have to do this yourself.

Panel Objects

Panel objects, as returned by `new_panel()` above, are windows with a stacking order. There's always a window associated with a panel which determines the content, while the panel methods are responsible for the window's depth in the panel stack.

Panel objects have the following methods:

`Panel.above()`

Returns the panel above the current panel.

`Panel.below()`

Returns the panel below the current panel.

`Panel.bottom()`

Push the panel to the bottom of the stack.

`Panel.hidden()`

Returns `True` if the panel is hidden (not visible), `False` otherwise.

`Panel.hide()`

Hide the panel. This does not delete the object, it just makes the window on screen invisible.

`Panel.move(y,x)`

Move the panel to the screen coordinates `(y,x)`.

Panel.replace(win)

Change the window associated with the panel to the windowwin.

Panel.set_userptr(obj)

Set the panel's user pointer toobj. This is used to associate an arbitrary piece of data with the panel, and can be any Python object.

Panel.show()

Display the panel (which might have been hidden).

Panel.top()

Push panel to the top of the stack.

Panel.userptr()

Returns the user pointer for the panel. This might be any Python object.

Panel.window()

Returns the window object associated with the panel.

cmd— Support for line-oriented command interpreters

Source code: `Lib/cmd.py`

The `Cmd` class provides a simple framework for writing line-oriented command interpreters. These are often useful for test harnesses, administrative tools, and prototypes that will later be wrapped in a more sophisticated interface.

`class cmd.Cmd(completekey='tab', stdin=None, stdout=None)`

A `Cmd` instance or subclass instance is a line-oriented interpreter framework. There is no good reason to instantiate `Cmd` itself; rather, it's useful as a superclass of an interpreter class you define yourself in order to inherit `Cmd`'s methods and encapsulate action methods. The optional argument `completekey` is the readline name of a completion key; it defaults to `Tab`. If `completekey` is not `None` and `readline` is available, command completion is done automatically. The default, `'tab'`, is treated specially, so that i

Cmd Objects

A `Cmd` instance has the following methods:

`Cmd.cmdloop(intro=None)`

Repeatedly issue a prompt, accept input, parse an initial prefix off the received input, and dispatch to action methods, passing them the remainder of the line as argument. The optional argument is a banner or intro string to be issued before the first prompt (this overrides the `intro` class attribute). If the `readline` module is loaded, input will automatically inherit `bash`-like history-list editing (e.g. `Control-P` scrolls back to the last command, `Control-N` forward to the next one, `Control-F` moves the cursor

`Cmd.do_help(arg)`

All subclasses of `Cmd` inherit a predefined `do_help()`. This method, called with an argument `'bar'`, invokes the corresponding method `help_bar()`, and if that is not present, prints the docstring of `do_bar()`, if available. With no argument, `do_help()` lists all available help topics (that is, all commands with corresponding `help_*` methods or commands that have docstrings), and also lists any undocumented commands.

`Cmd.onecmd(str)`

Interpret the argument as though it had been typed in response to the prompt. This may be overridden, but should not normally need to be; see the `precmd()` and `postcmd()` methods for useful execution hooks. The return value is a flag indicating whether interpretation of commands by the interpreter should stop. If there is a `do_*` method for the command `str`, the return value of that method is returned, otherwise the return value from the `default()` method is returned.

`Cmd.emptyline()`

Method called when an empty line is entered in response to the prompt. If this method is not overridden, it repeats the last nonempty command entered.

`Cmd.default(line)`

Method called on an input line when the command prefix is not recognized. If this method is not overridden, it prints an error message and returns.

`Cmd.completedefault(text, line, begidx, endidx)`

Method called to complete an input line when no command-specific `complete_*` method is available. By default, it returns an empty list.

Cmd.columnize(list,displaywidth=80)

Method called to display a list of strings as a compact set of columns. Each column is only as wide as necessary. Columns are separated by two spaces for readability.

Cmd.precmd(line)

Hook method executed just before the command `line` is interpreted, but after the input prompt is generated and issued. This method is a stub in `Cmd`; it exists to be overridden by subclasses. The return value is used as the command which will be executed by the `onecmd()` method; the `precmd()` implementation may re-write the command or simply return `line` unchanged.

Cmd.postcmd(stop,line)

Hook method executed just after a command dispatch is finished. This method is a stub in `Cmd`; it exists to be overridden by subclasses. `line` is the command line which was executed, and `stop` is a flag which indicates whether execution will be terminated after the call to `postcmd()`; this will be the return value of the `onecmd()` method. The return value of this method will be used as the new value for the internal flag which corresponds to `stop`; returning false will cause interpretation to continue.

Cmd.preloop()

Hook method executed once when `cmdloop()` is called. This method is a stub in `Cmd`; it exists to be overridden by subclasses.

Cmd.postloop()

Hook method executed once when `cmdloop()` is about to return. This method is a stub in `Cmd`; it exists to be overridden by subclasses.

Instances of `Cmd` subclasses have some public instance variables:

Cmd.prompt

The prompt issued to solicit input.

Cmd.identchars

The string of characters accepted for the command prefix.

Cmd.lastcmd

The last nonempty command prefix seen.

Cmd.cmdqueue

A list of queued input lines. The `cmdqueue` list is checked in `cmdloop()` when new input is needed; if it is nonempty, its elements will be processed in order, as if entered at the prompt.

Cmd.intro

A string to issue as an intro or banner. May be overridden by giving the `cmdloop()` method an argument.

Cmd.doc_header

The header to issue if the help output has a section for documented commands.

Cmd.misc_header

The header to issue if the help output has a section for miscellaneous help topics (that is, there are `help_*` methods without corresponding `do_*` methods).

Cmd.undoc_header

The header to issue if the help output has a section for undocumented commands (that is, there are `do_*` methods without corresponding `help_*` methods).

Cmd.ruler

The character used to draw separator lines under the help-message headers. If empty, no ruler line is drawn. It defaults to '='.

Cmd.use_rawinput

A flag, defaulting to true. If true, `cmdloop()` uses `input()` to display a prompt and read the next command; if false, `sys.stdout.write()` and `sys.stdin.readline()` are used. (This means that by importing `readline`, on systems that support it, the interpreter will automatically support Emacs-like line editing and command-history keystrokes.)

Cmd Example

The `cmd` module is mainly useful for building custom shells that let a user work with a program interactively.

This section presents a simple example of how to build a shell around a few of the commands in the `turtle` module.

Basic turtle commands such as `forward()` are added to a `Cmd` subclass with method named `do_forward()`. The argument is converted to a number and dispatched to the turtle module. The docstring is used in the help utility provided by the shell.

The example also includes a basic record and playback facility implemented with the `precmd()` method which is responsible for converting the input to lowercase and writing the commands to a file. The `do_playback()` method reads the file and adds the recorded commands to the `cmd` queue for immediate playback:

```
import cmd, sys
from turtle import *

class TurtleShell(cmd.Cmd):
    intro = 'Welcome to the turtle shell.  Type help or ? to list commands.\n'
    prompt = '(turtle) '
    file = None

    # ----- basic turtle commands -----
    def do_forward(self, arg):
        'Move the turtle forward by the specified distance:  FORWARD 10'
        forward(*parse(arg))
    def do_right(self, arg):
        'Turn turtle right by given number of degrees:  RIGHT 20'
        right(*parse(arg))
    def do_left(self, arg):
        'Turn turtle left by given number of degrees:  LEFT 90'
        left(*parse(arg))
    def do_goto(self, arg):
        'Move turtle to an absolute position with changing orientation.  GOTO 100 200'
        goto(*parse(arg))
    def do_home(self, arg):
        'Return turtle to the home position:  HOME'
        home()
    def do_circle(self, arg):
        'Draw circle with given radius an options extent and steps:  CIRCLE 50'
        circle(*parse(arg))
    def do_position(self, arg):
        'Print the current turtle position:  POSITION'
        print('Current position is %d %d\n' % position())
    def do_heading(self, arg):
        'Print the current turtle heading in degrees:  HEADING'
        print('Current heading is %d\n' % (heading(),))
    def do_color(self, arg):
        'Set the color:  COLOR BLUE'
        color(arg.lower())
```

```

def do_undo(self, arg):
    'Undo (repeatedly) the last turtle action(s):  UNDO'
def do_reset(self, arg):
    'Clear the screen and return turtle to center:  RESET'
    reset()
def do_bye(self, arg):
    'Stop recording, close the turtle window, and exit:  BYE'
    print('Thank you for using Turtle')
    self.close()
    bye()
    return True

# ----- record and playback -----
def do_record(self, arg):
    'Save future commands to filename:  RECORD rose.cmd'
    self.file = open(arg, 'w')
def do_playback(self, arg):
    'Playback commands from a file:  PLAYBACK rose.cmd'
    self.close()
    with open(arg) as f:
        self.cmdqueue.extend(f.read().splitlines())
def precmd(self, line):
    line = line.lower()
    if self.file and 'playback' not in line:
        print(line, file=self.file)
    return line
def close(self):
    if self.file:
        self.file.close()
        self.file = None

def parse(arg):
    'Convert a series of zero or more numbers to an argument tuple'
    return tuple(map(int, arg.split()))

if __name__ == '__main__':
    TurtleShell().cmdloop()

```

Here is a sample session with the turtle shell showing the help functions, using blank lines to repeat commands, and the simple record and playback facility:

Welcome to the turtle shell. Type help or ? to list commands.

(turtle) ?

Documented commands (type help <topic>):

```

=====
bye      color      goto      home      playback  record  right
circle  forward  heading  left      position  reset   undo

```

(turtle) help forward

Move the turtle forward by the specified distance: FORWARD 10

(turtle) record spiral.cmd

(turtle) position

Current position is 0 0

(turtle) heading

Current heading is 0

(turtle) reset

(turtle) circle 20

(turtle) right 30

(turtle) circle 40

(turtle) right 30

(turtle) circle 60

```
(turtle) right 30
(turtle) circle 80
(turtle) right 30
(turtle) circle 100
(turtle) right 30
(turtle) circle 120
(turtle) right 30
(turtle) circle 120
(turtle) heading
Current heading is 180
```

```
(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 500
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 300
(turtle) playback spiral.cmd
Current position is 0 0
```

Current heading is 0

Current heading is 180

```
(turtle) bye
Thank you for using Turtle
```

Concurrent Execution

The modules described in this chapter provide support for concurrent execution of code. The appropriate choice of tool will depend on the task to be executed (CPU bound vs IO bound) and preferred style of development (event driven cooperative multitasking vs preemptive multitasking). Here's an overview:

- **threading**— Thread-based parallelismIntroductionGIL and performance considerationsReferenceThread-local dataThread objectsLock objectsRLock objectsCondition objectsSemaphore objectsSemaphoreexampleEvent objectsTimer objectsBarrier objectsUsing locks, conditions, and semaphores in thewithstatement
- **multiprocessing**— Process-based parallelismIntroductionTheProcessclassContexts and start methodsExchanging objects between processesSynchronization between processesSharing state between processesUsing a pool of workersReferenceGlobal start methodProcessand exceptionsPipes and QueuesMiscellaneousConnection ObjectsSynchronization primitivesSharedctypesObjectsThemultiprocessing.sharedctypesmoduleMana
- **multiprocessing.shared_memory**— Shared memory for direct access across processes
- **Theconcurrentpackage**
- **concurrent.futures**— Launching parallel tasksExecutor ObjectsThreadPoolExecutorThreadPoolExecutor ExampleInterpreterPoolExecutorProcessPoolExecutorProcessPoolExecutor ExampleFuture ObjectsModule FunctionsException classes
- **concurrent.interpreters**— Multiple interpreters in the same processKey detailsIntroductionMultiple Interpreters and IsolationRunning in an InterpreterConcurrency and ParallelismCommunication Between Interpreters“Sharing” ObjectsReferenceInterpreter objectsExceptionsCommunicating Between InterpretersBasic usage
- **subprocess**— Subprocess managementUsing thesubprocessModuleFrequently Used ArgumentsPopen ConstructorExceptionsSecurity ConsiderationsPopen ObjectsWindows Popen HelpersWindows ConstantsOlder high-level APIReplacing Older Functions with thesubprocessModuleReplacing/bin/shshell command substitutionReplacing shell pipelineReplacingos.system()Replacing theos.spawnfamilyReplacingos.popen()Legacy Shell I
- **sched**— Event schedulerScheduler Objects
- **queue**— A synchronized queue classQueue ObjectsWaiting for task completionTerminating queuesSimpleQueue Objects
- **contextvars**— Context VariablesContext VariablesManual Context Managementasyncio support

The following are support modules for some of the above services:

- **_thread**— Low-level threading API

threading— Thread-based parallelism

Source code: `Lib/threading.py`

This module constructs higher-level threading interfaces on top of the lower level `_thread` module.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Introduction

The `threading` module provides a way to run multiple threads (smaller units of a process) concurrently within a single process. It allows for the creation and management of threads, making it possible to execute tasks in parallel, sharing memory space. Threads are particularly useful when tasks are I/O bound, such as file operations or making network requests, where much of the time is spent waiting for external resources.

A typical use case for `threading` includes managing a pool of worker threads that can process multiple tasks concurrently. Here's a basic example of creating and starting threads using `Thread`:

```
import threading
import time

def crawl(link, delay=3):
    print(f"crawl started for {link}")
    time.sleep(delay) # Blocking I/O (simulating a network request)
    print(f"crawl ended for {link}")

links = [
    "https://python.org",
    "https://docs.python.org",
    "https://peps.python.org",
]

# Start threads for each link
threads = []
for link in links:
    # Using `args` to pass positional arguments and `kwargs` for keyword arguments
    t = threading.Thread(target=crawl, args=(link,), kwargs={"delay": 2})
    threads.append(t)

# Start each thread
for t in threads:
    t.start()

# Wait for all threads to finish
for t in threads:
    t.join()
```

Changed in version 3.7: This module used to be optional, it is now always available.

See also

`concurrent.futures.ThreadPoolExecutor` offers a higher level interface to push tasks to a background thread without blocking execution of the calling thread, while still being able to retrieve their results when needed.

`queue` provides a thread-safe interface for exchanging data between running threads.

`asyncio` offers an alternative approach to achieving task level concurrency without requiring the use of multiple operating system threads.

■ Nota: NoteIn the Python 2.x series, this module containedcamelCasenames for some methods and functions. These are deprecated as of Python 3.10, but they are still supported for compatibility with Python 2.5 and lower.

CPython implementation detail:In CPython, due to theGlobal Interpreter Lock, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multi-core machines, you are advised to usemultiprocessingorconcurrent.futures.ProcessPoolExecutor. However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

GIL and performance considerations

Unlike themultiprocessingmodule, which uses separate processes to bypass theglobal interpreter lock(GIL), the threading module operates within a single process, meaning that all threads share the same memory space. However, the GIL limits the performance gains of threading when it comes to CPU-bound tasks, as only one thread can execute Python bytecode at a time. Despite this, threads remain a useful tool for achieving concurrency in many scenarios.

As of Python 3.13,free-threadedbuilds can disable the GIL, enabling true parallel execution of threads, but this feature is not available by default (seePEP 703).

Reference

This module defines the following functions:

threading.active_count()

Return the number ofThreadobjects currently alive. The returned count is equal to the length of the list returned byenumerate().The functionactiveCountis a deprecated alias for this function.

threading.current_thread()

Return the currentThreadobject, corresponding to the caller's thread of control. If the caller's thread of control was not created through thethreadingmodule, a dummy thread object with limited functionality is returned.The functioncurrentThreadis a deprecated alias for this function.

threading.excepthook(args, /)

Handle uncaught exception raised byThread.run().Theargsargument has the following attributes:exc_type: Exception type.exc_value: Exception value, can beNone.exc_traceback: Exception traceback, can beNone.thread: Thread which raised the exception, can beNone.Ifexc_typeisSystemExit, the exception is silently ignored. Otherwise, the exception is printed out onsys.stderr.If this function raises an exception,sys.excepthook()is called to handle it.threading.excepthook()can be overridden to control ho

threading.__excepthook__

Holds the original value ofthreading.excepthook(). It is saved so that the original value can be restored in case they happen to get replaced with broken or alternative objects.Added in version 3.10.

threading.get_ident()

Return the 'thread identifier' of the current thread. This is a nonzero integer. Its value has no direct meaning; it is intended as a magic cookie to be used e.g. to index a dictionary of thread-specific data. Thread identifiers may be recycled when a thread exits and another thread is created.Added in version 3.3.

threading.get_native_id()

Return the native integral Thread ID of the current thread assigned by the kernel. This is a non-negative integer. Its value may be used to uniquely identify this particular thread system-wide (until the thread terminates,

after which the value may be recycled by the OS).Availability: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD.Added in version 3.8.Changed in version 3.13:Added support for GNU/kFreeBSD.

threading.enumerate()

Return a list of allThreadobjects currently active. The list includes daemonic threads and dummy thread objects created bycurrent_thread(). It excludes terminated threads and threads that have not yet been started. However, the main thread is always part of the result, even when terminated.

threading.main_thread()

Return the mainThreadobject. In normal conditions, the main thread is the thread from which the Python interpreter was started.Added in version 3.4.

threading.settrace(func)

Set a trace function for all threads started from thethreadingmodule. Thefuncwill be passed tosys.settrace()for each thread, before itsrun()method is called.

threading.settrace_all_threads(func)

Set a trace function for all threads started from thethreadingmodule and all Python threads that are currently executing.Thefuncwill be passed tosys.settrace()for each thread, before itsrun()method is called.Added in version 3.12.

threading.gettrace()

Get the trace function as set bysettrace().Added in version 3.10.

threading.setprofile(func)

Set a profile function for all threads started from thethreadingmodule. Thefuncwill be passed tosys.setprofile()for each thread, before itsrun()method is called.

threading.setprofile_all_threads(func)

Set a profile function for all threads started from thethreadingmodule and all Python threads that are currently executing.Thefuncwill be passed tosys.setprofile()for each thread, before itsrun()method is called.Added in version 3.12.

threading.getprofile()

Get the profiler function as set bysetprofile().Added in version 3.10.

threading.stack_size([size])

Return the thread stack size used when creating new threads. The optionalsizeargument specifies the stack size to be used for subsequently created threads, and must be 0 (use platform or configured default) or a positive integer value of at least 32,768 (32 KiB). Ifsizeis not specified, 0 is used. If changing the thread stack size is unsupported, aRuntimeErroris raised. If the specified stack size is invalid, aValueErroris raised and the stack size is unmodified. 32 KiB is currently the mini

This module also defines the following constant:

threading.TIMEOUT_MAX

The maximum value allowed for thetimeoutparameter of blocking functions (Lock.acquire(),RLock.acquire(),Condition.wait(), etc.). Specifying a timeout greater than this value will raise anOverflowError.Added in version 3.2.

This module defines a number of classes, which are detailed in the sections below.

The design of this module is loosely based on Java's threading model. However, where Java makes locks and condition variables basic behavior of every object, they are separate objects in Python. Python's `Thread` class supports a subset of the behavior of Java's `Thread` class; currently, there are no priorities, no thread groups, and threads cannot be destroyed, stopped, suspended, resumed, or interrupted. The static methods of Java's `Thread` class, when implemented, are mapped to module-level functions.

All of the methods described below are executed atomically.

Thread-local data

Thread-local data is data whose values are thread specific. If you have data that you want to be local to a thread, create `localobject` and use its attributes:

```
>>> mydata = local()
>>> mydata.number = 42
>>> mydata.number
42
```

You can also access the `local`-object's dictionary:

```
>>> mydata.__dict__
{'number': 42}
>>> mydata.__dict__.setdefault('widgets', [])
[]
>>> mydata.widgets
[]
```

If we access the data in a different thread:

```
>>> log = []
>>> def f():
...     items = sorted(mydata.__dict__.items())
...     log.append(items)
...     mydata.number = 11
...     log.append(mydata.number)

>>> import threading
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[[], 11]
```

we get different data. Furthermore, changes made in the other thread don't affect data seen in this thread:

```
>>> mydata.number
42
```

Of course, values you get from `localobject`, including their `__dict__` attribute, are for whatever thread was current at the time the attribute was read. For that reason, you generally don't want to save these values across threads, as they apply only to the thread they came from.

You can create custom `local` objects by subclassing `localclass`:

```
>>> class MyLocal(local):
...     number = 2
...     def __init__(self, /, **kw):
...         self.__dict__.update(kw)
...     def squared(self):
...         return self.number ** 2
```

This can be useful to support default values, methods and initialization. Note that if you define an `__init__()` method, it will be called each time the `localobject` is used in a separate thread. This is necessary to initialize each thread's dictionary.

Now if we create a local object:

```
>>> mydata = MyLocal(color='red')
```

we have a default number:

```
>>> mydata.number
2
```

an initial color:

```
>>> mydata.color
'red'
>>> del mydata.color
```

And a method that operates on the data:

```
>>> mydata.squared()
4
```

As before, we can access the data in a separate thread:

```
>>> log = []
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[['color', 'red']], 11]
```

without affecting this thread's data:

```
>>> mydata.number
2
>>> mydata.color
Traceback (most recent call last):
...
AttributeError: 'MyLocal' object has no attribute 'color'
```

Note that subclasses can define `__slots__`, but they are not thread local. They are shared across threads:

```
>>> class MyLocal(local):
...     __slots__ = 'number'

>>> mydata = MyLocal()
>>> mydata.number = 42
>>> mydata.color = 'red'
```

So, the separate thread:

```
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
```

affects what we see:

```
>>> mydata.number
11
```

classthreading.local

A class that represents thread-local data.

Thread objects

The `Thread` class represents an activity that is run in a separate thread of control. There are two ways to specify the activity: by passing a callable object to the constructor, or by overriding the `run()` method in a subclass. No other methods (except for the constructor) should be overridden in a subclass. In other words, only override the `__init__()` and `run()` methods of this class.

Once a thread object is created, its activity must be started by calling the thread's `start()` method. This invokes the `run()` method in a separate thread of control.

Once the thread's activity is started, the thread is considered 'alive'. It stops being alive when its `run()` method terminates – either normally, or by raising an unhandled exception. The `is_alive()` method tests whether the thread is alive.

Other threads can call a thread's `join()` method. This blocks the calling thread until the thread whose `join()` method is called is terminated.

A thread has a name. The name can be passed to the constructor, and read or changed through the `name` attribute.

If the `run()` method raises an exception, `threading.excepthook()` is called to handle it. By default, `threading.excepthook()` ignores silently `SystemExit`.

A thread can be flagged as a “daemon thread”. The significance of this flag is that the entire Python program exits when only daemon threads are left. The initial value is inherited from the creating thread. The flag can be set through the `daemon` property or the `daemon` constructor argument.

■ **Nota:** Daemon threads are abruptly stopped at shutdown. Their resources (such as open files, database transactions, etc.) may not be released properly. If you want your threads to stop gracefully, make them non-daemonic and use a suitable signalling mechanism such as an `Event`.

There is a “main thread” object; this corresponds to the initial thread of control in the Python program. It is not a daemon thread.

There is the possibility that “dummy thread objects” are created. These are thread objects corresponding to “alien threads”, which are threads of control started outside the threading module, such as directly from C code. Dummy thread objects have limited functionality; they are always considered alive and daemonic, and cannot be joined. They are never deleted, since it is impossible to detect the termination of alien threads.

`threading.Thread(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None, context=None)`

`start()`

`run()`

`join(timeout=None)`

`name`

`getName()`

`setName()`

`ident`

`native_id`

`is_alive()`

`daemon`

`isDaemon()`

`setDaemon()`

This constructor should always be called with keyword arguments. Arguments are: `group` should be `None`; reserved for future extension when a `ThreadGroup` class is implemented. `target` is the callable object to be invoked by the `run()` method. Defaults to `None`, meaning nothing is called. `name` is the thread name. By default, a unique name is constructed of the form “Thread-N” where `N` is a small decimal number, or “Thread-N(`target`)” where “`target`” is `target.__name__` if the `target` argument is specified. `args` is a list or tuple

Start the thread's activity. It must be called at most once per thread object. It arranges for the object's `run()` method to be invoked in a separate thread of control. This method will raise a `RuntimeError` if called more than once on the same thread object. If supported, set the operating system thread name to `threading.Thread.name`. The name can be truncated depending on the operating system thread name limits. Changed in version 3.14: Set the operating system thread name.

Method representing the thread's activity. You may override this method in a subclass. The standard `run()` method invokes the callable object passed to the object's constructor as the `target` argument, if any, with positional and keyword arguments taken from the `args` and `kwargs` arguments, respectively. Using list or tuple as the `args` argument which passed to the `Thread` could achieve the same effect. Example:

```
>>>from threading import Thread>>>t=Thread(target=print,args=[1])>>>t.run()1>>>t=Thread(target=print,args=(1
```

Wait until the thread terminates. This blocks the calling thread until the thread whose `join()` method is called terminates – either normally or through an unhandled exception – or until the optional timeout occurs. When the `timeout` argument is present and not `None`, it should be a floating-point number specifying a timeout for the operation in seconds (or fractions thereof). As `join()` always returns `None`, you must call `is_alive()` after `join()` to decide whether a timeout happened – if the thread is still alive

A string used for identification purposes only. It has no semantics. Multiple threads may be given the same name. The initial name is set by the constructor. On some platforms, the thread name is set at the operating system level when the thread starts, so that it is visible in task managers. This name may be truncated to fit in a system-specific limit (for example, 15 bytes on Linux or 63 bytes on macOS). Changes to name are only reflected at the OS level when the currently running thread is renamed

Deprecated getter/setter API for `name`; use it directly as a property instead. Deprecated since version 3.10.

The 'thread identifier' of this thread or `None` if the thread has not been started. This is a nonzero integer. See the `get_ident()` function. Thread identifiers may be recycled when a thread exits and another thread is created. The identifier is available even after the thread has exited.

The Thread ID (TID) of this thread, as assigned by the OS (kernel). This is a non-negative integer, or `None` if the thread has not been started. See the `get_native_id()` function. This value may be used to uniquely identify this particular thread system-wide (until the thread terminates, after which the value may be recycled by the OS). Note: Similar to Process IDs, Thread IDs are only valid (guaranteed unique system-wide) from the time the thread is created until the thread has been terminated. Available

Return whether the thread is alive. This method returns `True` just before the `run()` method starts until just after the `run()` method terminates. The module function `enumerate()` returns a list of all alive threads.

A boolean value indicating whether this thread is a daemon thread (`True`) or not (`False`). This must be set before `start()` is called, otherwise `RuntimeError` is raised. Its initial value is inherited from the creating thread; the main thread is not a daemon thread and therefore all threads created in the main thread default to `daemon=False`. The entire Python program exits when no alive non-daemon threads are left.

Deprecated getter/setter API for `daemon`; use it directly as a property instead. Deprecated since version 3.10.

Lock objects

A primitive lock is a synchronization primitive that is not owned by a particular thread when locked. In Python, it is currently the lowest level synchronization primitive available, implemented directly by the `_thread` extension module.

A primitive lock is in one of two states, "locked" or "unlocked". It is created in the unlocked state. It has two basic methods, `acquire()` and `release()`. When the state is unlocked, `acquire()` changes the state to locked and returns immediately. When the state is locked, `acquire()` blocks until a call to `release()` in another thread changes it to unlocked, then the `acquire()` call resets it to locked and returns. The `release()` method should only be called in the locked state; it changes the state to unlocked and returns immediately. If an attempt is made to release an

unlocked lock, a `RuntimeError` will be raised.

Locks also support the context management protocol.

When more than one thread is blocked in `acquire()` waiting for the state to turn to unlocked, only one thread proceeds when a `release()` call resets the state to unlocked; which one of the waiting threads proceeds is not defined, and may vary across implementations.

All methods are executed atomically.

`threading.Lock`

`acquire(blocking=True, timeout=-1)`

`release()`

`locked()`

The class implementing primitive lock objects. Once a thread has acquired a lock, subsequent attempts to acquire it block, until it is released; any thread may release it. Changed in version 3.13: `Lock` is now a class. In earlier Pythons, `Lock` was a factory function which returned an instance of the underlying private lock type. `acquire(blocking=True, timeout=-1)` Acquire a lock, blocking or non-blocking. When invoked with the `blocking` argument set to `True` (the default), block until the lock is unlocked, then

Acquire a lock, blocking or non-blocking. When invoked with the `blocking` argument set to `True` (the default), block until the lock is unlocked, then set it to locked and return `True`. When invoked with the `blocking` argument set to `False`, do not block. If a call with `blocking` set to `True` would block, return `False` immediately; otherwise, set the lock to locked and return `True`. When invoked with the floating-point `timeout` argument set to a positive value, block for at most the number of seconds specified by `timeout` and as

Release a lock. This can be called from any thread, not only the thread which has acquired the lock. When the lock is locked, reset it to unlocked, and return. If any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed. When invoked on an unlocked lock, a `RuntimeError` is raised. There is no return value.

Return `True` if the lock is acquired.

`RLock` objects

A reentrant lock is a synchronization primitive that may be acquired multiple times by the same thread. Internally, it uses the concepts of “owning thread” and “recursion level” in addition to the locked/unlocked state used by primitive locks. In the locked state, some thread owns the lock; in the unlocked state, no thread owns it. Threads call a lock’s `acquire()` method to lock it, and its `release()` method to unlock it.

■ **Nota:** Note that reentrant locks support the context management protocol, so it is recommended to use `with` instead of manually calling `acquire()` and `release()` to handle acquiring and releasing the lock for a block of code.

`RLock`’s `acquire()/release()` call pairs may be nested, unlike `Lock`’s `acquire()/release()`. Only the final `release()` (the `release()` of the outermost pair) resets the lock to an unlocked state and allows another thread blocked in `acquire()` to proceed.

`acquire()/release()` must be used in pairs: each `acquire` must have a `release` in the thread that has acquired the lock. Failing to call `release` as many times the lock has been acquired can lead to deadlock.

`threading.RLock`

`acquire(blocking=True, timeout=-1)`

Using `RLock` as a context manager

`release()`

locked()

This class implements reentrant lock objects. A reentrant lock must be released by the thread that acquired it. Once a thread has acquired a reentrant lock, the same thread may acquire it again without blocking; the thread must release it once for each time it has acquired it. Note that `RLock` is actually a factory function which returns an instance of the most efficient version of the concrete `RLock` class that is supported by the platform. `acquire(blocking=True, timeout=-1)` Acquire a lock, blocking

Acquire a lock, blocking or non-blocking. See also `Using RLock as a context manager`. Recommended over `manualacquire()` and `release()` calls whenever practical. When invoked with the `blocking` argument set to `True` (the default): If no thread owns the lock, acquire the lock and return immediately. If another thread owns the lock, block until we are able to acquire lock, or `timeout`, if set to a positive float value. If the same thread owns the lock, acquire the lock again, and return immediately. This is the differen

Recommended over `manualacquire()` and `release()` calls whenever practical.

Release a lock, decrementing the recursion level. If after the decrement it is zero, reset the lock to unlocked (not owned by any thread), and if any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed. If after the decrement the recursion level is still nonzero, the lock remains locked and owned by the calling thread. Only call this method when the calling thread owns the lock. A `RuntimeError` is raised if this method is called when the lock is

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

Condition objects

A condition variable is always associated with some kind of lock; this can be passed in or one will be created by default. Passing one in is useful when several condition variables must share the same lock. The lock is part of the condition object: you don't have to track it separately.

A condition variable obeys the context management protocol: using the `with` statement acquires the associated lock for the duration of the enclosed block. The `acquire()` and `release()` methods also call the corresponding methods of the associated lock.

Other methods must be called with the associated lock held. The `wait()` method releases the lock, and then blocks until another thread awakens it by calling `notify()` or `notify_all()`. Once awakened, `wait()` re-acquires the lock and returns. It is also possible to specify a timeout.

The `notify()` method wakes up one of the threads waiting for the condition variable, if any are waiting. The `notify_all()` method wakes up all threads waiting for the condition variable.

Note: the `notify()` and `notify_all()` methods don't release the lock; this means that the thread or threads awakened will not return from their `wait()` call immediately, but only when the thread that called `notify()` or `notify_all()` finally relinquishes ownership of the lock.

The typical programming style using condition variables uses the lock to synchronize access to some shared state; threads that are interested in a particular change of state call `wait()` repeatedly until they see the desired state, while threads that modify the state call `notify()` or `notify_all()` when they change the state in such a way that it could possibly be a desired state for one of the waiters. For example, the following code is a generic producer-consumer situation with unlimited buffer capacity:

```
# Consume one item
with cv:
    while not an_item_is_available():
        cv.wait()
    get_an_available_item()

# Produce one item
with cv:
    make_an_item_available()
```



```
cv.notify()
```

The while loop checking for the application's condition is necessary because `wait()` can return after an arbitrary long time, and the condition which prompted the `notify()` call may no longer hold true. This is inherent to multi-threaded programming. The `wait_for()` method can be used to automate the condition checking, and eases the computation of timeouts:

```
# Consume an item
with cv:
    cv.wait_for(an_item_is_available)
    get_an_available_item()
```

To choose between `notify()` and `notify_all()`, consider whether one state change can be interesting for only one or several waiting threads. E.g. in a typical producer-consumer situation, adding one item to the buffer only needs to wake up one consumer thread.

`class threading.Condition(lock=None)`

`acquire(*args)`

`release()`

`locked()`

`wait(timeout=None)`

`wait_for(predicate, timeout=None)`

`notify(n=1)`

`notify_all()`

This class implements condition variable objects. A condition variable allows one or more threads to wait until they are notified by another thread. If the `lock` argument is given and not `None`, it must be a `Lock` or `RLock` object, and it is used as the underlying lock. Otherwise, a new `RLock` object is created and used as the underlying lock. Changed in version 3.3: changed from a factory function to a class. `acquire(*args)` Acquire the underlying lock. This method calls the corresponding method on the underlying

lock. Acquire the underlying lock. This method calls the corresponding method on the underlying lock; the return value is whatever that method returns.

Release the underlying lock. This method calls the corresponding method on the underlying lock; there is no return value.

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

Wait until notified or until a timeout occurs. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised. This method releases the underlying lock, and then blocks until it is awakened by a `notify()` or `notify_all()` call for the same condition variable in another thread, or until the optional timeout occurs. Once awakened or timed out, it re-acquires the lock and returns. When the `timeout` argument is present and not `None`, it should be a floating-point number speci

Wait until a condition evaluates to true. `predicate` should be a callable which result will be interpreted as a boolean value. A `timeout` may be provided giving the maximum time to wait. This utility method may call `wait()` repeatedly until the predicate is satisfied, or until a timeout occurs. The return value is the last return value of the predicate and will evaluate to `False` if the method timed out. Ignoring the timeout feature, calling this method is roughly equivalent to writing: `while not predicate(): cv.`

By default, wake up one thread waiting on this condition, if any. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised. This method wakes up at most `n` of the threads waiting for the condition variable; it is a no-op if no threads are waiting. The current implementation wakes up exactly `n` threads, if at least `n` threads are waiting. However, it's not safe to rely on this behavior. A future, optimized implementation

may occasionally wake up more than `n` threads.

Wake up all threads waiting on this condition. This method acts like `notify()`, but wakes up all waiting threads instead of one. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised. The method `notifyAll` is a deprecated alias for this method.

Semaphore objects

This is one of the oldest synchronization primitives in the history of computer science, invented by the early Dutch computer scientist Edsger W. Dijkstra (he used the names `P()` and `V()` instead of `acquire()` and `release()`).

A semaphore manages an internal counter which is decremented by each `acquire()` call and incremented by each `release()` call. The counter can never go below zero; when `acquire()` finds that it is zero, it blocks, waiting until some other thread calls `release()`.

Semaphores also support the context management protocol.

`class threading.Semaphore(value=1)`

`acquire(blocking=True, timeout=None)`

`release(n=1)`

This class implements semaphore objects. A semaphore manages an atomic counter representing the number of `release()` calls minus the number of `acquire()` calls, plus an initial value. The `acquire()` method blocks if necessary until it can return without making the counter negative. If not given, `value` defaults to 1. The optional argument gives the initial value for the internal counter; it defaults to 1. If the value given is less than 0, `ValueError` is raised. Changed in version 3.3: changed from a factory function

Acquire a semaphore. When invoked without arguments: If the internal counter is larger than zero on entry, decrement it by one and return `True` immediately. If the internal counter is zero on entry, block until awoken by a call to `release()`. Once awoken (and the counter is greater than 0), decrement the counter by 1 and return `True`. Exactly one thread will be awoken by each call to `release()`. The order in which threads are awoken should not be relied on. When invoked with `blocking` set to `False`, do not block.

Release a semaphore, incrementing the internal counter by `n`. When it was zero on entry and other threads are waiting for it to become larger than zero again, wake up `n` of those threads. Changed in version 3.9: Added `then` parameter to release multiple waiting threads at once.

`class threading.BoundedSemaphore(value=1)`

Class implementing bounded semaphore objects. A bounded semaphore checks to make sure its current value doesn't exceed its initial value. If it does, `ValueError` is raised. In most situations semaphores are used to guard resources with limited capacity. If the semaphore is released too many times it's a sign of a bug. If not given, `value` defaults to 1. Changed in version 3.3: changed from a factory function to a class.

Semaphore example

Semaphores are often used to guard resources with limited capacity, for example, a database server. In any situation where the size of the resource is fixed, you should use a bounded semaphore. Before spawning any worker threads, your main thread would initialize the semaphore:

```
maxconnections = 5
# ...
pool_sema = BoundedSemaphore(value=maxconnections)
```

Once spawned, worker threads call the semaphore's `acquire` and `release` methods when they need to connect to the server:

```
with pool_sema:
    conn = connectdb()
    try:
```

```
# ... use connection ...
finally:
    conn.close()
```

The use of a bounded semaphore reduces the chance that a programming error which causes the semaphore to be released more than it's acquired will go undetected.

Event objects

This is one of the simplest mechanisms for communication between threads: one thread signals an event and other threads wait for it.

An event object manages an internal flag that can be set to true with the `set()` method and reset to false with the `clear()` method. The `wait()` method blocks until the flag is true.

class threading.Event

is_set()

set()

clear()

wait(timeout=None)

Class implementing event objects. An event manages a flag that can be set to true with the `set()` method and reset to false with the `clear()` method. The `wait()` method blocks until the flag is true. The flag is initially false. Changed in version 3.3: changed from a factory function to a class. `is_set()` Return True if and only if the internal flag is true. The method `isSet` is a deprecated alias for this method. `set()` Set the internal flag to true. All threads waiting for it to become true are awakened. Threads t

Return True if and only if the internal flag is true. The method `isSet` is a deprecated alias for this method.

Set the internal flag to true. All threads waiting for it to become true are awakened. Threads that call `wait()` once the flag is true will not block at all.

Reset the internal flag to false. Subsequently, threads calling `wait()` will block until `set()` is called to set the internal flag to true again.

Block as long as the internal flag is false and the timeout, if given, has not expired. The return value represents the reason that this blocking method returned; True if returning because the internal flag is set to true, or False if a timeout is given and the internal flag did not become true within the given wait time. When the timeout argument is present and not None, it should be a floating-point number specifying a timeout for the operation in seconds, or fractions thereof. Changed in version 3.1

Timer objects

This class represents an action that should be run only after a certain amount of time has passed — a timer. `Timer` is a subclass of `Thread` and as such also functions as an example of creating custom threads.

Timers are started, as with threads, by calling their `Timer.start` method. The timer can be stopped (before its action has begun) by calling the `cancel()` method. The interval the timer will wait before executing its action may not be exactly the same as the interval specified by the user.

For example:

```
def hello():
    print("hello, world")

t = Timer(30.0, hello)
t.start() # after 30 seconds, "hello, world" will be printed
```

class threading.Timer(interval,function,args=None,kwargs=None)

cancel()

Create a timer that will run `function` with arguments `sargs` and keyword arguments `skwargs`, after `interval` seconds have passed. If `sargs` is `None` (the default) then an empty list will be used. If `kwargs` is `None` (the default) then an empty dict will be used. Changed in version 3.3: changed from a factory function to a class. `cancel()` Stop the timer, and cancel the execution of the timer's action. This will only work if the timer is still in its waiting stage.

Stop the timer, and cancel the execution of the timer's action. This will only work if the timer is still in its waiting stage.

Barrier objects

Added in version 3.2.

This class provides a simple synchronization primitive for use by a fixed number of threads that need to wait for each other. Each of the threads tries to pass the barrier by calling the `wait()` method and will block until all of the threads have made their `wait()` calls. At this point, the threads are released simultaneously.

The barrier can be reused any number of times for the same number of threads.

As an example, here is a simple way to synchronize a client and server thread:

```
b = Barrier(2, timeout=5)

def server():
    start_server()
    b.wait()
    while True:
        connection = accept_connection()
        process_server_connection(connection)

def client():
    b.wait()
    while True:
        connection = make_connection()
        process_client_connection(connection)
```

`class threading.Barrier(parties, action=None, timeout=None)`

`wait(timeout=None)`

`reset()`

`abort()`

`parties`

`n_waiting`

`broken`

Create a barrier object for `parties` number of threads. An `action`, when provided, is a callable to be called by one of the threads when they are released. `timeout` is the default timeout value if none is specified for the `wait()` method. `wait(timeout=None)` Pass the barrier. When all the threads party to the barrier have called this function, they are all released simultaneously. If `timeout` is provided, it is used in preference to any that was supplied to the class constructor. The return value is an integer

Pass the barrier. When all the threads party to the barrier have called this function, they are all released simultaneously. If `timeout` is provided, it is used in preference to any that was supplied to the class constructor. The return value is an integer in the range 0 to `parties - 1`, different for each thread. This can be used to select a thread to do some special housekeeping, e.g. `i = barrier.wait() if i == 0: # Only one thread needs to print this print("passed the barrier")` If an `action` was provided t

Return the barrier to the default, empty state. Any threads waiting on it will receive theBrokenBarrierErrorexception. Note that using this function may require some external synchronization if there are other threads whose state is unknown. If a barrier is broken it may be better to just leave it and create a new one.

Put the barrier into a broken state. This causes any active or future calls towait()to fail with theBrokenBarrierError. Use this for example if one of the threads needs to abort, to avoid deadlocking the application. It may be preferable to simply create the barrier with a sensibletimeoutvalue to automatically guard against one of the threads going awry.

The number of threads required to pass the barrier.

The number of threads currently waiting in the barrier.

A boolean that isTrueif the barrier is in the broken state.

exceptionthreading.BrokenBarrierError

This exception, a subclass ofRuntimeError, is raised when theBarrierobject is reset or broken.

Using locks, conditions, and semaphores in thewithstatement

All of the objects provided by this module that haveacquireandrelease methods can be used as context managers for awithstatement. Theacquiremethod will be called when the block is entered, andreleasewill be called when the block is exited. Hence, the following snippet:

```
with some_lock:
    # do something...
```

is equivalent to:

```
some_lock.acquire()
try:
    # do something...
finally:
    some_lock.release()
```

Currently,Lock,RLock,Condition,Semaphore, andBoundedSemaphoreobjects may be used aswithstatement context managers.

multiprocessing— Process-based parallelism

Source code: [Lib/multiprocessing/](#)

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Introduction

`multiprocessing` is a package that supports spawning processes using an API similar to the `threading` module. The `multiprocessing` package offers both local and remote concurrency, effectively side-stepping the Global Interpreter Lock by using subprocesses instead of threads. Due to this, the `multiprocessing` module allows the programmer to fully leverage multiple processors on a given machine. It runs on both POSIX and Windows.

The `multiprocessing` module also introduces the `Pool` object which offers a convenient means of parallelizing the execution of a function across multiple input values, distributing the input data across processes (data parallelism). The following example demonstrates the common practice of defining such functions in a module so that child processes can successfully import that module. This basic example of data parallelism using `Pool`,

```
from multiprocessing import Pool

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(5) as p:
        print(p.map(f, [1, 2, 3]))
```

will print to standard output

```
[1, 4, 9]
```

The `multiprocessing` module also introduces APIs which do not have analogs in the `threading` module, like the ability to `terminate`, `interrupt` or `kill` a running process.

See also

`concurrent.futures.ProcessPoolExecutor` offers a higher level interface to push tasks to a background process without blocking execution of the calling process. Compared to using the `Pool` interface directly, the `concurrent.futures` API more readily allows the submission of work to the underlying process pool to be separated from waiting for the results.

The `Process` class

In `multiprocessing`, processes are spawned by creating a `Process` object and then calling its `start()` method. `Process` follows the API of `threading.Thread`. A trivial example of a multiprocess program is

```
from multiprocessing import Process

def f(name):
    print('hello', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

To show the individual process IDs involved, here is an expanded example:

```

from multiprocessing import Process
import os

def info(title):
    print(title)
    print('module name:', __name__)
    print('parent process:', os.getppid())
    print('process id:', os.getpid())

def f(name):
    info('function f')
    print('hello', name)

if __name__ == '__main__':
    info('main line')
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()

```

For an explanation of why their `__name__ == '__main__'` part is necessary, see [Programming guidelines](#).

The arguments to `Process` usually need to be unpickleable from within the child process. If you tried typing the above example directly into a REPL it could lead to an `AttributeError` in the child process trying to locate the function in the `__main__` module.

Contexts and start methods

Depending on the platform, `multiprocessing` supports three ways to start a process. These start methods are

spawn

The parent process starts a fresh Python interpreter process. The child process will only inherit those resources necessary to run the process object's `run()` method. In particular, unnecessary file descriptors and handles from the parent process will not be inherited. Starting a process using this method is rather slow compared to using `fork` or `forkserver`. Available on POSIX and Windows platforms. The default on Windows and macOS.

fork

The parent process uses `os.fork()` to fork the Python interpreter. The child process, when it begins, is effectively identical to the parent process. All resources of the parent are inherited by the child process. Note that safely forking a multithreaded process is problematic. Available on POSIX systems. Changed in version 3.14: This is no longer the default start method on any platform. Code that requires `fork` must explicitly specify that via `get_context()` or `set_start_method()`. Changed in version 3.12

forkserver

When the program starts and selects the `forkserver` start method, a server process is spawned. From then on, whenever a new process is needed, the parent process connects to the server and requests that it fork a new process. The fork server process is single threaded unless system libraries or preloaded imports spawn threads as a side-effect so it is generally safe for it to use `os.fork()`. No unnecessary resources are inherited. Available on POSIX platforms which support passing file descriptors o

Changed in version 3.4: `spawn` added on all POSIX platforms, and `forkserver` added for some POSIX platforms. Child processes no longer inherit all of the parents inheritable handles on Windows.

Changed in version 3.8: On macOS, the `spawn` start method is now the default. The `fork` start method should be considered unsafe as it can lead to crashes of the subprocess as macOS system libraries may start threads. See [bpo-33725](#).

Changed in version 3.14: On POSIX platforms the default start method was changed from `fork` to `forkserver` to retain the performance but avoid common multithreaded process incompatibilities. See [gh-84559](#).

On POSIX using the `spawn` or `forkserver` start methods will also start a resource tracker process which tracks the unlinked named system resources (such as named semaphores or `SharedMemory` objects) created by processes of the program. When all processes have exited the resource tracker unlinks any remaining tracked object. Usually there should be none, but if a process was killed by a signal there may be some “leaked” resources. (Neither leaked semaphores nor shared memory segments will be automatically unlinked until the next reboot. This is problematic for both objects because the system allows only a limited number of named semaphores, and shared memory segments occupy some space in the main memory.)

To select a start method you use the `set_start_method()` in their `__name__ == '__main__'` clause of the main module. For example:

```
import multiprocessing as mp

def foo(q):
    q.put('hello')

if __name__ == '__main__':
    mp.set_start_method('spawn')
    q = mp.Queue()
    p = mp.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

`set_start_method()` should not be used more than once in the program.

Alternatively, you can use `get_context()` to obtain a context object. Context objects have the same API as the multiprocessing module, and allow one to use multiple start methods in the same program.

```
import multiprocessing as mp

def foo(q):
    q.put('hello')

if __name__ == '__main__':
    ctx = mp.get_context('spawn')
    q = ctx.Queue()
    p = ctx.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

Note that objects related to one context may not be compatible with processes for a different context. In particular, locks created using the `fork` context cannot be passed to processes started using the `spawn` or `forkserver` start methods.

Libraries using `multiprocessing` or `ProcessPoolExecutor` should be designed to allow their users to provide their own multiprocessing context. Using a specific context of your own within a library can lead to incompatibilities with the rest of the library user’s application. Always document if your library requires a specific start method.

■■ Advertencia: Warning The ‘spawn’ and ‘forkserver’ start methods generally cannot be used with “frozen” executables (i.e., binaries produced by packages like `PyInstaller` and `cx_Freeze`) on POSIX systems. The ‘fork’ start method may work if code does not use threads.

Exchanging objects between processes

`multiprocessing` supports two types of communication channel between processes:

Queues

The `Queue` class is a near clone of `queue.Queue`. For example:

```
from multiprocessing import Process, Queue
```



```
def f(q):
    q.put([42, None, 'hello'])

if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
    print(q.get())    # prints "[42, None, 'hello']"
    p.join()
```

Queues are thread and process safe. Any object put into a multiprocessing queue will be serialized.

Pipes

The `Pipe()` function returns a pair of connection objects connected by a pipe which by default is duplex (two-way).

For example:

```
from multiprocessing import Process, Pipe

def f(conn):
    conn.send([42, None, 'hello'])
    conn.close()

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p = Process(target=f, args=(child_conn,))
    p.start()
    print(parent_conn.recv())    # prints "[42, None, 'hello']"
    p.join()
```

The two connection objects returned by `Pipe()` represent the two ends of the pipe. Each connection object has `send()` and `recv()` methods (among others). Note that data in a pipe may become corrupted if two processes (or threads) try to read from or write to the same end of the pipe at the same time. Of course there is no risk of corruption from processes using different ends of the pipe at the same time.

The `send()` method serializes the object and `recv()` re-creates the object.

Synchronization between processes

multiprocessing contains equivalents of all the synchronization primitives from threading. For instance one can use a lock to ensure that only one process prints to standard output at a time:

```
from multiprocessing import Process, Lock

def f(l, i):
    l.acquire()
    try:
        print('hello world', i)
    finally:
        l.release()

if __name__ == '__main__':
    lock = Lock()

    for num in range(10):
        Process(target=f, args=(lock, num)).start()
```

Without using the lock output from the different processes is liable to get all mixed up.

Sharing state between processes

As mentioned above, when doing concurrent programming it is usually best to avoid using shared state as far as possible. This is particularly true when using multiple processes.

However, if you really do need to use some shared data then multiprocessing provides a couple of ways of doing so.

Shared memory

Data can be stored in a shared memory map using `Value` or `Array`. For example, the following code

```
from multiprocessing import Process, Value, Array
```

```
def f(n, a):
    n.value = 3.1415927
    for i in range(len(a)):
        a[i] = -a[i]

if __name__ == '__main__':
    num = Value('d', 0.0)
    arr = Array('i', range(10))

    p = Process(target=f, args=(num, arr))
    p.start()
    p.join()

    print(num.value)
    print(arr[:])
```

will print

```
3.1415927
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

The `'d'` and `'i'` arguments used when creating `num` and `arr` are typecodes of the kind used by the `array` module: `'d'` indicates a double precision float and `'i'` indicates a signed integer. These shared objects will be process and thread-safe.

For more flexibility in using shared memory one can use the `multiprocessing.sharedctypes` module which supports the creation of arbitrary ctypes objects allocated from shared memory.

Server process

A manager object returned by `Manager()` controls a server process which holds Python objects and allows other processes to manipulate them using proxies.

A manager returned by `Manager()` will support types `list`, `dict`, `set`, `Namespace`, `Lock`, `RLock`, `Semaphore`, `BoundedSemaphore`, `Condition`, `Event`, `Barrier`, `Queue`, `Value` and `Array`. For example,

```
from multiprocessing import Process, Manager
```

```
def f(d, l, s):
    d[1] = '1'
    d['2'] = 2
    d[0.25] = None
    l.reverse()
    s.add('a')
    s.add('b')

if __name__ == '__main__':
    with Manager() as manager:
        d = manager.dict()
        l = manager.list(range(10))
        s = manager.set()

        p = Process(target=f, args=(d, l, s))
        p.start()
        p.join()
```

```

print(d)
print(l)
print(s)

```

will print

```

{0.25: None, 1: '1', '2': 2}
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
{'a', 'b'}

```

Server process managers are more flexible than using shared memory objects because they can be made to support arbitrary object types. Also, a single manager can be shared by processes on different computers over a network. They are, however, slower than using shared memory.

Using a pool of workers

The `Pool` class represents a pool of worker processes. It has methods which allows tasks to be offloaded to the worker processes in a few different ways.

For example:

```

from multiprocessing import Pool, TimeoutError
import time
import os

def f(x):
    return x*x

if __name__ == '__main__':
    # start 4 worker processes
    with Pool(processes=4) as pool:

        # print "[0, 1, 4, ..., 81]"
        print(pool.map(f, range(10)))

        # print same numbers in arbitrary order
        for i in pool.imap_unordered(f, range(10)):
            print(i)

        # evaluate "f(20)" asynchronously
        res = pool.apply_async(f, (20,))    # runs in *only* one process
        print(res.get(timeout=1))           # prints "400"

        # evaluate "os.getpid()" asynchronously
        res = pool.apply_async(os.getpid, ()) # runs in *only* one process
        print(res.get(timeout=1))           # prints the PID of that process

        # launching multiple evaluations asynchronously *may* use more processes
        multiple_results = [pool.apply_async(os.getpid, ()) for i in range(4)]
        print([res.get(timeout=1) for res in multiple_results])

        # make a single worker sleep for 10 seconds
        res = pool.apply_async(time.sleep, (10,))
        try:
            print(res.get(timeout=1))
        except TimeoutError:
            print("We lacked patience and got a multiprocessing.TimeoutError")

        print("For the moment, the pool remains available for more work")

    # exiting the 'with'-block has stopped the pool
    print("Now the pool is closed and no longer available")

```

Note that the methods of a pool should only ever be used by the process which created it.

■ Nota: NoteFunctionality within this package requires that the `__main__` module be importable by the children. This is covered in Programming guidelines however it is worth pointing out here. This means that some examples, such as the `multiprocessing.pool.Pool` examples will not work in the interactive interpreter. For example: `>>>from multiprocessing import Pool>>>p=Pool(5)>>>def f(x):...return x*x...>>>with p:...p.map(f,[1,2,3])` Process PoolWorker-1:Process PoolWorker-2:Process PoolWorker-3:Traceback (most recent call last):Traceback (most recent call last):Traceback (most recent call last):AttributeError:Can't ge

Reference

The `multiprocessing` package mostly replicates the API of the `threading` module.

Global start method

Python supports several ways to create and initialize a process. The global start method sets the default mechanism for creating a process.

Several multiprocessing functions and methods that may also instantiate certain objects will implicitly set the global start method to the system's default, if it hasn't been set already. The global start method can only be set once. If you need to change the start method from the system default, you must proactively set the global start method before calling functions or methods, or creating these objects.

Process and exceptions

`class multiprocessing.Process(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)`

`run()`

`start()`

`join([timeout])`

`name`

`is_alive()`

`daemon`

`pid`

`exitcode`

`authkey`

`sentinel`

`interrupt()`

`terminate()`

`kill()`

`close()`

Process objects represent activity that is run in a separate process. The `Process` class has equivalents of all the methods of `threading.Thread`. The constructor should always be called with keyword arguments. `group` should always be `None`; it exists solely for compatibility with `threading.Thread`. `target` is the callable object to be invoked by the `run()` method. It defaults to `None`, meaning nothing is called. `name` is the process name (see `name` for more details). `args` is the argument tuple for the target invocation. `kwa`

Method representing the process's activity. You may override this method in a subclass. The standard `run()` method invokes the callable object passed to the object's constructor as the target argument, if

any, with sequential and keyword arguments taken from the args and kwargs arguments, respectively. Using a list or tuple as the args argument passed to Process achieves the same effect. Example: >>> from multiprocessing import Process >>> p = Process(target=print, args=[1]) >>> p.run() 1 >>> p = Process(target=print, args=(1

Start the process's activity. This must be called at most once per process object. It arranges for the object's run() method to be invoked in a separate process.

If the optional argument timeout is None (the default), the method blocks until the process whose join() method is called terminates. If timeout is a positive number, it blocks at most timeout seconds. Note that the method returns None if its process terminates or if the method times out. Check the process's exitcode to determine if it terminated. A process can be joined many times. A process cannot join itself because this would cause a deadlock. It is an error to attempt to join a process before it has been

The process's name. The name is a string used for identification purposes only. It has no semantics. Multiple processes may be given the same name. The initial name is set by the constructor. If no explicit name is provided to the constructor, a name of the form 'Process-N1:N2:...:Nk' is constructed, where each Nk is the N-th child of its parent.

Return whether the process is alive. Roughly, a process object is alive from the moment the start() method returns until the child process terminates.

The process's daemon flag, a Boolean value. This must be set before start() is called. The initial value is inherited from the creating process. When a process exits, it attempts to terminate all of its daemon child processes. Note that a daemon process is not allowed to create child processes. Otherwise a daemon process would leave its children orphaned if it gets terminated when its parent process exits. Additionally, these are not Unix daemons or services, they are normal processes that will

Return the process ID. Before the process is spawned, this will be None.

The child's exit code. This will be None if the process has not yet terminated. If the child's run() method returned normally, the exit code will be 0. If it terminated via sys.exit() with an integer argument N, the exit code will be N. If the child terminated due to an exception not caught within run(), the exit code will be 1. If it was terminated by signal N, the exit code will be the negative value -N.

The process's authentication key (a byte string). When multiprocessing is initialized the main process is assigned a random string using os.urandom(). When a Process object is created, it will inherit the authentication key of its parent process, although this may be changed by setting authkey to another byte string. See Authentication keys.

A numeric handle of a system object which will become "ready" when the process ends. You can use this value if you want to wait on several events at once using multiprocessing.connection.wait(). Otherwise calling join() is simpler. On Windows, this is an OS handle usable with the WaitForSingleObject and WaitForMultipleObjects family of API calls. On POSIX, this is a file descriptor usable with primitives from the select module. Added in version 3.3.

Terminate the process. Works on POSIX using the SIGINT signal. Behavior on Windows is undefined. By default, this terminates the child process by raising KeyboardInterrupt. This behavior can be altered by setting the respective signal handler in the child process signal.signal() for SIGINT. Note: if the child process catches and discards KeyboardInterrupt, the process will not be terminated. Note: the default behavior will also set exitcode to 1 as if an uncaught exception was raised in the child process. To

Terminate the process. On POSIX this is done using the SIGTERM signal; on Windows TerminateProcess() is used. Note that exit handlers and finally clauses, etc., will not be executed. Note that descendant processes of the process will not be terminated – they will simply become orphaned. Warning! If this method is used when the associated process is using a pipe or queue then the pipe or queue is liable to become corrupted and may become unusable by other process. Similarly, if the process has acquired

Same as terminate() but using the SIGKILL signal on POSIX. Added in version 3.7.

Close the `Process` object, releasing all resources associated with it. `ValueError` is raised if the underlying process is still running. Once `close()` returns successfully, most other methods and attributes of the `Process` object will raise `ValueError`. Added in version 3.7.

`exception multiprocessing.ProcessError`

The base class of all multiprocessing exceptions.

`exception multiprocessing.BufferTooShort`

Exception raised by `Connection.recv_bytes_into()` when the supplied buffer object is too small for the message read. If `e` is an instance of `BufferTooShort` then `e.args[0]` will give the message as a byte string.

`exception multiprocessing.AuthenticationError`

Raised when there is an authentication error.

`exception multiprocessing.TimeoutError`

Raised by methods with a timeout when the timeout expires.

Pipes and Queues

When using multiple processes, one generally uses message passing for communication between processes and avoids having to use any synchronization primitives like locks.

For passing messages one can use `Pipe()` (for a connection between two processes) or a queue (which allows multiple producers and consumers).

The `Queue`, `SimpleQueue` and `JoinableQueue` types are multi-producer, multi-consumer FIFO queues modelled on the `queue.Queue` class in the standard library. They differ in that `Queue` lacks the `task_done()` and `join()` methods introduced into Python 2.5's `queue.Queue` class.

If you use `JoinableQueue` then you must call `JoinableQueue.task_done()` for each task removed from the queue or else the semaphore used to count the number of unfinished tasks may eventually overflow, raising an exception.

One difference from other Python queue implementations, is that multiprocessing queues serialize all objects that are put into them using pickle. The object returned by the `get` method is a re-created object that does not share memory with the original object.

Note that one can also create a shared queue by using a manager object – see `Managers`.

■ Nota: Note that multiprocessing uses the usual `queue.Empty` and `queue.Full` exceptions to signal a timeout. They are not available in the multiprocessing namespace so you need to import them from `queue`.

■ Nota: Note When an object is put on a queue, the object is pickled and a background thread later flushes the pickled data to an underlying pipe. This has some consequences which are a little surprising, but should not cause any practical difficulties – if they really bother you then you can instead use a queue created with a manager. After putting an object on an empty queue there may be an infinitesimal delay before the queue's `empty()` method returns `False` and `get_nowait()` can return without raising `queue.Empty`. If multiple processes are enqueueing objects, it is possible for the objects to be received at the

■■ Advertencia: Warning If a process is killed using `Process.terminate()` or `os.kill()` while it is trying to use a `Queue`, then the data in the queue is likely to become corrupted. This may cause any other process to get an exception when it tries to use the queue later on.

■■ Advertencia: Warning As mentioned above, if a child process has put items on a queue (and it has not used `JoinableQueue.cancel_join_thread()`), then that process will not terminate until all buffered items have been flushed to the pipe. This means that if you try joining that process you may get a deadlock unless you are sure that all items which have been put on the queue have been consumed. Similarly, if the child process is non-daemonic then the parent process may hang on exit when it tries to join all its non-daemonic children. Note that a queue created using a manager does not have this issue. See `Programmi`

For an example of the usage of queues for interprocess communication see `Examples`.

multiprocessing.Pipe(duplex=True)

Returns a pair(conn1,conn2) of Connection objects representing the ends of a pipe. If duplex is True (the default) then the pipe is bidirectional. If duplex is False then the pipe is unidirectional: conn1 can only be used for receiving messages and conn2 can only be used for sending messages. The send() method serializes the object using pickle and the recv() re-creates the object.

class multiprocessing.Queue([maxsize])

qsize()

empty()

full()

put(obj[,block[,timeout]])

put_nowait(obj)

get([block[,timeout]])

get_nowait()

close()

join_thread()

cancel_join_thread()

Returns a process shared queue implemented using a pipe and a few locks/semaphores. When a process first puts an item on the queue a feeder thread is started which transfers objects from a buffer into the pipe. Instantiating this class may set the global start method. See Global start method for more details. The usual queue.Empty and queue.Full exceptions from the standard library's queue module are raised to signal timeouts. Queue implements all the methods of queue.Queue except for task_done() and join().

Return the approximate size of the queue. Because of multithreading/multiprocessing semantics, this number is not reliable. Note that this may raise NotImplementedError on platforms like macOS where sem_getvalue() is not implemented.

Return True if the queue is empty, False otherwise. Because of multithreading/multiprocessing semantics, this is not reliable. May raise an OSError on closed queues. (not guaranteed)

Return True if the queue is full, False otherwise. Because of multithreading/multiprocessing semantics, this is not reliable.

Put obj into the queue. If the optional argument block is True (the default) and timeout is None (the default), block if necessary until a free slot is available. If timeout is a positive number, it blocks at most timeout seconds and raises the queue.Full exception if no free slot was available within that time. Otherwise (block is False), put an item on the queue if a free slot is immediately available, else raise the queue.Full exception (timeout is ignored in that case). Changed in version 3.8: If the queue is

Equivalent to put(obj, False).

Remove and return an item from the queue. If optional args block is True (the default) and timeout is None (the default), block if necessary until an item is available. If timeout is a positive number, it blocks at most timeout seconds and raises the queue.Empty exception if no item was available within that time. Otherwise (block is False), return an item if one is immediately available, else raise the queue.Empty exception (timeout is ignored in that case). Changed in version 3.8: If the queue is closed, ValueError

Equivalent to get(False).

Close the queue: release internal resources. A queue must not be used anymore after it is closed. For example, get(), put() and empty() methods must no longer be called. The background thread will quit once it has

flushed all buffered data to the pipe. This is called automatically when the queue is garbage collected.

Join the background thread. This can only be used after `close()` has been called. It blocks until the background thread exits, ensuring that all data in the buffer has been flushed to the pipe. By default if a process is not the creator of the queue then on exit it will attempt to join the queue's background thread. The process can call `cancel_join_thread()` to make `join_thread()` do nothing.

Prevent `join_thread()` from blocking. In particular, this prevents the background thread from being joined automatically when the process exits – see `join_thread()`. A better name for this method might be `allow_exit_without_flush()`. It is likely to cause enqueued data to be lost, and you almost certainly will not need to use it. It is really only there if you need the current process to exit immediately without waiting to flush enqueued data to the underlying pipe, and you don't care about lost data.

class multiprocessing.SimpleQueue

close()

empty()

get()

put(item)

It is a simplified Queue type, very close to a `lockedPipe`. Instantiating this class may set the global start method. See `Global start method` for more details. `close()` Close the queue: release internal resources. A queue must not be used anymore after it is closed. For example, `get()`, `put()` and `empty()` methods must no longer be called. Added in version 3.9. `empty()` Return `True` if the queue is empty, `False` otherwise. Always raises an `OSError` if the `SimpleQueue` is closed. `get()` Remove and return an item from the queue. `put(`

Close the queue: release internal resources. A queue must not be used anymore after it is closed. For example, `get()`, `put()` and `empty()` methods must no longer be called. Added in version 3.9.

Return `True` if the queue is empty, `False` otherwise. Always raises an `OSError` if the `SimpleQueue` is closed.

Remove and return an item from the queue.

Put item into the queue.

class multiprocessing.JoinableQueue([maxsize])

task_done()

join()

`JoinableQueue`, a `Queue` subclass, is a queue which additionally has `task_done()` and `join()` methods. Instantiating this class may set the global start method. See `Global start method` for more details. `task_done()` Indicate that a formerly enqueued task is complete. Used by queue consumers. For each `get()` used to fetch a task, a subsequent call to `task_done()` tells the queue that the processing on the task is complete. If `join()` is currently blocking, it will resume when all items have been processed (meaning that

Indicate that a formerly enqueued task is complete. Used by queue consumers. For each `get()` used to fetch a task, a subsequent call to `task_done()` tells the queue that the processing on the task is complete. If `join()` is currently blocking, it will resume when all items have been processed (meaning that a `task_done()` call was received for every item that had been `put()` into the queue). Raises a `ValueError` if called more times than there were items placed in the queue.

Block until all items in the queue have been gotten and processed. The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer calls `task_done()` to indicate that the item was retrieved and all work on it is complete. When the count of unfinished tasks drops to zero, `join()` unblocks.

Miscellaneous

multiprocessing.active_children()

Return list of all live children of the current process. Calling this has the side effect of “joining” any processes which have already finished.

multiprocessing.cpu_count()

Return the number of CPUs in the system. This number is not equivalent to the number of CPUs the current process can use. The number of usable CPUs can be obtained with `os.process_cpu_count()` or `len(os.sched_getaffinity(0))`. When the number of CPUs cannot be determined a `NotImplementedError` is raised. See also `os.cpu_count()` and `os.process_cpu_count()`. Changed in version 3.13: The return value can also be overridden using the `Xcpu_count` flag or `PYTHON_CPU_COUNT` as this is merely a wrapper around the `os.cpu_count` A

multiprocessing.current_process()

Return the `Process` object corresponding to the current process. An analogue of `threading.current_thread()`.

multiprocessing.parent_process()

Return the `Process` object corresponding to the parent process of the `current_process()`. For the main process, `parent_process` will be `None`. Added in version 3.8.

multiprocessing.freeze_support()

Add support for when a program which uses `multiprocessing` has been frozen to produce an executable. (Has been tested with `py2exe`, `PyInstaller` and `cx_Freeze`.) One needs to call this function straight after the `if __name__ == '__main__':` line of the main module. For example: `from multiprocessing import Process, freeze_support; def f(): print('hello world!'); if __name__ == '__main__': freeze_support(); p = Process(target=f); p.start()` If the `freeze_support()` line is omitted then trying to run the frozen executable will raise `RuntimeError`

multiprocessing.get_all_start_methods()

Returns a list of the supported start methods, the first of which is the default. The possible start methods are `'fork'`, `'spawn'` and `'forkserver'`. Not all platforms support all methods. See `Contexts` and `start methods`. Added in version 3.4.

multiprocessing.get_context(method=None)

Return a context object which has the same attributes as the `multiprocessing` module. If `method` is `None` then the default context is returned. Note that if the global start method has not been set, this will set it to the system default. See `Global start method` for more details. Otherwise `method` should be `'fork'`, `'spawn'`, `'forkserver'`. `ValueError` is raised if the specified start method is not available. See `Contexts` and `start methods`. Added in version 3.4.

multiprocessing.get_start_method(allow_none=False)

Return the name of start method used for starting processes. If the global start method is not set and `allow_none` is `False`, the global start method is set to the default, and its name is returned. See `Global start method` for more details. The return value can be `'fork'`, `'spawn'`, `'forkserver'` or `None`. See `Contexts` and `start methods`. Added in version 3.4. Changed in version 3.8: On macOS, the `spawn` start method is now the default. The `fork` start method should be considered unsafe as it can lead to crashes of the `su`

multiprocessing.set_executable(executable)

Set the path of the Python interpreter to use when starting a child process. (By default `sys.executable` is used). Embedders will probably need to do something like `set_executable(os.path.join(sys.exec_prefix, 'pythonw.exe'))` before they can create child processes. Changed

in version 3.4: Now supported on POSIX when the 'spawn' start method is used. Changed in version 3.11: Accepts a path-like object.

multiprocessing.set_forkserver_preload(module_names)

Set a list of module names for the forkserver main process to attempt to import so that their already imported state is inherited by forked processes. Any `ImportError` when doing so is silently ignored. This can be used as a performance enhancement to avoid repeated work in every process. For this to work, it must be called before the forkserver process has been launched (before creating a `Pool` or starting a `Process`). Only meaningful when using the 'forkserver' start method. See `Contexts` and `start` methods.

multiprocessing.set_start_method(method, force=False)

Set the method which should be used to start child processes. The `method` argument can be 'fork', 'spawn' or 'forkserver'. Raises `RuntimeError` if the start method has already been set and `force` is not `True`. If `method` is `None` and `force` is `True` then the start method is set to `None`. If `method` is `None` and `force` is `False` then the context is set to the default context. Note that this should be called at most once, and it should be protected inside their `if __name__ == '__main__':` clause of the main module. See `Contexts` and `start` methods. A

■ Nota: Note `multiprocessing` contains no analogues of `threading.active_count()`, `threading.enumerate()`, `threading.settrace()`, `threading.setprofile()`, `threading.Timer`, or `threading.local`.

Connection Objects

Connection objects allow the sending and receiving of picklable objects or strings. They can be thought of as message oriented connected sockets.

Connection objects are usually created using `Pipe`— see also `Listeners` and `Clients`.

class multiprocessing.connection.Connection

send(obj)

recv()

fileno()

close()

poll([timeout])

send_bytes(buffer[, offset[, size]])

recv_bytes([maxlength])

recv_bytes_into(buffer[, offset])

send(obj) Send an object to the other end of the connection which should be read using `recv()`. The object must be picklable. Very large pickles (approximately 32 MiB+, though it depends on the OS) may raise a `ValueError` exception. **recv()** Return an object sent from the other end of the connection using `send()`. Blocks until there is something to receive. Raises `EOFError` if there is nothing left to receive and the other end was closed. **fileno()** Return the file descriptor or handle used by the connection. **clo**

Send an object to the other end of the connection which should be read using `recv()`. The object must be picklable. Very large pickles (approximately 32 MiB+, though it depends on the OS) may raise a `ValueError` exception.

Return an object sent from the other end of the connection using `send()`. Blocks until there is something to receive. Raises `EOFError` if there is nothing left to receive and the other end was closed.

Return the file descriptor or handle used by the connection.

Close the connection. This is called automatically when the connection is garbage collected.

Return whether there is any data available to be read. If `timeout` is not specified then it will return immediately. If `timeout` is a number then this specifies the maximum time in seconds to block. If `timeout` is `None` then an infinite timeout is used. Note that multiple connection objects may be polled at once by using `multiprocessing.connection.wait()`.

Send byte data from a bytes-like object as a complete message. If `offset` is given then data is read from that position in buffer. If `size` is given then that many bytes will be read from buffer. Very large buffers (approximately 32 MiB+, though it depends on the OS) may raise a `ValueError` exception

Return a complete message of byte data sent from the other end of the connection as a string. Blocks until there is something to receive. Raises `EOFError` if there is nothing left to receive and the other end has closed. If `maxlength` is specified and the message is longer than `maxlength` then `OSError` is raised and the connection will no longer be readable. Changed in version 3.3: This function used to raise `IOError`, which is now an alias of `OSError`.

Read into buffer a complete message of byte data sent from the other end of the connection and return the number of bytes in the message. Blocks until there is something to receive. Raises `EOFError` if there is nothing left to receive and the other end was closed. `buffer` must be a writable bytes-like object. If `offset` is given then the message will be written into the buffer from that position. Offset must be a non-negative integer less than the length of `buffer` (in bytes). If the buffer is too short the

For example:

```
>>> from multiprocessing import Pipe
>>> a, b = Pipe()
>>> a.send([1, 'hello', None])
>>> b.recv()
[1, 'hello', None]
>>> b.send_bytes(b'thank you')
>>> a.recv_bytes()
b'thank you'
>>> import array
>>> arr1 = array.array('i', range(5))
>>> arr2 = array.array('i', [0] * 10)
>>> a.send_bytes(arr1)
>>> count = b.recv_bytes_into(arr2)
>>> assert count == len(arr1) * arr1.itemsize
>>> arr2
array('i', [0, 1, 2, 3, 4, 0, 0, 0, 0, 0])
```

■■ Advertencia: Warning `TheConnection.recv()` method automatically unpickles the data it receives, which can be a security risk unless you can trust the process which sent the message. Therefore, unless the connection object was produced using `Pipe()` you should only use `recv()` and `send()` methods after performing some sort of authentication. See `Authentication keys`.

■■ Advertencia: Warning If a process is killed while it is trying to read or write to a pipe then the data in the pipe is likely to become corrupted, because it may become impossible to be sure where the message boundaries lie.

Synchronization primitives

Generally synchronization primitives are not as necessary in a multiprocess program as they are in a multithreaded program. See the documentation for `threading` module.

Note that one can also create synchronization primitives by using a manager object – see `Managers`.

`class multiprocessing.Barrier(parties[, action[, timeout]])`

A barrier object: a clone of `threading.Barrier`. Instantiating this class may set the global start method. See `Global start method` for more details. Added in version 3.3.

`class multiprocessing.BoundedSemaphore([value])`

locked()

A bounded semaphore object: a close analog of `threading.BoundedSemaphore`. Instantiating this class may set the global start method. See `Global start method` for more details. A solitary difference from its close analog exists: its `acquire` method's first argument is named `block`, as is consistent with `Lock.acquire()`. `locked()` Return a boolean indicating whether this object is locked right now. Added in version 3.14. Note On macOS, this is indistinguishable from `Semaphore` because `sem_getvalue()` is not implemented on

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

class multiprocessing.Condition([lock])

A condition variable: an alias for `threading.Condition`. If `lock` is specified then it should be a `Lock` or `RLock` object from `multiprocessing`. Instantiating this class may set the global start method. See `Global start method` for more details. Changed in version 3.3: The `wait_for()` method was added.

class multiprocessing.Event

A clone of `threading.Event`. Instantiating this class may set the global start method. See `Global start method` for more details.

class multiprocessing.Lock

acquire(block=True, timeout=None)

release()

locked()

A non-recursive lock object: a close analog of `threading.Lock`. Once a process or thread has acquired a lock, subsequent attempts to acquire it from any process or thread will block until it is released; any process or thread may release it. The concepts and behaviors of `threading.Lock` as it applies to threads are replicated here in `multiprocessing.Lock` as it applies to either processes or threads, except as noted. Note that `Lock` is actually a factory function which returns an instance of `multiprocessing`

Acquire a lock, blocking or non-blocking. With the `block` argument set to `True` (the default), the method call will block until the lock is in an unlocked state, then set it to locked and return `True`. Note that the name of this first argument differs from that in `threading.Lock.acquire()`. With the `block` argument set to `False`, the method call does not block. If the lock is currently in a locked state, return `False`; otherwise set the lock to a locked state and return `True`. When invoked with a positive, floating

Release a lock. This can be called from any process or thread, not only the process or thread which originally acquired the lock. Behavior is the same as in `threading.Lock.release()` except that when invoked on an unlocked lock, a `ValueError` is raised.

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

class multiprocessing.RLock

acquire(block=True, timeout=None)

release()

locked()

A recursive lock object: a close analog of `threading.RLock`. A recursive lock must be released by the process or thread that acquired it. Once a process or thread has acquired a recursive lock, the same process or thread may acquire it again without blocking; that process or thread must release it once for each time it has been acquired. Note that `RLock` is actually a factory function which returns an instance of `multiprocessing.synchronize.RLock` initialized with a default context. Instantiating this cl

Acquire a lock, blocking or non-blocking. When invoked with the `block` argument set to `True`, block until the lock is in an unlocked state (not owned by any process or thread) unless the lock is already owned by the current process or thread. The current process or thread then takes ownership of the lock (if it does not already have ownership) and the recursion level inside the lock increments by one, resulting in a return value of `True`. Note that there are several differences in this first argument'

Release a lock, decrementing the recursion level. If after the decrement the recursion level is zero, reset the lock to unlocked (not owned by any process or thread) and if any other processes or threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed. If after the decrement the recursion level is still nonzero, the lock remains locked and owned by the calling process or thread. Only call this method when the calling process or thread owns the lock. An

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

`class multiprocessing.Semaphore([value])`

`get_value()`

`locked()`

A semaphore object: a close analog of `threading.Semaphore`. Instantiating this class may set the global start method. See `Global start method` for more details. A solitary difference from its close analog exists: its `acquire` method's first argument is named `block`, as is consistent with `Lock.acquire().get_value()`. Return the current value of semaphore. Note that this may raise `NotImplementedError` on platforms like macOS where `sem_getvalue()` is not implemented. `locked()` Return a boolean indicating whether this object

Return the current value of semaphore. Note that this may raise `NotImplementedError` on platforms like macOS where `sem_getvalue()` is not implemented.

Return a boolean indicating whether this object is locked right now. Added in version 3.14.

■ Nota: Note On macOS, `sem_timedwait` is unsupported, so calling `acquire()` with a timeout will emulate that function's behavior using a sleeping loop.

■ Nota: Note Some of this package's functionality requires a functioning shared semaphore implementation on the host operating system. Without one, the `multiprocessing.synchronize` module will be disabled, and attempts to import it will result in an `ImportError`. See [bpo-3770](#) for additional information.

Shared ctypes Objects

It is possible to create shared objects using shared memory which can be inherited by child processes.

`multiprocessing.Value(typecode_or_type, *args, lock=True)`

Return a `ctypes` object allocated from shared memory. By default the return value is actually a synchronized wrapper for the object. The object itself can be accessed via the `value` attribute of `aValue.typecode_or_type` determines the type of the returned object: it is either a `ctypes` type or a one character typecode of the kind used by the `array` module. `*args` is passed on to the constructor for the type. If `lock` is `True` (the default) then a new recursive lock object is created to synchronize access to the val

`multiprocessing.Array(typecode_or_type, size_or_initializer, *, lock=True)`

Return a `ctypes` array allocated from shared memory. By default the return value is actually a synchronized wrapper for the array. `typecode_or_type` determines the type of the elements of the returned array: it is either a `ctypes` type or a one character typecode of the kind used by the `array` module with the exception of 'w', which is not supported. In addition, the 'c' typecode is an alias for `ctypes.c_char`. If `size_or_initializer` is an integer, then it determines the length of the array, and the array will

The multiprocessing.sharedctypes module

The `multiprocessing.sharedctypes` module provides functions for allocating ctypes objects from shared memory which can be inherited by child processes.

■ **Nota:** Note Although it is possible to store a pointer in shared memory remember that this will refer to a location in the address space of a specific process. However, the pointer is quite likely to be invalid in the context of a second process and trying to dereference the pointer from the second process may cause a crash.

`multiprocessing.sharedctypes.RawArray(typecode_or_type, size_or_initializer)`

Return a ctypes array allocated from shared memory. `typecode_or_type` determines the type of the elements of the returned array: it is either a ctypes type or a one character typecode of the kind used by the `array` module. If `size_or_initializer` is an integer then it determines the length of the array, and the array will be initially zeroed. Otherwise `size_or_initializer` is a sequence which is used to initialize the array and whose length determines the length of the array. Note that setting and getting a

`multiprocessing.sharedctypes.RawValue(typecode_or_type, *args)`

Return a ctypes object allocated from shared memory. `typecode_or_type` determines the type of the returned object: it is either a ctypes type or a one character typecode of the kind used by the `array` module. `*args` is passed on to the constructor for the type. Note that setting and getting the value is potentially non-atomic – use `value()` instead to make sure that access is automatically synchronized using a lock. Note that an array of `ctypes.c_char` has `value` and `draw` attributes which allow one to use it to store

`multiprocessing.sharedctypes.Array(typecode_or_type, size_or_initializer, *, lock=True, ctx=None)`

The same as `RawArray()` except that depending on the value of `lock` a process-safe synchronization wrapper may be returned instead of a raw ctypes array. If `lock` is `True` (the default) then a new lock object is created to synchronize access to the value. If `lock` is a `Lock` or `RLock` object then that will be used to synchronize access to the value. If `lock` is `False` then access to the returned object will not be automatically protected by a lock, so it will not necessarily be “process-safe”. `ctx` is a context object, or `None`

`multiprocessing.sharedctypes.Value(typecode_or_type, *args, lock=True, ctx=None)`

The same as `RawValue()` except that depending on the value of `lock` a process-safe synchronization wrapper may be returned instead of a raw ctypes object. If `lock` is `True` (the default) then a new lock object is created to synchronize access to the value. If `lock` is a `Lock` or `RLock` object then that will be used to synchronize access to the value. If `lock` is `False` then access to the returned object will not be automatically protected by a lock, so it will not necessarily be “process-safe”. `ctx` is a context object, or `None`

`multiprocessing.sharedctypes.copy(obj)`

Return a ctypes object allocated from shared memory which is a copy of the ctypes object `obj`.

`multiprocessing.sharedctypes.synchronized(obj, lock=None, ctx=None)`

Return a process-safe wrapper object for a ctypes object which uses `lock` to synchronize access. If `lock` is `None` (the default) then a `multiprocessing.RLock` object is created automatically. `ctx` is a context object, or `None` (use the current context). If `None`, calling this may set the global start method. See `Global start method` for more details. A synchronized wrapper will have two methods in addition to those of the object it wraps: `get_obj()` returns the wrapped object and `get_lock()` returns the lock object used for

The table below compares the syntax for creating shared ctypes objects from shared memory with the normal ctypes syntax. (In the table `MyStruct` is some subclass of `ctypes.Structure`.)

```
ctypes | sharedctypes using type | sharedctypes using typecode | c_double(2.4) |
RawValue(c_double, 2.4) | RawValue('d', 2.4) | MyStruct(4, 6) | RawValue(MyStruct, 4, 6) |
(c_short * 7)() | RawArray(c_short, 7) | RawArray('h', 7) | (c_int * 3)(9, 2, 8) |
RawArray(c_int, (9, 2, 8)) | RawArray('i', (9, 2, 8))
```

Below is an example where a number of ctypes objects are modified by a child process:

```
from multiprocessing import Process, Lock
```

```

from multiprocessing.sharedctypes import Value, Array
from ctypes import Structure, c_double

class Point(Structure):
    _fields_ = [('x', c_double), ('y', c_double)]

def modify(n, x, s, A):
    n.value **= 2
    x.value **= 2
    s.value = s.value.upper()
    for a in A:
        a.x **= 2
        a.y **= 2

if __name__ == '__main__':
    lock = Lock()

    n = Value('i', 7)
    x = Value(c_double, 1.0/3.0, lock=False)
    s = Array('c', b'hello world', lock=lock)
    A = Array(Point, [(1.875, -6.25), (-5.75, 2.0), (2.375, 9.5)], lock=lock)

    p = Process(target=modify, args=(n, x, s, A))
    p.start()
    p.join()

    print(n.value)
    print(x.value)
    print(s.value)
    print([(a.x, a.y) for a in A])

```

The results printed are

```

49
0.11111111111111111
HELLO WORLD
[(3.515625, 39.0625), (33.0625, 4.0), (5.640625, 90.25)]

```

Managers

Managers provide a way to create data which can be shared between different processes, including sharing over a network between processes running on different machines. A manager object controls a server process which manages shared objects. Other processes can access the shared objects by using proxies.

multiprocessing.Manager()

Returns a `SyncManager` object which can be used for sharing objects between processes. The returned manager object corresponds to a spawned child process and has methods which will create shared objects and return corresponding proxies.

Manager processes will be shutdown as soon as they are garbage collected or their parent process exits. The manager classes are defined in the `multiprocessing.managers` module:

```

class multiprocessing.managers.BaseManager(address=None, authkey=None, serializer='pickle', ctx=None, *, shutdown_timeout=1.0)

```

```

start([initializer[, initargs]])

```

```

get_server()

```

```

connect()

```

```

shutdown()

```

```

register(typeid[, callable[, proxytype[, exposed[, method_to_typeid[, create_method]]]])

```

address

Create a `BaseManager` object. Once created one should call `start()` or `get_server().serve_forever()` to ensure that the manager object refers to a started manager process. `address` is the address on which the manager process listens for new connections. If `address` is `None` then an arbitrary one is chosen. `authkey` is the authentication key which will be used to check the validity of incoming connections to the server process. If `authkey` is `None` then `current_process().authkey` is used. Otherwise `authkey` is used and it must

Start a subprocess to start the manager. If `initializer` is not `None` then the subprocess will call `initializer(*initargs)` when it starts.

Returns a `Server` object which represents the actual server under the control of the Manager. The `Server` object supports the `serve_forever()` method: >>>from multiprocessing.managers import BaseManager>>>manager=BaseManager(address=('',50000),authkey=b'abc')>>>server=manager.get_server()>>>server.serve_forever() `Server` additionally has an `address` attribute.

Connect a local manager object to a remote manager process: >>>from multiprocessing.managers import BaseManager>>>m=BaseManager(address=('127.0.0.1',50000),authkey=b'abc')>>>m.connect()

Stop the process used by the manager. This is only available if `start()` has been used to start the server process. This can be called multiple times.

A class method which can be used for registering a type or callable with the manager class. `typeid` is a "type identifier" which is used to identify a particular type of shared object. This must be a string. `callable` is a callable used for creating objects for this type identifier. If a manager instance will be connected to the server using the `connect()` method, or if the `create_method` argument is `False` then this can be left as `None`. `proxytype` is a subclass of `BaseProxy` which is used to create proxies for shar

The address used by the manager.

class multiprocessing.managers.SyncManager

Barrier(parties[,action[,timeout]])

BoundedSemaphore([value])

Condition([lock])

Event()

Lock()

Namespace()

Queue([maxsize])

RLock()

Semaphore([value])

Array(typecode,sequence)

Value(typecode,value)

dict()

dict(mapping)

dict(sequence)

list()

list(sequence)

set()

set(sequence)

set(mapping)

A subclass of `BaseManager` which can be used for the synchronization of processes. Objects of this type are returned by `multiprocessing.Manager()`. Its methods create and return Proxy Objects for a number of commonly used data types to be synchronized across processes. This notably includes shared lists and dictionaries. `Barrier(parties[,action[,timeout]])` Create a `sharedthreading.Barrier` object and return a proxy for it. Added in version 3.3. `BoundedSemaphore([value])` Create a `sharedthreading.BoundedSemaphore`

Create a `sharedthreading.Barrier` object and return a proxy for it. Added in version 3.3.

Create a `sharedthreading.BoundedSemaphore` object and return a proxy for it.

Create a `sharedthreading.Condition` object and return a proxy for it. If `lock` is supplied then it should be a proxy for a `threading.Lock` or `threading.RLock` object. Changed in version 3.3: The `wait_for()` method was added.

Create a `sharedthreading.Event` object and return a proxy for it.

Create a `sharedthreading.Lock` object and return a proxy for it.

Create a `sharedNamespace` object and return a proxy for it.

Create a `sharedqueue.Queue` object and return a proxy for it.

Create a `sharedthreading.RLock` object and return a proxy for it.

Create a `sharedthreading.Semaphore` object and return a proxy for it.

Create an array and return a proxy for it.

Create an object with a `writable` value attribute and return a proxy for it.

Create a `shareddict` object and return a proxy for it.

Create a `sharedlist` object and return a proxy for it.

Create a `sharedset` object and return a proxy for it. Added in version 3.14: `set` support was added.

class multiprocessing.managers.Namespace

A type that can register with `SyncManager`. A namespace object has no public methods, but does have writable attributes. Its representation shows the values of its attributes. However, when using a proxy for a namespace object, an attribute beginning with `'_'` will be an attribute of the proxy and not an attribute of the referent:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> Global = manager.Namespace()
>>> Global.x = 10
>>> Global.y = 'hello'
>>> Global._z = 12.3
# this is an attribute of the proxy, not the referent
```

Customized managers

To create one's own manager, one creates a subclass of `BaseManager` and uses the `register()` class method to register new types or callables with the manager class. For example:

```
from multiprocessing.managers import BaseManager
```

```
class MathsClass:
    def add(self, x, y):
        return x + y
    def mul(self, x, y):
        return x * y
```

```
class MyManager(BaseManager):
    pass
```

```
MyManager.register('Maths', MathsClass)
```

```
if __name__ == '__main__':
```

```

with MyManager() as manager:
    maths = manager.Maths()
    print(maths.add(4, 3))          # prints 7
    print(maths.mul(7, 8))         # prints 56

```

Using a remote manager

It is possible to run a manager server on one machine and have clients use it from other machines (assuming that the firewalls involved allow it).

Running the following commands creates a server for a single shared queue which remote clients can access:

```

>>> from multiprocessing.managers import BaseManager
>>> from queue import Queue
>>> queue = Queue()
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue', callable=lambda: queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra')
>>> s = m.get_server()
>>> s.serve_forever()

```

One client can access the server as follows:

```

>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'abracadabra')
>>> m.connect()
>>> queue = m.get_queue()
>>> queue.put('hello')

```

Another client can also use it:

```

>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'abracadabra')
>>> m.connect()
>>> queue = m.get_queue()
>>> queue.get()
'hello'

```

Local processes can also access that queue, using the code from above on the client to access it remotely:

```

>>> from multiprocessing import Process, Queue
>>> from multiprocessing.managers import BaseManager
>>> class Worker(Process):
...     def __init__(self, q):
...         self.q = q
...         super().__init__()
...     def run(self):
...         self.q.put('local hello')
...
>>> queue = Queue()
>>> w = Worker(queue)
>>> w.start()
>>> class QueueManager(BaseManager): pass
...
>>> QueueManager.register('get_queue', callable=lambda: queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra')
>>> s = m.get_server()
>>> s.serve_forever()

```

Proxy Objects

A proxy is an object which refers to a shared object which lives (presumably) in a different process. The shared object is said to be the referent of the proxy. Multiple proxy objects may have the same referent.

A proxy object has methods which invoke corresponding methods of its referent (although not every method of the referent will necessarily be available through the proxy). In this way, a proxy can be used just like its referent can:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> l = manager.list([i*i for i in range(10)])
>>> print(l)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> print(repr(l))
<ListProxy object, typeid 'list' at 0x...>
>>> l[4]
16
>>> l[2:5]
[4, 9, 16]
```

Notice that applying `str()` to a proxy will return the representation of the referent, whereas applying `repr()` will return the representation of the proxy.

An important feature of proxy objects is that they are picklable so they can be passed between processes. As such, a referent can contain Proxy Objects. This permits nesting of these managed lists, dicts, and other Proxy Objects:

```
>>> a = manager.list()
>>> b = manager.list()
>>> a.append(b)           # referent of a now contains referent of b
>>> print(a, b)
[<ListProxy object, typeid 'list' at ...>] []
>>> b.append('hello')
>>> print(a[0], b)
['hello'] ['hello']
```

Similarly, dict and list proxies may be nested inside one another:

```
>>> l_outer = manager.list([ manager.dict() for i in range(2) ])
>>> d_first_inner = l_outer[0]
>>> d_first_inner['a'] = 1
>>> d_first_inner['b'] = 2
>>> l_outer[1]['c'] = 3
>>> l_outer[1]['z'] = 26
>>> print(l_outer[0])
{'a': 1, 'b': 2}
>>> print(l_outer[1])
{'c': 3, 'z': 26}
```

If standard (non-proxy) list or dict objects are contained in a referent, modifications to those mutable values will not be propagated through the manager because the proxy has no way of knowing when the values contained within are modified. However, storing a value in a container proxy (which triggers a `__setitem__` on the proxy object) does propagate through the manager and so to effectively modify such an item, one could re-assign the modified value to the container proxy:

```
# create a list proxy and append a mutable object (a dictionary)
lproxy = manager.list()
lproxy.append({})
# now mutate the dictionary
d = lproxy[0]
d['a'] = 1
d['b'] = 2
# at this point, the changes to d are not yet synced, but by
# updating the dictionary, the proxy is notified of the change
lproxy[0] = d
```

This approach is perhaps less convenient than employing nestedProxy Objects for most use cases but also demonstrates a level of control over the synchronization.

■ Nota: NoteThe proxy types in multiprocessing do nothing to support comparisons by value. So, for instance, we have: >>> manager.list([1,2,3]) == [1,2,3] False One should just use a copy of the referent instead when making comparisons.

class multiprocessing.managers.BaseProxy

_callmethod(methodname[,args[,kwargs]])

_getvalue()

__repr__()

__str__()

Proxy objects are instances of subclasses of BaseProxy. **_callmethod(methodname[,args[,kwargs]])** Call and return the result of a method of the proxy's referent. If proxy is a proxy whose referent is obj then the expression proxy._callmethod(methodname,args,kwds) will evaluate the expression getattr(obj,methodname)(*args,**kwds) in the manager's process. The returned value will be a copy of the result of the call or a proxy to a new shared object – see documentation for the **method_to_typeid** argument of BaseManager.

Call and return the result of a method of the proxy's referent. If proxy is a proxy whose referent is obj then the expression proxy._callmethod(methodname,args,kwds) will evaluate the expression getattr(obj,methodname)(*args,**kwds) in the manager's process. The returned value will be a copy of the result of the call or a proxy to a new shared object – see documentation for the **method_to_typeid** argument of BaseManager.register(). If an exception is raised by the call, then is re-raised by _callmethod(). If so

Return a copy of the referent. If the referent is unpicklable then this will raise an exception.

Return a representation of the proxy object.

Return the representation of the referent.

Cleanup

A proxy object uses a weakref callback so that when it gets garbage collected it deregisters itself from the manager which owns its referent.

A shared object gets deleted from the manager process when there are no longer any proxies referring to it.

Process Pools

One can create a pool of processes which will carry out tasks submitted to it with the Pool class.

class multiprocessing.pool.Pool([processes[,initializer[,initargs[,maxtasksperchild[,context]]]])

apply(func[,args[,kwargs]])

apply_async(func[,args[,kwargs[,callback[,error_callback]]])

map(func,iterable[,chunksize])

map_async(func,iterable[,chunksize[,callback[,error_callback]]])

imap(func,iterable[,chunksize])

imap_unordered(func,iterable[,chunksize])

starmap(func,iterable[,chunksize])

starmap_async(func,iterable[,chunksize[,callback[,error_callback]]])

close()

terminate()

join()

A process pool object which controls a pool of worker processes to which jobs can be submitted. It supports asynchronous results with timeouts and callbacks and has a parallel map implementation. `processes` is the number of worker processes to use. If `processes` is `None` then the number returned by `os.process_cpu_count()` is used. If `initializer` is not `None` then each worker process will call `initializer(*initargs)` when it starts. `maxtasksperchild` is the number of tasks a worker process can complete before it will

Call `func` with arguments `sargs` and keyword arguments `skwds`. It blocks until the result is ready. Given this blocks, `apply_async()` is better suited for performing work in parallel. Additionally, `func` is only executed in one of the workers of the pool.

A variant of the `apply()` method which returns a `AsyncResult` object. If `callback` is specified then it should be a callable which accepts a single argument. When the result becomes ready `callback` is applied to it, that is unless the call failed, in which case `error_callback` is applied instead. If `error_callback` is specified then it should be a callable which accepts a single argument. If the target function fails, then `error_callback` is called with the exception instance. Callbacks should complete immedi

A parallel equivalent of the `map()` built-in function (it supports only one iterable argument though, for multiple iterables see `starmap()`). It blocks until the result is ready. This method chops the iterable into a number of chunks which it submits to the process pool as separate tasks. The (approximate) size of these chunks can be specified by setting `chunks` to a positive integer. Note that it may cause high memory usage for very long iterables. Consider using `imap()` or `imap_unordered()` with explicit `chunks`

A variant of the `map()` method which returns a `AsyncResult` object. If `callback` is specified then it should be a callable which accepts a single argument. When the result becomes ready `callback` is applied to it, that is unless the call failed, in which case `error_callback` is applied instead. If `error_callback` is specified then it should be a callable which accepts a single argument. If the target function fails, then `error_callback` is called with the exception instance. Callbacks should complete immediat

A lazier version of `map()`. The `chunks` argument is the same as the one used by the `map()` method. For very long iterables using a large value for `chunks` can make the job complete much faster than using the default value of 1. Also if `chunks` is 1 then the `next()` method of the iterator returned by the `imap()` method has an optional `timeout` parameter: `next(timeout)` will raise `multiprocessing.TimeoutError` if the result cannot be returned within `timeout` seconds.

The same as `imap()` except that the ordering of the results from the returned iterator should be considered arbitrary. (Only when there is only one worker process is the order guaranteed to be "correct".)

Like `map()` except that the elements of the iterable are expected to be iterables that are unpacked as arguments. Hence `an_iterable_of((1,2),(3,4))` results in `[func(1,2), func(3,4)]`. Added in version 3.3.

A combination of `starmap()` and `map_async()` that iterates over iterable of iterables and calls `func` with the iterables unpacked. Returns a result object. Added in version 3.3.

Prevents any more tasks from being submitted to the pool. Once all the tasks have been completed the worker processes will exit.

Stops the worker processes immediately without completing outstanding work. When the pool object is garbage collected `terminate()` will be called immediately.

Wait for the worker processes to exit. One must call `close()` or `terminate()` before using `join()`.

class multiprocessing.pool.AsyncResult

get([timeout])

wait([timeout])

ready()

successful()

The class of the result returned by `Pool.apply_async()` and `Pool.map_async().get([timeout])` Return the result when it arrives. If `timeout` is not `None` and the result does not arrive within `timeout` seconds then `multiprocessing.TimeoutError` is raised. If the remote call raised an exception then that exception will be reraised by `get().wait([timeout])`. Wait until the result is available or until `timeout` seconds pass. `ready()` Return whether the call has completed. `successful()` Return whether the call completed without r

Return the result when it arrives. If `timeout` is not `None` and the result does not arrive within `timeout` seconds then `multiprocessing.TimeoutError` is raised. If the remote call raised an exception then that exception will be reraised by `get()`.

Wait until the result is available or until `timeout` seconds pass.

Return whether the call has completed.

Return whether the call completed without raising an exception. Will raise `ValueError` if the result is not ready. Changed in version 3.7: If the result is not ready, `ValueError` is raised instead of `AssertionError`.

The following example demonstrates the use of a pool:

```
from multiprocessing import Pool
import time

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(processes=4) as pool:
        # start 4 worker processes
        result = pool.apply_async(f, (10,)) # evaluate "f(10)" asynchronously in a single process
        print(result.get(timeout=1))        # prints "100" unless your computer is *very* slow

        print(pool.map(f, range(10)))      # prints "[0, 1, 4, ..., 81]"

        it = pool.imap(f, range(10))
        print(next(it))                    # prints "0"
        print(next(it))                    # prints "1"
        print(it.next(timeout=1))           # prints "4" unless your computer is *very* slow

        result = pool.apply_async(time.sleep, (10,))
        print(result.get(timeout=1))        # raises multiprocessing.TimeoutError
```

Listeners and Clients

Usually message passing between processes is done using queues or by using `Connection` objects returned by `Pipe()`.

However, the `multiprocessing.connection` module allows some extra flexibility. It basically gives a high level message oriented API for dealing with sockets or Windows named pipes. It also has support for digest authentication using the `hmac` module, and for polling multiple connections at the same time.

`multiprocessing.connection.deliver_challenge(connection, authkey)`

Send a randomly generated message to the other end of the connection and wait for a reply. If the reply matches the digest of the message using `authkey` as the key then a welcome message is sent to the other end of the connection. Otherwise `AuthenticationError` is raised.

`multiprocessing.connection.answer_challenge(connection, authkey)`

Receive a message, calculate the digest of the message using `authkey` as the key, and then send the digest back. If a welcome message is not received, then `AuthenticationError` is raised.

`multiprocessing.connection.Client(address[,family[,authkey]])`

Attempt to set up a connection to the listener which is using `address`, returning a `Connection`. The type of the connection is determined by `family` argument, but this can generally be omitted since it can usually be inferred from the format of `address`. (See `Address Formats`) If `authkey` is given and not `None`, it should be a byte string and will be used as the secret key for an HMAC-based authentication challenge. No authentication is done if `authkey` is `None`. `AuthenticationError` is raised if authentication f

`class multiprocessing.connection.Listener([address[,family[,backlog[,authkey]]]])`

`accept()`

`close()`

`address`

`last_accepted`

A wrapper for a bound socket or Windows named pipe which is 'listening' for connections. `address` is the address to be used by the bound socket or named pipe of the listener object. Note if an address of '0.0.0.0' is used, the address will not be a connectable end point on Windows. If you require a connectable end-point, you should use '127.0.0.1'. `family` is the type of socket (or named pipe) to use. This can be one of the strings 'AF_INET' (for a TCP socket), 'AF_UNIX' (for a Unix domain socket) or 'AF_PIPE'.

Accept a connection on the bound socket or named pipe of the listener object and return a `Connection` object. If authentication is attempted and fails, then `AuthenticationError` is raised.

Close the bound socket or named pipe of the listener object. This is called automatically when the listener is garbage collected. However it is advisable to call it explicitly.

The address which is being used by the Listener object.

The address from which the last accepted connection came. If this is unavailable then it is `None`.

`multiprocessing.connection.wait(object_list, timeout=None)`

Wait till an object in `object_list` is ready. Returns the list of those objects in `object_list` which are ready. If `timeout` is a float then the call blocks for at most that many seconds. If `timeout` is `None` then it will block for an unlimited period. A negative `timeout` is equivalent to a zero `timeout`. For both POSIX and Windows, an object can appear in `object_list` if it is a readable `Connection` object; a connected and readable socket object; or the `sentinel` attribute of a `Process` object. A connection or socket ob

Examples

The following server code creates a listener which uses 'secret password' as an authentication key. It then waits for a connection and sends some data to the client:

```
from multiprocessing.connection import Listener
from array import array

address = ('localhost', 6000)    # family is deduced to be 'AF_INET'

with Listener(address, authkey=b'secret password') as listener:
    with listener.accept() as conn:
        print('connection accepted from', listener.last_accepted)

        conn.send([2.25, None, 'junk', float])

        conn.send_bytes(b'hello')

        conn.send_bytes(array('i', [42, 1729]))
```

The following code connects to the server and receives some data from the server:

```
from multiprocessing.connection import Client
from array import array

address = ('localhost', 6000)

with Client(address, authkey=b'secret password') as conn:
    print(conn.recv())          # => [2.25, None, 'junk', float]

    print(conn.recv_bytes())    # => 'hello'

    arr = array('i', [0, 0, 0, 0, 0])
    print(conn.recv_bytes_into(arr))  # => 8
    print(arr)                  # => array('i', [42, 1729, 0, 0, 0])
```

The following code uses `wait()` to wait for messages from multiple processes at once:

```
from multiprocessing import Process, Pipe, current_process
from multiprocessing.connection import wait

def foo(w):
    for i in range(10):
        w.send((i, current_process().name))
    w.close()

if __name__ == '__main__':
    readers = []

    for i in range(4):
        r, w = Pipe(duplex=False)
        readers.append(r)
        p = Process(target=foo, args=(w,))
        p.start()
        # We close the writable end of the pipe now to be sure that
        # p is the only process which owns a handle for it. This
        # ensures that when p closes its handle for the writable end,
        # wait() will promptly report the readable end as being ready.
        w.close()

    while readers:
        for r in wait(readers):
            try:
                msg = r.recv()
            except EOFError:
                readers.remove(r)
            else:
                print(msg)
```

Address Formats

- An `'AF_INET'` address is a tuple of the form `(hostname, port)` where `hostname` is a string and `port` is an integer.
- An `'AF_UNIX'` address is a string representing a filename on the filesystem.
- An `'AF_PIPE'` address is a string of the form `r'\\.\pipe\PipeName'`. To use `Client()` to connect to a named pipe on a remote computer called `ServerName` one should use an address of the form `r'\\ServerName\pipe\PipeName'` instead.

Note that any string beginning with two backslashes is assumed by default to be an `'AF_PIPE'` address rather than an `'AF_UNIX'` address.

Authentication keys

When one uses `Connection.recv`, the data received is automatically unpickled. Unfortunately unpickling data from an untrusted source is a security risk. Therefore `Listener` and `Client` use the `macmodule` to provide digest authentication.

An authentication key is a byte string which can be thought of as a password: once a connection is established both ends will demand proof that the other knows the authentication key. (Demonstrating that both ends are using the same key does not involve sending the key over the connection.)

If authentication is requested but no authentication key is specified then the return value of `current_process().authkey` is used (see `Process`). This value will be automatically inherited by any `Process` object that the current process creates. This means that (by default) all processes of a multi-process program will share a single authentication key which can be used when setting up connections between themselves.

Suitable authentication keys can also be generated by using `os.urandom()`.

Logging

Some support for logging is available. Note, however, that the `logging` package does not use process shared locks so it is possible (depending on the handler type) for messages from different processes to get mixed up.

`multiprocessing.get_logger()`

Returns the logger used by `multiprocessing`. If necessary, a new one will be created. When first created the logger has level `logging.NOTSET` and no default handler. Messages sent to this logger will not by default propagate to the root logger. Note that on Windows child processes will only inherit the level of the parent process's logger – any other customization of the logger will not be inherited.

`multiprocessing.log_to_stderr(level=None)`

This function performs a call to `get_logger()` but in addition to returning the logger created by `get_logger`, it adds a handler which sends output to `sys.stderr` using format `'%(levelname)s/%(processName)s'%(message)s'`. You can modify `levelname` of the logger by passing `level` argument.

Below is an example session with logging turned on:

```
>>> import multiprocessing, logging
>>> logger = multiprocessing.log_to_stderr()
>>> logger.setLevel(logging.INFO)
>>> logger.warning('doomed')
[WARNING/MainProcess] doomed
>>> m = multiprocessing.Manager()
[INFO/SyncManager-...] child process calling self.run()
[INFO/SyncManager-...] created temp directory /.../pypm-...
[INFO/SyncManager-...] manager serving at '/.../listener-...'
>>> del m
[INFO/MainProcess] sending shutdown message to manager
[INFO/SyncManager-...] manager exiting with exitcode 0
```

For a full table of logging levels, see the `logging` module.

The `multiprocessing.dummy` module

`multiprocessing.dummy` replicates the API of `multiprocessing` but is no more than a wrapper around the `threading` module.

In particular, the `Pool` function provided by `multiprocessing.dummy` returns an instance of `ThreadPool`, which is a subclass of `Pool` that supports all the same method calls but uses a pool of worker threads rather than worker processes.

`class multiprocessing.pool.ThreadPool([processes[, initializer[, initargs]])`

A thread pool object which controls a pool of worker threads to which jobs can be submitted. `ThreadPool` instances are fully interface compatible with `Pool` instances, and their resources must also be properly managed, either by using the pool as a context manager or by calling `close()` and `terminate()` manually. `processes` is the number of worker threads to use. If `processes` is `None` then the number returned by `os.process_cpu_count()` is used. If `initializer` is not `None` then each worker process will call `initializer(*init`

Programming guidelines

There are certain guidelines and idioms which should be adhered to when using multiprocessing.

All start methods

The following applies to all start methods.

Avoid shared state

As far as possible one should try to avoid shifting large amounts of data between processes.

It is probably best to stick to using queues or pipes for communication between processes rather than using the lower level synchronization primitives.

Picklability

Ensure that the arguments to the methods of proxies are picklable.

Thread safety of proxies

Do not use a proxy object from more than one thread unless you protect it with a lock.

(There is never a problem with different processes using the same proxy.)

Joining zombie processes

On POSIX when a process finishes but has not been joined it becomes a zombie. There should never be very many because each time a new process starts (or `active_children()` is called) all completed processes which have not yet been joined will be joined. Also calling a finished process's `Process.is_alive()` will join the process. Even so it is probably good practice to explicitly join all the processes that you start.

Better to inherit than pickle/unpickle

When using the `spawn` or `forkserver` start methods many types from multiprocessing need to be picklable so that child processes can use them. However, one should generally avoid sending shared objects to other processes using pipes or queues. Instead you should arrange the program so that a process which needs access to a shared resource created elsewhere can inherit it from an ancestor process.

Avoid terminating processes

Using the `Process.terminate()` method to stop a process is liable to cause any shared resources (such as locks, semaphores, pipes and queues) currently being used by the process to become broken or unavailable to other processes.

Therefore it is probably best to only consider using `Process.terminate()` on processes which never use any shared resources.

Joining processes that use queues

Bear in mind that a process that has put items in a queue will wait before terminating until all the buffered items are fed by the “feeder” thread to the underlying pipe. (The child process can call the `Queue.cancel_join_thread()` method of the queue to avoid this behaviour.)

This means that whenever you use a queue you need to make sure that all items which have been put on the queue will eventually be removed before the process is joined. Otherwise you cannot be sure that processes which have put items on the queue will terminate. Remember also that non-daemonic processes will be joined automatically.

An example which will deadlock is the following:

```
from multiprocessing import Process, Queue

def f(q):
    q.put('X' * 1000000)

if __name__ == '__main__':
    queue = Queue()
    p = Process(target=f, args=(queue,))
    p.start()
    p.join()                # this deadlocks
    obj = queue.get()
```

A fix here would be to swap the last two lines (or simply remove the `p.join()` line).

Explicitly pass resources to child processes

On POSIX using the `forkstart` method, a child process can make use of a shared resource created in a parent process using a global resource. However, it is better to pass the object as an argument to the constructor for the child process.

Apart from making the code (potentially) compatible with Windows and the other start methods this also ensures that as long as the child process is still alive the object will not be garbage collected in the parent process. This might be important if some resource is freed when the object is garbage collected in the parent process.

So for instance

```
from multiprocessing import Process, Lock

def f():
    ... do something using "lock" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f).start()
```

should be rewritten as

```
from multiprocessing import Process, Lock

def f(l):
    ... do something using "l" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f, args=(lock,)).start()
```

Beware of replacing `sys.stdin` with a “file like object”

`multiprocessing` originally unconditionally called:

```
os.close(sys.stdin.fileno())
```

in the `multiprocessing.Process._bootstrap()` method — this resulted in issues with processes-in-processes. This has been changed to:

```
sys.stdin.close()
sys.stdin = open(os.devnull, os.O_RDONLY, closefd=False)
```

Which solves the fundamental issue of processes colliding with each other resulting in a bad file descriptor error, but introduces a potential danger to applications which replaces `sys.stdin()` with a “file-like object” with output buffering. This danger is that if multiple processes call `close()` on this file-like object, it could result in the same data being flushed to the object multiple times, resulting in corruption.

If you write a file-like object and implement your own caching, you can make it fork-safe by storing the pid whenever you append to the cache, and discarding the cache when the pid changes. For example:

```
@property
def cache(self):
    pid = os.getpid()
    if pid != self._pid:
        self._pid = pid
        self._cache = []
    return self._cache
```

For more information, see [bpo-5155](#), [bpo-5313](#) and [bpo-5331](#)

Thespawnandforkserverstart methods

There are a few extra restrictions which don't apply to the `forkstart` method.

More picklability

Ensure that all arguments to `Process` are picklable. Also, if you subclass `Process.__init__`, you must make sure that instances will be picklable when the `Process.start` method is called.

Global variables

Bear in mind that if code run in a child process tries to access a global variable, then the value it sees (if any) may not be the same as the value in the parent process at the time that `Process.start` was called.

However, global variables which are just module level constants cause no problems.

Safe importing of main module

Make sure that the main module can be safely imported by a new Python interpreter without causing unintended side effects (such as starting a new process).

For example, using the `spawn` or `forkserver` start method running the following module would fail with a `RuntimeError`:

```
from multiprocessing import Process

def foo():
    print('hello')

p = Process(target=foo)
p.start()
```

Instead one should protect the “entry point” of the program by using `if __name__ == '__main__':` as follows:

```
from multiprocessing import Process, freeze_support, set_start_method

def foo():
    print('hello')

if __name__ == '__main__':
    freeze_support()
    set_start_method('spawn')
    p = Process(target=foo)
    p.start()
```

(The `freeze_support()` line can be omitted if the program will be run normally instead of frozen.)

This allows the newly spawned Python interpreter to safely import the module and then run the module'sfoo()function.

Similar restrictions apply if a pool or manager is created in the main module.

Examples

Demonstration of how to create and use customized managers and proxies:

```
from multiprocessing import freeze_support
from multiprocessing.managers import BaseManager, BaseProxy
import operator

##

class Foo:
    def f(self):
        print('you called Foo.f()')
    def g(self):
        print('you called Foo.g()')
    def _h(self):
        print('you called Foo._h()')

# A simple generator function
def baz():
    for i in range(10):
        yield i*i

# Proxy type for generator objects
class GeneratorProxy(BaseProxy):
    _exposed_ = ['__next__']
    def __iter__(self):
        return self
    def __next__(self):
        return self._callmethod('__next__')

# Function to return the operator module
def get_operator_module():
    return operator

##

class MyManager(BaseManager):
    pass

# register the Foo class; make `f()` and `g()` accessible via proxy
MyManager.register('Foo1', Foo)

# register the Foo class; make `g()` and `_h()` accessible via proxy
MyManager.register('Foo2', Foo, exposed=('g', '_h'))

# register the generator function baz; use `GeneratorProxy` to make proxies
MyManager.register('baz', baz, proxytype=GeneratorProxy)

# register get_operator_module(); make public functions accessible via proxy
MyManager.register('operator', get_operator_module)

##

def test():
    manager = MyManager()
    manager.start()
```

```

print('-' * 20)

f1 = manager.Foo1()
f1.f()
f1.g()
assert not hasattr(f1, '_h')
assert sorted(f1._exposed_) == sorted(['f', 'g'])

print('-' * 20)

f2 = manager.Foo2()
f2.g()
f2._h()
assert not hasattr(f2, 'f')
assert sorted(f2._exposed_) == sorted(['g', '_h'])

print('-' * 20)

it = manager.baz()
for i in it:
    print('< %d>' % i, end=' ')
print()

print('-' * 20)

op = manager.operator()

```

UsingPool:

```

import multiprocessing
import time
import random
import sys

#
# Functions used by test code
#

def calculate(func, args):
    result = func(*args)
    return '%s says that %s%s = %s' % (
        multiprocessing.current_process().name,
        func.__name__, args, result
    )

def calculatestar(args):
    return calculate(*args)

def mul(a, b):
    time.sleep(0.5 * random.random())
    return a * b

def plus(a, b):
    time.sleep(0.5 * random.random())
    return a + b

def f(x):
    return 1.0 / (x - 5.0)

def pow3(x):
    return x ** 3

def noop(x):
    pass

```

```

#
# Test code
#

def test():
    PROCESSES = 4
    print('Creating pool with %d processes\n' % PROCESSES)

    with multiprocessing.Pool(PROCESSES) as pool:
        #
        # Tests
        #

        TASKS = [(mul, (i, 7)) for i in range(10)] + \
            [(plus, (i, 8)) for i in range(10)]

        results = [pool.apply_async(calculate, t) for t in TASKS]
        imap_it = pool.imap(calculatestar, TASKS)
        imap_unordered_it = pool.imap_unordered(calculatestar, TASKS)

        print('Ordered results using pool.apply_async():')
        for r in results:
            print('\t', r.get())
        print()

        print('Ordered results using pool.imap():')
        for x in imap_it:
            print('\t', x)
        print()

        print('Unordered results using pool.imap_unordered():')
        for x in imap_unordered_it:
            print('\t', x)
        print()

        print('Ordered results using pool.map() --- will block till complete:')
        for x in pool.map(calculatestar, TASKS):
            print('\t', x)
        print()

        #
        # Test error handling
        #

```

An example showing how to use queues to feed tasks to a collection of worker processes and collect the results:

```

import time
import random

from multiprocessing import Process, Queue, current_process, freeze_support

#
# Function run by worker processes
#

def worker(input, output):
    for func, args in iter(input.get, 'STOP'):
        result = calculate(func, args)
        output.put(result)

#
# Function used to calculate result
#

def calculate(func, args):

```

```

    result = func(*args)
    return '%s says that %s%s = %s' % \
        (current_process().name, func.__name__, args, result)

#
# Functions referenced by tasks
#

def mul(a, b):
    time.sleep(0.5*random.random())
    return a * b

def plus(a, b):
    time.sleep(0.5*random.random())
    return a + b

#
#
#

def test():
    NUMBER_OF_PROCESSES = 4
    TASKS1 = [(mul, (i, 7)) for i in range(20)]
    TASKS2 = [(plus, (i, 8)) for i in range(10)]

    # Create queues
    task_queue = Queue()
    done_queue = Queue()

    # Submit tasks
    for task in TASKS1:
        task_queue.put(task)

    # Start worker processes
    for i in range(NUMBER_OF_PROCESSES):
        Process(target=worker, args=(task_queue, done_queue)).start()

    # Get and print results
    print('Unordered results:')
    for i in range(len(TASKS1)):
        print('\t', done_queue.get())

    # Add more tasks using `put()`
    for task in TASKS2:
        task_queue.put(task)

    # Get and print some more results
    for i in range(len(TASKS2)):
        print('\t', done_queue.get())

    # Tell child processes to stop
    for i in range(NUMBER_OF_PROCESSES):
        task_queue.put('STOP')

if __name__ == '__main__':
    freeze_support()
    test()

```

multiprocessing.shared_memory— Shared memory for direct access across processes

Source code: `Lib/multiprocessing/shared_memory.py`

Added in version 3.8.

This module provides a class, `SharedMemory`, for the allocation and management of shared memory to be accessed by one or more processes on a multicore or symmetric multiprocessor (SMP) machine. To assist with the life-cycle management of shared memory especially across distinct processes, a `BaseManager` subclass, `SharedMemoryManager`, is also provided in the `multiprocessing.managers` module.

In this module, shared memory refers to “POSIX style” shared memory blocks (though is not necessarily implemented explicitly as such) and does not refer to “distributed shared memory”. This style of shared memory permits distinct processes to potentially read and write to a common (or shared) region of volatile memory. Processes are conventionally limited to only have access to their own process memory space but shared memory permits the sharing of data between processes, avoiding the need to instead send messages between processes containing that data. Sharing data directly via memory can provide significant performance benefits compared to sharing data via disk or socket or other communications requiring the serialization/deserialization and copying of data.

`class multiprocessing.shared_memory.SharedMemory(name=None, create=False, size=0, *, track=True)`

Parameters:

`close()`

`unlink()`

`buf`

`name`

`size`

Create an instance of the `SharedMemory` class for either creating a new shared memory block or attaching to an existing shared memory block. Each shared memory block is assigned a unique name. In this way, one process can create a shared memory block with a particular name and a different process can attach to that same shared memory block using that same name. As a resource for sharing data across processes, shared memory blocks may outlive the original process that created them. When one process

`name(str|None)` – The unique name for the requested shared memory, specified as a string. When creating a new shared memory block, if `None` (the default) is supplied for the name, a novel name will be generated. `create(bool)` – Control whether a new shared memory block is created (`True`) or an existing shared memory block is attached (`False`). `size(int)` – The requested number of bytes when creating a new shared memory block. Because some platforms choose to allocate chunks of memory based upon that platf

Close the file descriptor/handle to the shared memory from this instance. `close()` should be called once access to the shared memory block from this instance is no longer needed. Depending on operating system, the underlying memory may or may not be freed even if all handles to it have been closed. To ensure proper cleanup, use the `unlink()` method.

Delete the underlying shared memory block. This should be called only once per shared memory block regardless of the number of handles to it, even in other processes. `unlink()` and `close()` can be called in any order, but trying to access data inside a shared memory block after `unlink()` may result in memory access errors, depending on platform. This method has no effect on Windows, where the only way to delete a shared memory block is to close all handles.

A memoryview of contents of the shared memory block.

Read-only access to the unique name of the shared memory block.

Read-only access to size in bytes of the shared memory block.

The following example demonstrates low-level use of SharedMemory instances:

```
>>> from multiprocessing import shared_memory
>>> shm_a = shared_memory.SharedMemory(create=True, size=10)
>>> type(shm_a.buf)
<class 'memoryview'>
>>> buffer = shm_a.buf
>>> len(buffer)
10
>>> buffer[4] = bytearray([22, 33, 44, 55]) # Modify multiple at once
>>> buffer[4] = 100 # Modify single byte at a time
>>> # Attach to an existing shared memory block
>>> shm_b = shared_memory.SharedMemory(shm_a.name)
>>> import array
>>> array.array('b', shm_b.buf[:5]) # Copy the data into a new array.array
array('b', [22, 33, 44, 55, 100])
>>> shm_b.buf[:5] = b'howdy' # Modify via shm_b using bytes
>>> bytes(shm_a.buf[:5]) # Access via shm_a
b'howdy'
>>> shm_b.close() # Close each SharedMemory instance
>>> shm_a.close()
>>> shm_a.unlink() # Call unlink only once to release the shared memory
```

The following example demonstrates a practical use of the SharedMemory class with NumPy arrays, accessing the same numpy.ndarray from two distinct Python shells:

```
>>> # In the first Python interactive shell
>>> import numpy as np
>>> a = np.array([1, 1, 2, 3, 5, 8]) # Start with an existing NumPy array
>>> from multiprocessing import shared_memory
>>> shm = shared_memory.SharedMemory(create=True, size=a.nbytes)
>>> # Now create a NumPy array backed by shared memory
>>> b = np.ndarray(a.shape, dtype=a.dtype, buffer=shm.buf)
>>> b[:] = a[:] # Copy the original data into shared memory
>>> b
array([1, 1, 2, 3, 5, 8])
>>> type(b)
<class 'numpy.ndarray'>
>>> type(a)
<class 'numpy.ndarray'>
>>> shm.name # We did not specify a name so one was chosen for us
'psm_21467_46075'

>>> # In either the same shell or a new Python shell on the same machine
>>> import numpy as np
>>> from multiprocessing import shared_memory
>>> # Attach to the existing shared memory block
>>> existing_shm = shared_memory.SharedMemory(name='psm_21467_46075')
>>> # Note that a.shape is (6,) and a.dtype is np.int64 in this example
>>> c = np.ndarray((6,), dtype=np.int64, buffer=existing_shm.buf)
>>> c
array([1, 1, 2, 3, 5, 8])
>>> c[-1] = 888
>>> c
array([ 1,  1,  2,  3,  5, 888])

>>> # Back in the first Python interactive shell, b reflects this change
>>> b
array([ 1,  1,  2,  3,  5, 888])
```

```
>>> # Clean up from within the second Python shell
>>> del c # Unnecessary; merely emphasizing the array is no longer used
>>> existing_shm.close()

>>> # Clean up from within the first Python shell
>>> del b # Unnecessary; merely emphasizing the array is no longer used
>>> shm.close()
>>> shm.unlink() # Free and release the shared memory block at the very end
```

class multiprocessing.managers.SharedMemoryManager([address[, authkey]])

SharedMemory(size)

ShareableList(sequence)

A subclass of `multiprocessing.managers.BaseManager` which can be used for the management of shared memory blocks across processes. A call to `start()` on a `SharedMemoryManager` instance causes a new process to be started. This new process's sole purpose is to manage the life cycle of all shared memory blocks created through it. To trigger the release of all shared memory blocks managed by that process, call `shutdown()` on the instance. This triggers a `unlink()` call on all of the `SharedMemory` objects managed by it.

Create and return a new `SharedMemory` object with the specified size in bytes.

Create and return a new `ShareableList` object, initialized by the values from the input sequence.

The following example demonstrates the basic mechanisms of a `SharedMemoryManager`:

```
>>> from multiprocessing.managers import SharedMemoryManager
>>> smm = SharedMemoryManager()
>>> smm.start() # Start the process that manages the shared memory blocks
>>> sl = smm.ShareableList(range(4))
>>> sl
ShareableList([0, 1, 2, 3], name='psm_6572_7512')
>>> raw_shm = smm.SharedMemory(size=128)
>>> another_sl = smm.ShareableList('alpha')
>>> another_sl
ShareableList(['a', 'l', 'p', 'h', 'a'], name='psm_6572_12221')
>>> smm.shutdown() # Calls unlink() on sl, raw_shm, and another_sl
```

The following example depicts a potentially more convenient pattern for using `SharedMemoryManager` objects via the `with` statement to ensure that all shared memory blocks are released after they are no longer needed:

```
>>> with SharedMemoryManager() as smm:
...     sl = smm.ShareableList(range(2000))
...     # Divide the work among two processes, storing partial results in sl
...     p1 = Process(target=do_work, args=(sl, 0, 1000))
...     p2 = Process(target=do_work, args=(sl, 1000, 2000))
...     p1.start()
...     p2.start() # A multiprocessing.Pool might be more efficient
...     p1.join()
...     p2.join() # Wait for all work to complete in both processes
...     total_result = sum(sl) # Consolidate the partial results now in sl
```

When using a `SharedMemoryManager` in a `with` statement, the shared memory blocks created using that manager are all released when the `with` statement's code block finishes execution.

class multiprocessing.shared_memory.ShareableList(sequence=None, *, name=None)

count(value)

index(value)

format

shm

Provide a mutable list-like object where all values stored within are stored in a shared memory block. This constrains storable values to the following built-in data types: `int` (signed 64-bit), `float`, `bool`, `str` (less than 10M bytes each when encoded as UTF-8), `bytes` (less than 10M bytes each), `None`. It also notably differs from the built-in `list` type in that these lists can not change their overall length (i.e. `noappend()`, `insert()`, etc.) and do not support the dynamic creation of new `ShareableList` instances via `sli`.

Return the number of occurrences of `value`.

Return first index position of `value`. Raise `ValueError` if `value` is not present.

Read-only attribute containing the `struct` packing format used by all currently stored values.

The `SharedMemory` instance where the values are stored.

The following example demonstrates basic use of a `ShareableList` instance:

```
>>> from multiprocessing import shared_memory
>>> a = shared_memory.ShareableList(['howdy', b'Howdy', -273.154, 100, None, True, 42])
>>> [ type(entry) for entry in a ]
[<class 'str'>, <class 'bytes'>, <class 'float'>, <class 'int'>, <class 'NoneType'>, <class 'bool'>, <class 'int'>]
>>> a[2]
-273.154
>>> a[2] = -78.5
>>> a[2]
-78.5
>>> a[2] = 'dry ice' # Changing data types is supported as well
>>> a[2]
'dry ice'
>>> a[2] = 'larger than previously allocated storage space'
Traceback (most recent call last):
...
ValueError: exceeds available storage for existing str
>>> a[2]
'dry ice'
>>> len(a)
7
>>> a.index(42)
6
>>> a.count(b'howdy')
0
>>> a.count(b'Howdy')
1
>>> a.shm.close()
>>> a.shm.unlink()
>>> del a # Use of a ShareableList after call to unlink() is unsupported
```

The following example depicts how one, two, or many processes may access the same `ShareableList` by supplying the name of the shared memory block behind it:

```
>>> b = shared_memory.ShareableList(range(5)) # In a first process
>>> c = shared_memory.ShareableList(name=b.shm.name) # In a second process
>>> c
ShareableList([0, 1, 2, 3, 4], name='...')
>>> c[-1] = -999
>>> b[-1]
-999
>>> b.shm.close()
>>> c.shm.close()
>>> c.shm.unlink()
```

The following examples demonstrate that `ShareableList` (and underlying `SharedMemory`) objects can be pickled and unpickled if needed. Note, that it will still be the same shared object. This happens, because the deserialized object has the same unique name and is just attached to an existing object with the same name (if the object is still alive):

```
>>> import pickle
>>> from multiprocessing import shared_memory
>>> sl = shared_memory.ShareableList(range(10))
>>> list(sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> deserialized_sl = pickle.loads(pickle.dumps(sl))
>>> list(deserialized_sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> sl[0] = -1
>>> deserialized_sl[1] = -2
>>> list(sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(deserialized_sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]

>>> sl.shm.close()
>>> sl.shm.unlink()
```

The concurrent package

This package contains the following modules:

- `concurrent.futures`— Launching parallel tasks
- `concurrent.interpreters`— Multiple interpreters in the same process

concurrent.futures— Launching parallel tasks

Added in version 3.2.

Source code: `Lib/concurrent/futures/thread.py`, `Lib/concurrent/futures/process.py`, and `Lib/concurrent/futures/interpreter.py`

The `concurrent.futures` module provides a high-level interface for asynchronously executing callables.

The asynchronous execution can be performed with threads, using `ThreadPoolExecutor` or `InterpreterPoolExecutor`, or separate processes, using `ProcessPoolExecutor`. Each implements the same interface, which is defined by the abstract `Executor` class.

`concurrent.futures.Future` must not be confused with `asyncio.Future`, which is designed for use with `asyncio` tasks and coroutines. See the `asyncio`'s `Future` documentation for a detailed comparison of the two.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See `WebAssembly` platforms for more information.

Executor Objects

class `concurrent.futures.Executor`

`submit(fn, /, *args, **kwargs)`

`map(fn, *iterables, timeout=None, chunksize=1, buffer_size=None)`

`shutdown(wait=True, *, cancel_futures=False)`

An abstract class that provides methods to execute calls asynchronously. It should not be used directly, but through its concrete subclasses. `submit(fn, /, *args, **kwargs)` Schedules the callable, `fn`, to be executed as `fn(*args, **kwargs)` and returns a `Future` object representing the execution of the callable. `with ThreadPoolExecutor(max_workers=1) as executor: future = executor.submit(pow, 323, 1235) print(future.result())` `map(fn, *iterables, timeout=None, chunksize=1, buffer_size=None)` Similar to `map(fn, *iterables)` except:

Schedules the callable, `fn`, to be executed as `fn(*args, **kwargs)` and returns a `Future` object representing the execution of the callable. `with ThreadPoolExecutor(max_workers=1) as executor: future = executor.submit(pow, 323, 1235) print(future.result())`

Similar to `map(fn, *iterables)` except: The iterables are collected immediately rather than lazily, unless `buffer_size` is specified to limit the number of submitted tasks whose results have not yet been yielded. If the buffer is full, iteration over the iterables pauses until a result is yielded from the buffer. `fn` is executed asynchronously and several calls to `fn` may be made concurrently. The returned iterator raises a `TimeoutError` if `__next__()` is called and the result isn't available after `timeout` seconds from `t`

Signal the executor that it should free any resources that it is using when the currently pending futures are done executing. Calls to `Executor.submit()` and `Executor.map()` made after shutdown will raise `RuntimeError`. If `wait` is `True` then this method will not return until all the pending futures are done executing and the resources associated with the executor have been freed. If `wait` is `False` then this method will return immediately and the resources associated with the executor will be freed when all pending

ThreadPoolExecutor

`ThreadPoolExecutor` is an `Executor` subclass that uses a pool of threads to execute calls asynchronously.

Deadlocks can occur when the callable associated with a Future waits on the results of another Future. For example:

```
import time
def wait_on_b():
    time.sleep(5)
    print(b.result()) # b will never complete because it is waiting on a.
    return 5

def wait_on_a():
    time.sleep(5)
    print(a.result()) # a will never complete because it is waiting on b.
    return 6

executor = ThreadPoolExecutor(max_workers=2)
a = executor.submit(wait_on_b)
b = executor.submit(wait_on_a)
```

And:

```
def wait_on_future():
    f = executor.submit(pow, 5, 2)
    # This will never complete because there is only one worker thread and
    # it is executing this function.
    print(f.result())

executor = ThreadPoolExecutor(max_workers=1)
future = executor.submit(wait_on_future)
# Note: calling future.result() would also cause a deadlock because
# the single worker thread is already waiting for wait_on_future().
```

concurrent.futures.ThreadPoolExecutor(max_workers=None, thread_name_prefix="", initializer=None, initargs=())

An `Executor` subclass that uses a pool of at most `max_workers` threads to execute calls asynchronously. All threads enqueued to `ThreadPoolExecutor` will be joined before the interpreter can exit. Note that the exit handler which does this is executed before any exit handlers added using `atexit`. This means exceptions in the main thread must be caught and handled in order to signal threads to exit gracefully. For this reason, it is recommended that `ThreadPoolExecutor` not be used for long-running tasks.

ThreadPoolExecutor Example

```
import concurrent.futures
import urllib.request

URLS = ['http://www.foxnews.com/',
        'http://www.cnn.com/',
        'http://europe.wsj.com/',
        'http://www.bbc.co.uk/',
        'http://nonexistent-subdomain.python.org/']

# Retrieve a single page and report the URL and contents
def load_url(url, timeout):
    with urllib.request.urlopen(url, timeout=timeout) as conn:
        return conn.read()

# We can use a with statement to ensure threads are cleaned up promptly
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    # Start the load operations and mark each future with its URL
    future_to_url = {executor.submit(load_url, url, 60): url for url in URLS}
    for future in concurrent.futures.as_completed(future_to_url):
        url = future_to_url[future]
        try:
```



```

        data = future.result()
    except Exception as exc:
        print('%r generated an exception: %s' % (url, exc))
    else:
        print('%r page is %d bytes' % (url, len(data)))

```

InterpreterPoolExecutor

Added in version 3.14.

The `InterpreterPoolExecutor` class uses a pool of interpreters to execute calls asynchronously. It is a `ThreadPoolExecutor` subclass, which means each worker is running in its own thread. The difference here is that each worker has its own interpreter, and runs each task using that interpreter.

The biggest benefit to using interpreters instead of only threads is true multi-core parallelism. Each interpreter has its own Global Interpreter Lock, so code running in one interpreter can run on one CPU core, while code in another interpreter runs unblocked on a different core.

The tradeoff is that writing concurrent code for use with multiple interpreters can take extra effort. However, this is because it forces you to be deliberate about how and when interpreters interact, and to be explicit about what data is shared between interpreters. This results in several benefits that help balance the extra effort, including true multi-core parallelism. For example, code written this way can make it easier to reason about concurrency. Another major benefit is that you don't have to deal with several of the big pain points of using threads, like race conditions.

Each worker's interpreter is isolated from all the other interpreters. "Isolated" means each interpreter has its own runtime state and operates completely independently. For example, if you redirect `sys.stdout` in one interpreter, it will not be automatically redirected to any other interpreter. If you import a module in one interpreter, it is not automatically imported in any other. You would need to import the module separately in interpreter where you need it. In fact, each module imported in an interpreter is a completely separate object from the same module in a different interpreter, including `sys`, `builtins`, and even `__main__`.

Isolation means a mutable object, or other data, cannot be used by more than one interpreter at the same time. That effectively means interpreters cannot actually share such objects or data. Instead, each interpreter must have its own copy, and you will have to synchronize any changes between the copies manually. Immutable objects and data, like the builtin singletons, strings, and tuples of immutable objects, don't have these limitations.

Communicating and synchronizing between interpreters is most effectively done using dedicated tools, like those proposed in PEP 734. One less efficient alternative is to serialize with `pickle` and then send the bytes over a shared socket or pipe.

`class concurrent.futures.InterpreterPoolExecutor(max_workers=None, thread_name_prefix="", initializer=None, initargs=())`

A `ThreadPoolExecutor` subclass that executes calls asynchronously using a pool of at most `max_workers` threads. Each thread runs tasks in its own interpreter. The worker interpreters are isolated from each other, which means each has its own runtime state and that they can't share any mutable objects or other data. Each interpreter has its own Global Interpreter Lock, which means code run with this executor has true multi-core parallelism. The optional `initializer` and `initargs` arguments have the same mea

`submit()` and `map()` work like normal, except the worker serializes the callable and arguments using `pickle` when sending them to its interpreter. The worker likewise serializes the return value when sending it back.

When a worker's current task raises an uncaught exception, the worker always tries to preserve the exception as-is. If that is successful then it also sets the `__cause__` to a corresponding `ExecutionFailed` instance, which contains a summary of the original exception. In the uncommon case that the worker is not able to preserve the

original as-is then it directly preserves the corresponding `ExecutionFailed` instance instead.

ProcessPoolExecutor

The `ProcessPoolExecutor` class is an `Executor` subclass that uses a pool of processes to execute calls asynchronously. `ProcessPoolExecutor` uses the multiprocessing module, which allows it to side-step the Global Interpreter Lock but also means that only picklable objects can be executed and returned.

The `__main__` module must be importable by worker subprocesses. This means that `ProcessPoolExecutor` will not work in the interactive interpreter.

Calling `Executor` or `Future` methods from a callable submitted to a `ProcessPoolExecutor` will result in deadlock.

Note that the restrictions on functions and arguments needing to be picklable as per multiprocessing. `Process` apply when using `submit()` and `map()` on a `ProcessPoolExecutor`. A function defined in a REPL or a lambda should not be expected to work.

`class concurrent.futures.ProcessPoolExecutor(max_workers=None, mp_context=None, initializer=None, initargs=(), max_tasks_per_child=None)`

`terminate_workers()`

`kill_workers()`

An `Executor` subclass that executes calls asynchronously using a pool of at most `max_workers` processes. If `max_workers` is `None` or not given, it will default to `os.process_cpu_count()`. If `max_workers` is less than or equal to 0, then a `ValueError` will be raised. On Windows, `max_workers` must be less than or equal to 61. If it is not, then `ValueError` will be raised. If `max_workers` is `None`, then the default chosen will be at most 61, even if more processors are available. `mp_context` can be a multiprocessing context or `None`. It w

Attempt to terminate all living worker processes immediately by calling `Process.terminate` on each of them. Internally, it will also call `Executor.shutdown()` to ensure that all other resources associated with the executor are freed. After calling this method the caller should no longer submit tasks to the executor. Added in version 3.14.

Attempt to kill all living worker processes immediately by calling `Process.kill` on each of them. Internally, it will also call `Executor.shutdown()` to ensure that all other resources associated with the executor are freed. After calling this method the caller should no longer submit tasks to the executor. Added in version 3.14.

ProcessPoolExecutor Example

```
import concurrent.futures
import math
```

```
PRIMES = [
    112272535095293,
    112582705942171,
    112272535095293,
    115280095190773,
    115797848077099,
    1099726899285419]
```

```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    sqrt_n = int(math.floor(math.sqrt(n)))
    for i in range(3, sqrt_n + 1, 2):
```

```

        if n % i == 0:
            return False
        return True

def main():
    with concurrent.futures.ProcessPoolExecutor() as executor:
        for number, prime in zip(PRIMES, executor.map(is_prime, PRIMES)):
            print('%d is prime: %s' % (number, prime))

if __name__ == '__main__':
    main()

```

Future Objects

The `Future` class encapsulates the asynchronous execution of a callable. `Future` instances are created by `Executor.submit()`.

class `concurrent.futures.Future`

`cancel()`

`cancelled()`

`running()`

`done()`

`result(timeout=None)`

`exception(timeout=None)`

`add_done_callback(fn)`

`set_running_or_notify_cancel()`

`set_result(result)`

`set_exception(exception)`

Encapsulates the asynchronous execution of a callable. `Future` instances are created by `Executor.submit()` and should not be created directly except for testing. `cancel()` Attempt to cancel the call. If the call is currently being executed or finished running and cannot be cancelled then the method will return `False`, otherwise the call will be cancelled and the method will return `True`. `cancelled()` Return `True` if the call was successfully cancelled. `running()` Return `True` if the call is currently being executed and

Attempt to cancel the call. If the call is currently being executed or finished running and cannot be cancelled then the method will return `False`, otherwise the call will be cancelled and the method will return `True`.

Return `True` if the call was successfully cancelled.

Return `True` if the call is currently being executed and cannot be cancelled.

Return `True` if the call was successfully cancelled or finished running.

Return the value returned by the call. If the call hasn't yet completed then this method will wait up to `timeout` seconds. If the call hasn't completed in `timeout` seconds, then a `TimeoutError` will be raised. `timeout` can be an int or float. If `timeout` is not specified or `None`, there is no limit to the wait time. If the future is cancelled before completing then `CancelledError` will be raised. If the call raised an exception, this method will raise the same exception.

Return the exception raised by the call. If the call hasn't yet completed then this method will wait up to `timeout` seconds. If the call hasn't completed in `timeout` seconds, then a `TimeoutError` will be raised. `timeout` can be an int or float. If `timeout` is not specified or `None`, there is no limit to the wait time. If the future is cancelled

before completing then `CancelledError` will be raised. If the call completed without raising, `None` is returned.

Attaches the callable `fn` to the future. `fn` will be called, with the future as its only argument, when the future is cancelled or finishes running. Added callables are called in the order that they were added and are always called in a thread belonging to the process that added them. If the callable raises an `Exception` subclass, it will be logged and ignored. If the callable raises a `BaseException` subclass, the behavior is undefined. If the future has already completed or been cancelled, `fn` will be called

This method should only be called by `Executor` implementations before executing the work associated with the future and by unit tests. If the method returns `False` then the future was cancelled, i.e. `Future.cancel()` was called and returned `True`. Any threads waiting on the future completing (i.e. through `has_completed()` or `wait()`) will be woken up. If the method returns `True` then the future was not cancelled and has been put in the running state, i.e. calls to `Future.running()` will return `True`. This method can only be called

Sets the result of the work associated with the future to `result`. This method should only be used by `Executor` implementations and unit tests. Changed in version 3.8: This method raises `concurrent.futures.InvalidStateError` if the future is already done.

Sets the result of the work associated with the future to the `Exception` `exception`. This method should only be used by `Executor` implementations and unit tests. Changed in version 3.8: This method raises `concurrent.futures.InvalidStateError` if the future is already done.

Module Functions

`concurrent.futures.wait(fs, timeout=None, return_when=ALL_COMPLETED)`

`concurrent.futures.FIRST_COMPLETED`

`concurrent.futures.FIRST_EXCEPTION`

`concurrent.futures.ALL_COMPLETED`

Wait for the future instances (possibly created by different `Executor` instances) given by `fs` to complete. Duplicate futures given to `fs` are removed and will be returned only once. Returns a named 2-tuple of sets. The first set, named `done`, contains the futures that completed (finished or cancelled futures) before the wait completed. The second set, named `not_done`, contains the futures that did not complete (pending or running futures). `timeout` can be used to control the maximum number of seconds to wait

`concurrent.futures.as_completed(fs, timeout=None)`

Returns an iterator over the future instances (possibly created by different `Executor` instances) given by `fs` that yields futures as they complete (finished or cancelled futures). Any futures given by `fs` that are duplicated will be returned once. Any futures that completed before `as_completed()` is called will be yielded first. The returned iterator raises a `TimeoutError` if `__next__()` is called and the result isn't available after `timeout` seconds from the original call to `as_completed()`. `timeout` can be an `int` or `float`

See also

PEP 3148– futures - execute computations asynchronously

The proposal which described this feature for inclusion in the Python standard library.

Exception classes

`exception concurrent.futures.CancelledError`

Raised when a future is cancelled.

exceptionconcurrent.futures.TimeoutError

A deprecated alias of `TimeoutError`, raised when a future operation exceeds the given timeout. Changed in version 3.11: This class was made an alias of `TimeoutError`.

exceptionconcurrent.futures.BrokenExecutor

Derived from `RuntimeError`, this exception class is raised when an executor is broken for some reason, and cannot be used to submit or execute new tasks. Added in version 3.7.

exceptionconcurrent.futures.InvalidStateError

Raised when an operation is performed on a future that is not allowed in the current state. Added in version 3.8.

exceptionconcurrent.futures.thread.BrokenThreadPool

Derived from `BrokenExecutor`, this exception class is raised when one of the workers of a `ThreadPoolExecutor` has failed initializing. Added in version 3.7.

exceptionconcurrent.futures.interpreter.BrokenInterpreterPool

Derived from `BrokenThreadPool`, this exception class is raised when one of the workers of a `InterpreterPoolExecutor` has failed initializing. Added in version 3.14.

exceptionconcurrent.futures.process.BrokenProcessPool

Derived from `BrokenExecutor` (formerly `RuntimeError`), this exception class is raised when one of the workers of a `ProcessPoolExecutor` has terminated in a non-clean fashion (for example, if it was killed from the outside). Added in version 3.3.

concurrent.interpreters— Multiple interpreters in the same process

Added in version 3.14.

Source code: `Lib/concurrent/interpreters`

The `concurrent.interpreters` module constructs higher-level interfaces on top of the lower level `_interpreters` module.

The module is primarily meant to provide a basic API for managing interpreters (AKA “subinterpreters”) and running things in them. Running mostly involves switching to an interpreter (in the current thread) and calling a function in that execution context.

For concurrency, interpreters themselves (and this module) don’t provide much more than isolation, which on its own isn’t useful. Actual concurrency is available separately through `threads`. See below

See also

InterpreterPoolExecutor

Combines threads with interpreters in a familiar interface.

Isolating Extension Modules

How to update an extension module to support multiple interpreters.

PEP 554

PEP 734

PEP 684

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Key details

Before we dive in further, there are a small number of details to keep in mind about using multiple interpreters:

- isolated, by default
- no implicit threads
- not all PyPI packages support use in multiple interpreters yet

Introduction

An “interpreter” is effectively the execution context of the Python runtime. It contains all of the state the runtime needs to execute a program. This includes things like the import state and builtins. (Each thread, even if there’s only the main thread, has some extra runtime state, in addition to the current interpreter, related to the current exception and the bytecode eval loop.)

The concept and functionality of the interpreter have been a part of Python since version 2.2, but the feature was only available through the C-API and not well known, and the isolation was relatively incomplete until version 3.12.

Multiple Interpreters and Isolation

A Python implementation may support using multiple interpreters in the same process. CPython has this support. Each interpreter is effectively isolated from the others (with a limited number of carefully managed process-global exceptions to the rule).

That isolation is primarily useful as a strong separation between distinct logical components of a program, where you want to have careful control of how those components interact.

■ Nota: NoteInterpreters in the same process can technically never be strictly isolated from one another since there are few restrictions on memory access within the same process. The Python runtime makes a best effort at isolation but extension modules may easily violate that. Therefore, do not use multiple interpreters in security-sensitive situations, where they shouldn't have access to each other's data.

Running in an Interpreter

Running in a different interpreter involves switching to it in the current thread and then calling some function. The runtime will execute the function using the current interpreter's state. The `concurrent.interpreters` module provides a basic API for creating and managing interpreters, as well as the switch-and-call operation.

No other threads are automatically started for the operation. There is a helper for that though. There is another dedicated helper for calling the builtin `exec()` in an interpreter.

When `exec()` or `eval()` are called in an interpreter, they run using the interpreter's `__main__` module as the “globals” namespace. The same is true for functions that aren't associated with any module. This is the same as how scripts invoked from the command-line run in the `__main__` module.

Concurrency and Parallelism

As noted earlier, interpreters do not provide any concurrency on their own. They strictly represent the isolated execution context the runtime will use in the current thread. That isolation makes them similar to processes, but they still enjoy in-process efficiency, like threads.

All that said, interpreters do naturally support certain flavors of concurrency. There's a powerful side effect of that isolation. It enables a different approach to concurrency than you can take with `async` or threads. It's a similar concurrency model to CSP or the actor model, a model which is relatively easy to reason about.

You can take advantage of that concurrency model in a single thread, switching back and forth between interpreters, Stackless-style. However, this model is more useful when you combine interpreters with multiple threads. This mostly involves starting a new thread, where you switch to another interpreter and run what you want there.

Each actual thread in Python, even if you're only running in the main thread, has its own current execution context. Multiple threads can use the same interpreter or different ones.

At a high level, you can think of the combination of threads and interpreters as threads with opt-in sharing.

As a significant bonus, interpreters are sufficiently isolated that they do not share the GIL, which means combining threads with multiple interpreters enables full multi-core parallelism. (This has been the case since Python 3.12.)

Communication Between Interpreters

In practice, multiple interpreters are useful only if we have a way to communicate between them. This usually involves some form of message passing, but can even mean sharing data in some carefully managed way.

With this in mind, the `concurrent.interpreters` module provides a `queue.Queue` implementation, available through `create_queue()`.

“Sharing” Objects

Any data actually shared between interpreters loses the thread-safety provided by the GIL. There are various options for dealing with this in extension modules. However, from Python code the lack of thread-safety means objects can't actually be shared, with a few exceptions. Instead, a copy must be created, which means mutable objects won't stay in sync.

By default, most objects are copied with pickle when they are passed to another interpreter. Nearly all of the immutable builtin objects are either directly shared or copied efficiently. For example:

- None
- bool(True and False)
- bytes
- str
- int
- float
- tuple (of similarly supported objects)

There is a small number of Python types that actually share mutable data between interpreters:

- memoryview
- Queue

Reference

This module defines the following functions:

concurrent.interpreters.list_all()

Return a list of Interpreter objects, one for each existing interpreter.

concurrent.interpreters.get_current()

Return an Interpreter object for the currently running interpreter.

concurrent.interpreters.get_main()

Return an Interpreter object for the main interpreter. This is the interpreter the runtime created to run the REPL or the script given at the command-line. It is usually the only one.

concurrent.interpreters.create()

Initialize a new (idle) Python interpreter and return an Interpreter object for it.

concurrent.interpreters.create_queue()

Initialize a new cross-interpreter queue and return a Queue object for it.

Interpreter objects

class concurrent.interpreters.Interpreter(id)

id

whence

is_running()

close()

prepare_main(ns=None, **kwargs)

exec(code,/,dedent=True)

call(callable,/,*args,kwargs)**

call_in_thread(callable,/,*args,kwargs)**

A single interpreter in the current process. Generally, `Interpreters` shouldn't be called directly. Instead, use `create()` or one of the other module functions. `id(read-only)` The underlying interpreter's ID. `whence(read-only)` A string describing where the interpreter came from. `is_running()` Return `True` if the interpreter is currently executing code in its `__main__` module and `False` otherwise. `close()` Finalize and destroy the interpreter. `prepare_main(ns=None,**kwargs)` Bind objects in the interpreter's `__main__` module. Some

(read-only) The underlying interpreter's ID.

(read-only) A string describing where the interpreter came from.

Return `True` if the interpreter is currently executing code in its `__main__` module and `False` otherwise.

Finalize and destroy the interpreter.

Bind objects in the interpreter's `__main__` module. Some objects are actually shared and some are copied efficiently, but most are copied via pickle. See "Sharing" Objects.

Run the given source code in the interpreter (in the current thread).

Return the result of calling running the given function in the interpreter (in the current thread).

Run the given function in the interpreter (in a new thread).

Exceptions

exception concurrent.interpreters.InterpreterError

This exception, a subclass of `Exception`, is raised when an interpreter-related error happens.

exception concurrent.interpreters.InterpreterNotFoundError

This exception, a subclass of `InterpreterError`, is raised when the targeted interpreter no longer exists.

exception concurrent.interpreters.ExecutionFailed

excinfo

This exception, a subclass of `InterpreterError`, is raised when the running code raised an uncaught exception. `excinfo` A basic snapshot of the exception raised in the other interpreter.

A basic snapshot of the exception raised in the other interpreter.

exception concurrent.interpreters.NotShareableError

This exception, a subclass of `TypeError`, is raised when an object cannot be sent to another interpreter.

Communicating Between Interpreters

class concurrent.interpreters.Queue(id)

id

A wrapper around a low-level, cross-interpreter queue, which implements the `queue.Queue` interface. The underlying queue can only be created through `create_queue()`. Some objects are actually shared and some are copied efficiently, but most are copied via pickle. See "Sharing" Objects. `id(read-only)` The queue's ID.

(read-only) The queue's ID.

exception concurrent.interpreters.QueueEmptyError

This exception, a subclass of `queue.Empty`, is raised from `Queue.get()` and `Queue.get_nowait()` when the queue is empty.

exception `concurrent.interpreters.QueueFullError`

This exception, a subclass of `queue.Full`, is raised from `Queue.put()` and `Queue.put_nowait()` when the queue is full.

Basic usage

Creating an interpreter and running code in it:

```
from concurrent import interpreters
```

```
interp = interpreters.create()
```

```
# Run in the current OS thread.
```

```
interp.exec('print("spam!")')
```

```
interp.exec("""if True:
    print('spam!')
""")
```

```
from textwrap import dedent
interp.exec(dedent("""
    print('spam!')
"""))
```

```
def run(arg):
    return arg
```

```
res = interp.call(run, 'spam!')
print(res)
```

```
def run():
    print('spam!')
```

```
interp.call(run)
```

```
# Run in new OS thread.
```

```
t = interp.call_in_thread(run)
t.join()
```

subprocess— Subprocess management

Source code: `Lib/subprocess.py`

The `subprocess` module allows you to spawn new processes, connect to their input/output/error pipes, and obtain their return codes. This module intends to replace several older modules and functions:

`os.system`
`os.spawn*`

Information about how the `subprocess` module can be used to replace these modules and functions can be found in the following sections.

See also

PEP 324— PEP proposing the `subprocess` module

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Using the `subprocess` Module

The recommended approach to invoking subprocesses is to use the `run()` function for all use cases it can handle. For more advanced use cases, the underlying `Popen` interface can be used directly.

`subprocess.run(args, *, stdin=None, input=None, stdout=None, stderr=None, capture_output=False, shell=False, cwd=None, timeout=None, check=False, encoding=None, errors=None, text=None, env=None, universal_newlines=None, **other_popen_kwargs)`

Run the command described by `args`. Wait for command to complete, then return a `CompletedProcess` instance. The arguments shown above are merely the most common ones, described below in *Frequently Used Arguments* (hence the use of keyword-only notation in the abbreviated signature). The full function signature is largely the same as that of the `Popen` constructor - most of the arguments to this function are passed through to that interface. (`timeout`, `input`, `check`, and `capture_output` are not.) If `capture_output` is

`class subprocess.CompletedProcess`

`args`

`returncode`

`stdout`

`stderr`

`check_returncode()`

The return value from `run()`, representing a process that has finished. `args` The arguments used to launch the process. This may be a list or a string. `returncode` Exit status of the child process. Typically, an exit status of 0 indicates that it ran successfully. A negative value -N indicates that the child was terminated by signal N (POSIX only). `stdout` Captured stdout from the child process. A bytes sequence, or a string if `run()` was called with an encoding, errors, or `text=True`. `None` if stdout was not captured

The arguments used to launch the process. This may be a list or a string.

Exit status of the child process. Typically, an exit status of 0 indicates that it ran successfully. A negative value -N indicates that the child was terminated by signal N (POSIX only).

Captured stdout from the child process. A bytes sequence, or a string if `run()` was called with an encoding, errors, or `text=True`. None if stdout was not captured. If you ran the process with `stderr=subprocess.STDOUT`, stdout and stderr will be combined in this attribute, and stderr will be None.

Captured stderr from the child process. A bytes sequence, or a string if `run()` was called with an encoding, errors, or `text=True`. None if stderr was not captured.

If `returncode` is non-zero, raise a `CalledProcessError`.

subprocess.DEVNULL

Special value that can be used as the `stdin`, `stdout` or `stderr` argument to `Popen` and indicates that the special file `/dev/null` will be used. Added in version 3.3.

subprocess.PIPE

Special value that can be used as the `stdin`, `stdout` or `stderr` argument to `Popen` and indicates that a pipe to the standard stream should be opened. Most useful with `Popen.communicate()`.

subprocess.STDOUT

Special value that can be used as the `stderr` argument to `Popen` and indicates that standard error should go into the same handle as standard output.

exceptions.subprocess.SubprocessError

Base class for all other exceptions from this module. Added in version 3.3.

exceptions.subprocess.TimeoutExpired

cmd

timeout

output

stdout

stderr

Subclass of `SubprocessError`, raised when a timeout expires while waiting for a child process. `cmd` Command that was used to spawn the child process. `timeout` Timeout in seconds. `output` Output of the child process if it was captured by `run()` or `check_output()`. Otherwise, None. This is always bytes when any output was captured regardless of the `text=True` setting. It may remain None instead of b'' when no output was observed. `stdout` Alias for output, for symmetry with `stderr`. `stderr` Stderr output of the child process if

Command that was used to spawn the child process.

Timeout in seconds.

Output of the child process if it was captured by `run()` or `check_output()`. Otherwise, None. This is always bytes when any output was captured regardless of the `text=True` setting. It may remain None instead of b'' when no output was observed.

Alias for output, for symmetry with `stderr`.

Stderr output of the child process if it was captured by `run()`. Otherwise, None. This is always bytes when `stderr` output was captured regardless of the `text=True` setting. It may remain None instead of b'' when no `stderr` output was observed.

exceptions.subprocess.CalledProcessError

returncode

cmd

output

stdout

stderr

Subclass of `SubprocessError`, raised when a process run by `check_call()`, `check_output()`, `run()` (with `check=True`) returns a non-zero exit status. `returncode` Exit status of the child process. If the process exited due to a signal, this will be the negative signal number. `cmd` Command that was used to spawn the child process. `output` Output of the child process if it was captured by `run()` or `check_output()`. Otherwise, `None`. `stdout` Alias for output, for symmetry with `stderr`. `stderr` Stderr output of the child process if it was captured by `run()` or `check_output()`. Otherwise, `None`.

Exit status of the child process. If the process exited due to a signal, this will be the negative signal number.

Command that was used to spawn the child process.

Output of the child process if it was captured by `run()` or `check_output()`. Otherwise, `None`.

Alias for output, for symmetry with `stderr`.

Stderr output of the child process if it was captured by `run()`. Otherwise, `None`.

Frequently Used Arguments

To support a wide variety of use cases, the `Popen` constructor (and the convenience functions) accept a large number of optional arguments. For most typical use cases, many of these arguments can be safely left at their default values. The arguments that are most commonly needed are:

`args` is required for all calls and should be a string, or a sequence of program arguments. Providing a sequence of arguments is generally preferred, as it allows the module to take care of any required escaping and quoting of arguments (e.g. to permit spaces in file names). If passing a single string, either `shell` must be `True` (see below) or else the string must simply name the program to be executed without specifying any arguments.

`stdin`, `stdout` and `stderr` specify the executed program's standard input, standard output and standard error file handles, respectively. Valid values are `None`, `PIPE`, `DEVNULL`, an existing file descriptor (a positive integer), and an existing file object with a valid file descriptor. With the default settings of `None`, no redirection will occur. `PIPE` indicates that a new pipe to the child should be created. `DEVNULL` indicates that the special files `/dev/null` will be used. Additionally, `stderr` can be `STDOUT`, which indicates that the `stderr` data from the child process should be captured into the same file handle as for `stdout`.

If `encoding` or `errors` are specified, or `text` (also known as `universal_newlines`) is `True`, the file objects `stdin`, `stdout` and `stderr` will be opened in text mode using the `encoding` and `errors` specified in the call or the defaults for `io.TextIOWrapper`.

For `stdin`, line ending characters `\n` in the input will be converted to the default line separator `os.linesep`. For `stdout` and `stderr`, all line endings in the output will be converted to `\n`. For more information see the documentation of the `io.TextIOWrapper` class when the `newline` argument to its constructor is `None`.

If text mode is not used, `stdin`, `stdout` and `stderr` will be opened as binary streams. No encoding or line ending conversion is performed.

Changed in version 3.6: Added the `encoding` and `errors` parameters.

Changed in version 3.7: Added the `text` parameter as an alias for `universal_newlines`.

■ **Nota:** Note The `newline` attribute of the file objects `Popen.stdin`, `Popen.stdout` and `Popen.stderr` are not updated by the `Popen.communicate()` method.

If `shell` is `True`, the specified command will be executed through the shell. This can be useful if you are using Python primarily for the enhanced control flow it offers over most system shells and still want convenient access to other shell features such as shell pipes, filename wildcards, environment variable expansion, and expansion of `~` to a user's home directory. However, note that Python itself offers implementations of many shell-like

features (in particular, `glob`, `fnmatch`, `os.walk()`, `os.path.expandvars()`, `os.path.expanduser()`, and `shutil`).

Changed in version 3.3: When `universal_newlines=True`, the class uses the `encodinglocale.getpreferredencoding(False)` instead of `locale.getpreferredencoding()`. See the `io.TextIOWrapper` class for more information on this change.

■ **Nota:** NoteRead the `Security Considerations` section before using `shell=True`.

These options, along with all of the other options, are described in more detail in the `Popen` constructor documentation.

Popen Constructor

The underlying process creation and management in this module is handled by the `Popen` class. It offers a lot of flexibility so that developers are able to handle the less common cases not covered by the convenience functions.

`class subprocess.Popen(args, bufsize=-1, executable=None, stdin=None, stdout=None, stderr=None, preexec_fn=None, close_fds=True, shell=False, cwd=None, env=None, universal_newlines=None, startupinfo=None, creationflags=0, restore_signals=True, start_new_session=False, pass_fds=(), *, group=None, extra_groups=None, user=`

Execute a child program in a new process. On POSIX, the class uses `os.execvpe()`-like behavior to execute the child program. On Windows, the class uses the `WindowsCreateProcess()` function. The arguments to `Popen` are as follows. `args` should be a sequence of program arguments or else a single string or path-like object. By default, the program to execute is the first item in `args` if `args` is a sequence. If `args` is a string, the interpretation is platform-dependent and described below. See the `shell` and `executa`

Exceptions

Exceptions raised in the child process, before the new program has started to execute, will be re-raised in the parent.

The most common exception raised is `OSError`. This occurs, for example, when trying to execute a non-existent file. Applications should prepare for `OSError` exceptions. Note that, when `shell=True`, `OSError` will be raised by the child only if the selected shell itself was not found. To determine if the shell failed to find the requested application, it is necessary to check the return code or output from the subprocess.

`AValueError` will be raised if `Popen` is called with invalid arguments.

`check_call()` and `check_output()` will raise `CalledProcessError` if the called process returns a non-zero return code.

All of the functions and methods that accept a `timeout` parameter, such as `run()` and `Popen.communicate()` will raise `TimeoutExpired` if the timeout expires before the process exits.

Exceptions defined in this module all inherit from `SubprocessError`.

Added in version 3.3: The `SubprocessError` base class was added.

Security Considerations

Unlike some other `popen` functions, this library will not implicitly choose to call a system shell. This means that all characters, including shell metacharacters, can safely be passed to child processes. If the shell is invoked explicitly, via `shell=True`, it is the application's responsibility to ensure that all whitespace and metacharacters are quoted appropriately to avoid shell injection vulnerabilities. On some platforms, it is possible to use `shlex.quote()` for this escaping.

On Windows, batch files (`*.bat` or `*.cmd`) may be launched by the operating system in a system shell regardless of the arguments passed to this library. This could result in arguments being parsed according to shell rules, but

without any escaping added by Python. If you are intentionally launching a batch file with arguments from untrusted sources, consider passing `shell=True` to allow Python to escape special characters. See [gh-114539](#) for additional discussion.

Popen Objects

Instances of the `Popen` class have the following methods:

`Popen.poll()`

Check if child process has terminated. Set and return `returncode` attribute. Otherwise, returns `None`.

`Popen.wait(timeout=None)`

Wait for child process to terminate. Set and return `returncode` attribute. If the process does not terminate after `timeout` seconds, raise a `TimeoutExpired` exception. It is safe to catch this exception and retry the wait. Note This will deadlock when using `stdout=PIPE` or `stderr=PIPE` and the child process generates enough output to a pipe such that it blocks waiting for the OS pipe buffer to accept more data. Use `Popen.communicate()` when using pipes to avoid that. Note When the `timeout` parameter is not `None`, then (o

`Popen.communicate(input=None, timeout=None)`

Interact with process: Send data to `stdin`. Read data from `stdout` and `stderr`, until end-of-file is reached. Wait for process to terminate and set the `returncode` attribute. The optional `input` argument should be data to be sent to the child process, or `None`, if no data should be sent to the child. If streams were opened in text mode, `input` must be a string. Otherwise, it must be bytes. `communicate()` returns a tuple(`stdout_data`, `stderr_data`). The data will be strings if streams were opened in text mode; o

`Popen.send_signal(signal)`

Sends the `signal` to the child. Do nothing if the process completed. Note On Windows, `SIGTERM` is an alias for `terminate()`. `CTRL_C_EVENT` and `CTRL_BREAK_EVENT` can be sent to processes started with a `creationflags` parameter which includes `CREATE_NEW_PROCESS_GROUP`.

`Popen.terminate()`

Stop the child. On POSIX OSs the method sends `SIGTERM` to the child. On Windows the Win32 API function `TerminateProcess()` is called to stop the child.

`Popen.kill()`

Kills the child. On POSIX OSs the function sends `SIGKILL` to the child. On Windows `kill()` is an alias for `terminate()`.

The following attributes are also set by the class for you to access. Reassigning them to new values is unsupported:

`Popen.args`

The `args` argument as it was passed to `Popen`— a sequence of program arguments or else a single string. Added in version 3.3.

`Popen.stdin`

If the `stdin` argument was `PIPE`, this attribute is a writeable stream object as returned by `open()`. If the `encoding` or `errors` arguments were specified or the `text` or `universal_newlines` argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the `stdin` argument was not `PIPE`, this attribute is `None`.

`Popen.stdout`

If the `stdout` argument was `PIPE`, this attribute is a readable stream object as returned by `open()`. Reading from the stream provides output from the child process. If the `encoding` or `errors` arguments were specified or the `text` or `universal_newlines` argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the `stdout` argument was not `PIPE`, this attribute is `None`.

`Popen.stderr`

If the `stderr` argument was `PIPE`, this attribute is a readable stream object as returned by `open()`. Reading from the stream provides error output from the child process. If the `encoding` or `errors` arguments were specified or the `text` or `universal_newlines` argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the `stderr` argument was not `PIPE`, this attribute is `None`.

■■ Advertencia: Warning Use `communicate()` rather than `stdin.write()`, `stdout.read()` or `stderr.read()` to avoid deadlocks due to any of the other OS pipe buffers filling up and blocking the child process.

`Popen.pid`

The process ID of the child process. Note that if you set the `shell` argument to `True`, this is the process ID of the spawned shell.

`Popen.returncode`

The child return code. Initially `None`, `returncode` is set by a call to the `poll()`, `wait()`, or `communicate()` methods if they detect that the process has terminated. A `None` value indicates that the process hadn't yet terminated at the time of the last method call. A negative value `-N` indicates that the child was terminated by signal `N` (POSIX only). When `shell=True`, the return code reflects the exit status of the shell itself (e.g. `/bin/sh`), which may map signals to codes such as `128+N`. See the documentation of the `sh`

Windows Popen Helpers

The `STARTUPINFO` class and following constants are only available on Windows.

`class subprocess.STARTUPINFO(*, dwFlags=0, hStdInput=None, hStdOutput=None, hStdError=None, wShowWindow=0, lpAttributeList=None)`

`dwFlags`

`hStdInput`

`hStdOutput`

`hStdError`

`wShowWindow`

`lpAttributeList`

`handle_list`

Partial support of the Windows `STARTUPINFO` structure is used for `Popen` creation. The following attributes can be set by passing them as keyword-only arguments. Changed in version 3.7: Keyword-only argument support was added. `dwFlags` A bit field that determines whether certain `STARTUPINFO` attributes are used when the process creates a window. `si=subprocess.STARTUPINFO()` `si.dwFlags=subprocess.STARTF_USESTDHANDLES` `S` `si=subprocess.STARTF_USESHOWWINDOW` `hStdInput` `if dwFlags specifies STARTF_USESTDHANDLES`, this attribute is

A bit field that determines whether certain `STARTUPINFO` attributes are used when the process creates a window. `si=subprocess.STARTUPINFO()` `si.dwFlags=subprocess.STARTF_USESTDHANDLES` `S` `si=subprocess.STARTF_USESHOWWINDOW`

If `ldwFlagsspecifiesSTARTF_USESTDHANDLES`, this attribute is the standard input handle for the process. If `STARTF_USESTDHANDLES` is not specified, the default for standard input is the keyboard buffer.

If `ldwFlagsspecifiesSTARTF_USESTDHANDLES`, this attribute is the standard output handle for the process. Otherwise, this attribute is ignored and the default for standard output is the console window's buffer.

If `ldwFlagsspecifiesSTARTF_USESTDHANDLES`, this attribute is the standard error handle for the process. Otherwise, this attribute is ignored and the default for standard error is the console window's buffer.

If `ldwFlagsspecifiesSTARTF_USESHOWWINDOW`, this attribute can be any of the values that can be specified in `thenCmdShowparameter` for the `ShowWindowfunction`, except for `SW_SHOWDEFAULT`. Otherwise, this attribute is ignored. `SW_HIDE` is provided for this attribute. It is used when `Popen` is called with `shell=True`.

A dictionary of additional attributes for process creation as given in `STARTUPINFOEX`, see `UpdateProcThreadAttribute`. Supported attributes: `handle_list` Sequence of handles that will be inherited. `close_fds` must be true if non-empty. The handles must be temporarily made inheritable by `os.set_handle_inheritable()` when passed to the `Popen` constructor, else `OSError` will be raised with Windows error `ERROR_INVALID_PARAMETER(87)`. Warning In a multithreaded process, use caution to avoid leaking handles that are marked in

Sequence of handles that will be inherited. `close_fds` must be true if non-empty. The handles must be temporarily made inheritable by `os.set_handle_inheritable()` when passed to the `Popen` constructor, else `OSError` will be raised with Windows error `ERROR_INVALID_PARAMETER(87)`. Warning In a multithreaded process, use caution to avoid leaking handles that are marked inheritable when combining this feature with concurrent calls to other process creation functions that inherit all handles such as `os.system()`. This

Windows Constants

The `subprocess` module exposes the following constants.

subprocess.STD_INPUT_HANDLE

The standard input device. Initially, this is the console input buffer, `CONIN$`.

subprocess.STD_OUTPUT_HANDLE

The standard output device. Initially, this is the active console screen buffer, `CONOUT$`.

subprocess.STD_ERROR_HANDLE

The standard error device. Initially, this is the active console screen buffer, `CONOUT$`.

subprocess.SW_HIDE

Hides the window. Another window will be activated.

subprocess.STARTF_USESTDHANDLES

Specifies that the `STARTUPINFO.hStdInput`, `STARTUPINFO.hStdOutput`, and `STARTUPINFO.hStdError` attributes contain additional information.

subprocess.STARTF_USESHOWWINDOW

Specifies that the `STARTUPINFO.wShowWindow` attribute contains additional information.

subprocess.STARTF_FORCEONFEEDBACK

A `STARTUPINFO.dwFlags` parameter to specify that the Working in Background mouse cursor will be displayed while a process is launching. This is the default behavior for GUI processes. Added in version 3.13.

subprocess.STARTF_FORCEOFFFEEDBACK

A `STARTUPINFO.dwFlags` parameter to specify that the mouse cursor will not be changed when launching a process. Added in version 3.13.

subprocess.CREATE_NEW_CONSOLE

The new process has a new console, instead of inheriting its parent's console (the default).

subprocess.CREATE_NEW_PROCESS_GROUP

A `subprocess.Popen` creation flag parameter to specify that a new process group will be created. This flag is necessary for using `subprocess.kill()` on the subprocess. This flag is ignored if `CREATE_NEW_CONSOLE` is specified.

subprocess.ABOVE_NORMAL_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have an above average priority. Added in version 3.7.

subprocess.BELOW_NORMAL_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have a below average priority. Added in version 3.7.

subprocess.HIGH_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have a high priority. Added in version 3.7.

subprocess.IDLE_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have an idle (lowest) priority. Added in version 3.7.

subprocess.NORMAL_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have a normal priority. (default) Added in version 3.7.

subprocess.REALTIME_PRIORITY_CLASS

A `subprocess.Popen` creation flag parameter to specify that a new process will have realtime priority. You should almost never use `REALTIME_PRIORITY_CLASS`, because this interrupts system threads that manage mouse input, keyboard input, and background disk flushing. This class can be appropriate for applications that “talk” directly to hardware or that perform brief tasks that should have limited interruptions. Added in version 3.7.

subprocess.CREATE_NO_WINDOW

A `subprocess.Popen` creation flag parameter to specify that a new process will not create a window. Added in version 3.7.

subprocess.DETACHED_PROCESS

A `subprocess.Popen` creation flag parameter to specify that a new process will not inherit its parent's console. This value cannot be used with `CREATE_NEW_CONSOLE`. Added in version 3.7.

subprocess.CREATE_DEFAULT_ERROR_MODE

A `subprocess.Popen` creation flag parameter to specify that a new process does not inherit the error mode of the calling process. Instead, the new process gets the default error mode. This feature is particularly useful for multithreaded shell applications that run with hard errors disabled. Added in version 3.7.

subprocess.CREATE_BREAKAWAY_FROM_JOB

A `subprocess.Popen` creation flag parameter to specify that a new process is not associated with the job. Added in version 3.7.

Older high-level API

Prior to Python 3.5, these three functions comprised the high level API to subprocess. You can now use `userun()` in many cases, but lots of existing code calls these functions.

`subprocess.call(args,*,stdin=None,stdout=None,stderr=None,shell=False,cwd=None,timeout=None,**other_popen_kwargs)`

Run the command described by `args`. Wait for command to complete, then return the `returncode` attribute. Code needing to capture `stdout` or `stderr` should use `run()` instead: `run(...).returncode`. To suppress `stdout` or `stderr`, supply a value of `DEVNULL`. The arguments shown above are merely some common ones. The full function signature is the same as that of the `Popen` constructor - this function passes all supplied arguments other than `timeout` directly through to that interface. Note Do not use `stdout=PIPE` or `stderr=PIPE`

`subprocess.check_call(args,*,stdin=None,stdout=None,stderr=None,shell=False,cwd=None,timeout=None,**other_popen_kwargs)`

Run command with arguments. Wait for command to complete. If the return code was zero then return, otherwise raise `CalledProcessError`. The `CalledProcessError` object will have the return code in the `returncode` attribute. If `check_call()` was unable to start the process it will propagate the exception that was raised. Code needing to capture `stdout` or `stderr` should use `run()` instead: `run(...,check=True)`. To suppress `stdout` or `stderr`, supply a value of `DEVNULL`. The arguments shown above are merely some common one

`subprocess.check_output(args,*,stdin=None,stderr=None,shell=False,cwd=None,encoding=None,errors=None,universal_newlines=None,timeout=None,text=None,**other_popen_kwargs)`

Run command with arguments and return its output. If the return code was non-zero it raises a `CalledProcessError`. The `CalledProcessError` object will have the return code in the `returncode` attribute and any output in the `output` attribute. This is equivalent to: `run(...,check=True,stdout=PIPE).stdout`. The arguments shown above are merely some common ones. The full function signature is largely the same as that of `run()` - most arguments are passed directly through to that interface. One API deviation from `run()` is

Replacing Older Functions with the `subprocess` Module

In this section, “a becomes b” means that b can be used as a replacement for a.

■ Nota: Note All “a” functions in this section fail (more or less) silently if the executed program cannot be found; the “b” replacements raise `OSError` instead. In addition, the replacements using `check_output()` will fail with a `CalledProcessError` if the requested operation produces a non-zero return code. The output is still available as the `output` attribute of the raised exception.

In the following examples, we assume that the relevant functions have already been imported from the `subprocess` module.

Replacing `/bin/sh` shell command substitution

```
output=$(mycmd myarg)
```

becomes:

```
output = check_output(["mycmd", "myarg"])
```

Replacing shell pipeline

```
output=$(dmesg | grep hda)
```

becomes:

```
p1 = Popen(["dmesg"], stdout=PIPE)
p2 = Popen(["grep", "hda"], stdin=p1.stdout, stdout=PIPE)
p1.stdout.close() # Allow p1 to receive a SIGPIPE if p2 exits.
output = p2.communicate()[0]
```

Thep1.stdout.close()call after starting the p2 is important in order for p1 to receive a SIGPIPE if p2 exits before p1.

Alternatively, for trusted input, the shell's own pipeline support may still be used directly:

```
output=$(dmesg | grep hda)
```

becomes:

```
output = check_output("dmesg | grep hda", shell=True)
```

Replacingos.system()

```
sts = os.system("mycmd" + " myarg")
# becomes
retcode = call("mycmd" + " myarg", shell=True)
```

Notes:

- Calling the program through the shell is usually not required.
- Thecall()return value is encoded differently to that ofos.system().
- Theos.system()function ignores SIGINT and SIGQUIT signals while the command is running, but the caller must do this separately when using thesubprocessmodule.

A more realistic example would look like this:

```
try:
    retcode = call("mycmd" + " myarg", shell=True)
    if retcode < 0:
        print("Child was terminated by signal", -retcode, file=sys.stderr)
    else:
        print("Child returned", retcode, file=sys.stderr)
except OSError as e:
    print("Execution failed:", e, file=sys.stderr)
```

Replacing theos.spawnfamily

P_NOWAIT example:

```
pid = os.spawnlp(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg")
==>
pid = Popen(["/bin/mycmd", "myarg"]).pid
```

P_WAIT example:

```
retcode = os.spawnlp(os.P_WAIT, "/bin/mycmd", "mycmd", "myarg")
==>
retcode = call(["/bin/mycmd", "myarg"])
```

Vector example:

```
os.spawnvp(os.P_NOWAIT, path, args)
==>
Popen([path] + args[1:])
```

Environment example:

```
os.spawnlpe(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg", env)
==>
Popen(["/bin/mycmd", "myarg"], env={"PATH": "/usr/bin"})
```

Replacingos.popen()

Return code handling translates as follows:

```
pipe = os.popen(cmd, 'w')
...
```

```

rc = pipe.close()
if rc is not None and rc >> 8:
    print("There were some errors")
==>
process = Popen(cmd, stdin=PIPE)
...
process.stdin.close()
if process.wait() != 0:
    print("There were some errors")

```

Legacy Shell Invocation Functions

This module also provides the following legacy functions from the `2.x.commands` module. These operations implicitly invoke the system shell and none of the guarantees described above regarding security and exception handling consistency are valid for these functions.

`subprocess.getstatusoutput(cmd,*,encoding=None,errors=None)`

Return (exitcode,output) of executing `cmd` in a shell. Execute the string `cmd` in a shell with `check_output()` and return a 2-tuple (exitcode,output). `encoding` and `errors` are used to decode output; see the notes on `Frequently Used Arguments` for more details. A trailing newline is stripped from the output. The exit code for the command can be interpreted as the return code of `subprocess`. Example: `>>> subprocess.getstatusoutput('ls /bin/ls')(0, '/bin/ls')` `>>> subprocess.getstatusoutput('cat /bin/junk')(1, 'cat: /bin/jun`

`subprocess.getoutput(cmd,*,encoding=None,errors=None)`

Return output (stdout and stderr) of executing `cmd` in a shell. Like `getstatusoutput()`, except the exit code is ignored and the return value is a string containing the command's output. Example: `>>> subprocess.getoutput('ls /bin/ls')'/bin/ls'` Availability: Unix, Windows. Changed in version 3.3.4: Windows support added Changed in version 3.11: Added the `encoding` and `errors` parameters.

Notes

Timeout Behavior

When using the `timeout` parameter in functions like `run()`, `Popen.wait()`, or `Popen.communicate()`, users should be aware of the following behaviors:

- **Process Creation Delay:** The initial process creation itself cannot be interrupted on many platform APIs. This means that even when specifying a timeout, you are not guaranteed to see a timeout exception until at least after however long process creation takes.
- **Extremely Small Timeout Values:** Setting very small timeout values (such as a few milliseconds) may result in almost immediate `TimeoutExpired` exceptions because process creation and system scheduling inherently require time.

Converting an argument sequence to a string on Windows

On Windows, an `args` sequence is converted to a string that can be parsed using the following rules (which correspond to the rules used by the MS C runtime):

- Arguments are delimited by white space, which is either a space or a tab.
- A string surrounded by double quotation marks is interpreted as a single argument, regardless of white space contained within. A quoted string can be embedded in an argument.
- A double quotation mark preceded by a backslash is interpreted as a literal double quotation mark.

- Backslashes are interpreted literally, unless they immediately precede a double quotation mark.
- If backslashes immediately precede a double quotation mark, every pair of backslashes is interpreted as a literal backslash. If the number of backslashes is odd, the last backslash escapes the next double quotation mark as described in rule 3.

See also

shlex

Module which provides function to parse and escape command lines.

Disable use of `posix_spawn()`

On Linux, `subprocess` defaults to using the `fork()` system call internally when it is safe to do so rather than `fork()`. This greatly improves performance.

```
subprocess._USE_POSIX_SPAWN = False # See CPython issue gh-NNNNNN.
```

It is safe to set this to false on any Python version. It will have no effect on older or newer versions where unsupported. Do not assume the attribute is available to read. Despite the name, a true value does not indicate the corresponding function will be used, only that it may be.

Please file issues any time you have to use these private knobs with a way to reproduce the issue you were seeing. Link to that issue from a comment in your code.

Added in version 3.8: `_USE_POSIX_SPAWN`

sched— Event scheduler

Source code: `Lib/sched.py`

The `sched` module defines a class which implements a general purpose event scheduler:

`class sched.scheduler(timefunc=time.monotonic, delayfunc=time.sleep)`

The `scheduler` class defines a generic interface to scheduling events. It needs two functions to actually deal with the “outside world” — `timefunc` should be callable without arguments, and return a number (the “time”, in any units whatsoever). The `delayfunc` function should be callable with one argument, compatible with the output of `timefunc`, and should delay that many time units. `delayfunc` will also be called with the argument 0 after each event is run to allow other threads an opportunity to run in mult

Example:

```
>>> import sched, time
>>> s = sched.scheduler(time.time, time.sleep)
>>> def print_time(a='default'):
...     print("From print_time", time.time(), a)
...
>>> def print_some_times():
...     print(time.time())
...     s.enter(10, 1, print_time)
...     s.enter(5, 2, print_time, argument=('positional',))
...     # despite having higher priority, 'keyword' runs after 'positional' as enter() is relative
...     s.enter(5, 1, print_time, kwargs={'a': 'keyword'})
...     s.enterabs(1_650_000_000, 10, print_time, argument=("first enterabs",))
...     s.enterabs(1_650_000_000, 5, print_time, argument=("second enterabs",))
...     s.run()
...     print(time.time())
...
>>> print_some_times()
1652342830.3640375
From print_time 1652342830.3642538 second enterabs
From print_time 1652342830.3643398 first enterabs
From print_time 1652342835.3694863 positional
From print_time 1652342835.3696074 keyword
From print_time 1652342840.369612 default
1652342840.3697174
```

Scheduler Objects

`scheduler` instances have the following methods and attributes:

`scheduler.enterabs(time, priority, action, argument=(), kwargs={})`

Schedule a new event. The `time` argument should be a numeric type compatible with the return value of the `timefunc` function passed to the constructor. Events scheduled for the same time will be executed in the order of their priority. A lower number represents a higher priority. Executing the event means executing `action(*argument, **kwargs)`. `argument` is a sequence holding the positional arguments for `action`. `kwargs` is a dictionary holding the keyword arguments for `action`. Return value is an event which may be u

`scheduler.enter(delay, priority, action, argument=(), kwargs={})`

Schedule an event for `delay` more time units. Other than the relative time, the other arguments, the effect and the return value are the same as those for `enterabs()`. Changed in version 3.3: `argument` parameter is optional. Changed in version 3.3: `kwargs` parameter was added.

scheduler.cancel(event)

Remove the event from the queue. If event is not an event currently in the queue, this method will raise a ValueError.

scheduler.empty()

Return True if the event queue is empty.

scheduler.run(blocking=True)

Run all scheduled events. This method will wait (using the delay function passed to the constructor) for the next event, then execute it and so on until there are no more scheduled events. If blocking is false executes the scheduled events due to expire soonest (if any) and then return the deadline of the next scheduled call in the scheduler (if any). Either action or delay function can raise an exception. In either case, the scheduler will maintain a consistent state and propagate the exception. If an

scheduler.queue

Read-only attribute returning a list of upcoming events in the order they will be run. Each event is shown as a named tuple with the following fields: time, priority, action, argument, kwargs.

queue— A synchronized queue class

Source code: `Lib/queue.py`

The `queue` module implements multi-producer, multi-consumer queues. It is especially useful in threaded programming when information must be exchanged safely between multiple threads. The `Queue` class in this module implements all the required locking semantics.

The module implements three types of queue, which differ only in the order in which the entries are retrieved. In a `FIFOQueue`, the first tasks added are the first retrieved. In a `LIFOQueue`, the most recently added entry is the first retrieved (operating like a stack). With a priority queue, the entries are kept sorted (using the `heapq` module) and the lowest valued entry is retrieved first.

Internally, those three types of queues use locks to temporarily block competing threads; however, they are not designed to handle reentrancy within a thread.

In addition, the module implements a “simple” `FIFOQueue` type, `SimpleQueue`, whose specific implementation provides additional guarantees in exchange for the smaller functionality.

The `queue` module defines the following classes and exceptions:

`class queue.Queue(maxsize=0)`

Constructor for a `FIFOQueue`. `maxsize` is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If `maxsize` is less than or equal to zero, the queue size is infinite.

`class queue.LifoQueue(maxsize=0)`

Constructor for a `LIFOQueue`. `maxsize` is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If `maxsize` is less than or equal to zero, the queue size is infinite.

`class queue.PriorityQueue(maxsize=0)`

Constructor for a priority queue. `maxsize` is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If `maxsize` is less than or equal to zero, the queue size is infinite. The lowest valued entries are retrieved first (the lowest valued entry is the one that would be returned by `min(entries)`). A typical pattern for entries is a tuple in the form: `(priority_number, data)`. If th

`class queue.SimpleQueue`

Constructor for an unbounded `FIFOQueue`. Simple queues lack advanced functionality such as task tracking. Added in version 3.7.

`exception queue.Empty`

Exception raised when `non-blockingget()` (or `get_nowait()`) is called on a `Queue` object which is empty.

`exception queue.Full`

Exception raised when `non-blockingput()` (or `put_nowait()`) is called on a `Queue` object which is full.

`exception queue.ShutDown`

Exception raised when `put()` (or `get()`) is called on a `Queue` object which has been shut down. Added in version 3.13.

Queue Objects

Queue objects (`Queue`, `LifoQueue`, or `PriorityQueue`) provide the public methods described below.

Queue.qsize()

Return the approximate size of the queue. Note, `qsize() > 0` doesn't guarantee that a subsequent `get()` will not block, nor will `qsize() < maxsize` guarantee that `put()` will not block.

Queue.empty()

Return `True` if the queue is empty, `False` otherwise. If `empty()` returns `True` it doesn't guarantee that a subsequent call to `put()` will not block. Similarly, if `empty()` returns `False` it doesn't guarantee that a subsequent call to `get()` will not block.

Queue.full()

Return `True` if the queue is full, `False` otherwise. If `full()` returns `True` it doesn't guarantee that a subsequent call to `get()` will not block. Similarly, if `full()` returns `False` it doesn't guarantee that a subsequent call to `put()` will not block.

Queue.put(item, block=True, timeout=None)

Put `item` into the queue. If optional arg `block` is `True` and `timeout` is `None` (the default), block if necessary until a free slot is available. If `timeout` is a positive number, it blocks at most `timeout` seconds and raises the `Full` exception if no free slot was available within that time. Otherwise (`block` is `False`), put an item on the queue if a free slot is immediately available, else raise the `Full` exception (`timeout` is ignored in that case). Raises `Shutdown` if the queue has been shut down.

Queue.put_nowait(item)

Equivalent to `put(item, block=False)`.

Queue.get(block=True, timeout=None)

Remove and return an item from the queue. If optional arg `block` is `True` and `timeout` is `None` (the default), block if necessary until an item is available. If `timeout` is a positive number, it blocks at most `timeout` seconds and raises the `Empty` exception if no item was available within that time. Otherwise (`block` is `False`), return an item if one is immediately available, else raise the `Empty` exception (`timeout` is ignored in that case). Prior to 3.0 on POSIX systems, and for all versions on Windows, if `block` is `True`

Queue.get_nowait()

Equivalent to `get(False)`.

Two methods are offered to support tracking whether enqueued tasks have been fully processed by daemon consumer threads.

Queue.task_done()

Indicate that a formerly enqueued task is complete. Used by queue consumer threads. For each `get()` used to fetch a task, a subsequent call to `task_done()` tells the queue that the processing on the task is complete. If `join()` is currently blocking, it will resume when all items have been processed (meaning that a `task_done()` call was received for every item that had been `put()` into the queue). Raises `ValueError` if called more times than there were items placed in the queue.

Queue.join()

Blocks until all items in the queue have been gotten and processed. The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer thread calls `task_done()` to indicate that the item was retrieved and all work on it is complete. When the count of unfinished tasks drops to zero, `join()` unblocks.

Waiting for task completion

Example of how to wait for enqueued tasks to be completed:

```
import threading
import queue

q = queue.Queue()

def worker():
    while True:
        item = q.get()
        print(f'Working on {item}')
        print(f'Finished {item}')
        q.task_done()

# Turn-on the worker thread.
threading.Thread(target=worker, daemon=True).start()

# Send thirty task requests to the worker.
for item in range(30):
    q.put(item)

# Block until all tasks are done.
q.join()
print('All work completed')
```

Terminating queues

When no longer needed, Queue objects can be wound down until empty or terminated immediately with a hard shutdown.

Queue.shutdown(immediate=False)

Put a Queue instance into a shutdown mode. The queue can no longer grow. Future calls to put() will raise ShutdownError. Currently blocked callers of get() will be unblocked and will raise ShutdownError in the formerly blocked thread. If immediate is false (the default), the queue can be wound down normally with get() calls to extract tasks that have already been loaded. And if task_done() is called for each remaining task, a pending join() will be unblocked normally. Once the queue is empty, future calls to get() will raise ShutdownError.

SimpleQueue Objects

SimpleQueue objects provide the public methods described below.

SimpleQueue.qsize()

Return the approximate size of the queue. Note, qsize() > 0 doesn't guarantee that a subsequent get() will not block.

SimpleQueue.empty()

Return True if the queue is empty, False otherwise. If empty() returns False it doesn't guarantee that a subsequent call to get() will not block.

SimpleQueue.put(item, block=True, timeout=None)

Put item into the queue. The method never blocks and always succeeds (except for potential low-level errors such as failure to allocate memory). The optional args block and timeout are ignored and only provided for compatibility with Queue.put(). CPython implementation detail: This method has a C implementation which is reentrant. That is, a put() or get() call can be interrupted by another put() call in the same thread without deadlocking or corrupting internal state inside the queue. This makes it appropriate

SimpleQueue.put_nowait(item)

Equivalent to `put(item, block=False)`, provided for compatibility with `Queue.put_nowait()`.

SimpleQueue.get(block=True, timeout=None)

Remove and return an item from the queue. If optional arg `block` is true and `timeout` is `None` (the default), block if necessary until an item is available. If `timeout` is a positive number, it blocks at most `timeout` seconds and raises the `EmptyException` if no item was available within that time. Otherwise (`block` is false), return an item if one is immediately available, else raise the `EmptyException` (`timeout` is ignored in that case).

SimpleQueue.get_nowait()

Equivalent to `get(False)`.

See also

Class `multiprocessing.Queue`

A queue class for use in a multi-processing (rather than multi-threading) context.

`collections.deque` is an alternative implementation of unbounded queues with fast `atomicappend()` and `popleft()` operations that do not require locking and also support indexing.

contextvars— Context Variables

This module provides APIs to manage, store, and access context-local state. The `ContextVar` class is used to declare and work with Context Variables. The `copy_context()` function and the `Context` class should be used to manage the current context in asynchronous frameworks.

Context managers that have state should use Context Variables instead of `threading.local()` to prevent their state from bleeding to other code unexpectedly, when used in concurrent code.

See also [PEP 567](#) for additional details.

Added in version 3.7.

Context Variables

class `contextvars.ContextVar(name[,*,default])`

name

get([default])

set(value)

reset(token)

This class is used to declare a new Context Variable, e.g.: `var:ContextVar[int]=ContextVar('var',default=42)` The required `name` parameter is used for introspection and debug purposes. The optional keyword-only `default` parameter is returned by `ContextVar.get()` when no value for the variable is found in the current context. Important: Context Variables should be created at the top module level and never in closures. Context objects hold strong references to context variables which prevents context variables from

The name of the variable. This is a read-only property. Added in version 3.7.1.

Return a value for the context variable for the current context. If there is no value for the variable in the current context, the method will: return the value of the `default` argument of the method, if provided; or return the default value for the context variable, if it was created with one; or raise a `LookupError`.

Call to set a new value for the context variable in the current context. The required `value` argument is the new value for the context variable. Returns a `Token` object that can be used to restore the variable to its previous value via the `ContextVar.reset()` method. For convenience, the token object can be used as a context manager to avoid calling `ContextVar.reset()` manually: `var=ContextVar('var',default='default value')` with `var.set('new value')`: `assert var.get()=='new value'` `assert var.get()=='default value'` It is

Reset the context variable to the value it had before the `ContextVar.set()` that created the `token` was used. For example: `var=ContextVar('var')` `token=var.set('new value')` # code that uses 'var'; `var.get()` returns 'new value'. `var.reset(token)` # After the reset call the `var` has no value again, so `var.get()` would raise a `LookupError`. The same `token` cannot be used twice.

class `contextvars.Token`

var

old_value

MISSING

`Token` objects are returned by the `ContextVar.set()` method. They can be passed to the `ContextVar.reset()` method to revert the value of the variable to what it was before the corresponding `set`. A single token cannot reset a context variable more than once. Tokens support the context manager protocol to

automatically reset context variables. See `ContextVar.set()`. Added in version 3.14: Added support for usage as a context manager. `var` a read-only property. Points to the `ContextVar` object that created the token.

A read-only property. Points to the `ContextVar` object that created the token.

A read-only property. Set to the value the variable had before the `ContextVar.set()` method call that created the token. It points to `Token.MISSING` if the variable was not set before the call.

A marker object used by `Token.old_value`.

Manual Context Management

`contextvars.copy_context()`

Returns a copy of the current `Context` object. The following snippet gets a copy of the current context and prints all variables and their values that are set in it: `ctx: Context = copy_context(); print(list(ctx.items()))`. The function has an $O(1)$ complexity, i.e. works equally fast for contexts with a few context variables and for contexts that have a lot of them.

`class contextvars.Context`

`run(callable, *args, **kwargs)`

`copy()`

`var_in_context`

`context[var]`

`get(var[, default])`

`iter(context)`

`len(proxy)`

`keys()`

`values()`

`items()`

A mapping of `ContextVar`s to their values. `Context()` creates an empty context with no values in it. To get a copy of the current context use the `copy_context()` function. Each thread has its own effective stack of `Context` objects. The current context is the `Context` object at the top of the current thread's stack. All `Context` objects in the stacks are considered to be entered. Entering a context, which can be done by calling its `run()` method, makes the context the current context by pushing it onto the top of the cu

Enters the Context, executes `callable(*args, **kwargs)`, then exits the Context. Returns `callable`'s return value, or propagates an exception if one

occurred. Example:

```
import contextvars
var = contextvars.ContextVar('var')
var.set('spam')
print(var.get()) # 'spam'
ctx = contextvars.copy_context()
def main():
    # 'var' was set to 'spam' before # calling 'copy_context()' and
    # 'ctx.run(main)', so:
    print(var.get()) # 'spam'
    print(ctx[var]) # 'spam'
    var.set('ham')
    # Now, after setting 'var' to 'ham':
    print(var.get()) # 'ham'
    print(
```

Return a shallow copy of the context object.

Return `True` if the context has a value for `var` set; return `False` otherwise.

Return the value of the `var` `ContextVar` variable. If the variable is not set in the context object, a `KeyError` is raised.

Return the value for `var` if `var` has the value in the context object. Return `default` otherwise. If `default` is not given, return `None`.

Return an iterator over the variables stored in the context object.

Return the number of variables set in the context object.

Return a list of all variables in the context object.

Return a list of all variables' values in the context object.

Return a list of 2-tuples containing all variables and their values in the context object.

asyncio support

Context variables are natively supported in `asyncio` and are ready to be used without any extra configuration. For example, here is a simple echo server, that uses a context variable to make the address of a remote client available in the Task that handles that client:

```
import asyncio
import contextvars

client_addr_var = contextvars.ContextVar('client_addr')

def render_goodbye():
    # The address of the currently handled client can be accessed
    # without passing it explicitly to this function.

    client_addr = client_addr_var.get()
    return f'Good bye, client @ {client_addr}\r\n'.encode()

async def handle_request(reader, writer):
    addr = writer.transport.get_extra_info('socket').getpeername()
    client_addr_var.set(addr)

    # In any code that we call is now possible to get
    # client's address by calling 'client_addr_var.get()'.

    while True:
        line = await reader.readline()
        print(line)
        if not line.strip():
            break

    writer.write(b'HTTP/1.1 200 OK\r\n') # status line
    writer.write(b'\r\n') # headers
    writer.write(render_goodbye()) # body
    writer.close()

async def main():
    srv = await asyncio.start_server(
        handle_request, '127.0.0.1', 8081)

    async with srv:
        await srv.serve_forever()

asyncio.run(main())

# To test it you can use telnet or curl:
#     telnet 127.0.0.1 8081
#     curl 127.0.0.1:8081
```

`_thread`— Low-level threading API

This module provides low-level primitives for working with multiple threads (also called light-weight processes or tasks) — multiple threads of control sharing their global data space. For synchronization, simple locks (also called mutexes or binary semaphores) are provided. The `threading` module provides an easier to use and higher-level threading API built on top of this module.

Changed in version 3.7: This module used to be optional, it is now always available.

This module defines the following constants and functions:

`exception _thread.error`

Raised on thread-specific errors. Changed in version 3.3: This is now a synonym of the built-in `RuntimeError`.

`_thread.LockType`

This is the type of lock objects.

`_thread.start_new_thread(function, args[, kwargs])`

Start a new thread and return its identifier. The thread executes the `function` with the argument `args` (which must be a tuple). The optional `kwargs` argument specifies a dictionary of keyword arguments. When the function returns, the thread silently exits. When the function terminates with an unhandled exception, `sys.unraisablehook()` is called to handle the exception. The `object` attribute of the `hook` argument is `function`. By default, a stack trace is printed and then the thread exits (but other

`_thread.interrupt_main(signum=signal.SIGINT, /)`

Simulate the effect of a signal arriving in the main thread. A thread can use this function to interrupt the main thread, though there is no guarantee that the interruption will happen immediately. If given, `signum` is the number of the signal to simulate. If `signum` is not given, `signal.SIGINT` is simulated. If the given signal isn't handled by Python (it was set to `signal.SIG_DFL` or `signal.SIG_IGN`), this function does nothing. Changed in version 3.10: The `signum` argument is added to customize the signal number.

`_thread.exit()`

Raise the `SystemExit` exception. When not caught, this will cause the thread to exit silently.

`_thread.allocate_lock()`

Return a new lock object. Methods of locks are described below. The lock is initially unlocked.

`_thread.get_ident()`

Return the 'thread identifier' of the current thread. This is a nonzero integer. Its value has no direct meaning; it is intended as a magic cookie to be used e.g. to index a dictionary of thread-specific data. Thread identifiers may be recycled when a thread exits and another thread is created.

`_thread.get_native_id()`

Return the native integral Thread ID of the current thread assigned by the kernel. This is a non-negative integer. Its value may be used to uniquely identify this particular thread system-wide (until the thread terminates, after which the value may be recycled by the OS). Availability: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD. Added in version 3.8. Changed in version 3.13: Added support for GNU/kFreeBSD.

`_thread.stack_size([size])`

Return the thread stack size used when creating new threads. The optional `size` argument specifies the stack size to be used for subsequently created threads, and must be 0 (use platform or configured default) or a

positive integer value of at least 32,768 (32 KiB). If `size` is not specified, 0 is used. If changing the thread stack size is unsupported, a `RuntimeError` is raised. If the specified stack size is invalid, a `ValueError` is raised and the stack size is unmodified. 32 KiB is currently the mini

`_thread.TIMEOUT_MAX`

The maximum value allowed for the `timeout` parameter of `Lock.acquire`. Specifying a timeout greater than this value will raise an `OverflowError`. Added in version 3.2.

Lock objects have the following methods:

`lock.acquire(blocking=True, timeout=-1)`

Without any optional argument, this method acquires the lock unconditionally, if necessary waiting until it is released by another thread (only one thread at a time can acquire a lock — that's their reason for existence). If the `blocking` argument is present, the action depends on its value: if it is false, the lock is only acquired if it can be acquired immediately without waiting, while if it is true, the lock is acquired unconditionally as above. If the floating-point `timeout` argument is present and

`lock.release()`

Releases the lock. The lock must have been acquired earlier, but not necessarily by the same thread.

`lock.locked()`

Return the status of the lock: `True` if it has been acquired by some thread, `False` if not.

In addition to these methods, lock objects can also be used via the `with` statement, e.g.:

```
import _thread

a_lock = _thread.allocate_lock()

with a_lock:
    print("a_lock is locked while this executes")
```

Caveats:

- Interrupts always go to the main thread (the `KeyboardInterrupt` exception will be received by that thread.)
- Calling `sys.exit()` or raising the `SystemExit` exception is equivalent to calling `_thread.exit()`.
- When the main thread exits, it is system defined whether the other threads survive. On most systems, they are killed without executing `try...finally` clauses or executing object destructors.

Networking and Interprocess Communication

The modules described in this chapter provide mechanisms for networking and inter-processes communication.

Some modules only work for two processes that are on the same machine, e.g. `signal` and `mmap`. Other modules support networking protocols that two or more processes can use to communicate across machines.

The list of modules described in this chapter is:

- `asyncio`— Asynchronous I/O
- `socket`— Low-level networking interface
- `ssl`— TLS/SSL wrapper for socket objects
- `select`— Waiting for I/O completion
- `selectors`— High-level I/O multiplexing
- `signal`— Set handlers for asynchronous events
- `mmap`— Memory-mapped file support

asyncio— Asynchronous I/O

Hello World!

```
import asyncio

async def main():
    print('Hello ...')
    await asyncio.sleep(1)
    print('... World!')

asyncio.run(main())
```

asyncio is a library to write concurrent code using the `async/await` syntax.

asyncio is used as a foundation for multiple Python asynchronous frameworks that provide high-performance network and web-servers, database connection libraries, distributed task queues, etc.

asyncio is often a perfect fit for IO-bound and high-level structured network code.

See also

A Conceptual Overview of asyncio

Explanation of the fundamentals of asyncio.

asyncio provides a set of high-level APIs to:

- run Python coroutines concurrently and have full control over their execution;
- perform network IO and IPC;
- control subprocesses;
- distribute tasks via queues;
- synchronize concurrent code;

Additionally, there are low-level APIs for library and framework developers to:

- create and manage event loops, which provide asynchronous APIs for networking, running subprocesses, handling OS signals, etc;
- implement efficient protocols using transports;
- bridge callback-based libraries and code with `async/await` syntax.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

asyncio REPL

You can experiment with an asyncio concurrent context in the REPL:

```
$ python -m asyncio
asyncio REPL ...
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license" for more information.
>>> import asyncio
>>> await asyncio.sleep(10, result='hello')
'hello'
```

This REPL provides limited compatibility with `PYTHON_BASIC_REPL`. It is recommended that the default REPL is used for full functionality and the latest features.

Raises an `audit event` `python.run_stdin` with no arguments.

Changed in version 3.12.5: (also 3.11.10, 3.10.15, 3.9.20, and 3.8.20) Emits audit events.

Changed in version 3.13: Uses PyREPL if possible, in which case `PYTHONSTARTUP` is also executed. Emits audit events.

Reference

High-level APIs

- [Runners](#)
- [Coroutines and tasks](#)
- [Streams](#)
- [Synchronization Primitives](#)
- [Subprocesses](#)
- [Queues](#)
- [Exceptions](#)
- [Call Graph Introspection](#)

Low-level APIs

- [Event loop](#)
- [Futures](#)
- [Transports and Protocols](#)
- [Policies](#)
- [Platform Support](#)
- [Extending](#)

Guides and Tutorials

- [High-level API Index](#)
- [Low-level API Index](#)
- [Developing with asyncio](#)

■ **Nota:** The source code for `asyncio` can be found in `Lib/asyncio/`.

socket— Low-level networking interface

Source code: `Lib/socket.py`

This module provides access to the BSD socket interface. It is available on all modern Unix systems, Windows, MacOS, and probably additional platforms.

■ Nota: NoteSome behavior may be platform dependent, since calls are made to the operating system socket APIs.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See WebAssembly platforms for more information.

The Python interface is a straightforward transliteration of the Unix system call and library interface for sockets to Python's object-oriented style: the `socket()` function returns a socket object whose methods implement the various socket system calls. Parameter types are somewhat higher-level than in the C interface: as `withread()` and `write()` operations on Python files, buffer allocation on receive operations is automatic, and buffer length is implicit on send operations.

See also

Module `socketserver`

Module `ssl`

Classes that simplify writing network servers.

A TLS/SSL wrapper for socket objects.

Socket families

Depending on the system and the build options, various socket families are supported by this module.

The address format required by a particular socket object is automatically selected based on the address family specified when the socket object was created. Socket addresses are represented as follows:

- The address of an `AF_UNIX` socket bound to a file system node is represented as a string, using the file system encoding and the 'surrogateescape' error handler (see PEP 383). An address in Linux's abstract namespace is returned as a bytes-like object with an initial null byte; note that sockets in this namespace can communicate with normal file system sockets, so programs intended to run on Linux may need
- A pair `(host, port)` is used for the `AF_INET` address family, where `host` is a string representing either a hostname in internet domain notation like `'daring.cwi.nl'` or an IPv4 address like `'100.50.200.5'`, and `port` is an integer. For IPv4 addresses, two special forms are accepted instead of a host address: `''` represents `INADDR_ANY`, which is used to bind to all interfaces, and the string `'<broadcast>'` represents `INADDR_MULTICAST`
- For `AF_INET6` address family, a four-tuple `(host, port, flowinfo, scope_id)` is used, where `flowinfo` and `scope_id` represent the `sin6_flowinfo` and `sin6_scope_id` members in `struct sockaddr_in6` in C. For socket module methods, `flowinfo` and `scope_id` can be omitted just for backward compatibility. Note, however, omission of `scope_id` can cause problems in manipulating scoped IPv6 addresses. Changed in version 3.7: For multicast address
- `AF_NETLINK` sockets are represented as pairs `(pid, groups)`.
- Linux-only support for TIPC is available using the `AF_TIPC` address family. TIPC is an open, non-IP based networked protocol designed for use in clustered computer environments. Addresses are represented by a

tuple, and the fields depend on the address type. The general tuple form is (addr_type, v1, v2, v3[, scope]), where: addr_type is one of TIPC_ADDR_NAMESEQ, TIPC_ADDR_NAME, or TIPC_ADDR_ID. scope is one of

- A tuple (interface,) is used for the AF_CAN address family, where interface is a string representing a network interface name like 'can0'. The network interface name "can" can be used to receive packets from all network interfaces of this family. CAN_ISOTP protocol requires a tuple (interface, rx_addr, tx_addr) where both additional parameters are unsigned long integer that represent a CAN identifier (standard or extended).
 - A string or a tuple (id, unit) is used for the SYSPROTO_CONTROL protocol of the PF_SYSTEM family. The string is the name of a kernel control using a dynamically assigned ID. The tuple can be used if ID and unit number of the kernel control are known or if a registered ID is used. Added in version 3.3.
 - AF_BLUETOOTH supports the following protocols and address formats: BTPROTO_L2CAP accepts a tuple (bdaddr, psm[, cid[, bdaddr_type]]) where: bdaddr is a string specifying the Bluetooth address. psm is an integer specifying the Protocol/Service Multiplexer. cid is an optional integer specifying the Channel Identifier. If not given, defaults to zero. bdaddr_type is an optional integer specifying the address type; on
 - AF_ALG is a Linux-only socket based interface to Kernel cryptography. An algorithm socket is configured with a tuple of two to four elements (type, name[, feat[, mask]]), where: type is the algorithm type as string, e.g. aead, hash, skcipher or rng. name is the algorithm name and operation mode as string, e.g. sha256, hmac (sha256), cbc (aes) or drbg_nopr_ctr_aes256. feat and mask are unsigned 32bit integers. Availability:
 - AF_VSOCK allows communication between virtual machines and their hosts. The sockets are represented as a (CID, port) tuple where the context ID or CID and port are integers. Availability: Linux >= 3.9. See vsock(7). Added in version 3.7.
 - AF_PACKET is a low-level interface directly to network devices. The addresses are represented by the tuple (ifname, proto[, pkttype[, hatype[, addr]]]) where: ifname- String specifying the device name. proto- The Ethernet protocol number. May be ETH_P_ALL to capture all protocols, one of the ETHertype_* constants or any other Ethernet protocol number. pkttype- Optional integer specifying the packet type: PACKET_
 - AF_QIPCRTR is a Linux-only socket based interface for communicating with services running on co-processors in Qualcomm platforms. The address family is represented as a (node, port) tuple where the node and port are non-negative integers. Availability: Linux >= 4.7. Added in version 3.8.
 - IPPROTO_UDPLITE is a variant of UDP which allows you to specify what portion of a packet is covered with the checksum. It adds two socket options that you can change. self.setsockopt(IPPROTO_UDPLITE, UDPLITE_SEND_CSCOV, length) will change what portion of outgoing packets are covered by the checksum and self.setsockopt(IPPROTO_UDPLITE, UDPLITE_RECV_CSCOV, length) will filter out packets which cover too little.
 - AF_HYPERV is a Windows-only socket based interface for communicating with Hyper-V hosts and guests. The address family is represented as a (vm_id, service_id) tuple where the vm_id and service_id are UUID strings. The vm_id is the virtual machine identifier or a set of known VMID values if the target is not a specific virtual machine. Known VMID constants defined on socket are: HV_GUID_ZERO, HV_GUID_BROADCAST, HV_G
- If you use a hostname in the host portion of IPv4/v6 socket address, the program may show a nondeterministic behavior, as Python uses the first address returned from the DNS resolution. The socket address will be resolved differently into an actual IPv4/v6 address, depending on the results from DNS resolution and/or the host configuration. For deterministic behavior use a numeric address in the host portion.

All errors raise exceptions. The normal exceptions for invalid argument types and out-of-memory conditions can be raised. Errors related to socket or address semantics raise OSError or one of its subclasses.

Non-blocking mode is supported through setblocking(). A generalization of this based on timeouts is supported through settimeout().

Module contents

The module `socket` exports the following elements.

Exceptions

`exception.socket.error`

A deprecated alias of `OSError`. Changed in version 3.3: Following PEP 3151, this class was made an alias of `OSError`.

`exception.socket.herror`

A subclass of `OSError`, this exception is raised for address-related errors, i.e. for functions that use `h_errno` in the POSIX C API, including `gethostbyname_ex()` and `gethostbyaddr()`. The accompanying value is a pair `(h_errno, string)` representing an error returned by a library call. `h_errno` is a numeric value, while `string` represents the description of `h_errno`, as returned by the `hstrerror()` C function. Changed in version 3.3: This class was made a subclass of `OSError`.

`exception.socket.gaierror`

A subclass of `OSError`, this exception is raised for address-related errors by `getaddrinfo()` and `getnameinfo()`. The accompanying value is a pair `(error, string)` representing an error returned by a library call. `string` represents the description of error, as returned by the `gai_strerror()` C function. The numeric error value will match one of the `EAI_*` constants defined in this module. Changed in version 3.3: This class was made a subclass of `OSError`.

`exception.socket.timeout`

A deprecated alias of `TimeoutError`. A subclass of `OSError`, this exception is raised when a timeout occurs on a socket which has had timeouts enabled via a prior call to `settimeout()` (or implicitly through `setdefaulttimeout()`). The accompanying value is a string whose value is currently always “timed out”. Changed in version 3.3: This class was made a subclass of `OSError`. Changed in version 3.10: This class was made an alias of `TimeoutError`.

Constants

The `AF_*` and `SOCK_*` constants are now `AddressFamily` and `SocketKind` `IntEnum` collections.

Added in version 3.4.

`socket.AF_UNIX`

`socket.AF_INET`

`socket.AF_INET6`

These constants represent the address (and protocol) families, used for the first argument to `socket()`. If the `AF_UNIX` constant is not defined then this protocol is unsupported. More constants may be available depending on the system.

`socket.AF_UNSPEC`

`AF_UNSPEC` means that `getaddrinfo()` should return socket addresses for any address family (either IPv4, IPv6, or any other) that can be used.

`socket.SOCK_STREAM`

`socket.SOCK_DGRAM`

`socket.SOCK_RAW`

`socket.SOCK_RDM`

socket.SOCK_SEQPACKET

These constants represent the socket types, used for the second argument to `socket()`. More constants may be available depending on the system. (Only `SOCK_STREAM` and `SOCK_DGRAM` appear to be generally useful.)

socket.SOCK_CLOEXEC

socket.SOCK_NONBLOCK

These two constants, if defined, can be combined with the socket types and allow you to set some flags atomically (thus avoiding possible race conditions and the need for separate calls). See also [Secure File Descriptor Handling](#) for a more thorough explanation. Availability: Linux $\geq$ 2.6.27. Added in version 3.2.

SO_*

socket.SOMAXCONN

MSG_*

SOL_*

SCM_*

IPPROTO_*

IPPORT_*

INADDR_*

IP_*

IPV6_*

EAI_*

AI_*

NI_*

TCP_*

Many constants of these forms, documented in the Unix documentation on sockets and/or the IP protocol, are also defined in the socket module. They are generally used in arguments to `setsockopt()` and `getsockopt()` methods of socket objects. In most cases, only those symbols that are defined in the Unix header files are defined; for a few symbols, default values are provided. Changed in version 3.6: `SO_DOMAIN`, `SO_PROTOCOL`, `SO_PEERSEC`, `SO_PASSSEC`, `TCP_USER_TIMEOUT`, `TCP_CONGESTION` were added. Changed in version 3.14: Restored missing `CAN_RAW_ERR_FILTER` on Linux.

socket.AF_CAN

socket.PF_CAN

SOL_CAN_*

CAN_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module. Availability: Linux $\geq$ 2.6.25, NetBSD $\geq$ 8. Added in version 3.3. Changed in version 3.11: NetBSD support was added. Changed in version 3.14: Restored missing `CAN_RAW_ERR_FILTER` on Linux.

socket.CAN_BCM

CAN_BCM_*

`CAN_BCM`, in the CAN protocol family, is the broadcast manager (BCM) protocol. Broadcast manager constants, documented in the Linux documentation, are also defined in the socket module. Availability: Linux $\geq$

2.6.25. Note The `CAN_BCM_CAN_FD_FRAME` flag is only available on Linux ≥ 4.8 . Added in version 3.4.

socket.CAN_RAW_FD_FRAMES

Enables CAN FD support in a `CAN_RAW` socket. This is disabled by default. This allows your application to send both CAN and CAN FD frames; however, you must accept both CAN and CAN FD frames when reading from the socket. This constant is documented in the Linux documentation. Availability: Linux ≥ 3.6 . Added in version 3.5.

socket.CAN_RAW_JOIN_FILTERS

Joins the applied CAN filters such that only CAN frames that match all given CAN filters are passed to user space. This constant is documented in the Linux documentation. Availability: Linux ≥ 4.1 . Added in version 3.9.

socket.CAN_ISOTP

`CAN_ISOTP`, in the CAN protocol family, is the ISO-TP (ISO 15765-2) protocol. ISO-TP constants, documented in the Linux documentation. Availability: Linux $\geq 2.6.25$. Added in version 3.7.

socket.CAN_J1939

`CAN_J1939`, in the CAN protocol family, is the SAE J1939 protocol. J1939 constants, documented in the Linux documentation. Availability: Linux ≥ 5.4 . Added in version 3.9.

socket.AF_DIVERT

socket.PF_DIVERT

These two constants, documented in the FreeBSD `divert(4)` manual page, are also defined in the socket module. Availability: FreeBSD ≥ 14.0 . Added in version 3.12.

socket.AF_PACKET

socket.PF_PACKET

PACKET_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module. Availability: Linux ≥ 2.2 .

socket.ETH_P_ALL

`ETH_P_ALL` can be used in the `socket` constructor as proto for the `AF_PACKET` family in order to capture every packet, regardless of protocol. For more information, see the `packet(7)` manpage. Availability: Linux. Added in version 3.12.

socket.AF_RDS

socket.PF_RDS

socket.SOL_RDS

RDS_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module. Availability: Linux $\geq 2.6.30$. Added in version 3.3.

socket.SIO_RCVALL

socket.SIO_KEEPA_LIVE_VALS

socket.SIO_LOOPBACK_FAST_PATH

RCVALL_*

Constants for Windows' WSAlctl(). The constants are used as arguments to the ioctl() method of socket objects. Changed in version 3.6: SIO_LOOPBACK_FAST_PATH was added.

TIPC_*

TIPC related constants, matching the ones exported by the C socket API. See the TIPC documentation for more information.

socket.AF_ALG

socket.SOL_ALG

ALG_*

Constants for Linux Kernel cryptography. Availability: Linux >= 2.6.38. Added in version 3.6.

socket.AF_VSOCK

socket.IOCTL_VM_SOCKETS_GET_LOCAL_CID

VMADDR*

SO_VM*

Constants for Linux host/guest communication. Availability: Linux >= 4.8. Added in version 3.7.

socket.AF_LINK

Availability: BSD, macOS. Added in version 3.4.

socket.has_ipv6

This constant contains a boolean value which indicates if IPv6 is supported on this platform.

socket.AF_BLUETOOTH

socket.BTPROTO_L2CAP

socket.BTPROTO_RFCOMM

socket.BTPROTO_HCI

socket.BTPROTO_SCO

Integer constants for use with Bluetooth addresses.

socket.BDADDR_ANY

socket.BDADDR_LOCAL

These are string constants containing Bluetooth addresses with special meanings. For example, BDADDR_ANY can be used to indicate any address when specifying the binding socket with BTPROTO_RFCOMM.

socket.BDADDR_BREDR

socket.BDADDR_LE_PUBLIC

socket.BDADDR_LE_RANDOM

These constants describe the Bluetooth address type when binding or connecting a BTPROTO_L2CAP socket. Availability: Linux, FreeBSD. Added in version 3.14.

socket.SOL_RFCOMM

socket.SOL_L2CAP

socket.SOL_HCI

socket.SOL_SCO

socket.SOL_BLUETOOTH

Used in the level argument to the `setsockopt()` and `getsockopt()` methods of Bluetooth socket objects. `SOL_BLUETOOTH` is only available on Linux. Other constants are available if the corresponding protocol is supported.

SO_L2CAP_*

socket.L2CAP_LM

L2CAP_LM_*

SO_RFCOMM_*

RFCOMM_LM_*

SO_SCO_*

SO_BTH_*

BT_*

Used in the option name and value argument to the `setsockopt()` and `getsockopt()` methods of Bluetooth socket objects. `BT_*` and `L2CAP_LM_*` are only available on Linux. `SO_BTH_*` are only available on Windows. Other constants may be available on Linux and various BSD platforms. Added in version 3.14.

socket.HCI_FILTER

socket.HCI_TIME_STAMP

socket.HCI_DATA_DIR

socket.SO_HCI_EVT_FILTER

socket.SO_HCI_PKT_FILTER

Option names for use with `BTPROTO_HCI`. Availability and format of the option values depend on platform. Changed in version 3.14: Added `SO_HCI_EVT_FILTER` and `SO_HCI_PKT_FILTER` on NetBSD and DragonFly BSD. Added `HCI_DATA_DIR` on FreeBSD, NetBSD and DragonFly BSD.

socket.HCI_DEV_NONE

The `device_id` value used to create an HCI socket that isn't specific to a single Bluetooth adapter. Availability: Linux. Added in version 3.14.

socket.HCI_CHANNEL_RAW

socket.HCI_CHANNEL_USER

socket.HCI_CHANNEL_MONITOR

socket.HCI_CHANNEL_CONTROL

socket.HCI_CHANNEL_LOGGING

Possible values for `channel` field in the `BTPROTO_HCI` address. Availability: Linux. Added in version 3.14.

socket.AF_QIPCRTR

Constant for Qualcomm's IPC router protocol, used to communicate with service providing remote processors. Availability: Linux >= 4.7.

socket.SCM_CREDS2

socket.LOCAL_CREDS

socket.LOCAL_CREDS_PERSISTENT

LOCAL_CREDS and LOCAL_CREDS_PERSISTENT can be used with SOCK_DGRAM, SOCK_STREAM sockets, equivalent to Linux/DragonFlyBSD SO_PASSCRED, while LOCAL_CREDS sends the credentials at first read, LOCAL_CREDS_PERSISTENT sends for each read, SCM_CREDS2 must be then used for the latter for the message type. Added in version 3.11. Availability: FreeBSD.

socket.SO_INCOMING_CPU

Constant to optimize CPU locality, to be used in conjunction with SO_REUSEPORT. Added in version 3.11. Availability: Linux >= 3.9

socket.SO_REUSEPORT_LB

Constant to enable duplicate address and port bindings with load balancing. Added in version 3.14. Availability: FreeBSD >= 12.0

socket.AF_HYPERV

socket.HV_PROTOCOL_RAW

socket.HVSOCKET_CONNECT_TIMEOUT

socket.HVSOCKET_CONNECT_TIMEOUT_MAX

socket.HVSOCKET_CONNECTED_SUSPEND

socket.HVSOCKET_ADDRESS_FLAG_PASSTHRU

socket.HV_GUID_ZERO

socket.HV_GUID_WILDCARD

socket.HV_GUID_BROADCAST

socket.HV_GUID_CHILDREN

socket.HV_GUID_LOOPBACK

socket.HV_GUID_PARENT

Constants for Windows Hyper-V sockets for host/guest communications. Availability: Windows. Added in version 3.12.

socket.ETHERTYPE_ARP

socket.ETHERTYPE_IP

socket.ETHERTYPE_IPV6

socket.ETHERTYPE_VLAN

IEEE 802.3 protocol number. constants. Availability: Linux, FreeBSD, macOS. Added in version 3.12.

socket.SHUT_RD

socket.SHUT_WR

socket.SHUT_RDWR

These constants are used by the shutdown() method of socket objects. Availability: not WASI.

Functions

Creating sockets

The following functions all create socket objects.

socket.socket(family=AF_INET,type=SOCK_STREAM,proto=0,fileno=None)

Create a new socket using the given address family, socket type and protocol number. The address family should be `AF_INET` (the default), `AF_INET6`, `AF_UNIX`, `AF_CAN`, `AF_PACKET`, or `AF_RDS`. The socket type should be `SOCK_STREAM` (the default), `SOCK_DGRAM`, `SOCK_RAW` or perhaps one of the other `SOCK_*` constants. The protocol number is usually zero and may be omitted or in the case where the address family is `AF_CAN` the protocol should be one of `CAN_RAW`, `CAN_BCM`, `CAN_ISOTP` or `CAN_J1939`. If `fileno` is specified, the values for `family`

socket.socketpair([family[,type[,proto]]])

Build a pair of connected socket objects using the given address family, socket type, and protocol number. Address family, socket type, and protocol number are as for the `socket()` function. The default family is `AF_UNIX` if defined on the platform; otherwise, the default is `AF_INET`. The newly created sockets are non-inheritable. Changed in version 3.2: The returned socket objects now support the whole socket API, rather than a subset. Changed in version 3.4: The returned sockets are now non-inheritable. Cha

socket.create_connection(address,timeout=GLOBAL_DEFAULT,source_address=None,*,all_errors=False)

Connect to a TCP service listening on the internet address (a 2-tuple (host,port)), and return the socket object. This is a higher-level function than `socket.connect()`: if `host` is a non-numeric hostname, it will try to resolve it for both `AF_INET` and `AF_INET6`, and then try to connect to all possible addresses in turn until a connection succeeds. This makes it easy to write clients that are compatible to both IPv4 and IPv6. Passing the optional `timeout` parameter will set the timeout on the socket instance

socket.create_server(address,*,family=AF_INET,backlog=None,reuse_port=False,dualstack_ipv6=False)

Convenience function which creates a TCP socket bound to address (a 2-tuple (host,port)) and returns the socket object. `family` should be either `AF_INET` or `AF_INET6`. `backlog` is the queue size passed to `socket.listen()`; if not specified, a default reasonable value is chosen. `reuse_port` dictates whether to set the `SO_REUSEPORT` socket option. If `dualstack_ipv6` is true, `family` is `AF_INET6` and the platform supports it the socket will be able to accept both IPv4 and IPv6 connections, else it will raise `ValueError`. Most POSI

socket.has_dualstack_ipv6()

Return `True` if the platform supports creating a TCP socket which can handle both IPv4 and IPv6 connections. Added in version 3.8.

socket.fromfd(fd,family,type,proto=0)

Duplicate the file descriptor `fd` (an integer as returned by a file object's `fileno()` method) and build a socket object from the result. Address family, socket type and protocol number are as for the `socket()` function. The file descriptor should refer to a socket, but this is not checked — subsequent operations on the object may fail if the file descriptor is invalid. This function is rarely needed, but can be used to get or set socket options on a socket passed to a program as standard input or output

socket.fromshare(data)

Instantiate a socket from data obtained from the `socket.share()` method. The socket is assumed to be in blocking mode. Availability: Windows. Added in version 3.3.

socket.SocketType

This is a Python type object that represents the socket object type. It is the same as `type(socket(...))`.

Other functions

The `socket` module also offers various network-related services:

socket.close(fd)

Close a socket file descriptor. This is like `os.close()`, but for sockets. On some platforms (most notably Windows) `os.close()` does not work for socket file descriptors. Added in version 3.7.

`socket.getaddrinfo(host,port,family=AF_UNSPEC,type=0,proto=0,flags=0)`

This function wraps the C function `getaddrinfo` of the underlying system. Translate the `host/port` argument into a sequence of 5-tuples that contain all the necessary arguments for creating a socket connected to that service. `host` is a domain name, a string representation of an IPv4/v6 address or `None`. `port` is a string service name such as 'http', a numeric port number or `None`. By passing `None` as the value of `host` and `port`, you can pass `NULL` to the underlying C API. The `family`, `type` and `proto` arguments can be optionally

`socket.getfqdn([name])`

Return a fully qualified domain name for `name`. If `name` is omitted or empty, it is interpreted as the local host. To find the fully qualified name, the `hostname` returned by `gethostbyaddr()` is checked, followed by aliases for the host, if available. The first name which includes a period is selected. In case no fully qualified domain name is available and `name` was provided, it is returned unchanged. If `name` was empty or equal to '0.0.0.0', the `hostname` from `gethostname()` is returned.

`socket.gethostbyname(hostname)`

Translate a host name to IPv4 address format. The IPv4 address is returned as a string, such as '100.50.200.5'. If the host name is an IPv4 address itself it is returned unchanged. See `gethostbyname_ex()` for a more complete interface. `gethostbyname()` does not support IPv6 name resolution, and `getaddrinfo()` should be used instead for IPv4/v6 dual stack support. Raises an `auditing event` `socket.gethostbyname` with argument `hostname`. Availability: not WASI.

`socket.gethostbyname_ex(hostname)`

Translate a host name to IPv4 address format, extended interface. Return a 3-tuple (`hostname`, `aliaslist`, `ipaddrlist`) where `hostname` is the host's primary host name, `aliaslist` is a (possibly empty) list of alternative host names for the same address, and `ipaddrlist` is a list of IPv4 addresses for the same interface on the same host (often but not always a single address). `gethostbyname_ex()` does not support IPv6 name resolution, and `getaddrinfo()` should be used instead for IPv4/v6 dual stack support. Raises an

`socket.gethostname()`

Return a string containing the `hostname` of the machine where the Python interpreter is currently executing. Raises an `auditing event` `socket.gethostname` with no arguments. Note: `gethostname()` doesn't always return the fully qualified domain name; use `getfqdn()` for that. Availability: not WASI.

`socket.gethostbyaddr(ip_address)`

Return a 3-tuple (`hostname`, `aliaslist`, `ipaddrlist`) where `hostname` is the primary host name responding to the given `ip_address`, `aliaslist` is a (possibly empty) list of alternative host names for the same address, and `ipaddrlist` is a list of IPv4/v6 addresses for the same interface on the same host (most likely containing only a single address). To find the fully qualified domain name, use the function `getfqdn()`. `gethostbyaddr()` supports both IPv4 and IPv6. Raises an `auditing event` `socket.gethostbyaddr` with argumen

`socket.getnameinfo(sockaddr,flags)`

Translate a socket address `sockaddr` into a 2-tuple (`host`, `port`). Depending on the settings of `flags`, the result can contain a fully qualified domain name or numeric address representation in `host`. Similarly, `port` can contain a string port name or a numeric port number. For IPv6 addresses, `%scope_id` is appended to the host part if `sockaddr` contains meaningful `scope_id`. Usually this happens for multicast addresses. For more information about `flags` you can consult `getnameinfo(3)`. Raises an `auditing event` `socket.getnam`

`socket.getprotobyname(protocolname)`

Translate an internet protocol name (for example, 'icmp') to a constant suitable for passing as the (optional) third argument to the `socket()` function. This is usually only needed for sockets opened in "raw" mode (`SOCK_RAW`); for the normal socket modes, the correct protocol is chosen automatically if the protocol is omitted or zero. Availability: not WASI.

`socket.getservbyname(servicename[,protocolname])`

Translate an internet service name and protocol name to a port number for that service. The optional protocol name, if given, should be 'tcp' or 'udp', otherwise any protocol will match. Raises an `auditing` event `socket.getservbyname` with arguments `servicename`, `protocolname`. Availability: not WASI.

`socket.getservbyport(port[,protocolname])`

Translate an internet port number and protocol name to a service name for that service. The optional protocol name, if given, should be 'tcp' or 'udp', otherwise any protocol will match. Raises an `auditing` event `socket.getservbyport` with arguments `port`, `protocolname`. Availability: not WASI.

`socket.ntohl(x)`

Convert 32-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

`socket.ntohs(x)`

Convert 16-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation. Changed in version 3.10: Raises `OverflowError` if `x` does not fit in a 16-bit unsigned integer.

`socket.htonl(x)`

Convert 32-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

`socket.htons(x)`

Convert 16-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation. Changed in version 3.10: Raises `OverflowError` if `x` does not fit in a 16-bit unsigned integer.

`socket.inet_aton(ip_string)`

Convert an IPv4 address from dotted-quad string format (for example, '123.45.67.89') to 32-bit packed binary format, as a bytes object four characters in length. This is useful when conversing with a program that uses the standard C library and needs objects of type `in_addr`, which is the C type for the 32-bit packed binary this function returns. `inet_aton()` also accepts strings with less than three dots; see the Unix manual page `inet(3)` for details. If the IPv4 address string passed to this function

`socket.inet_ntoa(packed_ip)`

Convert a 32-bit packed IPv4 address (a bytes-like object four bytes in length) to its standard dotted-quad string representation (for example, '123.45.67.89'). This is useful when conversing with a program that uses the standard C library and needs objects of type `in_addr`, which is the C type for the 32-bit packed binary data this function takes as an argument. If the byte sequence passed to this function is not exactly 4 bytes in length, `OSError` will be raised. `inet_ntoa()` does not support IPv6, and

`socket.inet_pton(address_family,ip_string)`

Convert an IP address from its family-specific string format to a packed, binary format. `inet_pton()` is useful when a library or network protocol calls for an object of type `in_addr` (similar to `inet_aton()`) or `in6_addr`. Supported values for `address_family` are currently `AF_INET` and `AF_INET6`. If the IP address string `ip_string` is invalid, `OSError` will be raised. Note that exactly what is valid depends on both the value of `address_family` and the

underlying implementation of `inet_pton()`. Availability: Unix, Windows. Cha

socket.inet_ntop(address_family,packed_ip)

Convert a packed IP address (a bytes-like object of some number of bytes) to its standard, family-specific string representation (for example, '7.10.0.5' or '5aef:2b::8'). `inet_ntop()` is useful when a library or network protocol returns an object of type `in_addr` (similar to `inet_ntoa()`) or `in6_addr`. Supported values for `address_family` are currently `AF_INET` and `AF_INET6`. If the bytes object `packed_ip` is not the correct length for the specified address family, `ValueError` will be raised. `OSError` is raised for errors from

socket.CMSG_LEN(length)

Return the total length, without trailing padding, of an ancillary data item with associated data of the given `length`. This value can often be used as the buffer size for `recvmsg()` to receive a single item of ancillary data, but RFC 3542 requires portable applications to use `CMSG_SPACE()` and thus include space for padding, even when the item will be the last in the buffer. Raises `OverflowError` if `length` is outside the permissible range of values. Availability: Unix, not WASI. Most Unix platforms. Added in v

socket.CMSG_SPACE(length)

Return the buffer size needed for `recvmsg()` to receive an ancillary data item with associated data of the given `length`, along with any trailing padding. The buffer space needed to receive multiple items is the sum of the `CMSG_SPACE()` values for their associated data lengths. Raises `OverflowError` if `length` is outside the permissible range of values. Note that some systems might support ancillary data without providing this function. Also note that setting the buffer size using the results of this functi

socket.getdefaulttimeout()

Return the default timeout in seconds (float) for new socket objects. A value of `None` indicates that new socket objects have no timeout. When the socket module is first imported, the default is `None`.

socket.setdefaulttimeout(timeout)

Set the default timeout in seconds (float) for new socket objects. When the socket module is first imported, the default is `None`. See `settimeout()` for possible values and their respective meanings.

socket.sethostname(name)

Set the machine's hostname to `name`. This will raise an `OSError` if you don't have enough rights. Raises an auditing event `socket.sethostname` with argument `name`. Availability: Unix, not Android. Added in version 3.3.

socket.if_nameindex()

Return a list of network interface information (index int, name string) tuples. `OSError` if the system call fails. Availability: Unix, Windows, not WASI. Added in version 3.3. Changed in version 3.8: Windows support was added. Note On Windows network interfaces have different names in different contexts (all names are examples): `UUID:{FB605B73-AAC2-49A6-9A2F-25416AEA0573}name:ethernet_32770friendlyname:vEthernet(nat)description:Hyper-V Virtual Ethernet Adapter` This function returns names of the second form f

socket.if_nameindex(if_name)

Return a network interface index number corresponding to an interface name. `OSError` if no interface with the given name exists. Availability: Unix, Windows, not WASI. Added in version 3.3. Changed in version 3.8: Windows support was added. See also "Interface name" is a name as documented in `if_nameindex()`.

socket.if_indextoname(if_index)

Return a network interface name corresponding to an interface index number. `OSError` if no interface with the given index exists. Availability: Unix, Windows, not WASI. Added in version 3.3. Changed in version 3.8: Windows support was added. See also "Interface name" is a name as documented in `if_nameindex()`.

socket.send_fds(sock,buffers,fds[,flags[,address]])

Send the list of file descriptors `fdsover` an `AF_UNIX` socket `sock`. The `fdsover` parameter is a sequence of file descriptors. Consult `sendmsg()` for the documentation of these parameters. Availability: Unix, not WASI. Unix platforms supporting `sendmsg()` and `SCM_RIGHTS` mechanism. Added in version 3.9.

socket.recv_fds(sock, bufsize, maxfds[, flags])

Receive up to `maxfds` file descriptors from an `AF_UNIX` socket `sock`. Return `(msg, list(fds), flags, addr)`. Consult `recvmsg()` for the documentation of these parameters. Availability: Unix, not WASI. Unix platforms supporting `recvmsg()` and `SCM_RIGHTS` mechanism. Added in version 3.9. Note Any truncated integers at the end of the list of file descriptors.

Socket Objects

Socket objects have the following methods. Except for `makefile()`, these correspond to Unix system calls applicable to sockets.

Changed in version 3.2: Support for the context manager protocol was added. Exiting the context manager is equivalent to calling `close()`.

socket.accept()

Accept a connection. The socket must be bound to an address and listening for connections. The return value is a pair `(conn, address)` where `conn` is a new socket object usable to send and receive data on the connection, and `address` is the address bound to the socket on the other end of the connection. The newly created socket is non-inheritable. Changed in version 3.4: The socket is now non-inheritable. Changed in version 3.5: If the system call is interrupted and the signal handler does not raise an exception,

socket.bind(address)

Bind the socket to `address`. The socket must not already be bound. The format of `address` depends on the address family — see `Socket families`. Raises an `auditing event socket.bind` with arguments `self, address`. Availability: not WASI.

socket.close()

Mark the socket closed. The underlying system resource (e.g. a file descriptor) is also closed when all file objects from `makefile()` are closed. Once that happens, all future operations on the socket object will fail. The remote end will receive no more data (after queued data is flushed). Sockets are automatically closed when they are garbage-collected, but it is recommended to `close()` them explicitly, or to use a `with` statement around them. Changed in version 3.6: `OSError` is now raised if an error occurs.

socket.connect(address)

Connect to a remote socket at `address`. The format of `address` depends on the address family — see `Socket families`. If the connection is interrupted by a signal, the method waits until the connection completes, or raises a `TimeoutError` on timeout, if the signal handler doesn't raise an exception and the socket is blocking or has a timeout. For non-blocking sockets, the method raises an `InterruptedError` exception if the connection is interrupted by a signal (or the exception raised by the signal handler). Raises

socket.connect_ex(address)

Like `connect(address)`, but return an error indicator instead of raising an exception for errors returned by the C-level `connect()` call (other problems, such as “host not found,” can still raise exceptions). The error indicator is 0 if the operation succeeded, otherwise the value of the `errno` variable. This is useful to support, for example, asynchronous connects. Raises an `auditing event socket.connect` with arguments `self, address`. Availability: not WASI.

socket.detach()

Put the socket object into closed state without actually closing the underlying file descriptor. The file descriptor is returned, and can be reused for other purposes. Added in version 3.2.

socket.dup()

Duplicate the socket. The newly created socket is non-inheritable. Changed in version 3.4: The socket is now non-inheritable. Availability: not WASI.

socket.fileno()

Return the socket's file descriptor (a small integer), or -1 on failure. This is useful with `select.select()`. Under Windows the small integer returned by this method cannot be used where a file descriptor can be used (such as `os.fdopen()`). Unix does not have this limitation.

socket.get_inheritable()

Get the inheritable flag of the socket's file descriptor or socket's handle: `True` if the socket can be inherited in child processes, `False` if it cannot. Added in version 3.4.

socket.getpeername()

Return the remote address to which the socket is connected. This is useful to find out the port number of a remote IPv4/v6 socket, for instance. The format of the address returned depends on the address family — see `Socket` families. On some systems this function is not supported.

socket.getsockname()

Return the socket's own address. This is useful to find out the port number of an IPv4/v6 socket, for instance. The format of the address returned depends on the address family — see `Socket` families.

socket.getsockopt(level, optname[, buflen])

Return the value of the given socket option (see the Unix man page `getsockopt(2)`). The needed symbolic constants (`SO_*` etc.) are defined in this module. If `buflen` is absent, an integer option is assumed and its integer value is returned by the function. If `buflen` is present, it specifies the maximum length of the buffer used to receive the option in, and this buffer is returned as a bytes object. It is up to the caller to decode the contents of the buffer (see the optional built-in module `struct` for

socket.getblocking()

Return `True` if socket is in blocking mode, `False` if in non-blocking. This is equivalent to checking `socket.gettimeout() != 0`. Added in version 3.7.

socket.gettimeout()

Return the timeout in seconds (float) associated with socket operations, or `None` if no timeout is set. This reflects the last call to `setblocking()` or `settimeout()`.

socket.ioctl(control, option)

The `ioctl()` method is a limited interface to the `WSAIoctl` system interface. Please refer to the `Win32` documentation for more information. On other platforms, the generic `fcntl.fcntl()` and `fcntl.ioctl()` functions may be used; they accept a socket object as their first argument. Currently only the following control codes are supported: `SIO_RCVALL`, `SIO_KEEPA_LIVE_VALS`, and `SIO_LOOPBACK_FAST_PATH`. Availability: Windows. Changed in version 3.6: `SIO_LOOPBACK_FAST_PATH` was added.

socket.listen([backlog])

Enable a server to accept connections. If `backlog` is specified, it must be at least 0 (if it is lower, it is set to 0); it specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen. Availability: not WASI. Changed in version 3.5: The `backlog` parameter is now optional.

socket.makefile(mode='r',buffering=None,*,encoding=None,errors=None,newline=None)

Return a file object associated with the socket. The exact returned type depends on the arguments given to `makefile()`. These arguments are interpreted the same way as by the built-in `open()` function, except the only supported mode values are 'r' (default), 'w', 'b', or a combination of those. The socket must be in blocking mode; it can have a timeout, but the file object's internal buffer may end up in an inconsistent state if a timeout occurs. Closing the file object returned by `makefile()` won't close the o

socket.recv(bufsize[,flags])

Receive data from the socket. The return value is a bytes object representing the data received. The maximum amount of data to be received at once is specified by `bufsize`. A returned empty bytes object indicates that the client has disconnected. See the Unix manual `recv(2)` for the meaning of the optional argument `flags`; it defaults to zero. Changed in version 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead

socket.recvfrom(bufsize[,flags])

Receive data from the socket. The return value is a pair (bytes, address) where bytes is a bytes object representing the data received and address is the address of the socket sending the data. See the Unix manual `recv(2)` for the meaning of the optional argument `flags`; it defaults to zero. The format of address depends on the address family — see `Socket families`. Changed in version 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the syste

socket.recvmsg(bufsize[,ancbufsize[,flags]])

Receive normal data (up to `bufsize` bytes) and ancillary data from the socket. The `ancbufsize` argument sets the size in bytes of the internal buffer used to receive the ancillary data; it defaults to 0, meaning that no ancillary data will be received. Appropriate buffer sizes for ancillary data can be calculated using `MSG_SPACE()` or `MSG_LEN()`, and items which do not fit into the buffer might be truncated or discarded. The `flags` argument defaults to 0 and has the same meaning as for `recv()`. The return v

socket.recvmsg_into(buffers[,ancbufsize[,flags]])

Receive normal data and ancillary data from the socket, behaving as `recvmsg()` would, but scatter the non-ancillary data into a series of buffers instead of returning a new bytes object. The `buffers` argument must be an iterable of objects that export writable buffers (e.g. bytearray objects); these will be filled with successive chunks of the non-ancillary data until it has all been written or there are no more buffers. The operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of

socket.recvfrom_into(buffer[,nbytes[,flags]])

Receive data from the socket, writing it into `buffer` instead of creating a new bytestring. The return value is a pair (nbytes, address) where `nbytes` is the number of bytes received and `address` is the address of the socket sending the data. See the Unix manual `recv(2)` for the meaning of the optional argument `flags`; it defaults to zero. The format of address depends on the address family — see `Socket families`.

socket.recv_into(buffer[,nbytes[,flags]])

Receive up to `nbytes` bytes from the socket, storing the data into a buffer rather than creating a new bytestring. If `nbytes` is not specified (or 0), receive up to the size available in the given buffer. Returns the number of bytes received. See the Unix manual `recv(2)` for the meaning of the optional argument `flags`; it defaults to zero.

socket.send(bytes[,flags])

Send data to the socket. The socket must be connected to a remote socket. The optional `flags` argument has the same meaning as for `recv()`. Returns the number of bytes sent. Applications are responsible for checking that all data has been sent; if only some of the data was transmitted, the application needs to attempt delivery of the remaining data. For further information on this topic, consult the `Socket Programming HOWTO`. Changed in

version 3.5: If the system call is interrupted and the signal handler

socket.sendall(bytes[, flags])

Send data to the socket. The socket must be connected to a remote socket. The optional `flags` argument has the same meaning as for `recv()`. Unlike `send()`, this method continues to send data from `bytes` until either all data has been sent or an error occurs. `None` is returned on success. On error, an exception is raised, and there is no way to determine how much data, if any, was successfully sent. Changed in version 3.5: The socket timeout is no longer reset each time data is sent successfully. The socket

socket.sendto(bytes, address)

socket.sendto(bytes, flags, address)

Send data to the socket. The socket should not be connected to a remote socket, since the destination socket is specified by `address`. The optional `flags` argument has the same meaning as for `recv()`. Return the number of bytes sent. The format of `address` depends on the address family — see `Socket families`. Raises an `auditing` event `socket.sendto` with arguments `self, address`. Changed in version 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the

socket.sendmsg(buffers[, ancdata[, flags[, address]]])

Send normal and ancillary data to the socket, gathering the non-ancillary data from a series of buffers and concatenating it into a single message. The `buffers` argument specifies the non-ancillary data as an iterable of bytes-like objects (e.g. `bytes` objects); the operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of buffers that can be used. The `ancdata` argument specifies the ancillary data (control messages) as an iterable of zero or more tuples (`cmsg_level, cmsg_type, cmsg_data`)

socket.sendmsg_afalg([msg,]*, op[, iv[, assoclen[, flags]]])

Specialized version of `sendmsg()` for `AF_ALG` socket. Set mode, IV, AEAD associated data length and flags for `AF_ALG` socket. Availability: Linux >= 2.6.38. Added in version 3.6.

socket.sendfile(file, offset=0, count=None)

Send a file until EOF is reached by using high-performance `os.sendfile` and return the total number of bytes which were sent. `file` must be a regular file object opened in binary mode. If `os.sendfile` is not available (e.g. Windows) or `file` is not a regular file `send()` will be used instead. `offset` tells from where to start reading the file. If specified, `count` is the total number of bytes to transmit as opposed to sending the file until EOF is reached. File position is updated on return or also in case of error

socket.set_inheritable(inheritable)

Set the `inheritable` flag of the socket's file descriptor or socket's handle. Added in version 3.4.

socket.setblocking(flag)

Set blocking or non-blocking mode of the socket: if `flag` is `False`, the socket is set to non-blocking, else to blocking mode. This method is a shorthand for certain `settimeout()` calls: `sock.setblocking(True)` is equivalent to `sock.settimeout(None)`; `sock.setblocking(False)` is equivalent to `sock.settimeout(0.0)`. Changed in version 3.7: The method no longer applies `SOCK_NONBLOCK` flag on `socket.type`.

socket.settimeout(value)

Set a timeout on blocking socket operations. The `value` argument can be a nonnegative floating-point number expressing seconds, or `None`. If a non-zero value is given, subsequent socket operations will raise a `TimeoutException` if the timeout period `value` has elapsed before the operation has completed. If zero is given, the socket is put in non-blocking mode. If `None` is given, the socket is put in blocking mode. For further information, please consult the notes on socket timeouts. Changed in version 3.7: The

socket.setsockopt(level, optname, value: int|Buffer)

socket.setsockopt(level,optname,None,optlen:int)

Set the value of the given socket option (see the Unix manual `setsockopt(2)`). The needed symbolic constants are defined in this module (`SO_*` etc. `<socket-unix-constants>`). The value can be an integer, `None` or a bytes-like object representing a buffer. In the latter case it is up to the caller to ensure that the bytestring contains the proper bits (see the optional built-in module `struct` for a way to encode C structures as bytestrings). When `value` is set to `None`, `optlen` argument is required. It's equi

socket.shutdown(how)

Shut down one or both halves of the connection. If `how` is `SHUT_RD`, further receives are disallowed. If `how` is `SHUT_WR`, further sends are disallowed. If `how` is `SHUT_RDWR`, further sends and receives are disallowed. Availability: not WASI.

socket.share(process_id)

Duplicate a socket and prepare it for sharing with a target process. The target process must be provided with `process_id`. The resulting bytes object can then be passed to the target process using some form of interprocess communication and the socket can be recreated there using `fromshare()`. Once this method has been called, it is safe to close the socket since the operating system has already duplicated it for the target process. Availability: Windows. Added in version 3.3.

Note that there are no methods `read()` or `write()`; `recv()` and `send()` without `flags` argument instead.

Socket objects also have these (read-only) attributes that correspond to the values given to the socket constructor.

socket.family

The socket family.

socket.type

The socket type.

socket.proto

The socket protocol.

Notes on socket timeouts

A socket object can be in one of three modes: blocking, non-blocking, or timeout. Sockets are by default always created in blocking mode, but this can be changed by calling `setdefaulttimeout()`.

- In blocking mode, operations block until complete or the system returns an error (such as connection timed out).
- In non-blocking mode, operations fail (with an error that is unfortunately system-dependent) if they cannot be completed immediately: functions from the `select` module can be used to know when and whether a socket is available for reading or writing.
- In timeout mode, operations fail if they cannot be completed within the timeout specified for the socket (they raise a `TimeoutException`) or if the system returns an error.

■ **Nota:** At the operating system level, sockets in timeout mode are internally set in non-blocking mode. Also, the blocking and timeout modes are shared between file descriptors and socket objects that refer to the same network endpoint. This implementation detail can have visible consequences if e.g. you decide to use the `fileno()` of a socket.

Timeouts and the connect method

The `connect()` operation is also subject to the timeout setting, and in general it is recommended to call `settimeout()` before calling `connect()` or pass a timeout parameter to `create_connection()`. However, the system network stack may also return a connection timeout error of its own regardless of any Python socket timeout setting.

Timeouts and the `accept` method

If `getdefaulttimeout()` is not `None`, sockets returned by the `accept()` method inherit that timeout. Otherwise, the behaviour depends on settings of the listening socket:

- if the listening socket is in blocking mode or in timeout mode, the socket returned by `accept()` is in blocking mode;
- if the listening socket is in non-blocking mode, whether the socket returned by `accept()` is in blocking or non-blocking mode is operating system-dependent. If you want to ensure cross-platform behaviour, it is recommended you manually override this setting.

Example

Here are four minimal example programs using the TCP/IP protocol: a server that echoes all data that it receives back (servicing only one client), and a client using it. Note that a server must perform the sequence `socket(), bind(), listen(), accept()` (possibly repeating the `accept()` to service more than one client), while a client only needs the sequence `socket(), connect()`. Also note that the server does not `sendall()/recv()` on the socket it is listening on but on the new socket returned by `accept()`.

The first two examples support IPv4 only.

```
# Echo server program
import socket

HOST = ''          # Symbolic name meaning all available interfaces
PORT = 50007       # Arbitrary non-privileged port
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    conn, addr = s.accept()
    with conn:
        print('Connected by', addr)
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)

# Echo client program
import socket

HOST = 'daring.cwi.nl' # The remote host
PORT = 50007           # The same port as used by the server
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))
```

The next two examples are identical to the above two, but support both IPv4 and IPv6. The server side will listen to the first address family available (it should listen to both instead). On most of IPv6-ready systems, IPv6 will take precedence and the server may not accept IPv4 traffic. The client side will try to connect to all the addresses returned as a result of the name resolution, and sends traffic to the first one connected successfully.

```
# Echo server program
import socket
import sys
```

```

HOST = None                # Symbolic name meaning all available interfaces
PORT = 50007               # Arbitrary non-privileged port
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC,
                              socket.SOCK_STREAM, 0, socket.AI_PASSIVE):
    af, socktype, proto, canonname, sa = res
    try:
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.bind(sa)
        s.listen(1)
    except OSError as msg:
        s.close()
        s = None
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
conn, addr = s.accept()
with conn:
    print('Connected by', addr)
    while True:
        data = conn.recv(1024)
        if not data: break
        conn.send(data)

# Echo client program
import socket
import sys

HOST = 'daring.cwi.nl'     # The remote host
PORT = 50007               # The same port as used by the server
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC, socket.SOCK_STREAM):
    af, socktype, proto, canonname, sa = res
    try:
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.connect(sa)
    except OSError as msg:
        s.close()
        s = None
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
with s:
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))

```

The next example shows how to write a very simple network sniffer with raw sockets on Windows. The example requires administrator privileges to modify the interface:

```

import socket

# the public network interface

```

```

HOST = socket.gethostbyname(socket.gethostname())

# create a raw socket and bind it to the public interface
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind((HOST, 0))

# Include IP headers
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# receive all packets
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

# receive a packet
print(s.recvfrom(65565))

# disabled promiscuous mode
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

```

The next example shows how to use the socket interface to communicate to a CAN network using the raw socket protocol. To use CAN with the broadcast manager protocol instead, open a socket with:

```
socket.socket(socket.AF_CAN, socket.SOCK_DGRAM, socket.CAN_BCM)
```

After binding (CAN_RAW) or connecting (CAN_BCM) the socket, you can use the socket.send() and socket.recv() operations (and their counterparts) on the socket object as usual.

This last example might require special privileges:

```

import socket
import struct

# CAN frame packing/unpacking (see 'struct can_frame' in <linux/can.h>)

can_frame_fmt = "=IB3x8s"
can_frame_size = struct.calcsize(can_frame_fmt)

def build_can_frame(can_id, data):
    can_dlc = len(data)
    data = data.ljust(8, b'\x00')
    return struct.pack(can_frame_fmt, can_id, can_dlc, data)

def dissect_can_frame(frame):
    can_id, can_dlc, data = struct.unpack(can_frame_fmt, frame)
    return (can_id, can_dlc, data[:can_dlc])

# create a raw socket and bind it to the 'vcan0' interface
s = socket.socket(socket.AF_CAN, socket.SOCK_RAW, socket.CAN_RAW)
s.bind(('vcan0',))

while True:
    cf, addr = s.recvfrom(can_frame_size)

    print('Received: can_id=%x, can_dlc=%x, data=%s' % dissect_can_frame(cf))

    try:
        s.send(cf)
    except OSError:
        print('Error sending CAN frame')

    try:
        s.send(build_can_frame(0x01, b'\x01\x02\x03'))
    except OSError:
        print('Error sending CAN frame')

```


Running an example several times with too small delay between executions, could lead to this error:

```
OSError: [Errno 98] Address already in use
```

This is because the previous execution has left the socket in `TIME_WAIT` state, and can't be immediately reused.

There is a socket flag to set, in order to prevent this, `socket.SO_REUSEADDR`:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT))
```

the `SO_REUSEADDR` flag tells the kernel to reuse a local socket in `TIME_WAIT` state, without waiting for its natural timeout to expire.

See also

For an introduction to socket programming (in C), see the following papers:

- An Introductory 4.3BSD Interprocess Communication Tutorial, by Stuart Sechrest
- An Advanced 4.3BSD Interprocess Communication Tutorial, by Samuel J. Leffler et al,

both in the UNIX Programmer's Manual, Supplementary Documents 1 (sections PS1:7 and PS1:8). The platform-specific reference material for the various socket-related system calls are also a valuable source of information on the details of socket semantics. For Unix, refer to the manual pages; for Windows, see the WinSock (or Winsock 2) specification. For IPv6-ready APIs, readers may want to refer to RFC 3493 titled Basic Socket Interface Extensions for IPv6.

ssl— TLS/SSL wrapper for socket objects

Source code: `Lib/ssl.py`

This module provides access to Transport Layer Security (often known as “Secure Sockets Layer”) encryption and peer authentication facilities for network sockets, both client-side and server-side. This module uses the OpenSSL library.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

■ **Nota:** Some behavior may be platform dependent, since calls are made to the operating system socket APIs. The installed version of OpenSSL may also cause variations in behavior. For example, TLSv1.3 comes with OpenSSL version 1.1.1.

■■ **Advertencia:** Warning Don't use this module without reading the [Security considerations](#). Doing so may lead to a false sense of security, as the default settings of the `ssl` module are not necessarily appropriate for your application.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

This section documents the objects and functions in the `ssl` module; for more general information about TLS, SSL, and certificates, the reader is referred to the documents in the “See Also” section at the bottom.

This module provides a class, `ssl.SSLSocket`, which is derived from `socket.socket`, and provides a socket-like wrapper that also encrypts and decrypts the data going over the socket with SSL. It supports additional methods such as `getpeercert()`, which retrieves the certificate of the other side of the connection, `cipher()`, which retrieves the cipher being used for the secure connection or `get_verified_chain()`, `get_unverified_chain()` which retrieves certificate chain.

For more sophisticated applications, the `ssl.SSLContext` class helps manage settings and certificates, which can then be inherited by SSL sockets created through the `SSLContext.wrap_socket()` method.

Changed in version 3.5.3: Updated to support linking with OpenSSL 1.1.0

Changed in version 3.6: OpenSSL 0.9.8, 1.0.0 and 1.0.1 are deprecated and no longer supported. In the future the `ssl` module will require at least OpenSSL 1.0.2 or 1.1.0.

Changed in version 3.10: PEP 644 has been implemented. The `ssl` module requires OpenSSL 1.1.1 or newer.

Use of deprecated constants and functions result in deprecation warnings.

Functions, constants, and exceptions

Socket creation

Instances of `SSLSocket` must be created using the `SSLContext.wrap_socket()` method. The helper function `create_default_context()` returns a new context with secure default settings.

Client socket example with default context and IPv4/IPv6 dual stack:

```
import socket
import ssl

hostname = 'www.python.org'
context = ssl.create_default_context()
```

```
with socket.create_connection((hostname, 443)) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as ssock:
        print(ssock.version())
```

Client socket example with custom context and IPv4:

```
hostname = 'www.python.org'
# PROTOCOL_TLS_CLIENT requires valid cert chain and hostname
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.load_verify_locations('path/to/cabundle.pem')

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as ssock:
        print(ssock.version())
```

Server socket example listening on localhost IPv4:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain('path/to/certchain.pem', 'path/to/private.key')

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as sock:
    sock.bind(('127.0.0.1', 8443))
    sock.listen(5)
    with context.wrap_socket(sock, server_side=True) as ssock:
        conn, addr = ssock.accept()
    ...
```

Context creation

A convenience function helps create `SSLContext` objects for common purposes.

`ssl.create_default_context(purpose=Purpose.SERVER_AUTH, *, cafile=None, capath=None, cadata=None)`

Return a new `SSLContext` object with default settings for the given purpose. The settings are chosen by the `ssl` module, and usually represent a higher security level than when calling the `SSLContext` constructor directly. `cafile`, `capath`, `cadata` represent optional CA certificates to trust for certificate verification, as in `SSLContext.load_verify_locations()`. If all three are `None`, this function can choose to trust the system's default CA certificates instead. The settings are: `PROTOCOL_TLS_CLIENT` or `PROTOCOL_TLS`

Exceptions

exception `ssl.SSLError`

library

reason

Raised to signal an error from the underlying SSL implementation (currently provided by the OpenSSL library). This signifies some problem in the higher-level encryption and authentication layer that's superimposed on the underlying network connection. This error is a subtype of `OSError`. The error code and message of `SSLError` instances are provided by the OpenSSL library. Changed in version 3.3: `SSLError` used to be a subtype of `socket.error`. library A string mnemonic designating the OpenSSL submodule i

A string mnemonic designating the OpenSSL submodule in which the error occurred, such as `SSL`, `PEM` or `X509`. The range of possible values depends on the OpenSSL version. Added in version 3.3.

A string mnemonic designating the reason this error occurred, for example `CERTIFICATE_VERIFY_FAILED`. The range of possible values depends on the OpenSSL version. Added in version 3.3.

exception `ssl.SSLZeroReturnError`

A subclass of `SSLError` raised when trying to read or write and the SSL connection has been closed cleanly. Note that this doesn't mean that the underlying transport (read TCP) has been closed. Added in version 3.3.

exceptionssl.SSLWantReadError

A subclass ofSSLErrorraised by anon-blocking SSL socketwhen trying to read or write data, but more data needs to be received on the underlying TCP transport before the request can be fulfilled.Added in version 3.3.

exceptionssl.SSLWantWriteError

A subclass ofSSLErrorraised by anon-blocking SSL socketwhen trying to read or write data, but more data needs to be sent on the underlying TCP transport before the request can be fulfilled.Added in version 3.3.

exceptionssl.SSLSyscallError

A subclass ofSSLErrorraised when a system error was encountered while trying to fulfill an operation on a SSL socket. Unfortunately, there is no easy way to inspect the original errno number.Added in version 3.3.

exceptionssl.SSLEOFError

A subclass ofSSLErrorraised when the SSL connection has been terminated abruptly. Generally, you shouldn't try to reuse the underlying transport when this error is encountered.Added in version 3.3.

exceptionssl.SSLCertVerificationError

verify_code

verify_message

A subclass ofSSLErrorraised when certificate validation has failed.Added in version 3.7.verify_codeA numeric error number that denotes the verification error.verify_messageA human readable string of the verification error.

A numeric error number that denotes the verification error.

A human readable string of the verification error.

exceptionssl.CertificateError

An alias forSSLCertVerificationError.Changed in version 3.7:The exception is now an alias forSSLCertVerificationError.

Random generation

ssl.RAND_bytes(num, /)

Returnnumcryptographically strong pseudo-random bytes. Raises anSSLErrorif the PRNG has not been seeded with enough data or if the operation is not supported by the current RAND method.RAND_status()can be used to check the status of the PRNG andRAND_add()can be used to seed the PRNG.For almost all applicationsos.urandom()is preferable.Read the Wikipedia article,Cryptographically secure pseudorandom number generator (CSPRNG), to get the requirements of a cryptographically strong generator.Added i

ssl.RAND_status()

ReturnTrueif the SSL pseudo-random number generator has been seeded with 'enough' randomness, andFalseotherwise. You can usessl.RAND_egd()andssl.RAND_add()to increase the randomness of the pseudo-random number generator.

ssl.RAND_add(bytes, entropy, /)

Mix the givenbytesinto the SSL pseudo-random number generator. The parameterentropy(a float) is a lower bound on the entropy contained in string (so you can always use0.0). SeeRFC 1750for more information on sources of entropy.Changed in version 3.5:Writablebytes-like objectis now accepted.

Certificate handling

ssl.cert_time_to_seconds(cert_time)

Return the time in seconds since the epoch, given the `cert_timestring` representing the “notBefore” or “notAfter” date from a certificate in “%b%d%H:%M:%S%Y%Z” strptime format (C locale). Here’s an example: >>> import ssl >>> import datetime as dt >>> timestamp = ssl.cert_time_to_seconds("Jan 5 09:34:43 2018 GMT") >>> timestamp 1515144883 >>> print(dt.datetime.fromtimestamp(timestamp, dt.UTC)) 2018-01-05 09:34:43+00:00 “notBefore” or “notAfter” dates must use GMT (RFC 5280). Changed in version 3.5: Interpret the input `ti`

ssl.get_server_certificate(addr, ssl_version=PROTOCOL_TLS_CLIENT, ca_certs=None[, timeout])

Given the address `addr` of an SSL-protected server, as a (hostname, port-number) pair, fetches the server’s certificate, and returns it as a PEM-encoded string. If `ssl_version` is specified, uses that version of the SSL protocol to attempt to connect to the server. If `ca_certs` is specified, it should be a file containing a list of root certificates, the same format as used for the `cafile` parameter in `SSLContext.load_verify_locations()`. The call will attempt to validate the server certificate against that

ssl.DER_cert_to_PEM_cert(der_cert_bytes)

Given a certificate as a DER-encoded blob of bytes, returns a PEM-encoded string version of the same certificate.

ssl.PEM_cert_to_DER_cert(pem_cert_string)

Given a certificate as an ASCII PEM string, returns a DER-encoded sequence of bytes for that same certificate.

ssl.get_default_verify_paths()

Returns a named tuple with paths to OpenSSL’s default `cafile` and `capath`. The paths are the same as used by `SSLContext.set_default_verify_paths()`. The return value is a named tuple `DefaultVerifyPaths: cafile-` resolved path to `cafile` or `None` if the file doesn’t exist, `capath-` resolved path to `capath` or `None` if the directory doesn’t exist, `openssl_cafile_env-` OpenSSL’s environment key that points to a `cafile`, `openssl_cafile-` hard coded path to a `cafile`, `openssl_capath_env-` OpenSSL’s environment key that points

ssl.enum_certificates(store_name)

Retrieve certificates from Windows’ system cert store. `store_name` may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too. The function returns a list of (`cert_bytes`, `encoding_type`, `trust`) tuples. The `encoding_type` specifies the encoding of `cert_bytes`. It is either `x509_asn` for X.509 ASN.1 data or `pkcs_7_asn` for PKCS#7 ASN.1 data. `trust` specifies the purpose of the certificate as a set of OIDs or exactly `True` if the certificate is trustworthy for all purposes. Example: >>> ssl.enum_certifi

ssl.enum_crls(store_name)

Retrieve CRLs from Windows’ system cert store. `store_name` may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too. The function returns a list of (`cert_bytes`, `encoding_type`, `trust`) tuples. The `encoding_type` specifies the encoding of `cert_bytes`. It is either `x509_asn` for X.509 ASN.1 data or `pkcs_7_asn` for PKCS#7 ASN.1 data. Availability: Windows. Added in version 3.4.

Constants

All constants are now `enum.IntEnum` or `enum.IntFlag` collections.

Added in version 3.6.

ssl.CERT_NONE

Possible value for `SSLContext.verify_mode`. Except for `PROTOCOL_TLS_CLIENT`, it is the default mode. With client-side sockets, just about any cert is accepted. Validation errors, such as untrusted or expired cert, are ignored and do not abort the TLS/SSL handshake. In server mode, no certificate is requested from the client, so the client does not send any for client cert authentication. See the discussion of [Security considerations](#) below.

ssl.CERT_OPTIONAL

Possible value for `SSLContext.verify_mode`. In client mode, `CERT_OPTIONAL` has the same meaning as `CERT_REQUIRED`. It is recommended to use `CERT_REQUIRED` for client-side sockets instead. In server mode, a client certificate request is sent to the client. The client may either ignore the request or send a certificate in order to perform TLS client cert authentication. If the client chooses to send a certificate, it is verified. Any verification error immediately aborts the TLS handshake. Use of this setting

ssl.CERT_REQUIRED

Possible value for `SSLContext.verify_mode`. In this mode, certificates are required from the other side of the socket connection; an `SSLError` will be raised if no certificate is provided, or if its validation fails. This mode is not sufficient to verify a certificate in client mode as it does not match hostnames. `check_hostname` must be enabled as well to verify the authenticity of a cert. `PROTOCOL_TLS_CLIENT` uses `CERT_REQUIRED` and enables `check_hostname` by default. With server socket, this mode provides mandat

class ssl.VerifyMode

enum.IntEnum collection of `CERT_*` constants. Added in version 3.6.

ssl.VERIFY_DEFAULT

Possible value for `SSLContext.verify_flags`. In this mode, certificate revocation lists (CRLs) are not checked. By default OpenSSL does neither require nor verify CRLs. Added in version 3.4.

ssl.VERIFY_CRL_CHECK_LEAF

Possible value for `SSLContext.verify_flags`. In this mode, only the peer cert is checked but none of the intermediate CA certificates. The mode requires a valid CRL that is signed by the peer cert's issuer (its direct ancestor CA). If no proper CRL has been loaded with `SSLContext.load_verify_locations`, validation will fail. Added in version 3.4.

ssl.VERIFY_CRL_CHECK_CHAIN

Possible value for `SSLContext.verify_flags`. In this mode, CRLs of all certificates in the peer cert chain are checked. Added in version 3.4.

ssl.VERIFY_X509_STRICT

Possible value for `SSLContext.verify_flag` to disable workarounds for broken X.509 certificates. Added in version 3.4.

ssl.VERIFY_ALLOW_PROXY_CERTS

Possible value for `SSLContext.verify_flag` to enable proxy certificate verification. Added in version 3.10.

ssl.VERIFY_X509_TRUSTED_FIRST

Possible value for `SSLContext.verify_flags`. It instructs OpenSSL to prefer trusted certificates when building the trust chain to validate a certificate. This flag is enabled by default. Added in version 3.4.4.

ssl.VERIFY_X509_PARTIAL_CHAIN

Possible value for `SSLContext.verify_flags`. It instructs OpenSSL to accept intermediate CAs in the trust store to be treated as trust-anchors, in the same way as the self-signed root CA certificates. This makes it possible to trust certificates issued by an intermediate CA without having to trust its ancestor root CA. Added in version 3.10.

class ssl.VerifyFlags

enum.IntFlag collection of `VERIFY_*` constants. Added in version 3.6.

ssl.PROTOCOL_TLS

Selects the highest protocol version that both the client and server support. Despite the name, this option can select both “SSL” and “TLS” protocols. Added in version 3.6. Deprecated since version 3.10: TLS clients and servers require different default settings for secure communication. The generic TLS protocol constant is deprecated in favor of `PROTOCOL_TLS_CLIENT` and `PROTOCOL_TLS_SERVER`.

ssl.PROTOCOL_TLS_CLIENT

Auto-negotiate the highest protocol version that both the client and server support, and configure the context client-side connections. The protocol enables `CERT_REQUIRED` and `check_hostname` by default. Added in version 3.6.

ssl.PROTOCOL_TLS_SERVER

Auto-negotiate the highest protocol version that both the client and server support, and configure the context server-side connections. Added in version 3.6.

ssl.PROTOCOL_SSLv23

Alias for `PROTOCOL_TLS`. Deprecated since version 3.6: Use `PROTOCOL_TLS` instead.

ssl.PROTOCOL_SSLv3

Selects SSL version 3 as the channel encryption protocol. This protocol is not available if OpenSSL is compiled with the `no-ssl3` option. Warning: SSL version 3 is insecure. Its use is highly discouraged. Deprecated since version 3.6: OpenSSL has deprecated all version specific protocols. Use the default protocol `PROTOCOL_TLS_SERVER` or `PROTOCOL_TLS_CLIENT` with `SSLContext.minimum_version` and `SSLContext.maximum_version` instead.

ssl.PROTOCOL_TLSv1

Selects TLS version 1.0 as the channel encryption protocol. Deprecated since version 3.6: OpenSSL has deprecated all version specific protocols.

ssl.PROTOCOL_TLSv1_1

Selects TLS version 1.1 as the channel encryption protocol. Available only with openssl version 1.0.1+. Added in version 3.4. Deprecated since version 3.6: OpenSSL has deprecated all version specific protocols.

ssl.PROTOCOL_TLSv1_2

Selects TLS version 1.2 as the channel encryption protocol. Available only with openssl version 1.0.1+. Added in version 3.4. Deprecated since version 3.6: OpenSSL has deprecated all version specific protocols.

ssl.OP_ALL

Enables workarounds for various bugs present in other SSL implementations. This option is set by default. It does not necessarily set the same flags as OpenSSL's `SSL_OP_ALL` constant. Added in version 3.2.

ssl.OP_NO_SSLv2

Prevents an SSLv2 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing SSLv2 as the protocol version. Added in version 3.2. Deprecated since version 3.6: SSLv2 is deprecated

ssl.OP_NO_SSLv3

Prevents an SSLv3 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing SSLv3 as the protocol version. Added in version 3.2. Deprecated since version 3.6: SSLv3 is deprecated

ssl.OP_NO_TLSv1

Prevents a TLSv1 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1 as the protocol version. Added in version 3.2. Deprecated since version 3.7: The option is deprecated since OpenSSL 1.1.0, use the `newSSLContext.minimum_version` and `SSLContext.maximum_version` instead.

`ssl.OP_NO_TLSv1_1`

Prevents a TLSv1.1 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.1 as the protocol version. Available only with openssl version 1.0.1+. Added in version 3.4. Deprecated since version 3.7: The option is deprecated since OpenSSL 1.1.0.

`ssl.OP_NO_TLSv1_2`

Prevents a TLSv1.2 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.2 as the protocol version. Available only with openssl version 1.0.1+. Added in version 3.4. Deprecated since version 3.7: The option is deprecated since OpenSSL 1.1.0.

`ssl.OP_NO_TLSv1_3`

Prevents a TLSv1.3 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.3 as the protocol version. TLS 1.3 is available with OpenSSL 1.1.1 or later. When Python has been compiled against an older version of OpenSSL, the flag defaults to 0. Added in version 3.6.3. Deprecated since version 3.7: The option is deprecated since OpenSSL 1.1.0. It was added to 2.7.15 and 3.6.3 for backwards compatibility with OpenSSL 1.0.2.

`ssl.OP_NO_RENEGOTIATION`

Disable all renegotiation in TLSv1.2 and earlier. Do not send HelloRequest messages, and ignore renegotiation requests via ClientHello. This option is only available with OpenSSL 1.1.0h and later. Added in version 3.7.

`ssl.OP_CIPHER_SERVER_PREFERENCE`

Use the server's cipher ordering preference, rather than the client's. This option has no effect on client sockets and SSLv2 server sockets. Added in version 3.3.

`ssl.OP_SINGLE_DH_USE`

Prevents reuse of the same DH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets. Added in version 3.3.

`ssl.OP_SINGLE_ECDH_USE`

Prevents reuse of the same ECDH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets. Added in version 3.3.

`ssl.OP_ENABLE_MIDDLEBOX_COMPAT`

Send dummy Change Cipher Spec (CCS) messages in TLS 1.3 handshake to make a TLS 1.3 connection look more like a TLS 1.2 connection. This option is only available with OpenSSL 1.1.1 and later. Added in version 3.8.

`ssl.OP_NO_COMPRESSION`

Disable compression on the SSL channel. This is useful if the application protocol supports its own compression scheme. Added in version 3.3.

`class ssl.Options`

enum.IntFlag collection of `OP_*` constants.

`ssl.OP_NO_TICKET`

Prevent client side from requesting a session ticket. Added in version 3.6.

ssl.OP_IGNORE_UNEXPECTED_EOF

Ignore unexpected shutdown of TLS connections. This option is only available with OpenSSL 3.0.0 and later. Added in version 3.10.

ssl.OP_ENABLE_KTLS

Enable the use of the kernel TLS. To benefit from the feature, OpenSSL must have been compiled with support for it, and the negotiated cipher suites and extensions must be supported by it (a list of supported ones may vary by platform and kernel version). Note that with enabled kernel TLS some cryptographic operations are performed by the kernel directly and not via any available OpenSSL Providers. This might be undesirable if, for example, the application requires all cryptographic operations to

ssl.OP_LEGACY_SERVER_CONNECT

Allow legacy insecure renegotiation between OpenSSL and unpatched servers only. Added in version 3.12.

ssl.HAS_ALPN

Whether the OpenSSL library has built-in support for the Application-Layer Protocol Negotiation TLS extension as described in RFC 7301. Added in version 3.5.

ssl.HAS_NEVER_CHECK_COMMON_NAME

Whether the OpenSSL library has built-in support not checking subject common name and `SSLContext.hostname_checks_common_name` is writeable. Added in version 3.7.

ssl.HAS_ECDH

Whether the OpenSSL library has built-in support for the Elliptic Curve-based Diffie-Hellman key exchange. This should be true unless the feature was explicitly disabled by the distributor. Added in version 3.3.

ssl.HAS_SNI

Whether the OpenSSL library has built-in support for the Server Name Indication extension (as defined in RFC 6066). Added in version 3.2.

ssl.HAS_NPN

Whether the OpenSSL library has built-in support for the Next Protocol Negotiation as described in the Application Layer Protocol Negotiation. When true, you can use the `SSLContext.set_npn_protocols()` method to advertise which protocols you want to support. Added in version 3.3.

ssl.HAS_SSLv2

Whether the OpenSSL library has built-in support for the SSL 2.0 protocol. Added in version 3.7.

ssl.HAS_SSLv3

Whether the OpenSSL library has built-in support for the SSL 3.0 protocol. Added in version 3.7.

ssl.HAS_TLSv1

Whether the OpenSSL library has built-in support for the TLS 1.0 protocol. Added in version 3.7.

ssl.HAS_TLSv1_1

Whether the OpenSSL library has built-in support for the TLS 1.1 protocol. Added in version 3.7.

ssl.HAS_TLSv1_2

Whether the OpenSSL library has built-in support for the TLS 1.2 protocol. Added in version 3.7.

ssl.HAS_TLSv1_3

Whether the OpenSSL library has built-in support for the TLS 1.3 protocol. Added in version 3.7.

ssl.HAS_PSK

Whether the OpenSSL library has built-in support for TLS-PSK. Added in version 3.13.

ssl.HAS_PHA

Whether the OpenSSL library has built-in support for TLS-PHA. Added in version 3.14.

ssl.CHANNEL_BINDING_TYPES

List of supported TLS channel binding types. Strings in this list can be used as arguments to `SSLContext.get_channel_binding()`. Added in version 3.3.

ssl.OPENSSL_VERSION

The version string of the OpenSSL library loaded by the interpreter: `>>>ssl.OPENSSL_VERSION` 'OpenSSL 1.0.2k 26 Jan 2017' Added in version 3.2.

ssl.OPENSSL_VERSION_INFO

A tuple of five integers representing version information about the OpenSSL library: `>>>ssl.OPENSSL_VERSION_INFO(1, 0, 2, 11, 15)` Added in version 3.2.

ssl.OPENSSL_VERSION_NUMBER

The raw version number of the OpenSSL library, as a single integer: `>>>ssl.OPENSSL_VERSION_NUMBER` 268443839 `>>>hex(ssl.OPENSSL_VERSION_NUMBER)` '0x100020bf' Added in version 3.2.

ssl.ALERT_DESCRIPTION_HANDSHAKE_FAILURE

ssl.ALERT_DESCRIPTION_INTERNAL_ERROR

ssl.ALERT_DESCRIPTION_*

Alert Descriptions from RFC 5246 and others. The IANA TLS Alert Registry contains this list and references to the RFCs where their meaning is defined. Used as the return value of the callback function in `SSLContext.set_servername_callback()`. Added in version 3.4.

ssl.AlertDescription

enum.IntEnum collection of `ssl.ALERT_DESCRIPTION_*` constants. Added in version 3.6.

ssl.Purpose.SERVER_AUTH

Option for `create_default_context()` and `SSLContext.load_default_certs()`. This value indicates that the context may be used to authenticate web servers (therefore, it will be used to create client-side sockets). Added in version 3.4.

ssl.Purpose.CLIENT_AUTH

Option for `create_default_context()` and `SSLContext.load_default_certs()`. This value indicates that the context may be used to authenticate web clients (therefore, it will be used to create server-side sockets). Added in version 3.4.

ssl.SSLErrorNumber

enum.IntEnum collection of `ssl.SSL_ERROR_*` constants. Added in version 3.6.

ssl.TLSVersion

enum.IntEnum collection of SSL and TLS versions for `SSLContext.maximum_version` and `SSLContext.minimum_version`. Added in version 3.7.

ssl.TLSVersion.MINIMUM_SUPPORTED

TLSVersion.MAXIMUM_SUPPORTED

The minimum or maximum supported SSL or TLS version. These are magic constants. Their values don't reflect the lowest and highest available TLS/SSL versions.

TLSVersion.SSLv3

TLSVersion.TLSv1

TLSVersion.TLSv1_1

TLSVersion.TLSv1_2

TLSVersion.TLSv1_3

SSL 3.0 to TLS 1.3. Deprecated since version 3.10: All `TLSVersion` members except `TLSVersion.TLSv1_2` and `TLSVersion.TLSv1_3` are deprecated.

SSL sockets

class ssl.SSLSocket(socket.socket)

SSL sockets provide the following methods of Socket Objects: `accept()`, `bind()`, `close()`, `connect()`, `detach()`, `fileno()`, `getpeername()`, `getsockname()`, `getsockopt()`, `setsockopt()`, `gettimeout()`, `settimeout()`, `setblocking()`, `listen()`, `makefile()`, `recv()`, `recv_into()` (but passing a non-zero `flags` argument is not allowed), `send()`, `sendall()` (with the same limitation), `sendfile()` (but `os.sendfile` will be used for plain-text sockets only, else `send()` will be used), `shutdown()`. However, since the SSL (and TLS) protocol has its own framing atop of TCP,

SSL sockets also have the following additional methods and attributes:

SSLSocket.read(len=1024,buffer=None)

Read up to `len` bytes of data from the SSL socket and return the result as a `bytes` instance. If `buffer` is specified, then read into the buffer instead, and return the number of bytes read. Raise `SSLWantReadError` or `SSLWantWriteError` if the socket is non-blocking and the read would block. As at any time a re-negotiation is possible, a call to `read()` can also cause write operations. Changed in version 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum

SSLSocket.write(data)

Write `data` to the SSL socket and return the number of bytes written. The `data` argument must be an object supporting the buffer interface. Raise `SSLWantReadError` or `SSLWantWriteError` if the socket is non-blocking and the write would block. As at any time a re-negotiation is possible, a call to `write()` can also cause read operations. Changed in version 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration to write `data`. Deprecated since

■ **Note:** Note that `read()` and `write()` methods are the low-level methods that read and write unencrypted, application-level data and decrypt/encrypt it to encrypted, wire-level data. These methods require an active SSL connection, i.e. the handshake was completed and `SSLSocket.unwrap()` was not called. Normally you should use the socket API methods like `recv()` and `send()` instead of these methods.

SSLSocket.do_handshake(block=False)

Perform the SSL setup handshake. If `block` is `True` and the timeout obtained by `gettimeout()` is zero, the socket is set in blocking mode until the handshake is performed. Changed in version 3.4: The handshake method also performs `match_hostname()` when the `check_hostname` attribute of the socket's context is `True`. Changed in version 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration of the handshake. Changed in version 3.7: Hostname o

SSLSocket.getpeercert(binary_form=False)

If there is no certificate for the peer on the other end of the connection, returnNone. If the SSL handshake hasn't been done yet, raiseValueError.If thebinary_formparameter isFalse, and a certificate was received from the peer, this method returns adictinstance. If the certificate was not validated, the dict is empty. If the certificate was validated, it returns a dict with several keys, amongst themsubject(the principal for which the certificate was issued) andissuer(the principal issuing t

SSLSocket.get_verified_chain()

Returns verified certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes. If certificate verification was disabled method acts the same asget_unverified_chain().Added in version 3.13.

SSLSocket.get_unverified_chain()

Returns raw certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes.Added in version 3.13.

SSLSocket.cipher()

Returns a three-value tuple containing the name of the cipher being used, the version of the SSL protocol that defines its use, and the number of secret bits being used. If no connection has been established, returnsNone.

SSLSocket.shared_ciphers()

Return the list of ciphers available in both the client and server. Each entry of the returned list is a three-value tuple containing the name of the cipher, the version of the SSL protocol that defines its use, and the number of secret bits the cipher uses.shared_ciphers()returnsNoneif no connection has been established or the socket is a client socket.Added in version 3.5.

SSLSocket.compression()

Return the compression algorithm being used as a string, orNoneif the connection isn't compressed.If the higher-level protocol supports its own compression mechanism, you can useOP_NO_COMPRESSIONto disable SSL-level compression.Added in version 3.3.

SSLSocket.get_channel_binding(cb_type='tls-unique')

Get channel binding data for current connection, as a bytes object. ReturnsNoneif not connected or the handshake has not been completed.Thecb_typeparameter allow selection of the desired channel binding type. Valid channel binding types are listed in theCHANNEL_BINDING_TYPESlist. Currently only the 'tls-unique' channel binding, defined byRFC 5929, is supported.ValueErrorwill be raised if an unsupported channel binding type is requested.Added in version 3.3.

SSLSocket.selected_alpn_protocol()

Return the protocol that was selected during the TLS handshake. IfSSLContext.set_alpn_protocols()was not called, if the other party does not support ALPN, if this socket does not support any of the client's proposed protocols, or if the handshake has not happened yet,Noneis returned.Added in version 3.5.

SSLSocket.selected_npn_protocol()

Return the higher-level protocol that was selected during the TLS/SSL handshake. IfSSLContext.set_npn_protocols()was not called, or if the other party does not support NPN, or if the handshake has not yet happened, this will returnNone.Added in version 3.3.Deprecated since version 3.10:NPN has been superseded by ALPN

SSLSocket.unwrap()

Performs the SSL shutdown handshake, which removes the TLS layer from the underlying socket, and returns the underlying socket object. This can be used to go from encrypted operation over a connection to unencrypted. The returned socket should always be used for further communication with the other side of the connection, rather than the original socket.

SSLSocket.verify_client_post_handshake()

Requests post-handshake authentication (PHA) from a TLS 1.3 client. PHA can only be initiated for a TLS 1.3 connection from a server-side socket, after the initial TLS handshake and with PHA enabled on both sides, see `SSLContext.post_handshake_auth`. The method does not perform a cert exchange immediately. The server-side sends a `CertificateRequest` during the next write event and expects the client to respond with a certificate on the next read event. If any precondition isn't met (e.g. not TLS 1.3,

SSLSocket.version()

Return the actual SSL protocol version negotiated by the connection as a string, or `None` if no secure connection is established. As of this writing, possible return values include `"SSLv2"`, `"SSLv3"`, `"TLSv1"`, `"TLSv1.1"` and `"TLSv1.2"`. Recent OpenSSL versions may define more return values. Added in version 3.5.

SSLSocket.pending()

Returns the number of already decrypted bytes available for read, pending on the connection.

SSLSocket.context

The `SSLContext` object this SSL socket is tied to. Added in version 3.2.

SSLSocket.server_side

A boolean which is `True` for server-side sockets and `False` for client-side sockets. Added in version 3.2.

SSLSocket.server_hostname

Hostname of the server: str type, or `None` for server-side socket or if the hostname was not specified in the constructor. Added in version 3.2. Changed in version 3.7: The attribute is now always ASCII text. When `server_hostname` is an internationalized domain name (IDN), this attribute now stores the A-label form (`"xn--pythn-mua.org"`), rather than the U-label form (`"python.org"`).

SSLSocket.session

The `SSLSession` for this SSL connection. The session is available for client and server side sockets after the TLS handshake has been performed. For client sockets the session can be set before `do_handshake()` has been called to reuse a session. Added in version 3.6.

SSLSocket.session_reused

Added in version 3.6.

SSL contexts

Added in version 3.2.

An SSL context holds various data longer-lived than single SSL connections, such as SSL configuration options, certificate(s) and private key(s). It also manages a cache of SSL sessions for server-side sockets, in order to speed up repeated connections from the same clients.

class ssl.SSLContext(protocol=None)

Create a new SSL context. You may pass `protocol` which must be one of the `PROTOCOL_*` constants defined in this module. The parameter specifies which version of the SSL protocol to use. Typically, the server chooses a particular protocol version, and the client must adapt to the server's choice. Most of the versions are not interoperable with the other versions. If not specified, the default is `PROTOCOL_TLS`; it provides the most compatibility with other versions. Here's a table showing which versio

`SSLContext` objects have the following methods and attributes:

SSLContext.cert_store_stats()

Get statistics about quantities of loaded X.509 certificates, count of X.509 certificates flagged as CA certificates and certificate revocation lists as dictionary. Example for a context with one CA cert and one other cert: >>> context.cert_store_stats() {'crl': 0, 'x509_ca': 1, 'x509': 2} Added in version 3.4.

SSLContext.load_cert_chain(certfile, keyfile=None, password=None)

Load a private key and the corresponding certificate. The certfile string must be the path to a single file in PEM format containing the certificate as well as any number of CA certificates needed to establish the certificate's authenticity. The keyfile string, if present, must point to a file containing the private key. Otherwise the private key will be taken from certfile as well. See the discussion of Certificates for more information on how the certificate is stored in the certfile. The password arg

SSLContext.load_default_certs(purpose=Purpose.SERVER_AUTH)

Load a set of default "certification authority" (CA) certificates from default locations. On Windows it loads CA certs from the CA and ROOT system stores. On all systems it calls SSLContext.set_default_verify_paths(). In the future the method may load CA certificates from other locations, too. The purpose flag specifies what kind of CA certificates are loaded. The default settings Purpose.SERVER_AUTH loads certificates, that are flagged and trusted for TLS web server authentication (client side sockets). P

SSLContext.load_verify_locations(cafile=None, capath=None, cadata=None)

Load a set of "certification authority" (CA) certificates used to validate other peers' certificates when verify_mode is other than CERT_NONE. At least one of cafile or capath must be specified. This method can also load certification revocation lists (CRLs) in PEM or DER format. In order to make use of CRLs, SSLContext.verify_flags must be configured properly. The cafile string, if present, is the path to a file of concatenated CA certificates in PEM format. See the discussion of Certificates for more inform

SSLContext.get_ca_certs(binary_form=False)

Get a list of loaded "certification authority" (CA) certificates. If the binary_form parameter is False each list entry is a dict like the output of SSLSocket.get_peer_cert(). Otherwise the method returns a list of DER-encoded certificates. The returned list does not contain certificates from capath unless a certificate was requested and loaded by a SSL connection. Note Certificates in a capath directory aren't loaded unless they have been used at least once. Added in version 3.4.

SSLContext.get_ciphers()

Get a list of enabled ciphers. The list is in order of cipher priority. See SSLContext.set_ciphers(). Example: >>> ctx = ssl.SSLContext(ssl.PROTOCOL_SSLv23) >>> ctx.set_ciphers('ECDHE+AESGCM:!ECDSA') >>> ctx.get_ciphers()[0] {'aead': True, 'alg_bits': 256, 'auth': 'auth-rsa', 'description': 'ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH Au=RSA "Enc=AESGCM(256) Mac=AEAD", 'digest': None, 'id': 50380848, 'kea': 'kx-ecdhe', 'name': 'ECDHE-RSA-AES256-GCM-SHA384', 'protocol': 'TLSv1.2', 'strength_bits': 256, 'symmetric'}

SSLContext.set_default_verify_paths()

Load a set of default "certification authority" (CA) certificates from a filesystem path defined when building the OpenSSL library. Unfortunately, there's no easy way to know whether this method succeeds: no error is returned if no certificates are to be found. When the OpenSSL library is provided as part of the operating system, though, it is likely to be configured properly.

SSLContext.set_ciphers(ciphers, /)

Set the available ciphers for sockets created with this context. It should be a string in the OpenSSL cipher list format. If no cipher can be selected (because compile-time options or other configuration forbids use of all the specified ciphers), an SSL error will be raised. Note when connected, the SSLSocket.cipher() method of SSL sockets will give the currently selected cipher. TLS 1.3 cipher suites cannot be disabled with set_ciphers().

SSLContext.set_alpn_protocols(alpn_protocols)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of ASCII strings, like['http/1.1','spdy/2'], ordered by preference. The selection of a protocol will happen during the handshake, and will play out according to RFC 7301. After a successful handshake, the `SSLSocket.selected_alpn_protocol()` method will return the agreed-upon protocol. This method will raise `NotImplementedError` if `HAS_ALPN` is `False`. Added in version 3.5.

SSLContext.set_npn_protocols(npn_protocols)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of strings, like['http/1.1','spdy/2'], ordered by preference. The selection of a protocol will happen during the handshake, and will play out according to the Application Layer Protocol Negotiation. After a successful handshake, the `SSLSocket.selected_npn_protocol()` method will return the agreed-upon protocol. This method will raise `NotImplementedError` if `HAS_NPN` is `False`. Added in version 3.3. Deprecated

SSLContext.sni_callback

Register a callback function that will be called after the TLS Client Hello handshake message has been received by the SSL/TLS server when the TLS client specifies a server name indication. The server name indication mechanism is specified in RFC 6066 section 3 - Server Name Indication. Only one callback can be set per `SSLContext`. If `sni_callback` is set to `None` then the callback is disabled. Calling this function a subsequent time will disable the previously registered callback. The callback function will

SSLContext.set_servername_callback(server_name_callback)

This is a legacy API retained for backwards compatibility. When possible, you should use `sni_callback` instead. The given `server_name_callback` is similar to `sni_callback`, except that when the server hostname is an IDN-encoded internationalized domain name, the `server_name_callback` receives a decoded U-label ("python.org"). If there is a decoding error on the server name, the TLS connection will terminate with an `ALERT_DESCRIPTION_INTERNAL_ERROR` fatal TLS alert message to the client. Added in version 3.4.

SSLContext.load_dh_params(dhfile, /)

Load the key generation parameters for Diffie-Hellman (DH) key exchange. Using DH key exchange improves forward secrecy at the expense of computational resources (both on the server and on the client). The `dhfile` parameter should be the path to a file containing DH parameters in PEM format. This setting doesn't apply to client sockets. You can also use the `OP_SINGLE_DH_USE` option to further improve security. Added in version 3.3.

SSLContext.set_ecdh_curve(curve_name, /)

SSL/TLS & Perfect Forward Secrecy

Set the curve name for Elliptic Curve-based Diffie-Hellman (ECDH) key exchange. ECDH is significantly faster than regular DH while arguably as secure. The `curve_name` parameter should be a string describing a well-known elliptic curve, for example `prime256v1` for a widely supported curve. This setting doesn't apply to client sockets. You can also use the `OP_SINGLE_ECDH_USE` option to further improve security. This method is not available if `HAS_ECDH` is `False`. Added in version 3.3. See also [SSL/TLS & Perfect F](#)

Vincent Bernat.

SSLContext.wrap_socket(sock, server_side=False, do_handshake_on_connect=True, suppress_ragged_eofs=True, server_hostname=None, session=None)

Wrap an existing Python socket `sock` and return an instance of `SSLContext.sslsocket_class` (default `SSLSocket`). The returned SSL socket is tied to the context, its settings and certificates. `sock` must be a `SOCK_STREAM` socket; other socket types are unsupported. The parameter `server_side` is a boolean which identifies whether server-side or client-side behavior is desired from this socket. For client-side sockets, the

context construction is lazy; if the underlying socket isn't connected yet, the context constru

SSLContext.sslsocket_class

The return type of `SSLContext.wrap_socket()`, defaults to `SSLSocket`. The attribute can be assigned to on instances of `SSLContext` in order to return a custom subclass of `SSLSocket`. Added in version 3.7.

SSLContext.wrap_bio(incoming,outgoing,server_side=False,server_hostname=None,session=None)

Wrap the BIO objects `incoming` and `outgoing` and return an instance of `SSLContext.sslobj_class` (default `SSLObject`). The SSL routines will read input data from the incoming BIO and write data to the outgoing BIO. The `server_side`, `server_hostname` and `session` parameters have the same meaning as in `SSLContext.wrap_socket()`. Changed in version 3.6: `session` argument was added. Changed in version 3.7: The method returns an instance of `SSLContext.sslobj_class` instead of hard-coded `SSLObject`.

SSLContext.sslobj_class

The return type of `SSLContext.wrap_bio()`, defaults to `SSLObject`. The attribute can be overridden on instance of class in order to return a custom subclass of `SSLObject`. Added in version 3.7.

SSLContext.session_stats()

Get statistics about the SSL sessions created or managed by this context. A dictionary is returned which maps the names of each piece of information to their numeric values. For example, here is the total number of hits and misses in the session cache since the context was created:

```
>>> stats=context.session_stats()>>> stats['hits'], stats['misses']
```

 (0, 0)

SSLContext.check_hostname

Whether to match the peer cert's hostname in `SSLSocket.do_handshake()`. The context's `verify_mode` must be set to `CERT_OPTIONAL` or `CERT_REQUIRED`, and you must pass `server_hostname` to `wrap_socket()` in order to match the hostname. Enabling hostname checking automatically sets `verify_mode` from `CERT_NONE` to `CERT_REQUIRED`. It cannot be set back to `CERT_NONE` as long as hostname checking is enabled. The `PROTOCOL_TLS_CLIENT` protocol enables hostname checking by default. With other protocols, hostname checking must be enabled.

SSLContext.keylog_filename

Write TLS keys to a keylog file, whenever key material is generated or received. The keylog file is designed for debugging purposes only. The file format is specified by NSS and used by many traffic analyzers such as Wireshark. The log file is opened in append-only mode. Writes are synchronized between threads, but not between processes. Added in version 3.8.

SSLContext.maximum_version

`ATLSVersion` enum member representing the highest supported TLS version. The value defaults to `TLSVersion.MAXIMUM_SUPPORTED`. The attribute is read-only for protocols other than `PROTOCOL_TLS`, `PROTOCOL_TLS_CLIENT`, and `PROTOCOL_TLS_SERVER`. The attributes `maximum_version`, `minimum_version` and `SSLContext.options` all affect the supported SSL and TLS versions of the context. The implementation does not prevent invalid combination. For example a context with `OP_NO_TLSv1_2` in options and `maximum_version` set to `TLSVersion.TL`

SSLContext.minimum_version

Like `SSLContext.maximum_version` except it is the lowest supported version or `TLSVersion.MINIMUM_SUPPORTED`. Added in version 3.7.

SSLContext.num_tickets

Control the number of TLS 1.3 session tickets of a `PROTOCOL_TLS_SERVER` context. The setting has no impact on TLS 1.0 to 1.2 connections. Added in version 3.8.

SSLContext.options

An integer representing the set of SSL options enabled on this context. The default value is `OP_ALL`, but you can specify other options such as `OP_NO_SSLv2` by ORing them together. Changed in version 3.6: `SSLContext.options` returns `Options.flags`:>>> `ssl.create_default_context().options`<`Options.OP_ALL|OP_NO_SSLv3|OP_NO_SSLv2|OP_NO_COMPRESSION: 2197947391`> Deprecated since version 3.7: `OP_NO_SSL` and `OP_NO_TLS` options have been deprecated since Python 3.7. Use `SSLContext.minimum_version` and `SSLContext.maximum_version`

SSLContext.post_handshake_auth

Enable TLS 1.3 post-handshake client authentication. Post-handshake auth is disabled by default and a server can only request a TLS client certificate during the initial handshake. When enabled, a server may request a TLS client certificate at any time after the handshake. When enabled on client-side sockets, the client signals the server that it supports post-handshake authentication. When enabled on server-side sockets, `SSLContext.verify_mode` must be set to `CERT_OPTIONAL` or `CERT_REQUIRED`, too. The ac

SSLContext.protocol

The protocol version chosen when constructing the context. This attribute is read-only.

SSLContext.hostname_checks_common_name

Whether `check_hostname` falls back to verify the cert's subject common name in the absence of a subject alternative name extension (default: `true`). Added in version 3.7. Changed in version 3.10: The flag had no effect with OpenSSL before version 1.1.1l. Python 3.8.9, 3.9.3, and 3.10 include workarounds for previous versions.

SSLContext.security_level

An integer representing the security level for the context. This attribute is read-only. Added in version 3.10.

SSLContext.verify_flags

The flags for certificate verification operations. You can set flags like `VERIFY_CRL_CHECK_LEAF` by ORing them together. By default OpenSSL does neither require nor verify certificate revocation lists (CRLs). Added in version 3.4. Changed in version 3.6: `SSLContext.verify_flags` returns `VerifyFlags.flags`:>>> `ssl.create_default_context().verify_flags`<`VerifyFlags.VERIFY_X509_TRUSTED_FIRST: 32768`>

SSLContext.verify_mode

Whether to try to verify other peers' certificates and how to behave if verification fails. This attribute must be one of `CERT_NONE`, `CERT_OPTIONAL` or `CERT_REQUIRED`. Changed in version 3.6: `SSLContext.verify_mode` returns `VerifyMode.enum`:>>> `ssl.create_default_context().verify_mode`<`VerifyMode.CERT_REQUIRED: 2`>

SSLContext.set_psk_client_callback(callback)

Enables TLS-PSK (pre-shared key) authentication on a client-side connection. In general, certificate based authentication should be preferred over this method. The parameter `callback` is a callable object with the signature: `def callback(hint: str|None) -> tuple[str|None, bytes]`. The `hint` parameter is an optional identity hint sent by the server. The return value is a tuple in the form (client-identity, psk). Client-identity is an optional string which may be used by the server to select a corresponding PSK

SSLContext.set_psk_server_callback(callback, identity_hint=None)

Enables TLS-PSK (pre-shared key) authentication on a server-side connection. In general, certificate based authentication should be preferred over this method. The parameter `callback` is a callable object with the signature: `def callback(identity: str|None) -> bytes`. The `identity` parameter is an optional identity sent by the client which can be used to select a corresponding PSK. The return value is a bytes-like object representing the pre-shared key. Return a zero length PSK to reject the connection. Setting c

Certificates

Certificates in general are part of a public-key / private-key system. In this system, each principal, (which may be a machine, or a person, or an organization) is assigned a unique two-part encryption key. One part of the key is public, and is called the public key; the other part is kept secret, and is called the private key. The two parts are related, in that if you encrypt a message with one of the parts, you can decrypt it with the other part, and only with the other part.

A certificate contains information about two principals. It contains the name of a subject, and the subject's public key. It also contains a statement by a second principal, the issuer, that the subject is who they claim to be, and that this is indeed the subject's public key. The issuer's statement is signed with the issuer's private key, which only the issuer knows. However, anyone can verify the issuer's statement by finding the issuer's public key, decrypting the statement with it, and comparing it to the other information in the certificate. The certificate also contains information about the time period over which it is valid. This is expressed as two fields, called "notBefore" and "notAfter".

In the Python use of certificates, a client or server can use a certificate to prove who they are. The other side of a network connection can also be required to produce a certificate, and that certificate can be validated to the satisfaction of the client or server that requires such validation. The connection attempt can be set to raise an exception if the validation fails. Validation is done automatically, by the underlying OpenSSL framework; the application need not concern itself with its mechanics. But the application does usually need to provide sets of certificates to allow this process to take place.

Python uses files to contain certificates. They should be formatted as "PEM" (see RFC 1422), which is a base-64 encoded form wrapped with a header line and a footer line:

```
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Certificate chains

The Python files which contain certificates can contain a sequence of certificates, sometimes called a certificate chain. This chain should start with the specific certificate for the principal who "is" the client or server, and then the certificate for the issuer of that certificate, and then the certificate for the issuer of that certificate, and so on up the chain till you get to a certificate which is self-signed, that is, a certificate which has the same subject and issuer, sometimes called a root certificate. The certificates should just be concatenated together in the certificate file. For example, suppose we had a three certificate chain, from our server certificate to the certificate of the certification authority that signed our server certificate, to the root certificate of the agency which issued the certification authority's certificate:

```
-----BEGIN CERTIFICATE-----
... (certificate for your server)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the certificate for the CA)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the root certificate for the CA's issuer)...
-----END CERTIFICATE-----
```

CA certificates

If you are going to require validation of the other side of the connection's certificate, you need to provide a "CA certs" file, filled with the certificate chains for each issuer you are willing to trust. Again, this file just contains these chains concatenated together. For validation, Python will use the first chain it finds in the file which matches. The platform's certificates file can be used by calling `SSLContext.load_default_certs()`, this is done automatically with `create_default_context()`.

Combined key and certificate

Often the private key is stored in the same file as the certificate; in this case, only the `certfile` parameter to `SSLContext.load_cert_chain()` needs to be passed. If the private key is stored with the certificate, it should come before the first certificate in the certificate chain:

```
-----BEGIN RSA PRIVATE KEY-----
... (private key in base64 encoding) ...
-----END RSA PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Self-signed certificates

If you are going to create a server that provides SSL-encrypted connection services, you will need to acquire a certificate for that service. There are many ways of acquiring appropriate certificates, such as buying one from a certification authority. Another common practice is to generate a self-signed certificate. The simplest way to do this is with the OpenSSL package, using something like the following:

```
% openssl req -new -x509 -days 365 -nodes -out cert.pem -keyout cert.pem
Generating a 1024 bit RSA private key
.....+++++
.....+++++
writing new private key to 'cert.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:MyState
Locality Name (eg, city) []:Some City
Organization Name (eg, company) [Internet Widgits Pty Ltd]:My Organization, Inc.
Organizational Unit Name (eg, section) []:My Group
Common Name (eg, YOUR name) []:myserver.mygroup.myorganization.com
Email Address []:ops@myserver.mygroup.myorganization.com
%
```

The disadvantage of a self-signed certificate is that it is its own root certificate, and no one else will have it in their cache of known (and trusted) root certificates.

Examples

Testing for SSL support

To test for the presence of SSL support in a Python installation, user code should use the following idiom:

```
try:
    import ssl
except ImportError:
    pass
else:
    ... # do something that requires SSL support
```

Client-side operation

This example creates a SSL context with the recommended security settings for client sockets, including automatic certificate verification:

```
>>> context = ssl.create_default_context()
```

If you prefer to tune security settings yourself, you might create a context from scratch (but beware that you might not get the settings right):

```
>>> context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> context.load_verify_locations("/etc/ssl/certs/ca-bundle.crt")
```

(this snippet assumes your operating system places a bundle of all CA certificates in/etc/ssl/certs/ca-bundle.crt; if not, you'll get an error and have to adjust the location)

The `PROTOCOL_TLS_CLIENT` protocol configures the context for cert validation and hostname verification. `verify_mode` is set to `CERT_REQUIRED` and `check_hostname` is set to `True`. All other protocols create SSL contexts with insecure defaults.

When you use the context to connect to a server, `CERT_REQUIRED` and `check_hostname` validate the server certificate: it ensures that the server certificate was signed with one of the CA certificates, checks the signature for correctness, and verifies other properties like validity and identity of the hostname:

```
>>> conn = context.wrap_socket(socket.socket(socket.AF_INET),
...                             server_hostname="www.python.org")
>>> conn.connect(("www.python.org", 443))
```

You may then fetch the certificate:

```
>>> cert = conn.getpeercert()
```

Visual inspection shows that the certificate does identify the desired service (that is, the HTTPS host `www.python.org`):

```
>>> pprint.pprint(cert)
{'OCSP': ('http://ocsp.digicert.com',),
 'caIssuers': ('http://cacerts.digicert.com/DigiCertSHA2ExtendedValidationServerCA.crt',),
 'crlDistributionPoints': ('http://crl3.digicert.com/sha2-ev-server-g1.crl',
                           'http://crl4.digicert.com/sha2-ev-server-g1.crl'),
 'issuer': (((('countryName', 'US'),),
               (('organizationName', 'DigiCert Inc'),),
               (('organizationalUnitName', 'www.digicert.com'),),
               (('commonName', 'DigiCert SHA2 Extended Validation Server CA'),)),
 'notAfter': 'Sep  9 12:00:00 2016 GMT',
 'notBefore': 'Sep  5 00:00:00 2014 GMT',
 'serialNumber': '01BB6F00122B177F36CAB49CEA8B6B26',
 'subject': (((('businessCategory', 'Private Organization'),),
                (('1.3.6.1.4.1.311.60.2.1.3', 'US'),),
                (('1.3.6.1.4.1.311.60.2.1.2', 'Delaware'),),
                (('serialNumber', '3359300'),),
                (('streetAddress', '16 Allen Rd'),),
                (('postalCode', '03894-4801'),),
                (('countryName', 'US'),),
                (('stateOrProvinceName', 'NH'),),
                (('localityName', 'Wolfeboro'),),
                (('organizationName', 'Python Software Foundation'),),
                (('commonName', 'www.python.org'),)),
 'subjectAltName': (('DNS', 'www.python.org'),
                    ('DNS', 'python.org'),
                    ('DNS', 'pypi.org'),
                    ('DNS', 'docs.python.org'),
                    ('DNS', 'testpypi.org'),
                    ('DNS', 'bugs.python.org'),
                    ('DNS', 'wiki.python.org'),
                    ('DNS', 'hg.python.org'),
                    ('DNS', 'mail.python.org'),
                    ('DNS', 'packaging.python.org'),
                    ('DNS', 'pythonhosted.org'),
                    ('DNS', 'www.pythonhosted.org'),
```

```

        ('DNS', 'test.pythonhosted.org'),
        ('DNS', 'us.pycon.org'),
        ('DNS', 'id.python.org')),
    'version': 3}

```

Now the SSL channel is established and the certificate verified, you can proceed to talk with the server:

```

>>> conn.sendall(b"HEAD / HTTP/1.0\r\nHost: linuxfr.org\r\n\r\n")
>>> pprint.pprint(conn.recv(1024).split(b"\r\n"))
[b'HTTP/1.1 200 OK',
 b'Date: Sat, 18 Oct 2014 18:27:20 GMT',
 b'Server: nginx',
 b'Content-Type: text/html; charset=utf-8',
 b'X-Frame-Options: SAMEORIGIN',
 b'Content-Length: 45679',
 b'Accept-Ranges: bytes',
 b'Via: 1.1 varnish',
 b'Age: 2188',
 b'X-Served-By: cache-lcy1134-LCY',
 b'X-Cache: HIT',
 b'X-Cache-Hits: 11',
 b'Vary: Cookie',
 b'Strict-Transport-Security: max-age=63072000; includeSubDomains',
 b'Connection: close',
 b'',
 b'']

```

See the discussion of [Security considerations](#) below.

Server-side operation

For server operation, typically you'll need to have a server certificate, and private key, each in a file. You'll first create a context holding the key and the certificate, so that clients can check your authenticity. Then you'll open a socket, bind it to a port, call `listen()` on it, and start waiting for clients to connect:

```

import socket, ssl

context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile="mycertfile", keyfile="mykeyfile")

bindsocket = socket.socket()
bindsocket.bind(('myaddr.example.com', 10023))
bindsocket.listen(5)

```

When a client connects, you'll call `accept()` on the socket to get the new socket from the other end, and use the context's `SSLContext.wrap_socket()` method to create a server-side SSL socket for the connection:

```

while True:
    newsocket, fromaddr = bindsocket.accept()
    connstream = context.wrap_socket(newsocket, server_side=True)
    try:
        deal_with_client(connstream)
    finally:
        connstream.shutdown(socket.SHUT_RDWR)
        connstream.close()

```

Then you'll read data from the `connstream` and do something with it till you are finished with the client (or the client is finished with you):

```

def deal_with_client(connstream):
    data = connstream.recv(1024)
    # empty data means the client is finished with us
    while data:
        if not do_something(connstream, data):
            # we'll assume do_something returns False

```

```

        # when we're finished with client
        break
    data = connstream.recv(1024)
    # finished with client

```

And go back to listening for new client connections (of course, a real server would probably handle each client connection in a separate thread, or put the sockets in non-blocking mode and use an event loop).

Notes on non-blocking sockets

SSL sockets behave slightly different than regular sockets in non-blocking mode. When working with non-blocking sockets, there are thus several things you need to be aware of:

- Most `SSLSocket` methods will raise either `SSLWantWriteError` or `SSLWantReadError` instead of `BlockingIOError` if an I/O operation would block. `SSLWantReadError` will be raised if a read operation on the underlying socket is necessary, and `SSLWantWriteError` for a write operation on the underlying socket. Note that attempts to write to an SSL socket may require reading from the underlying socket first, and attempts to read
- Calling `select()` tells you that the OS-level socket can be read from (or written to), but it does not imply that there is sufficient data at the upper SSL layer. For example, only part of an SSL frame might have arrived. Therefore, you must be ready to handle `SSLSocket.recv()` and `SSLSocket.send()` failures, and retry after another call to `select()`.
- Conversely, since the SSL layer has its own framing, a SSL socket may still have data available for reading without `select()` being aware of it. Therefore, you should first call `SSLSocket.recv()` to drain any potentially available data, and then only block on a `select()` call if still necessary. (Of course, similar provisions apply when using other primitives such as `poll()`, or those in the `selectors` module)
- The SSL handshake itself will be non-blocking: the `SSLSocket.do_handshake()` method has to be retried until it returns successfully. Here is a synopsis using `select()` to wait for the socket's readiness:


```
while True:
    try:
        sock.do_handshake()
    except ssl.SSLWantReadError:
        select.select([sock], [], [])
    except ssl.SSLWantWriteError:
        select.select([], [sock], [])
```

See also

The `asyncio` module supports non-blocking SSL sockets and provides a higher level Streams API. It polls for events using the `selectors` module and handles `SSLWantWriteError`, `SSLWantReadError` and `BlockingIOError` exceptions. It runs the SSL handshake asynchronously as well.

Memory BIO support

Added in version 3.5.

Ever since the SSL module was introduced in Python 2.6, the `SSLSocket` class has provided two related but distinct areas of functionality:

- SSL protocol handling
- Network IO

The network IO API is identical to that provided by `socket.socket`, from which `SSLSocket` also inherits. This allows an SSL socket to be used as a drop-in replacement for a regular socket, making it very easy to add SSL support to an existing application.

Combining SSL protocol handling and network IO usually works well, but there are some cases where it doesn't. An example is async IO frameworks that want to use a different IO multiplexing model than the "select/poll on a

file descriptor” (readiness based) model that is assumed by `socket.socket` and by the internal OpenSSL socket IO routines. This is mostly relevant for platforms like Windows where this model is not efficient. For this purpose, a reduced scope variant of `SSL.Socket` called `SSL.Object` is provided.

class `ssl.SSLObject`

A reduced-scope variant of `SSL.Socket` representing an SSL protocol instance that does not contain any network IO methods. This class is typically used by framework authors that want to implement asynchronous IO for SSL through memory buffers. This class implements an interface on top of a low-level SSL object as implemented by OpenSSL. This object captures the state of an SSL connection but does not provide any network IO itself. IO needs to be performed through separate “BIO” objects which are Open

An `SSLObject` communicates with the outside world using memory buffers. The class `MemoryBIO` provides a memory buffer that can be used for this purpose. It wraps an OpenSSL memory BIO (Basic IO) object:

class `ssl.MemoryBIO`

pending

eof

read(*n=-1, /*)

write(*buf, /*)

write_eof()

A memory buffer that can be used to pass data between Python and an SSL protocol instance.
pending Return the number of bytes currently in the memory buffer.
eof A boolean indicating whether the memory BIO is current at the end-of-file position.
read(*n=-1, /*) Read up to *n* bytes from the memory buffer. If *n* is not specified or negative, all bytes are returned.
write(*buf, /*) Write the bytes from *buf* to the memory BIO. The *buf* argument must be an object supporting the buffer protocol. The return value is the number of

Return the number of bytes currently in the memory buffer.

A boolean indicating whether the memory BIO is current at the end-of-file position.

Read up to *n* bytes from the memory buffer. If *n* is not specified or negative, all bytes are returned.

Write the bytes from *buf* to the memory BIO. The *buf* argument must be an object supporting the buffer protocol. The return value is the number of bytes written, which is always equal to the length of *buf*.

Write an EOF marker to the memory BIO. After this method has been called, it is illegal to call `write()`. The attribute `eof` will become true after all data currently in the buffer has been read.

SSL session

Added in version 3.6.

class `ssl.SSLSession`

id

time

timeout

ticket_lifetime_hint

has_ticket

Session object used by `session.id`, `session.time`, `session.timeout`, `session.ticket_lifetime_hint`, `session.has_ticket`

Security considerations

Best defaults

For client use, if you don't have any special requirements for your security policy, it is highly recommended that you use the `create_default_context()` function to create your SSL context. It will load the system's trusted CA certificates, enable certificate validation and hostname checking, and try to choose reasonably secure protocol and cipher settings.

For example, here is how you would use the `smtplib.SMTP` class to create a trusted, secure connection to a SMTP server:

```
>>> import ssl, smtplib
>>> smtp = smtplib.SMTP("mail.python.org", port=587)
>>> context = ssl.create_default_context()
>>> smtp.starttls(context=context)
(220, b'2.0.0 Ready to start TLS')
```

If a client certificate is needed for the connection, it can be added with `SSLContext.load_cert_chain()`.

By contrast, if you create the SSL context by calling the `SSLContext` constructor yourself, it will not have certificate validation nor hostname checking enabled by default. If you do so, please read the paragraphs below to achieve a good security level.

Manual settings

Verifying certificates

When calling the `SSLContext` constructor directly, `CERT_NONE` is the default. Since it does not authenticate the other peer, it can be insecure, especially in client mode where most of the time you would like to ensure the authenticity of the server you're talking to. Therefore, when in client mode, it is highly recommended to use `CERT_REQUIRED`. However, it is in itself not sufficient; you also have to check that the server certificate, which can be obtained by calling `SSLSocket.getpeercert()`, matches the desired service. For many protocols and applications, the service can be identified by the hostname. This common check is automatically performed when `SSLContext.check_hostname` is enabled.

Changed in version 3.7: Hostname matching is now performed by OpenSSL. Python no longer uses `match_hostname()`.

In server mode, if you want to authenticate your clients using the SSL layer (rather than using a higher-level authentication mechanism), you'll also have to specify `CERT_REQUIRED` and similarly check the client certificate.

Protocol versions

SSL versions 2 and 3 are considered insecure and are therefore dangerous to use. If you want maximum compatibility between clients and servers, it is recommended to use `PROTOCOL_TLS_CLIENT` or `PROTOCOL_TLS_SERVER` as the protocol version. SSLv2 and SSLv3 are disabled by default.

```
>>> client_context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> client_context.minimum_version = ssl.TLSVersion.TLSv1_3
>>> client_context.maximum_version = ssl.TLSVersion.TLSv1_3
```

The SSL context created above will only allow TLSv1.3 and later (if supported by your system) connections to a server. `PROTOCOL_TLS_CLIENT` implies certificate validation and hostname checks by default. You have to load certificates into the context.

Cipher selection

If you have advanced security requirements, fine-tuning of the ciphers enabled when negotiating a SSL session is possible through the `SSLContext.set_ciphers()` method. Starting from Python 3.2.3, the `ssl` module disables certain weak ciphers by default, but you may want to further restrict the cipher choice. Be sure to read OpenSSL's documentation about the cipher list format. If you want to check which ciphers are enabled by a given cipher list, use `SSLContext.get_ciphers()` or the `openssl ciphers` command on your system.

Multi-processing

If using this module as part of a multi-processed application (using, for example `multiprocessing` or `concurrent.futures` modules), be aware that OpenSSL's internal random number generator does not properly handle forked processes. Applications must change the PRNG state of the parent process if they use any SSL feature with `os.fork()`. Any successful call of `RAND_add()` or `RAND_bytes()` is sufficient.

TLS 1.3

Added in version 3.7.

The TLS 1.3 protocol behaves slightly differently than previous version of TLS/SSL. Some new TLS 1.3 features are not yet available.

- TLS 1.3 uses a disjunct set of cipher suites. All AES-GCM and ChaCha20 cipher suites are enabled by default. The method `SSLContext.set_ciphers()` cannot enable or disable any TLS 1.3 ciphers yet, but `SSLContext.get_ciphers()` returns them.
- Session tickets are no longer sent as part of the initial handshake and are handled differently. `SSLSocket.session` and `SSLSession` are not compatible with TLS 1.3.
- Client-side certificates are also no longer verified during the initial handshake. A server can request a certificate at any time. Clients process certificate requests while they send or receive application data from the server.
- TLS 1.3 features like early data, deferred TLS client cert request, signature algorithm configuration, and rekeying are not supported yet.

See also

`Classsocket.socket`

SSL/TLS Strong Encryption: An Introduction

RFC 1422: Privacy Enhancement for Internet Electronic Mail: Part II: Certificate-Based Key Management

RFC 4086: Randomness Requirements for Security

RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile

RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2

RFC 6066: Transport Layer Security (TLS) Extensions

IANA TLS: Transport Layer Security (TLS) Parameters

RFC 7525: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)

Mozilla's Server Side TLS recommendations

Documentation of underlying `socket` class

Intro from the Apache HTTP Server documentation

Steve Kent

Donald E. Eastlake, Jeffrey I. Schiller, Steve Crocker

David Cooper et al.

Tim Dierks and Eric Rescorla.

Donald E. Eastlake

IANA

IETF

Mozilla

select— Waiting for I/O completion

This module provides access to the `select()` and `poll()` functions available in most operating systems, `devpoll()` available on Solaris and derivatives, `epoll()` available on Linux 2.5+ and `kqueue()` available on most BSD. Note that on Windows, it only works for sockets; on other operating systems, it also works for other file types (in particular, on Unix, it works on pipes). It cannot be used on regular files to determine whether a file has grown since it was last read.

■ **Nota:** Note that the `selectors` module allows high-level and efficient I/O multiplexing, built upon the `select` module primitives. Users are encouraged to use the `selectors` module instead, unless they want precise control over the OS-level primitives used.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

The module defines the following:

exception`select.error`

A deprecated alias of `OSError`. Changed in version 3.3: Following PEP 3151, this class was made an alias of `OSError`.

select`.devpoll()`

(Only supported on Solaris and derivatives.) Returns a `dev/poll` polling object; see section `dev/poll` Polling Objects below for the methods supported by `devpoll` objects. `devpoll()` objects are linked to the number of file descriptors allowed at the time of instantiation. If your program reduces this value, `devpoll()` will fail. If your program increases this value, `devpoll()` may return an incomplete list of active file descriptors. The new file descriptor is non-inheritable. Added in version 3.3. Changed in v

select`.epoll(sizehint=-1, flags=0)`

(Only supported on Linux 2.5.44 and newer.) Return an edge polling object, which can be used as Edge or Level Triggered interface for I/O events. `sizehint` informs `epoll` about the expected number of events to be registered. It must be positive, or -1 to use the default. It is only used on older systems where `epoll_create1()` is not available; otherwise it has no effect (though its value is still checked). `flags` is deprecated and completely ignored. However, when supplied, its value must be 0 or `select.EPOL`.

select`.poll()`

(Not supported by all operating systems.) Returns a polling object, which supports registering and unregistering file descriptors, and then polling them for I/O events; see section `Polling Objects` below for the methods supported by polling objects.

select`.kqueue()`

(Only supported on BSD.) Returns a kernel queue object; see section `Kqueue Objects` below for the methods supported by `kqueue` objects. The new file descriptor is non-inheritable. Changed in version 3.4: The new file descriptor is now non-inheritable.

select`.kevent(ident, filter=KQ_FILTER_READ, flags=KQ_EV_ADD, fflags=0, data=0, udata=0)`

(Only supported on BSD.) Returns a kernel event object; see section `Kevent Objects` below for the methods supported by `kevent` objects.

select`.select(rlist, wlist, xlist, timeout=None)`

This is a straightforward interface to the `Unixselect()` system call. The first three arguments are iterables of 'waitable objects': either integers representing file descriptors or objects with a parameterless method `namedfileno()` returning such an integer: `rlist`: wait until ready for reading `wlist`: wait until ready for writing `xlist`: wait for an "exceptional condition" (see the manual page for what your system considers such a condition) Empty iterables are allowed, but acceptance of three empty iter

select.PIPE_BUF

The minimum number of bytes which can be written without blocking to a pipe when the pipe has been reported as ready for writing by `select()`, `poll()` or another interface in this module. This doesn't apply to other kinds of file-like objects such as sockets. This value is guaranteed by POSIX to be at least 512. Availability: Unix Added in version 3.2.

/dev/poll Polling Objects

Solaris and derivatives have `/dev/poll`. While `select()` is `O(highest file descriptor)` and `poll()` is `O(number of file descriptors)`, `/dev/poll` is `O(active file descriptors)`.

`/dev/poll` behaviour is very close to the standard `poll()` object.

devpoll.close()

Close the file descriptor of the polling object. Added in version 3.4.

devpoll.closed

True if the polling object is closed. Added in version 3.4.

devpoll.fileno()

Return the file descriptor number of the polling object. Added in version 3.4.

devpoll.register(fd[,eventmask])

Register a file descriptor with the polling object. Future calls to the `poll()` method will then check whether the file descriptor has any pending I/O events. `fd` can be either an integer, or an object with a `fileno()` method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument. `eventmask` is an optional bitmask describing the type of events you want to check for. The constants are the same as with `poll()` object. The default value is a combination of the constant

devpoll.modify(fd[,eventmask])

This method does an `unregister()` followed by `register()`. It is (a bit) more efficient than doing the same explicitly.

devpoll.unregister(fd)

Remove a file descriptor being tracked by a polling object. Just like the `register()` method, `fd` can be an integer or an object with a `fileno()` method that returns an integer. Attempting to remove a file descriptor that was never registered is safely ignored.

devpoll.poll([timeout])

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd,event)` 2-tuples for the descriptors that have events or errors to report. `fd` is the file descriptor, and `event` is a bitmask with bits set for the reported events for that descriptor — `POLLIN` for waiting input, `POLLOUT` to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If `timeout` is given, it specifies th

Edge and Level Trigger Polling (epoll) Objects

<https://linux.die.net/man/4/epoll>

eventmask

Constant | Meaning | EPOLLIN | Available for read | EPOLLOUT | Available for write | EPOLLPRI | Urgent data for read | EPOLLERR | Error condition happened on the assoc. fd | EPOLLHUP | Hang up happened on the assoc. fd | EPOLLET | Set Edge Trigger behavior, the default is Level Trigger behavior | EPOLLONESHOT | Set one-shot behavior. After one event is pulled out, the fd is internally disabled | EPOLLEXCLUSIVE | Wake only one epoll object when the associated fd has an event. The default (if this flag is not set) is to wake all epoll objects polling on a fd. | EPOLLRDHUP | Stream socket peer closed connection or shut down writing half of connection. | EPOLLRDNORM | Equivalent to | EPOLLIN | EPOLLRDBAND | Priority data band can be read. | EPOLLWRNORM | Equivalent to | EPOLLOUT | EPOLLWRBAND

Added in version 3.6:EPOLLEXCLUSIVEwas added. It's only supported by Linux Kernel 4.5 or later.

Added in version 3.14:EPOLLWAKEUPwas added. It's only supported by Linux Kernel 3.5 or later.

epoll.close()

Close the control file descriptor of the epoll object.

epoll.closed

True if the epoll object is closed.

epoll.fileno()

Return the file descriptor number of the control fd.

epoll.fromfd(fd)

Create an epoll object from a given file descriptor.

epoll.register(fd[,eventmask])

Register a fd descriptor with the epoll object.

epoll.modify(fd,eventmask)

Modify a registered file descriptor.

epoll.unregister(fd)

Remove a registered file descriptor from the epoll object.Changed in version 3.9:The method no longer ignores theEBADError.

epoll.poll(timeout=None,maxevents=-1)

Wait for events. timeout in seconds (float)Changed in version 3.5:The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (seePEP 475for the rationale), instead of raisingInterruptedError.

Polling Objects

Thepoll()system call, supported on most Unix systems, provides better scalability for network servers that service many, many clients at the same time.poll()scales better because the system call only requires listing the file descriptors of interest, whileselect()builds a bitmap, turns on bits for the fds of interest, and then afterward the whole bitmap has to be linearly scanned again.select()isO(highest file descriptor), whilepoll()isO(number of file descriptors).

poll.register(fd[,eventmask])

Register a file descriptor with the polling object. Future calls to the `poll()` method will then check whether the file descriptor has any pending I/O events. `fd` can be either an integer, or an object with a `fileno()` method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument. `eventmask` is an optional bitmask describing the type of events you want to check for, and can be a combination of the constants `POLLIN`, `POLLPR`, and `POLLOUT`, described in the table below.

`poll.modify(fd,eventmask)`

Modifies an already registered `fd`. This has the same effect as `register(fd,eventmask)`. Attempting to modify a file descriptor that was never registered causes an `OSError` exception with `errno.ENOENT` to be raised.

`poll.unregister(fd)`

Remove a file descriptor being tracked by a polling object. Just like the `register()` method, `fd` can be an integer or an object with a `fileno()` method that returns an integer. Attempting to remove a file descriptor that was never registered causes a `KeyError` exception to be raised.

`poll.poll([timeout])`

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd,event)` 2-tuples for the descriptors that have events or errors to report. `fd` is the file descriptor, and `event` is a bitmask with bits set for the reported events for that descriptor — `POLLIN` for waiting input, `POLLOUT` to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If `timeout` is given, it specifies th

Kqueue Objects

`kqueue.close()`

Close the control file descriptor of the `kqueue` object.

`kqueue.closed`

True if the `kqueue` object is closed.

`kqueue.fileno()`

Return the file descriptor number of the control `fd`.

`kqueue.fromfd(fd)`

Create a `kqueue` object from a given file descriptor.

`kqueue.control(changelist,max_events[,timeout])→eventlist`

Low level interface to `kevent`. `changelist` must be an iterable of `kevent` objects or `None`. `max_events` must be 0 or a positive integer. `timeout` in seconds (floats possible); the default is `None`, to wait forever. Changed in version 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see PEP 475 for the rationale), instead of raising `InterruptedError`.

Kevent Objects

<https://man.freebsd.org/cgi/man.cgi?query=kqueue&sektion=2>

`kevent.ident`

Value used to identify the event. The interpretation depends on the filter but it's usually the file descriptor. In the constructor `ident` can either be an int or an object with a `fileno()` method. `kevent` stores the integer internally.

`kevent.filter`

Name of the kernel filter. Constant Meaning
KQ_FILTER_READ Takes a descriptor and returns whenever there is data available to read
KQ_FILTER_WRITE Takes a descriptor and returns whenever there is data available to write
KQ_FILTER_AIO AIO requests
KQ_FILTER_VNODE Returns when one or more of the requested events watched in flag occurs
KQ_FILTER_PROC Watch for events on a process id
KQ_FILTER_NETDEV Watch for events on a network device [not available on macOS]
KQ_FILTER_SIGNAL Returns whenever the watched signal is

kevent.flags

Filter action. Constant Meaning
KQ_EV_ADD Adds or modifies an event
KQ_EV_DELETE Removes an event from the queue
KQ_EV_ENABLE Permits control() to return the event
KQ_EV_DISABLE Disables event
KQ_EV_ONESHOT Removes event after first occurrence
KQ_EV_CLEAR Reset the state after an event is retrieved
KQ_EV_SYSFLAGS Internal event
KQ_EV_FLAG1 Internal event
KQ_EV_EOF Filter specific EOF condition
KQ_EV_ERROR See return values

kevent.fflags

Filter specific flags. KQ_FILTER_READ and KQ_FILTER_WRITE filter
flags: Constant Meaning
KQ_NOTE_LOWAT low water mark of a socket buffer
KQ_FILTER_VNODE filter
flags: Constant Meaning
KQ_NOTE_DELETE unlink() was called
KQ_NOTE_WRITE a write occurred
KQ_NOTE_EXTEND the file was extended
KQ_NOTE_ATTRIB an attribute was changed
KQ_NOTE_LINK the link count has changed
KQ_NOTE_RENAME the file was renamed
KQ_NOTE_REVOKE access to the file was revoked
KQ_FILTER_PROC filter
flags: Constant Meaning
KQ_NOTE_EXIT the process has exited
KQ_

kevent.data

Filter specific data.

kevent.udata

User defined value.

selectors— High-level I/O multiplexing

Added in version 3.4.

Source code: [Lib/selectors.py](#)

Introduction

This module allows high-level and efficient I/O multiplexing, built upon the `select` module primitives. Users are encouraged to use this module instead, unless they want precise control over the OS-level primitives used.

It defines a `BaseSelector` abstract base class, along with several concrete implementations (`KqueueSelector`, `EpollSelector`...), that can be used to wait for I/O readiness notification on multiple file objects. In the following, “file object” refers to any object with a `fileno()` method, or a raw file descriptor. See `file` object.

`DefaultSelector` is an alias to the most efficient implementation available on the current platform: this should be the default choice for most users.

■ **Nota:** Note The type of file objects supported depends on the platform: on Windows, sockets are supported, but not pipes, whereas on Unix, both are supported (some other types may be supported as well, such as fifos or special file devices).

See also

select

Low-level I/O multiplexing module.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Classes

Classes hierarchy:

```
BaseSelector
+-- SelectSelector
+-- PollSelector
+-- EpollSelector
+-- DevpollSelector
+-- KqueueSelector
```

In the following, `events` is a bitwise mask indicating which I/O events should be waited for on a given file object. It can be a combination of the module's constants below:

Constant	Meaning	<code>selectors.EVENT_READ</code>	Available for read	<code>selectors.EVENT_WRITE</code>	Available for write
----------	---------	-----------------------------------	--------------------	------------------------------------	---------------------

selectors.SelectorKey

fileobj

fd

events

data

A `SelectorKey` is a named tuple used to associate a file object to its underlying file descriptor, selected event mask and attached data. It is returned by several `BaseSelector` methods. `fileobj` File object registered. `fd` Underlying file descriptor. `events` Events that must be waited for on this file object. `data` Optional opaque data associated to this file object: for example, this could be used to store a per-client session ID.

File object registered.

Underlying file descriptor.

Events that must be waited for on this file object.

Optional opaque data associated to this file object: for example, this could be used to store a per-client session ID.

`class selectors.BaseSelector`

`abstractmethod register(fileobj, events, data=None)`

`abstractmethod unregister(fileobj)`

`modify(fileobj, events, data=None)`

`abstractmethod select(timeout=None)`

`close()`

`get_key(fileobj)`

`abstractmethod get_map()`

A `BaseSelector` is used to wait for I/O event readiness on multiple file objects. It supports file stream registration, unregistration, and a method to wait for I/O events on those streams, with an optional timeout. It's an abstract base class, so cannot be instantiated. Use `DefaultSelector` instead, or one of `SelectSelector`, `KQueueSelector` etc. if you want to specifically use an implementation, and your platform supports it. `BaseSelector` and its concrete implementations support the context manager protocol.

Register a file object for selection, monitoring it for I/O events. `fileobj` is the file object to monitor. It may either be an integer file descriptor or an object with a `fileno()` method. `events` is a bitwise mask of events to monitor. `data` is an opaque object. This returns a new `SelectorKey` instance, or raises a `ValueError` in case of invalid event mask or file descriptor, or `KeyError` if the file object is already registered.

Unregister a file object from selection, removing it from monitoring. A file object shall be unregistered prior to being closed. `fileobj` must be a file object previously registered. This returns the associated `SelectorKey` instance, or raises a `KeyError` if `fileobj` is not registered. It will raise `ValueError` if `fileobj` is invalid (e.g. it has no `fileno()` method or its `fileno()` method has an invalid return value).

Change a registered file object's monitored events or attached data. This is equivalent to `BaseSelector.unregister(fileobj)` followed by `BaseSelector.register(fileobj, events, data)`, except that it can be implemented more efficiently. This returns a new `SelectorKey` instance, or raises a `ValueError` in case of invalid event mask or file descriptor, or `KeyError` if the file object is not registered.

Wait until some registered file objects become ready, or the timeout expires. If `timeout > 0`, this specifies the maximum wait time, in seconds. If `timeout ≤ 0`, the call won't block, and will report the currently ready file objects. If `timeout is None`, the call will block until a monitored file object becomes ready. This returns a list of `(key, events)` tuples, one for each ready file object. `key` is the `SelectorKey` instance corresponding to a ready file object. `events` is a bitmask of events ready on this file object.

Close the selector. This must be called to make sure that any underlying resource is freed. The selector shall not be used once it has been closed.

Return the key associated with a registered file object. This returns the `SelectorKey` instance associated to this file object, or raises `KeyError` if the file object is not registered.

Return a mapping of file objects to selector keys. This returns a `Mapping` instance mapping registered file objects to their associated `SelectorKey` instance.

selectors.DefaultSelector

The default selector class, using the most efficient implementation available on the current platform. This should be the default choice for most users.

selectors.SelectSelector

`select.select()`-based selector.

selectors.PollSelector

`select.poll()`-based selector.

selectors.EpollSelector

fileno()

`select.epoll()`-based selector. `fileno()` This returns the file descriptor used by the underlying `select.epoll()` object.

This returns the file descriptor used by the underlying `select.epoll()` object.

selectors.DevpollSelector

fileno()

`select.devpoll()`-based selector. `fileno()` This returns the file descriptor used by the underlying `select.devpoll()` object. Added in version 3.5.

This returns the file descriptor used by the underlying `select.devpoll()` object.

selectors.KqueueSelector

fileno()

`select.kqueue()`-based selector. `fileno()` This returns the file descriptor used by the underlying `select.kqueue()` object.

This returns the file descriptor used by the underlying `select.kqueue()` object.

Examples

Here is a simple echo server implementation:

```
import selectors
import socket

sel = selectors.DefaultSelector()

def accept(sock, mask):
    conn, addr = sock.accept() # Should be ready
    print('accepted', conn, 'from', addr)
    conn.setblocking(False)
    sel.register(conn, selectors.EVENT_READ, read)

def read(conn, mask):
    data = conn.recv(1000) # Should be ready
    if data:
        print('echoing', repr(data), 'to', conn)
        conn.send(data) # Hope it won't block
    else:
        print('closing', conn)
        sel.unregister(conn)
```

```
        conn.close()

sock = socket.socket()
sock.bind(('localhost', 1234))
sock.listen(100)
sock.setblocking(False)
sel.register(sock, selectors.EVENT_READ, accept)

while True:
    events = sel.select()
    for key, mask in events:
        callback = key.data
        callback(key.fileobj, mask)
```

signal— Set handlers for asynchronous events

Source code: `Lib/signal.py`

This module provides mechanisms to use signal handlers in Python.

General rules

The `signal.signal()` function allows defining custom handlers to be executed when a signal is received. A small number of default handlers are installed: `SIGPIPE` is ignored (so write errors on pipes and sockets can be reported as ordinary Python exceptions) and `SIGINT` is translated into a `KeyboardInterrupt` exception if the parent process has not changed it.

A handler for a particular signal, once set, remains installed until it is explicitly reset (Python emulates the BSD style interface regardless of the underlying implementation), with the exception of the handler for `SIGCHLD`, which follows the underlying implementation.

On WebAssembly platforms, signals are emulated and therefore behave differently. Several functions and signals are not available on these platforms.

Execution of Python signal handlers

A Python signal handler does not get executed inside the low-level (C) signal handler. Instead, the low-level signal handler sets a flag which tells the virtual machine to execute the corresponding Python signal handler at a later point (for example, at the next bytecode instruction). This has consequences:

- It makes little sense to catch synchronous errors like `SIGFPE` or `SIGSEGV` that are caused by an invalid operation in C code. Python will return from the signal handler to the C code, which is likely to raise the same signal again, causing Python to apparently hang. From Python 3.3 onwards, you can use the `faulthandler` module to report on synchronous errors.
- A long-running calculation implemented purely in C (such as regular expression matching on a large body of text) may run uninterrupted for an arbitrary amount of time, regardless of any signals received. The Python signal handlers will be called when the calculation finishes.
- If the handler raises an exception, it will be raised “out of thin air” in the main thread. See the note below for a discussion.

Signals and threads

Python signal handlers are always executed in the main Python thread of the main interpreter, even if the signal was received in another thread. This means that signals can't be used as a means of inter-thread communication. You can use the synchronization primitives from the `threading` module instead.

Besides, only the main thread of the main interpreter is allowed to set a new signal handler.

■■ Advertencia: Warning Synchronization primitives such as `threading.Lock` should not be used within signal handlers. Doing so can lead to unexpected deadlocks.

Module contents

Changed in version 3.5: `signal` (`SIG*`), `handler` (`SIG_DFL`, `SIG_IGN`) and `sigmask` (`SIG_BLOCK`, `SIG_UNBLOCK`, `SIG_SETMASK`) related constants listed below were turned into enums (`Signals`, `Handlers` and `Sigmasks` respectively). `getsignal()`, `pthread_sigmask()`, `sigpending()` and `sigwait()` functions return human-readable enums as `Signal` objects.

The signal module defines three enums:

class signal.Signals

enum.IntEnum collection of SIG* constants and the CTRL_* constants. Added in version 3.5.

class signal.Handlers

enum.IntEnum collection of the constants SIG_DFL and SIG_IGN. Added in version 3.5.

class signal.Sigmask

enum.IntEnum collection of the constants SIG_BLOCK, SIG_UNBLOCK and SIG_SETMASK. Availability: Unix. See the man pages `sigprocmask(2)` and `pthread_sigmask(3)` for further information. Added in version 3.5.

The variables defined in the `signal` module are:

signal.SIG_DFL

This is one of two standard signal handling options; it will simply perform the default function for the signal. For example, on most systems the default action for `SIGQUIT` is to dump core and exit, while the default action for `SIGCHLD` is to simply ignore it.

signal.SIG_IGN

This is another standard signal handler, which will simply ignore the given signal.

signal.SIGABRT

Abort signal from `abort(3)`.

signal.SIGALRM

Timer signal from `alarm(2)`. Availability: Unix.

signal.SIGBREAK

Interrupt from keyboard (CTRL + BREAK). Availability: Windows.

signal.SIGBUS

Bus error (bad memory access). Availability: Unix.

signal.SIGCHLD

Child process stopped or terminated. Availability: Unix.

signal.SIGCLD

Alias to `SIGCHLD`. Availability: not macOS.

signal.SIGCONT

Continue the process if it is currently stopped. Availability: Unix.

signal.SIGFPE

Floating-point exception. For example, division by zero. See also `ZeroDivisionError` is raised when the second argument of a division or modulo operation is zero.

signal.SIGHUP

Hangup detected on controlling terminal or death of controlling process. Availability: Unix.

signal.SIGILL

Illegal instruction.

signal.SIGINT

Interrupt from keyboard (CTRL + C).Default action is to raiseKeyboardInterrupt.

signal.SIGKILL

Kill signal.It cannot be caught, blocked, or ignored.Availability: Unix.

signal.SIGPIPE

Broken pipe: write to pipe with no readers.Default action is to ignore the signal.Availability: Unix.

signal.SIGPROF

Profiling timer expired.Availability: Unix.

signal.SIGQUIT

Terminal quit signal.Availability: Unix.

signal.SIGSEGV

Segmentation fault: invalid memory reference.

signal.SIGSTOP

Stop executing (cannot be caught or ignored).Availability: Unix.

signal.SIGSTKFLT

Stack fault on coprocessor. The Linux kernel does not raise this signal: it can only be raised in user space.Availability: Linux.On architectures where the signal is available. See the man pagesignal(7)for further information.Added in version 3.11.

signal.SIGTERM

Termination signal.

signal.SIGUSR1

User-defined signal 1.Availability: Unix.

signal.SIGUSR2

User-defined signal 2.Availability: Unix.

signal.SIGVTALRM

Virtual timer expired.Availability: Unix.

signal.SIGWINCH

Window resize signal.Availability: Unix.

signal.SIGXCPU

CPU time limit exceeded.Availability: Unix.

SIG*

All the signal numbers are defined symbolically. For example, the hangup signal is defined assignal.SIGHUP; the variable names are identical to the names used in C programs, as found in<signal.h>. The Unix man page for 'signal' lists the existing signals (on some systems this issignal(2), on others the list is insignal(7)). Note that not all systems define the same set of signal names; only those names defined by the system are defined by this module.

signal.CTRL_C_EVENT

The signal corresponding to theCtrl+Ckeystroke event. This signal can only be used withos.kill().Availability: Windows.Added in version 3.2.

signal.CTRL_BREAK_EVENT

The signal corresponding to the Ctrl+Break keystroke event. This signal can only be used with `os.kill()`. Availability: Windows. Added in version 3.2.

signal.NSIG

One more than the number of the highest signal number. Use `valid_signals()` to get valid signal numbers.

signal.ITIMER_REAL

Decrements interval timer in real time, and delivers `SIGALRM` upon expiration.

signal.ITIMER_VIRTUAL

Decrements interval timer only when the process is executing, and delivers `SIGVTALRM` upon expiration.

signal.ITIMER_PROF

Decrements interval timer both when the process executes and when the system is executing on behalf of the process. Coupled with `ITIMER_VIRTUAL`, this timer is usually used to profile the time spent by the application in user and kernel space. `SIGPROF` is delivered upon expiration.

signal.SIG_BLOCK

A possible value for the `how` parameter to `pthread_sigmask()` indicating that signals are to be blocked. Added in version 3.3.

signal.SIG_UNBLOCK

A possible value for the `how` parameter to `pthread_sigmask()` indicating that signals are to be unblocked. Added in version 3.3.

signal.SIG_SETMASK

A possible value for the `how` parameter to `pthread_sigmask()` indicating that the signal mask is to be replaced. Added in version 3.3.

The `signal` module defines one exception:

exceptions.signal.ITimerError

Raised to signal an error from the underlying `setitimer()` or `getitimer()` implementation. Expect this error if an invalid interval timer or a negative time is passed to `setitimer()`. This error is a subtype of `OSError`. Added in version 3.3: This error used to be a subtype of `IOError`, which is now an alias of `OSError`.

The `signal` module defines the following functions:

signal.alarm(time)

If `time` is non-zero, this function requests that a `SIGALRM` signal be sent to the process in `time` seconds. Any previously scheduled alarm is canceled (only one alarm can be scheduled at any time). The returned value is then the number of seconds before any previously set alarm was to have been delivered. If `time` is zero, no alarm is scheduled, and any scheduled alarm is canceled. If the return value is zero, no alarm is currently scheduled. Availability: Unix. See the man page `alarm(2)` for further information.

signal.getsignal(signalnum)

Return the current signal handler for the signal `signalnum`. The returned value may be a callable Python object, or one of the special values `signal.SIG_IGN`, `signal.SIG_DFL` or `None`. Here, `signal.SIG_IGN` means that the signal was previously ignored, `signal.SIG_DFL` means that the default way of handling the signal was previously in use, and `None` means that the previous signal handler was not installed from Python.

signal.strsignal(signalnum)

Returns the description of signal `signalnum`, such as "Interrupt" for `SIGINT`. Returns `None` if `signalnum` has no description. Raises `ValueError` if `signalnum` is invalid. Added in version 3.8.

signal.valid_signals()

Return the set of valid signal numbers on this platform. This can be less than `range(1, NSIG)` if some signals are reserved by the system for internal use. Added in version 3.8.

signal.pause()

Cause the process to sleep until a signal is received; the appropriate handler will then be called. Returns nothing. Availability: Unix. See the man page `signal(2)` for further information. See also `sigwait()`, `sigwaitinfo()`, `sigtimedwait()` and `sigpending()`.

signal.raise_signal(signalnum)

Sends a signal to the calling process. Returns nothing. Added in version 3.8.

signal.pidfd_send_signal(pidfd, sig, siginfo=None, flags=0)

Send signal `sig` to the process referred to by file descriptor `pidfd`. Python does not currently support the `siginfo` parameter; it must be `None`. The `flags` argument is provided for future extensions; no flag values are currently defined. See the `pidfd_send_signal(2)` man page for more information. Availability: Linux >= 5.1, Android >= build-time API level 31. Added in version 3.9.

signal.pthread_kill(thread_id, signalnum)

Send the signal `signalnum` to the thread `thread_id`, another thread in the same process as the caller. The target thread can be executing any code (Python or not). However, if the target thread is executing the Python interpreter, the Python signal handlers will be executed by the main thread of the main interpreter. Therefore, the only point of sending a signal to a particular Python thread would be to force a running system call to fail with `InterruptedError`. Use `threading.get_ident()` or the `ident` attr

signal.pthread_sigmask(how, mask)

Fetch and/or change the signal mask of the calling thread. The signal mask is the set of signals whose delivery is currently blocked for the caller. Return the old signal mask as a set of signals. The behavior of the call is dependent on the value of `how`, as follows. `SIG_BLOCK`: The set of blocked signals is the union of the current set and the `mask` argument. `SIG_UNBLOCK`: The signals in `mask` are removed from the current set of blocked signals. It is permissible to attempt to unblock a signal which is n

signal.setitimer(which, seconds, interval=0.0)

Sets given interval timer (one of `signal.ITIMER_REAL`, `signal.ITIMER_VIRTUAL` or `signal.ITIMER_PROF`) specified by `which` to fire after `seconds` (float is accepted, different from `alarm()`) and after that every `interval` seconds (if `interval` is non-zero). The interval timer specified by `which` can be cleared by setting `seconds` to zero. When an interval timer fires, a signal is sent to the process. The signal sent is dependent on the timer being used; `signal.ITIMER_REAL` will deliver `SIGALRM`, `signal.ITIMER_VIRTUAL` sends `SIGVTALR`

signal.getitimer(which)

Returns current value of a given interval timer specified by `which`. Availability: Unix.

signal.set_wakeup_fd(fd, *, warn_on_full_buffer=True)

Set the wakeup file descriptor to `fd`. When a signal your program has registered a signal handler for is received, the signal number is written as a single byte into the `fd`. If you haven't registered a signal handler for the signals you care about, then nothing will be written to the wakeup `fd`. This can be used by a library to wakeup a poll or select call, allowing the signal to be fully processed. The old wakeup `fd` is returned (or `-1` if file descriptor wakeup was not enabled). If `fd` is `-1`, file d

signal.siginterrupt(signalnum, flag)

Change system call restart behaviour: if `flag` is `False`, system calls will be restarted when interrupted by `signal.signalnum`, otherwise system calls will be interrupted. Returns nothing. Availability: Unix. See the man page `siginterrupt(3)` for further information. Note that installing a signal handler with `signal()` will reset the restart behaviour to interruptible by implicitly calling `siginterrupt()` with a `True` flag value for the given signal.

signal.signal(signalnum, handler)

Set the handler for `signalnum` to the function `handler`. `handler` can be a callable Python object taking two arguments (see below), or one of the special values `signal.SIG_IGN` or `signal.SIG_DFL`. The previous signal handler will be returned (see the description of `getsignal()` above). (See the Unix man page `signal(2)` for further information.) When threads are enabled, this function can only be called from the main thread of the main interpreter; attempting to call it from other threads will cause a `ValueError`.

signal.sigpending()

Examine the set of signals that are pending for delivery to the calling thread (i.e., the signals which have been raised while blocked). Return the set of the pending signals. Availability: Unix. See the man page `sigpending(2)` for further information. See also `pause()`, `pthread_sigmask()` and `sigwait()`. Added in version 3.3.

signal.sigwait(sigset)

Suspend execution of the calling thread until the delivery of one of the signals specified in the signal set `sigset`. The function accepts the signal (removes it from the pending list of signals), and returns the signal number. Availability: Unix. See the man page `sigwait(3)` for further information. See also `pause()`, `pthread_sigmask()`, `sigpending()`, `sigwaitinfo()` and `sigtimedwait()`. Added in version 3.3.

signal.sigwaitinfo(sigset)

Suspend execution of the calling thread until the delivery of one of the signals specified in the signal set `sigset`. The function accepts the signal and removes it from the pending list of signals. If one of the signals in `sigset` is already pending for the calling thread, the function will return immediately with information about that signal. The signal handler is not called for the delivered signal. The function raises an `InterruptedError` if it is interrupted by a signal that is not in `sigset`. The `r`

signal.sigtimedwait(sigset, timeout)

Like `sigwaitinfo()`, but takes an additional `timeout` argument specifying a timeout. If `timeout` is specified as `0`, a poll is performed. Returns `None` if a timeout occurs. Availability: Unix. See the man page `sigtimedwait(2)` for further information. See also `pause()`, `sigwait()` and `sigwaitinfo()`. Added in version 3.3. Changed in version 3.5: The function is now retried with the recomputed timeout if interrupted by a signal not in `sigset` and the signal handler does not raise an exception (see PEP 475 for the rationale).

Examples

Here is a minimal example program. It uses the `alarm()` function to limit the time spent waiting to open a file; this is useful if the file is for a serial device that may not be turned on, which would normally cause the `os.open()` to hang indefinitely. The solution is to set a 5-second alarm before opening the file; if the operation takes too long, the alarm signal will be sent, and the handler raises an exception.

```
import signal, os

def handler(signum, frame):
    signame = signal.Signals(signum).name
    print(f'Signal handler called with signal {signame} ({signum})')
    raise OSError("Couldn't open device!")

# Set the signal handler and a 5-second alarm
signal.signal(signal.SIGALRM, handler)
signal.alarm(5)
```

```
# This open() may hang indefinitely
fd = os.open('/dev/ttyS0', os.O_RDWR)

signal.alarm(0)          # Disable the alarm
```

Note on SIGPIPE

Piping output of your program to tools like `head(1)` will cause a `SIGPIPE` signal to be sent to your process when the receiver of its standard output closes early. This results in an exception like `BrokenPipeError: [Errno32] Broken pipe`. To handle this case, wrap your entry point to catch this exception as follows:

```
import os
import sys

def main():
    try:
        # simulate large output (your code replaces this loop)
        for x in range(10000):
            print("y")
        # flush output here to force SIGPIPE to be triggered
        # while inside this try block.
        sys.stdout.flush()
    except BrokenPipeError:
        # Python flushes standard streams on exit; redirect remaining output
        # to devnull to avoid another BrokenPipeError at shutdown
        devnull = os.open(os.devnull, os.O_WRONLY)
        os.dup2(devnull, sys.stdout.fileno())
        sys.exit(1) # Python exits with error code 1 on EPIPE

if __name__ == '__main__':
    main()
```

Do not set `SIGPIPE`'s disposition to `SIG_DFL` in order to avoid `BrokenPipeError`. Doing that would cause your program to exit unexpectedly whenever any socket connection is interrupted while your program is still writing to it.

Note on Signal Handlers and Exceptions

If a signal handler raises an exception, the exception will be propagated to the main thread and may be raised after any bytecode instruction. Most notably, a `KeyboardInterrupt` may appear at any point during execution. Most Python code, including the standard library, cannot be made robust against this, and so a `KeyboardInterrupt` (or any other exception resulting from a signal handler) may on rare occasions put the program in an unexpected state.

To illustrate this issue, consider the following code:

```
class SpamContext:
    def __init__(self):
        self.lock = threading.Lock()

    def __enter__(self):
        # If KeyboardInterrupt occurs here, everything is fine
        self.lock.acquire()
        # If KeyboardInterrupt occurs here, __exit__ will not be called
        ...
        # KeyboardInterrupt could occur just before the function returns

    def __exit__(self, exc_type, exc_val, exc_tb):
```

```
...
self.lock.release()
```

For many programs, especially those that merely want to exit on `KeyboardInterrupt`, this is not a problem, but applications that are complex or require high reliability should avoid raising exceptions from signal handlers. They should also avoid catching `KeyboardInterrupt` as a means of gracefully shutting down. Instead, they should install their own `SIGINT` handler. Below is an example of an HTTP server that avoids `KeyboardInterrupt`:

```
import signal
import socket
from selectors import DefaultSelector, EVENT_READ
from http.server import HTTPServer, SimpleHTTPRequestHandler

interrupt_read, interrupt_write = socket.socketpair()

def handler(signum, frame):
    print('Signal handler called with signal', signum)
    interrupt_write.send(b'\0')
signal.signal(signal.SIGINT, handler)

def serve_forever(httpd):
    sel = DefaultSelector()
    sel.register(interrupt_read, EVENT_READ)
    sel.register(httpd, EVENT_READ)

    while True:
        for key, _ in sel.select():
            if key.fileobj == interrupt_read:
                interrupt_read.recv(1)
                return
            if key.fileobj == httpd:
                httpd.handle_request()

print("Serving on port 8000")
httpd = HTTPServer(('', 8000), SimpleHTTPRequestHandler)
serve_forever(httpd)
print("Shutdown...")
```

mmap— Memory-mapped file support

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Memory-mapped file objects behave like both `bytearray` and `file` objects. You can use `mmap` objects in most places where `bytearray` are expected; for example, you can use `themodule` to search through a memory-mapped file. You can also change a single byte by doing `obj[index]=97`, or change a subsequence by assigning to a slice: `obj[1:12]=b'...'`. You can also read and write data starting at the current file position, and `seek()` through the file to different positions.

A memory-mapped file is created by the `mmap` constructor, which is different on Unix and on Windows. In either case you must provide a file descriptor for a file opened for update. If you wish to map an existing Python file object, use its `fileno()` method to obtain the correct value for the `fileno` parameter. Otherwise, you can open the file using the `os.open()` function, which returns a file descriptor directly (the file still needs to be closed when done).

■ **Nota:** Not if you want to create a memory-mapping for a writable, buffered file, you should `flush()` the file first. This is necessary to ensure that local modifications to the buffers are actually available to the mapping.

For both the Unix and Windows versions of the constructor, `access` may be specified as an optional keyword parameter. `access` accepts one of four values: `ACCESS_READ`, `ACCESS_WRITE`, or `ACCESS_COPY` to specify read-only, write-through or copy-on-write memory respectively, or `ACCESS_DEFAULT` to defer to `prot`. `access` can be used on both Unix and Windows. If `access` is not specified, Windows `mmap` returns a write-through mapping. The initial memory values for all three access types are taken from the specified file. Assignment to an `ACCESS_READ` memory map raises a `TypeError` exception. Assignment to an `ACCESS_WRITE` memory map affects both memory and the underlying file. Assignment to an `ACCESS_COPY` memory map affects memory but does not update the underlying file.

Changed in version 3.7: Added `ACCESS_DEFAULT` constant.

To map anonymous memory, `-1` should be passed as the `fileno` along with the length.

`class mmap.mmap(fileno, length, tagname=None, access=ACCESS_DEFAULT, offset=0)`

(Windows version) Maps `length` bytes from the file specified by the file handle `fileno`, and creates a `mmap` object. If `length` is larger than the current size of the file, the file is extended to contain `length` bytes. If `length` is 0, the maximum length of the map is the current size of the file, except that if the file is empty Windows raises an exception (you cannot create an empty mapping on Windows). `tagname`, if specified and not `None`, is a string giving a tag name for the mapping. Windows allows you to h

`class mmap.mmap(fileno, length, flags=MAP_SHARED, prot=PROT_WRITE|PROT_READ, access=ACCESS_DEFAULT, offset=0, *, trackfd=True)`

`close()`

`closed`

`find(sub[, start[, end]])`

`flush()`

`flush(offset, size, /)`

`madvice(option[, start[, length]])`

`move(dest, src, count)`

`read([n])`

read_byte()

readline()

resize(newsize)

rfind(sub[,start[,end]])

seek(pos[,whence])

seekable()

size()

tell()

write(bytes)

write_byte(byte)

(Unix version) Maps `length` bytes from the file specified by the file descriptor `fileno`, and returns a `mmap` object. If `length` is 0, the maximum length of the map will be the current size of the file when `mmap` is called. `flags` specifies the nature of the mapping. `MAP_PRIVATE` creates a private copy-on-write mapping, so changes to the contents of the `mmap` object will be private to this process, and `MAP_SHARED` creates a mapping that's shared with all other processes mapping the same areas of the file. The default

Closes the `mmap`. Subsequent calls to other methods of the object will result in a `ValueError` exception being raised. This will not close the open file.

True if the file is closed. Added in version 3.2.

Returns the lowest index in the object where the subsequence `sub` is found, such that `sub` is contained in the range `[start,end]`. Optional arguments `start` and `end` are interpreted as in slice notation. Returns -1 on failure. Changed in version 3.5: Writable bytes-like object is now accepted.

Flushes changes made to the in-memory copy of a file back to disk. Without use of this call there is no guarantee that changes are written back before the object is destroyed. If `offset` and `size` are specified, only changes to the given range of bytes will be flushed to disk; otherwise, the whole extent of the mapping is flushed. `offset` must be a multiple of the `PAGESIZE` or `ALLOCATIONGRANULARITY`. `None` is returned to indicate success. An exception is raised when the call failed. Changed in version 3.8: Previ

Send advice `option` to the kernel about the memory region beginning at `start` and extending `length` bytes. `option` must be one of the `MADV_*` constants available on the system. If `start` and `length` are omitted, the entire mapping is spanned. On some systems (including Linux), `start` must be a multiple of the `PAGESIZE`. Availability: Systems with the `madvise()` system call. Added in version 3.8.

Copy the `count` bytes starting at `offset` to the destination index `dest`. If the `mmap` was created with `ACCESS_READ`, then calls to move will raise a `TypeError` exception.

Return a bytes containing up to `n` bytes starting from the current file position. If the argument is omitted, `None` or negative, return all bytes from the current file position to the end of the mapping. The file position is updated to point after the bytes that were returned. Changed in version 3.3: Argument can be omitted or `None`.

Returns a byte at the current file position as an integer, and advances the file position by 1.

Returns a single line, starting at the current file position and up to the next newline. The file position is updated to point after the bytes that were returned.

Resizes the map and the underlying file, if any. Resizing a map created with `access` of `ACCESS_READ` or `ACCESS_COPY`, will raise a `TypeError` exception. Resizing a map created with `trackfd` set to `False`, will raise a `ValueError` exception. On Windows: Resizing the map will raise an `OSError` if there are other maps against the same named file. Resizing an anonymous map (ie against the pagefile) will

silently create a new map with the original data copied over up to the length of the new size.Changed in version 3.11:Cor

Returns the highest index in the object where the subsequencesubis found, such thatsubis contained in the range [start,end]. Optional argumentsstartandendare interpreted as in slice notation. Returns-1on failure.Changed in version 3.5:Writablebytes-like objectis now accepted.

Set the file's current position.whenceargument is optional and defaults toos.SEEK_SETor0(absolute file positioning); other values areos.SEEK_CURor1(seek relative to the current position) andos.SEEK_ENDor2(seek relative to the file's end).Changed in version 3.13:Return the new absolute position instead ofNone.

Return whether the file supports seeking, and the return value is alwaysTrue.Added in version 3.13.

Return the length of the file, which can be larger than the size of the memory-mapped area.

Returns the current position of the file pointer.

Write the bytes inbytesinto memory at the current position of the file pointer and return the number of bytes written (never less thanlen(bytes), since if the write fails, aValueErrorwill be raised). The file position is updated to point after the bytes that were written. If the mmap was created withACCESS_READ, then writing to it will raise aTypeErrorexception.Changed in version 3.5:Writablebytes-like objectis now accepted.Changed in version 3.6:The number of bytes written is now returned.

Write the integerbyteinto memory at the current position of the file pointer; the file position is advanced by1. If the mmap was created withACCESS_READ, then writing to it will raise aTypeErrorexception.

MADV_* Constants

mmap.MADV_NORMAL

mmap.MADV_RANDOM

mmap.MADV_SEQUENTIAL

mmap.MADV_WILLNEED

mmap.MADV_DONTNEED

mmap.MADV_REMOVE

mmap.MADV_DONTFORK

mmap.MADV_DOFORK

mmap.MADV_HWPOISON

mmap.MADV_MERGEABLE

mmap.MADV_UNMERGEABLE

mmap.MADV_SOFT_OFFLINE

mmap.MADV_HUGEPAGE

mmap.MADV_NOHUGEPAGE

mmap.MADV_DONTDUMP

mmap.MADV_DODUMP

mmap.MADV_FREE

mmap.MADV_NOSYNC

mmap.MADV_AUTOSYNC
mmap.MADV_NOCORE
mmap.MADV_CORE
mmap.MADV_PROTECT
mmap.MADV_FREE_REUSABLE
mmap.MADV_FREE_REUSE

These options can be passed to `mmap.madvise()`. Not every option will be present on every system. Availability: Systems with the `madvise()` system call. Added in version 3.8.

MAP_* Constants

mmap.MAP_SHARED
mmap.MAP_PRIVATE
mmap.MAP_32BIT
mmap.MAP_ALIGNED_SUPER
mmap.MAP_ANON
mmap.MAP_ANONYMOUS
mmap.MAP_CONCEAL
mmap.MAP_DENYWRITE
mmap.MAP_EXECUTABLE
mmap.MAP_HASSEMAPHORE
mmap.MAP_JIT
mmap.MAP_NOCACHE
mmap.MAP_NOEXTEND
mmap.MAP_NORESERVE
mmap.MAP_POPULATE
mmap.MAP_RESILIENT_CODESIGN
mmap.MAP_RESILIENT_MEDIA
mmap.MAP_STACK
mmap.MAP_TPRO
mmap.MAP_TRANSLATED_ALLOW_EXECUTE
mmap.MAP_UNIX03

These are the various flags that can be passed to `mmap.mmap()`. `MAP_ALIGNED_SUPER` is only available at FreeBSD and `MAP_CONCEAL` is only available at OpenBSD. Note that some options might not be present on some systems. Changed in version 3.10: Added `MAP_POPULATE` constant. Added in version 3.11: Added `MAP_STACK` constant. Added in version 3.12: Added `MAP_ALIGNED_SUPER` and `MAP_CONCEAL` constants. Added in version 3.13: Added `MAP_32BIT`, `MAP_HASSEMAPHORE`, `MAP_JIT`, `MAP_NOCACHE`, `MAP_NOEXTEND`, `MAP_NORESERVE`, `MAP_RESILIENT_CODESIGN`, `M`

Internet Data Handling

This chapter describes modules which support handling data formats commonly used on the internet.

- email— An email and MIME handling packageemail.message: Representing an email messageemail.parser: Parsing email messagesFeedParser APIParser APIAdditional notesemail.generator: Generating MIME documentsemail.policy: Policy Objectsemail.errors: Exception and Defect classesemail.headerregistry: Custom Header Objectsemail.contentmanager: Managing MIME ContentContent Manager Instancesemail: Examplese
- json— JSON encoder and decoderBasic UsageEncoders and DecodersExceptionsStandard Compliance and InteroperabilityCharacter EncodingsInfinite and NaN Number ValuesRepeated Names Within an ObjectTop-level Non-Object, Non-Array ValuesImplementation LimitationsCommand-line interfaceCommand-line options
- mailbox— Manipulate mailboxes in various formatsMailboxobjectsMaildirobjectsmbboxobjectsMHobjectsBabylobjectsMMDFObjetsMessageobjectsMaildirMessageobjectsmbboxMessageobjectsMHMessageobjectsBabyIMessageobjectsMMDFMessageobjectsExceptionsExamples
- mimetypes— Map filenames to MIME typesMimeTypes objectsCommand-line usageCommand-line example
- base64— Base16, Base32, Base64, Base85 Data EncodingsRFC 4648 EncodingsBase85 EncodingsLegacy InterfaceSecurity Considerations
- binascii— Convert between binary and ASCII
- quopri— Encode and decode MIME quoted-printable data

email— An email and MIME handling package

Source code: `Lib/email/__init__.py`

The `email` package is a library for managing email messages. It is specifically not designed to do any sending of email messages to SMTP (RFC 2821), NNTP, or other servers; those are functions of modules such as `smtpplib`. The `email` package attempts to be as RFC-compliant as possible, supporting RFC 5322 and RFC 6532, as well as such MIME-related RFCs as RFC 2045, RFC 2046, RFC 2047, RFC 2183, and RFC 2231.

The overall structure of the email package can be divided into three major components, plus a fourth component that controls the behavior of the other components.

The central component of the package is an “object model” that represents email messages. An application interacts with the package primarily through the object model interface defined in the `message` sub-module. The application can use this API to ask questions about an existing email, to construct a new email, or to add or remove email subcomponents that themselves use the same object model interface. That is, following the nature of email messages and their MIME subcomponents, the email object model is a tree structure of objects that all provide the `EmailMessage` API.

The other two major components of the package are the parser and the generator. The parser takes the serialized version of an email message (a stream of bytes) and converts it into a tree of `EmailMessage` objects. The generator takes an `EmailMessage` and turns it back into a serialized byte stream. (The parser and generator also handle streams of text characters, but this usage is discouraged as it is too easy to end up with messages that are not valid in one way or another.)

The control component is the `policy` module. Every `EmailMessage`, every generator, and every parser has an associated `policy` object that controls its behavior. Usually an application only needs to specify the policy when an `EmailMessage` is created, either by directly instantiating an `EmailMessage` to create a new email, or by parsing an input stream using a parser. But the policy can be changed when the message is serialized using a generator. This allows, for example, a generic email message to be parsed from disk, but to serialize it using standard SMTP settings when sending it to an email server.

The email package does its best to hide the details of the various governing RFCs from the application. Conceptually the application should be able to treat the email message as a structured tree of unicode text and binary attachments, without having to worry about how these are represented when serialized. In practice, however, it is often necessary to be aware of at least some of the rules governing MIME messages and their structure, specifically the names and nature of the MIME “content types” and how they identify multipart documents. For the most part this knowledge should only be required for more complex applications, and even then it should only be the high level structure in question, and not the details of how those structures are represented. Since MIME content types are used widely in modern internet software (not just email), this will be a familiar concept to many programmers.

The following sections describe the functionality of the `email` package. We start with the `message` object model, which is the primary interface an application will use, and follow that with the parser and generator components. Then we cover the `policy` controls, which completes the treatment of the main components of the library.

The next three sections cover the exceptions the package may raise and the defects (non-compliance with the RFCs) that the parser may detect. Then we cover the header registry and the content managers sub-components, which provide tools for doing more detailed manipulation of headers and payloads, respectively. Both of these components contain features relevant to consuming and producing non-trivial messages, but also document their extensibility APIs, which will be of interest to advanced applications.

Following those is a set of examples of using the fundamental parts of the APIs covered in the preceding sections.

The foregoing represent the modern (unicode friendly) API of the email package. The remaining sections, starting with the `Message` class, cover the legacy `compat32` API that deals much more directly with the details of how email messages are represented. The `compat32` API does not hide the details of the RFCs from the application, but for applications that need to operate at that level, they can be useful tools. This documentation is also relevant for applications that are still using the `compat32` API for backward compatibility reasons.

Changed in version 3.6: Docs reorganized and rewritten to promote the new `EmailMessage/EmailPolicy` API.

Contents of the email package documentation:

- `email.message`: Representing an email message
- `email.parser`: Parsing email messages `FeedParser` API `Parser` API Additional notes
- `email.generator`: Generating MIME documents
- `email.policy`: Policy Objects
- `email.errors`: Exception and Defect classes
- `email.headerregistry`: Custom Header Objects
- `email.contentmanager`: Managing MIME Content `ContentManager` Instances
- `email`: Examples

Legacy API:

- `email.message.Message`: Representing an email message using the `compat32` API
- `email.mime`: Creating email and MIME objects from scratch
- `email.header`: Internationalized headers
- `email.charset`: Representing character sets
- `email.encoders`: Encoders
- `email.utils`: Miscellaneous utilities
- `email.iterators`: Iterators

See also

Module `smtplib`

Module `poplib`

Module `imaplib`

Module `mailbox`

SMTP (Simple Mail Transport Protocol) client

POP (Post Office Protocol) client

IMAP (Internet Message Access Protocol) client

Tools for creating, reading, and managing collections of messages on disk using a variety standard formats.

json— JSON encoder and decoder

Source code: `Lib/json/__init__.py`

JSON (JavaScript Object Notation), specified by RFC 7159 (which obsoletes RFC 4627) and by ECMA-404, is a lightweight data interchange format inspired by JavaScript object literal syntax (although it is not a strict subset of JavaScript[1]).

■ **Nota:** Note The term “object” in the context of JSON processing in Python can be ambiguous. All values in Python are objects. In JSON, an object refers to any data wrapped in curly braces, similar to a Python dictionary.

■■ **Advertencia:** Warning Be cautious when parsing JSON data from untrusted sources. A malicious JSON string may cause the decoder to consume considerable CPU and memory resources. Limiting the size of data to be parsed is recommended.

This module exposes an API familiar to users of the standard library `marshal` and `pickle` modules.

Encoding basic Python object hierarchies:

```
>>> import json
>>> json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
'["foo", {"bar": ["baz", null, 1.0, 2]}]'
>>> print(json.dumps("\"foo\\bar\""))
"\"foo\\bar\""
>>> print(json.dumps('\u1234'))
"\u1234"
>>> print(json.dumps('\''))
"\'"
>>> print(json.dumps({'c': 0, "b": 0, "a": 0}, sort_keys=True))
{"a": 0, "b": 0, "c": 0}
>>> from io import StringIO
>>> io = StringIO()
>>> json.dump(['streaming API'], io)
>>> io.getvalue()
'["streaming API"]'
```

Compact encoding:

```
>>> import json
>>> json.dumps([1, 2, 3, {'4': 5, '6': 7}], separators=(',', ':'))
'[1,2,3,{"4":5,"6":7}]'
```

Pretty printing:

```
>>> import json
>>> print(json.dumps({'6': 7, '4': 5}, sort_keys=True, indent=4))
{
    "4": 5,
    "6": 7
}
```

Customizing JSON object encoding:

```
>>> import json
>>> def custom_json(obj):
...     if isinstance(obj, complex):
...         return {'__complex__': True, 'real': obj.real, 'imag': obj.imag}
...     raise TypeError(f'Cannot serialize object of {type(obj)}')
...
>>> json.dumps(1 + 2j, default=custom_json)
'{"__complex__": true, "real": 1.0, "imag": 2.0}'
```

Decoding JSON:

```
>>> import json
```

```
>>> json.loads('{"foo", {"bar":["baz", null, 1.0, 2]}}')
['foo', {'bar': ['baz', None, 1.0, 2]}]
>>> json.loads('"\\\\"foo\\bar"')
'"foo\x08ar'
>>> from io import StringIO
>>> io = StringIO(['streaming API'])
>>> json.load(io)
['streaming API']
```

Customizing JSON object decoding:

```
>>> import json
>>> def as_complex(dct):
...     if '__complex__' in dct:
...         return complex(dct['real'], dct['imag'])
...     return dct
...
>>> json.loads('{"__complex__": true, "real": 1, "imag": 2}',
...     object_hook=as_complex)
(1+2j)
>>> import decimal
>>> json.loads('1.1', parse_float=decimal.Decimal)
Decimal('1.1')
```

ExtendingJSONEncoder:

```
>>> import json
>>> class ComplexEncoder(json.JSONEncoder):
...     def default(self, obj):
...         if isinstance(obj, complex):
...             return [obj.real, obj.imag]
...         # Let the base class default method raise the TypeError
...         return super().default(obj)
...
>>> json.dumps(2 + 1j, cls=ComplexEncoder)
'[2.0, 1.0]'
>>> ComplexEncoder().encode(2 + 1j)
'[2.0, 1.0]'
>>> list(ComplexEncoder().iterencode(2 + 1j))
['[2.0', ', ', '1.0', ', ', ']']
```

Usingjsonfrom the shell to validate and pretty-print:

```
$ echo '{"json":"obj"}' | python -m json
{
  "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

SeeCommand-line interfacefor detailed documentation.

■ Nota: NoteJSON is a subset ofYAML1.2. The JSON produced by this module's default settings (in particular, the defaultseparatorsvalue) is also a subset of YAML 1.0 and 1.1. This module can thus also be used as a YAML serializer.

■ Nota: NoteThis module's encoders and decoders preserve input and output order by default. Order is only lost if the underlying containers are unordered.

Basic Usage

json.dump(obj,fp,*,skipkeys=False,ensure_ascii=True,check_circular=True,allow_nan=True,cls=None,indent=None,separators=None,default=None,sort_keys=False,kw)**

Parameters:

Serializeobjas a JSON formatted stream tofp(a.write()-supportingfile-like object) using thisPython-to-JSON conversion table.NoteUnlikepickleandmarshal, JSON is not a framed protocol, so trying to serialize multiple objects with repeated calls todump()using the samefpwill result in an invalid JSON file.Parameters:obj(object) – The Python object to be serialized.fp(file-like object) – The file-like objectobjwill be serialized to. Thejsonmodule always producesstrobjects, notbytesobjects, thereforefp.write()must

obj(object) – The Python object to be serialized.fp(file-like object) – The file-like objectobjwill be serialized to. Thejsonmodule always producesstrobjects, notbytesobjects, thereforefp.write()must supportstrinput.skipkeys(bool) – IfTrue, keys that are not of a basic type (str,int,float,bool,None) will be skipped instead of raising aTypeError. DefaultFalse.ensure_ascii(bool) – IfTrue(the default), the output is guaranteed to have all incoming non-ASCII and non-printable characters escaped. IfFalse

json.dumps(obj,*,skipkeys=False,ensure_ascii=True,check_circular=True,allow_nan=True,cls=None,indent=None,separators=None,default=None,sort_keys=False,kw)**

Serializeobjto a JSON formattedstrusing thisconversion table. The arguments have the same meaning as indump().NoteKeys in key/value pairs of JSON are always of the typestr. When a dictionary is converted into JSON, all the keys of the dictionary are coerced to strings. As a result of this, if a dictionary is converted into JSON and then back into a dictionary, the dictionary may not equal the original one. That is,loads(dumps(x))!=xif x has non-string keys.

json.load(fp,*,cls=None,object_hook=None,parse_float=None,parse_int=None,parse_constant=None,object_pairs_hook=None,kw)**

Parameters:

Raises:

Deserializefp to a Python object using theJSON-to-Python conversion table.Parameters:fp(file-like object) – A.read()-supportingtext fileorbinary filecontaining the JSON document to be deserialized.cls(aJSONDecodersubclass) – If set, a custom JSON decoder. Additional keyword arguments toload()will be passed to the constructor ofcls. IfNone(the default),JSONDecoderis used.object_hook(callable|None) – If set, a function that is called with the result of any JSON object literal decoded (adict). The

fp(file-like object) – A.read()-supportingtext fileorbinary filecontaining the JSON document to be deserialized.cls(aJSONDecodersubclass) – If set, a custom JSON decoder. Additional keyword arguments toload()will be passed to the constructor ofcls. IfNone(the default),JSONDecoderis used.object_hook(callable|None) – If set, a function that is called with the result of any JSON object literal decoded (adict). The return value of this function will be used instead of thedict. This feature can be u

JSONDecodeError– When the data being deserialized is not a valid JSON document.UnicodeDecodeError– When the data being deserialized does not contain UTF-8, UTF-16 or UTF-32 encoded data.

json.loads(s,*,cls=None,object_hook=None,parse_float=None,parse_int=None,parse_constant=None,object_pairs_hook=None,kw)**

Identical toload(), but instead of a file-like object, deserializes(astr,bytesorbytearrayinstance containing a JSON document) to a Python object using thisconversion table.Changed in version 3.6:scan now be of typebytesorbytearray. The input encoding should be UTF-8, UTF-16 or UTF-32.Changed in version 3.9:The keyword argumentencodinghas been removed.

Encoders and Decoders

classjson.JSONDecoder(*,object_hook=None,parse_float=None,parse_int=None,parse_constant=None,strict=True,object_pairs_hook=None)

decode(s)

raw_decode(s)

Simple JSON decoder. Performs the following translations in decoding by default: JSONPythonobjectdictarrayliststringstrnumber (int)intnumber (real)floattrueTruefalseFalse>nullNone. It also understands NaN, Infinity, and -Infinity as their corresponding float values, which is outside the JSON spec. object_hook is an optional function that will be called with the result of every JSON object decoded and its return value will be used in place of the given dict. This can be used to provide custom deserializations

Return the Python representation of s (a str instance containing a JSON document). JSONDecodeError will be raised if the given JSON document is not valid.

Decode a JSON document from s (a str beginning with a JSON document) and return a 2-tuple of the Python representation and the index in s where the document ended. This can be used to decode a JSON document from a string that may have extraneous data at the end.

class json.JSONEncoder(*, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, sort_keys=False, indent=None, separators=None, default=None)

default(o)

encode(o)

iterencode(o)

Extensible JSON encoder for Python data structures. Supports the following objects and types by default: PythonJSONdictobjectlist, tuplearraystrstringint, float, int- & float-derived EnumsnumberTruefalseFalseNone>null. Changed in version 3.4: Added support for int- and float-derived Enum classes. To extend this to recognize other objects, subclass and implement a default() method with another method that returns a serializable object for o if possible, otherwise it should call the superclass implementation

Implement this method in a subclass such that it returns a serializable object for o, or calls the base implementation (to raise a TypeError). For example, to support arbitrary iterators, you could implement default() like this: def default(self, o): try: iterable = iter(o) except TypeError: pass else: return list(iterable) # Let the base class default method raise the TypeError return super().default(o)

Return a JSON string representation of a Python data structure, o. For example: >>> json.JSONEncoder().encode({"foo": ["bar", "baz"]}) '{"foo": ["bar", "baz"]}'

Encode the given object, o, and yield each string representation as available. For example: for chunk in json.JSONEncoder().iterencode(bigobject): mysocket.write(chunk)

Exceptions

exception json.JSONDecodeError(msg, doc, pos)

msg

doc

pos

lineno

colno

Subclass of ValueError with the following additional attributes: msg The unformatted error message. doc The JSON document being parsed. pos The start index of doc where parsing failed. lineno The line corresponding to pos. colno The column corresponding to pos. Added in version 3.5.

The unformatted error message.

The JSON document being parsed.

The start index of doc where parsing failed.

The line corresponding to pos.

The column corresponding to pos.

Standard Compliance and Interoperability

The JSON format is specified by RFC 7159 and by ECMA-404. This section details this module's level of compliance with the RFC. For simplicity, `JSONEncoder` and `JSONDecoder` subclasses, and parameters other than those explicitly mentioned, are not considered.

This module does not comply with the RFC in a strict fashion, implementing some extensions that are valid JavaScript but not valid JSON. In particular:

- Infinite and NaN number values are accepted and output;
- Repeated names within an object are accepted, and only the value of the last name-value pair is used.

Since the RFC permits RFC-compliant parsers to accept input texts that are not RFC-compliant, this module's deserializer is technically RFC-compliant under default settings.

Character Encodings

The RFC requires that JSON be represented using either UTF-8, UTF-16, or UTF-32, with UTF-8 being the recommended default for maximum interoperability.

As permitted, though not required, by the RFC, this module's serializer sets `ensure_ascii=True` by default, thus escaping the output so that the resulting strings only contain printable ASCII characters.

Other than the `ensure_ascii` parameter, this module is defined strictly in terms of conversion between Python objects and `Unicode` strings, and thus does not otherwise directly address the issue of character encodings.

The RFC prohibits adding a byte order mark (BOM) to the start of a JSON text, and this module's serializer does not add a BOM to its output. The RFC permits, but does not require, JSON deserializers to ignore an initial BOM in their input. This module's deserializer raises a `ValueError` when an initial BOM is present.

The RFC does not explicitly forbid JSON strings which contain byte sequences that don't correspond to valid `Unicode` characters (e.g. unpaired UTF-16 surrogates), but it does note that they may cause interoperability problems. By default, this module accepts and outputs (when present in the original `str`) code points for such sequences.

Infinite and NaN Number Values

The RFC does not permit the representation of infinite or NaN number values. Despite that, by default, this module accepts and outputs `Infinity`, `-Infinity`, and `NaN` as if they were valid JSON number literal values:

```
>>> # Neither of these calls raises an exception, but the results are not valid JSON
>>> json.dumps(float('-inf'))
'-Infinity'
>>> json.dumps(float('nan'))
'NaN'
>>> # Same when deserializing
>>> json.loads('-Infinity')
-inf
>>> json.loads('NaN')
nan
```

In the serializer, the `allow_nan` parameter can be used to alter this behavior. In the deserializer, the `parse_constant` parameter can be used to alter this behavior.

Repeated Names Within an Object

The RFC specifies that the names within a JSON object should be unique, but does not mandate how repeated names in JSON objects should be handled. By default, this module does not raise an exception; instead, it ignores all but the last name-value pair for a given name:

```
>>> weird_json = '{"x": 1, "x": 2, "x": 3}'
>>> json.loads(weird_json)
{'x': 3}
```

The `object_pairs_hook` parameter can be used to alter this behavior.

Top-level Non-Object, Non-Array Values

The old version of JSON specified by the obsolete RFC 4627 required that the top-level value of a JSON text must be either a JSON object or array (Python dictionary or list), and could not be a JSON null, boolean, number, or string value. RFC 7159 removed that restriction, and this module does not and has never implemented that restriction in either its serializer or its deserializer.

Regardless, for maximum interoperability, you may wish to voluntarily adhere to the restriction yourself.

Implementation Limitations

Some JSON deserializer implementations may set limits on:

- the size of accepted JSON texts
- the maximum level of nesting of JSON objects and arrays
- the range and precision of JSON numbers
- the content and maximum length of JSON strings

This module does not impose any such limits beyond those of the relevant Python datatypes themselves or the Python interpreter itself.

When serializing to JSON, beware any such limitations in applications that may consume your JSON. In particular, it is common for JSON numbers to be deserialized into IEEE 754 double precision numbers and thus subject to that representation's range and precision limitations. This is especially relevant when serializing Python integers of extremely large magnitude, or when serializing instances of “exotic” numerical types such as `decimal.Decimal`.

Command-line interface

Source code: `Lib/json/tool.py`

The `json` module can be invoked as a script via `python -m json` to validate and pretty-print JSON objects. The `json.tool` submodule implements this interface.

If the optional `infile` and `outfile` arguments are not specified, `sys.stdin` and `sys.stdout` will be used respectively:

```
$ echo '{"json": "obj"}' | python -m json
{
  "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```


Changed in version 3.5: The output is now in the same order as the input. Use the `--sort-keys` option to sort the output of dictionaries alphabetically by key.

Changed in version 3.14: The `json` module may now be directly executed as `python-mjson`. For backwards compatibility, invoking the CLI as `python-mjson.tool` remains supported.

Command-line options

infile

The JSON file to be validated or pretty-printed: `$python-mjson mp_films.json [{"title": "And Now for Something Completely Different", "year": 1971}, {"title": "Monty Python and the Holy Grail", "year": 1975}]` If `infile` is not specified, read from `sys.stdin`.

outfile

Write the output of the `infile` to the given `outfile`. Otherwise, write it to `sys.stdout`.

--sort-keys

Sort the output of dictionaries alphabetically by key. Added in version 3.5.

--no-ensure-ascii

Disable escaping of non-ascii characters, see `json.dumps()` for more information. Added in version 3.9.

--json-lines

Parse every input line as separate JSON object. Added in version 3.8.

--indent, --tab, --no-indent, --compact

Mutually exclusive options for whitespace control. Added in version 3.9.

-h, --help

Show the help message.

Footnotes

As noted in the errata for RFC 7159, JSON permits literal U+2028 (LINE SEPARATOR) and U+2029 (PARAGRAPH SEPARATOR) characters in strings, whereas JavaScript (as of ECMAScript Edition 5.1) does not.

mailbox— Manipulate mailboxes in various formats

Source code: [Lib/mailbox.py](#)

This module defines two classes, `Mailbox` and `Message`, for accessing and manipulating on-disk mailboxes and the messages they contain. `Mailbox` offers a dictionary-like mapping from keys to messages. `Message` extends the `email.message` module's `Message` class with format-specific state and behavior. Supported mailbox formats are Maildir, mbox, MH, Babyl, and MMDF.

See also

Module `email`

Represent and manipulate messages.

Mailbox objects

`class mailbox.Mailbox`

`add(message)`

`remove(key)`

`__delitem__(key)`

`discard(key)`

`__setitem__(key, message)`

`iterkeys()`

`keys()`

`itervalues()`

`__iter__()`

`values()`

`iteritems()`

`items()`

`get(key, default=None)`

`__getitem__(key)`

`get_message(key)`

`get_bytes(key)`

`get_string(key)`

`get_file(key)`

`__contains__(key)`

`__len__()`

`clear()`

`pop(key, default=None)`

`popitem()`

update(arg)

flush()

lock()

unlock()

close()

A mailbox, which may be inspected and modified. The `Mailbox` class defines an interface and is not intended to be instantiated. Instead, format-specific subclasses should inherit from `Mailbox` and your code should instantiate a particular subclass. The `Mailbox` interface is dictionary-like, with small keys corresponding to messages. Keys are issued by the `Mailbox` instance with which they will be used and are only meaningful to that `Mailbox` instance. A key continues to identify a message even if the correspond

Add `message` to the mailbox and return the key that has been assigned to it. Parameter `message` may be a `Message` instance, an `email.message.Message` instance, a string, a byte string, or a file-like object (which should be open in binary mode). If `message` is an instance of the appropriate format-specific `Message` subclass (e.g., if it's an `anmbboxMessage` instance and this is an `anmbbox` instance), its format-specific information is used. Otherwise, reasonable defaults for format-specific information are used. Changed in ver

Delete the message corresponding to `key` from the mailbox. If no such message exists, a `KeyError` exception is raised if the method was called as `remove()` or `__delitem__()` but no exception is raised if the method was called as `discard()`. The behavior of `discard()` may be preferred if the underlying mailbox format supports concurrent modification by other processes.

Replace the message corresponding to `key` with `message`. Raise a `KeyError` exception if no message already corresponds to `key`. As with `add()`, parameter `message` may be a `Message` instance, an `email.message.Message` instance, a string, a byte string, or a file-like object (which should be open in binary mode). If `message` is an instance of the appropriate format-specific `Message` subclass (e.g., if it's an `anmbboxMessage` instance and this is an `anmbbox` instance), its format-specific information is used. Otherwise, the format-specif

Return an iterator over all keys

The same as `iterkeys()`, except that a list is returned rather than an iterator

Return an iterator over representations of all messages. The messages are represented as instances of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized. Note the behavior of `__iter__()` is unlike that of dictionaries, which iterate over keys.

The same as `itervalues()`, except that a list is returned rather than an iterator

Return an iterator over (key, message) pairs, where `key` is a key and `message` is a message representation. The messages are represented as instances of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

The same as `iteritems()`, except that a list of pairs is returned rather than an iterator of pairs.

Return a representation of the message corresponding to `key`. If no such message exists, `default` is returned if the method was called as `get()` and a `KeyError` exception is raised if the method was called as `__getitem__()`. The message is represented as an instance of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

Return a representation of the message corresponding to `key` as an instance of the appropriate format-specific `Message` subclass, or raise a `KeyError` exception if no such message exists.

Return a byte representation of the message corresponding tokey, or raise aKeyErrorexception if no such message exists.Added in version 3.2.

Return a string representation of the message corresponding tokey, or raise aKeyErrorexception if no such message exists. The message is processed throughemail.message.Messageeto convert it to a 7bit clean representation.

Return afile-likerepresentation of the message corresponding tokey, or raise aKeyErrorexception if no such message exists. The file-like object behaves as if open in binary mode. This file should be closed once it is no longer needed.Changed in version 3.2:The file object really is abinary file; previously it was incorrectly returned in text mode. Also, thefile-like objectnow supports thecontext managerprotocol: you can use awithstatement to automatically close it.NoteUnlike other representat

ReturnTrueifkeycorresponds to a message,Falseotherwise.

Return a count of messages in the mailbox.

Delete all messages from the mailbox.

Return a representation of the message corresponding tokeyand delete the message. If no such message exists, returndefault. The message is represented as an instance of the appropriate format-specificMessagesubclass unless a custom message factory was specified when theMailboxinstance was initialized.

Return an arbitrary (key,message) pair, wherekeyis a key andmessageis a message representation, and delete the corresponding message. If the mailbox is empty, raise aKeyErrorexception. The message is represented as an instance of the appropriate format-specificMessagesubclass unless a custom message factory was specified when theMailboxinstance was initialized.

Parameterargshould be akey-to-messagemapping or an iterable of (key,message) pairs. Updates the mailbox so that, for each givenkeyandmessage, the message corresponding tokeyis set tomessagetas if by using__setitem__(). As with__setitem__(), eachkeymust already correspond to a message in the mailbox or else aKeyErrorexception will be raised, so in general it is incorrect forargto be aMailboxinstance.NoteUnlike with dictionaries, keyword arguments are not supported.

Write any pending changes to the filesystem. For someMailboxsubclasses, changes are always written immediately andflush()does nothing, but you should still make a habit of calling this method.

Acquire an exclusive advisory lock on the mailbox so that other processes know not to modify it. AnExternalClashErroris raised if the lock is not available. The particular locking mechanisms used depend upon the mailbox format. You shouldalwayslock the mailbox before making any modifications to its contents.

Release the lock on the mailbox, if any.

Flush the mailbox, unlock it if necessary, and close any open files. For someMailboxsubclasses, this method does nothing.

Maildirobjects

classmailbox.Maildir(dirname,factory=None,create=True)

colon

list_folders()

get_folder(folder)

add_folder(folder)

remove_folder(folder)

clean()

get_flags(key)
set_flags(key,flags)
add_flag(key,flag)
remove_flag(key,flag)
get_info(key)
set_info(key,info)
add(message)
__setitem__(key,message)
update(arg)
flush()
lock()
unlock()
close()
get_file(key)

A subclass of Mailbox for mailboxes in Maildir format. Parameter factory is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If factory is None, MaildirMessage is used as the default message representation. If create is True, the mailbox is created if it does not exist. If create is True and the dir name path exists, it will be treated as an existing maildir without attempting to verify its directory layout. It is for h

The Maildir specification requires the use of a colon (':') in certain message file names. However, some operating systems do not permit this character in file names. If you wish to use a Maildir-like format on such an operating system, you should specify another character to use instead. The exclamation point ('!') is a popular choice. For example: `import mailbox; mailbox.Maildir.colon='!'` The colon attribute may also be set on a per-instance basis.

Return a list of the names of all folders.

Return a Maildir instance representing the folder whose name is folder. A NoSuchMailboxError exception is raised if the folder does not exist.

Create a folder whose name is folder and return a Maildir instance representing it.

Delete the folder whose name is folder. If the folder contains any messages, a NotEmptyError exception will be raised and the folder will not be deleted.

Delete temporary files from the mailbox that have not been accessed in the last 36 hours. The Maildir specification says that mail-reading programs should do this occasionally.

Return as a string the flags that are set on the message corresponding to key. This is the same as `get_message(key).get_flags()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get. If you do have a MaildirMessage object, use its `get_flags()` method instead, because changes made by the message's `set_flags()`, `add_flag()` and `remove_flag()` methods are not reflected here until the mailbox's `__setitem__()` method is

On the message corresponding to key, set the flags specified by flags and unset all others. Calling `some_mailbox.set_flags(key,flags)` is similar to `one_message=some_mailbox.get_message(key); one_message.set_flags(flags); some_mailbox[key]=one_message` but faster, because it does not open the message file. If

you do have a `MaildirMessage` object, use its `set_flags()` method instead, because changes made with this mailbox method will not be visible to the message object's `get_flags()`. Added in version 3.13.

On the message corresponding to `key`, set the flags specified by `flag` without changing other flags. To add more than one flag at a time, `flag` may be a string of more than one character. Considerations for using this method versus the message object's `sadd_flag()` method are similar to those for `set_flags()`; see the discussion there. Added in version 3.13.

On the message corresponding to `key`, unset the flags specified by `flag` without changing other flags. To remove more than one flag at a time, `flag` may be a string of more than one character. Considerations for using this method versus the message object's `sremove_flag()` method are similar to those for `set_flags()`; see the discussion there. Added in version 3.13.

Return a string containing the info for the message corresponding to `key`. This is the same as `get_message(key).get_info()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get. If you do have a `MaildirMessage` object, use its `get_info()` method instead, because changes made by the message's `set_info()` method are not reflected here until the mailbox's `__setitem__()` method is called. Added in version 3.13.

Set the info of the message corresponding to `key` to `info`. Calling `some_mailbox.set_info(key, flags)` is similar to `one_message=some_mailbox.get_message(key); one_message.set_info(info); some_mailbox[key]=one_message` but faster, because it does not open the message file. If you do have a `MaildirMessage` object, use its `set_info()` method instead, because changes made with this mailbox method will not be visible to the message object's `get_info()`. Added in version 3.13.

Warning These methods generate unique file names based upon the current process ID. When using multiple threads, undetected name clashes may occur and cause corruption of the mailbox unless threads are coordinated to avoid using these methods to manipulate the same mailbox simultaneously.

All changes to Maildir mailboxes are immediately applied, so this method does nothing.

Maildir mailboxes do not support (or require) locking, so these methods do nothing.

Maildir instances do not keep any open files and the underlying mailboxes do not support locking, so this method does nothing.

Depending upon the host platform, it may not be possible to modify or remove the underlying message while the returned file remains open.

See also

maildir man page from Courier

Using maildir format

A specification of the format. Describes a common extension for supporting folders.

Notes on Maildir by its inventor. Includes an updated name-creation scheme and details on “info” semantics.

mbxobjects

`class mailbox.mbox(path, factory=None, create=True)`

`get_bytes(key, from_=False)`

`get_file(key, from_=False)`

`get_string(key, from_=False)`

`lock()`

unlock()

A subclass of `Mailbox` for mailboxes in mbox format. `Parameterfactory` is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If `factory` is `None`, `mboxMessage` is used as the default message representation. If `create` is `True`, the mailbox is created if it does not exist. The mbox format is the classic format for storing mail on Unix systems. All messages in an mbox mailbox are stored in a single file with the beginning

Note: This method has an extra parameter (`from_`) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If `from_` is `False`, the first line of the file is dropped.

Using the file after calling `flush()` or `close()` on the `mbox` instance may yield unpredictable results or raise an exception. Note: This method has an extra parameter (`from_`) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If `from_` is `False`, the first line of the file is dropped.

Note: This method has an extra parameter (`from_`) compared with other classes. The first line of an mbox file entry is the Unix "From " line. If `from_` is `False`, the first line of the file is dropped.

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls.

See also

mbox man page from tin

Configuring Netscape Mail on Unix: Why The Content-Length Format is Bad

“mbox” is a family of several mutually incompatible mailbox formats

A specification of the format, with details on locking.

An argument for using the original mbox format rather than a variation.

A history of mbox variations.

MHobjects

`class mailbox.MH(path, factory=None, create=True)`

`list_folders()`

`get_folder(folder)`

`add_folder(folder)`

`remove_folder(folder)`

`get_sequences()`

`set_sequences(sequences)`

`pack()`

`remove(key)`

`__delitem__(key)`

`discard(key)`

`lock()`

`unlock()`

`get_file(key)`

`flush()`

`close()`

A subclass of `Mailbox` for mailboxes in MH format. Parameter `factory` is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If `factory` is `None`, `MHMessage` is used as the default message representation. If `create` is `True`, the mailbox is created if it does not exist. MH is a directory-based mailbox format invented for the MH Message Handling System, a mail user agent. Each message in an MH mailbox resides in its own file.

Return a list of the names of all folders.

Return an `MHInstance` representing the folder whose name is `folder`. A `NoSuchMailboxError` exception is raised if the folder does not exist.

Create a folder whose name is `folder` and return an `MHInstance` representing it.

Delete the folder whose name is `folder`. If the folder contains any messages, a `NotEmptyError` exception will be raised and the folder will not be deleted.

Return a dictionary of sequence names mapped to key lists. If there are no sequences, the empty dictionary is returned.

Re-define the sequences that exist in the mailbox based upon `sequences`, a dictionary of names mapped to key lists, like returned by `get_sequences()`.

Rename messages in the mailbox as necessary to eliminate gaps in numbering. Entries in the sequences list are updated correspondingly. Note already-issued keys are invalidated by this operation and should not be subsequently used.

These methods immediately delete the message. The MH convention of marking a message for deletion by prepending a comma to its name is not used.

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls. For MH mailboxes, locking the mailbox means locking the `.mh_sequences` file and, only for the duration of any operations that affect them, locking individual message files.

Depending upon the host platform, it may not be possible to remove the underlying message while the returned file remains open.

All changes to MH mailboxes are immediately applied, so this method does nothing.

`MHInstances` do not keep any open files, so this method is equivalent to `unlock()`.

See also

nmh - Message Handling System

MH & nmh: Email for Users & Programmers

Home page of nmh, an updated version of the original mh.

A GPL-licensed book on mhand and nmh, with some information on the mailbox format.

***Babyl*objects**

`class mailbox.Babyl(path, factory=None, create=True)`

`get_labels()`

`get_file(key)`

`lock()`

`unlock()`

A subclass of `Mailbox` for mailboxes in Babyl format. Parameter `factory` is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation.

If `factory` is `None`, `BabylMessage` is used as the default message representation. If `create` is `True`, the mailbox is created if it does not exist. `Babyl` is a single-file mailbox format used by the `Rmail` mail user agent included with Emacs. The beginning of a message is indicated by a line conta

Return a list of the names of all user-defined labels used in the mailbox. Note The actual messages are inspected to determine which labels exist in the mailbox rather than consulting the list of labels in the `Babyl` options section, but the `Babyl` section is updated whenever the mailbox is modified.

In `Babyl` mailboxes, the headers of a message are not stored contiguously with the body of the message. To generate a file-like representation, the headers and body are copied together into `anio.BytesIO` instance, which has an API identical to that of a file. As a result, the file-like object is truly independent of the underlying mailbox but does not save memory compared to a string representation.

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls.

See also

Format of Version 5 Babyl Files

Reading Mail with Rmail

A specification of the `Babyl` format.

The `Rmail` manual, with some information on `Babyl` semantics.

MMDF Objects

`class mailbox.MMDF(path, factory=None, create=True)`

`get_bytes(key, from_=False)`

`get_file(key, from_=False)`

`lock()`

`unlock()`

A subclass of `Mailbox` for mailboxes in `MMDF` format. Parameter `factory` is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If `factory` is `None`, `MMDFMessage` is used as the default message representation. If `create` is `True`, the mailbox is created if it does not exist. `MMDF` is a single-file mailbox format invented for the Multichannel Memorandum Distribution Facility, a mail transfer agent. Each message is in the sam

Note: This method has an extra parameter (`from_`) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If `from_` is `False`, the first line of the file is dropped.

Using the file after calling `flush()` or `close()` on the `MMDF` instance may yield unpredictable results or raise an exception. Note: This method has an extra parameter (`from_`) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If `from_` is `False`, the first line of the file is dropped.

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls.

See also

`mmdf` man page from tin

MMDF

A specification of `MMDF` format from the documentation of `tin`, a newsreader.

A Wikipedia article describing the Multichannel Memorandum Distribution Facility.

Messageobjects

class mailbox.Message(message=None)

A subclass of the email.message module's Message. Subclasses of mailbox.Message add mailbox-format-specific state and behavior. If message is omitted, the new instance is created in a default, empty state. If message is an email.message.Message instance, its contents are copied; furthermore, any format-specific information is converted insofar as possible if message is a Message instance. If message is a string, a byte string, or a file, it should contain an RFC 5322-compliant message, which is read and parsed. F

MaildirMessageobjects

class mailbox.MaildirMessage(message=None)

get_subdir()

set_subdir(subdir)

get_flags()

set_flags(flags)

add_flag(flag)

remove_flag(flag)

get_date()

set_date(date)

get_info()

set_info(info)

A message with Maildir-specific behaviors. Parameter message has the same meaning as with the Message constructor. Typically, a mail user agent application moves all of the messages in the newsubdirectory to the cursubdirectory after the first time the user opens and closes the mailbox, recording that the messages are old whether or not they've actually been read. Each message incurhas an "info" section added to its file name to store information about its state. (Some mail readers may also add an "inf

Return either "new" (if the message should be stored in the newsubdirectory) or "cur" (if the message should be stored in the cursubdirectory). Note A message is typically moved from new to cur after its mailbox has been accessed, whether or not the message has been read. A message msg has been read if "S" in msg.get_flags() is True.

Set the subdirectory the message should be stored in. Parameter subdir must be either "new" or "cur".

Return a string specifying the flags that are currently set. If the message complies with the standard Maildir format, the result is the concatenation in alphabetical order of zero or one occurrence of each of 'D', 'F', 'P', 'R', 'S', and 'T'. The empty string is returned if no flags are set or if "info" contains experimental semantics.

Set the flags specified by flags and unset all others.

Set the flag(s) specified by flag without changing other flags. To add more than one flag at a time, flag may be a string of more than one character. The current "info" is overwritten whether or not it contains experimental information rather than flags.

Unset the flag(s) specified by flag without changing other flags. To remove more than one flag at a time, flag may be a string of more than one character. If "info" contains experimental information rather than flags, the current "info" is not modified.

Return the delivery date of the message as a floating-point number representing seconds since the epoch.

Set the delivery date of the message to `date`, a floating-point number representing seconds since the epoch.

Return a string containing the “info” for a message. This is useful for accessing and modifying “info” that is experimental (i.e., not a list of flags).

Set “info” to `info`, which should be a string.

When a `MaildirMessage` instance is created based upon an `mbxMessage` or `MMDFMessage` instance, the `status` and `X-Status` headers are omitted and the following conversions take place:

```
Resulting state | mboxMessage | or | MMDMessage | state | "cur" subdirectory | 0 flag | F flag  
| F flag | R flag | A flag | S flag | R flag | T flag | D flag
```

When a `MaildirMessage` instance is created based upon an `MHMessage` instance, the following conversions take place:

```
Resulting state | MHMessage | state | "cur" subdirectory | "unseen" sequence | "cur"  
subdirectory and S flag | no "unseen" sequence | F flag | "flagged" sequence | R flag |  
"replied" sequence
```

When a `MaildirMessage` instance is created based upon a `BabylMessage` instance, the following conversions take place:

```
Resulting state | BabylMessage | state | "cur" subdirectory | "unseen" label | "cur"  
subdirectory and S flag | no "unseen" label | P flag | "forwarded" or "resent" label | R flag |  
"answered" label | T flag | "deleted" label
```

***mbxMessage*objects**

`class mailbox.mbxMessage(message=None)`

`get_from()`

`set_from(from_,time_=None)`

`get_flags()`

`set_flags(flags)`

`add_flag(flag)`

`remove_flag(flag)`

A message with mbox-specific behaviors. Parameter `message` has the same meaning as with the `Message` constructor. Messages in an mbox mailbox are stored together in a single file. The sender's envelope address and the time of delivery are typically stored in a line beginning with “From “ that is used to indicate the start of a message, though there is considerable variation in the exact format of this data among mbox implementations. Flags that indicate the state of the message, such as whether it has

Return a string representing the “From “ line that marks the start of the message in an mbox mailbox. The leading “From “ and the trailing newline are excluded.

Set the “From “ line to `from_`, which should be specified without a leading “From “ or trailing newline. For convenience, `time_` may be specified and will be formatted appropriately and appended to `from_`. If `time_` is specified, it should be a `time.struct_time` instance, a tuple suitable for passing to `time.strftime()`, or `True` (to use `time.gmtime()`).

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of 'R', 'O', 'D', 'F', and 'A'.

Set the flags specified by `flags` and unset all others. Parameter `flags` should be the concatenation in any order of zero or more occurrences of each of 'R', 'O', 'D', 'F', and 'A'.

Set the flag(s) specified by `flag` without changing other flags. To add more than one flag at a time, `flag` may be a string of more than one character.

Unset the flag(s) specified by `flag` without changing other flags. To remove more than one flag at a time, `flag` may be a string of more than one character.

When an `amboxMessageinstance` is created based upon a `MaildirMessageinstance`, a “From “ line is generated based upon the `MaildirMessageinstance`’s delivery date, and the following conversions take place:

Resulting state		MaildirMessage		state		R flag		S flag		O flag		“cur” subdirectory		D flag		T flag		F flag		F flag		A flag		R flag
-----------------	--	----------------	--	-------	--	--------	--	--------	--	--------	--	--------------------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------

When an `amboxMessageinstance` is created based upon an `MHMessageinstance`, the following conversions take place:

Resulting state		MHMessage		state		R flag and O flag		no “unseen” sequence		O flag		“unseen” sequence		F flag		“flagged” sequence		A flag		“replied” sequence
-----------------	--	-----------	--	-------	--	-------------------	--	----------------------	--	--------	--	-------------------	--	--------	--	--------------------	--	--------	--	--------------------

When an `amboxMessageinstance` is created based upon a `BabylMessageinstance`, the following conversions take place:

Resulting state		BabylMessage		state		R flag and O flag		no “unseen” label		O flag		“unseen” label		D flag		“deleted” label		A flag		“answered” label
-----------------	--	--------------	--	-------	--	-------------------	--	-------------------	--	--------	--	----------------	--	--------	--	-----------------	--	--------	--	------------------

When an `amboxMessageinstance` is created based upon an `MMDFMessageinstance`, the “From “ line is copied and all flags directly correspond:

Resulting state		MMDFMessage		state		R flag		R flag		O flag		O flag		D flag		D flag		F flag		F flag		A flag		A flag
-----------------	--	-------------	--	-------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------

MHMessageobjects

`class mailbox.MHMessage(message=None)`

`get_sequences()`

`set_sequences(sequences)`

`add_sequence(sequence)`

`remove_sequence(sequence)`

A message with MH-specific behaviors. Parameter `message` has the same meaning as with the `Message` constructor. MH messages do not support marks or flags in the traditional sense, but they do support sequences, which are logical groupings of arbitrary messages. Some mail reading programs (although not the standard `mh` and `nmh`) use sequences in much the same way flags are used with other formats, as follows: `SequenceExplanation` `unseen` Not read, but previously detected by MUA `replied` Replied to `flagged` Marked as im

Return a list of the names of sequences that include this message.

Set the list of sequences that include this message.

Add `sequence` to the list of sequences that include this message.

Remove `sequence` from the list of sequences that include this message.

When an `MHMessageinstance` is created based upon a `MaildirMessageinstance`, the following conversions take place:

Resulting state		MaildirMessage		state		“unseen” sequence		no S flag		“replied” sequence		R flag		“flagged” sequence		F flag
-----------------	--	----------------	--	-------	--	-------------------	--	-----------	--	--------------------	--	--------	--	--------------------	--	--------

When an `MHMessageinstance` is created based upon an `amboxMessage` or `MMDFMessageinstance`, the `Status` and `X-Status` headers are omitted and the following conversions take place:

```
Resulting state | mboxMessage | or | MMDFMessage | state | "unseen" sequence | no R flag |  
"replied" sequence | A flag | "flagged" sequence | F flag
```

When an `MMHMessage` instance is created based upon an `ABabylMessage` instance, the following conversions take place:

```
Resulting state | BabylMessage | state | "unseen" sequence | "unseen" label | "replied" sequence  
| "answered" label
```

***BabylMessage* objects**

`class mailbox.BabylMessage(message=None)`

`get_labels()`

`set_labels(labels)`

`add_label(label)`

`remove_label(label)`

`get_visible()`

`set_visible(visible)`

`update_visible()`

A message with Babyl-specific behaviors. Parameter `message` has the same meaning as with the `Message` constructor. Certain message labels, called attributes, are defined by convention to have special meanings. The attributes are as follows: `LabelExplanation` unseen Not read, but previously detected by MUA `deleted` Marked for subsequent deletion `filed` Copied to another file or mailbox `answered` Replied to `forwarded` Forwarded `edited` Modified by the user `resent` Resent By default, Rmail displays only visible headers. The Babyl

Return a list of labels on the message.

Set the list of labels on the message to `labels`.

Add `label` to the list of labels on the message.

Remove `label` from the list of labels on the message.

Return a `Message` instance whose headers are the message's visible headers and whose body is empty.

Set the message's visible headers to be the same as the headers in `message`. Parameter `visible` should be a `Message` instance, an `email.message.Message` instance, a string, or a file-like object (which should be open in text mode).

When an `ABabylMessage` instance's original headers are modified, the visible headers are not automatically modified to correspond. This method updates the visible headers as follows: each visible header with a corresponding original header is set to the value of the original header, each visible header without a corresponding original header is removed, and any of `Date`, `From`, `Reply-To`, `To`, `CC`, and `Subject` that are present in the original headers but not the visible headers are added to the visible headers.

When an `ABabylMessage` instance is created based upon an `AMaildirMessage` instance, the following conversions take place:

```
Resulting state | MaildirMessage | state | "unseen" label | no S flag | "deleted" label | T flag  
| "answered" label | R flag | "forwarded" label | P flag
```

When an `ABabylMessage` instance is created based upon an `mboxMessage` or `MMDFMessage` instance, the `Status` and `X-Status` headers are omitted and the following conversions take place:

```
Resulting state | mboxMessage | or | MMDFMessage | state | "unseen" label | no R flag |  
"deleted" label | D flag | "answered" label | A flag
```

When a `BabyMessage` instance is created based upon an `MHMessage` instance, the following conversions take place:

Resulting state	MHMessage	state	"unseen" label	"unseen" sequence	"answered" label	"replied" sequence

***MMDFMessage* objects**

`class mailbox.MMDFMessage(message=None)`

`get_from()`

`set_from(from_, time_=None)`

`get_flags()`

`set_flags(flags)`

`add_flag(flag)`

`remove_flag(flag)`

A message with MMDF-specific behaviors. Parameter `message` has the same meaning as with the `Message` constructor. As with `message` in an mbox mailbox, MMDF messages are stored with the sender's address and the delivery date in an initial line beginning with "From ". Likewise, flags that indicate the state of the message are typically stored in `Status` and `X-Status` headers. Conventional flags for MMDF messages are identical to those of mbox message and are as follows: `FlagMeaningExplanationRRReadReadOOldPreviou`

Return a string representing the "From " line that marks the start of the message in an mbox mailbox. The leading "From " and the trailing newline are excluded.

Set the "From " line to `from_`, which should be specified without a leading "From " or trailing newline. For convenience, `time_` may be specified and will be formatted appropriately and appended to `from_`. If `time_` is specified, it should be a `time.struct_time` instance, a tuple suitable for passing to `time.strftime()`, or `True` (to use `time.gmtime()`).

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of 'R', 'O', 'D', 'F', and 'A'.

Set the flags specified by `flags` and unset all others. Parameter `flags` should be the concatenation in any order of zero or more occurrences of each of 'R', 'O', 'D', 'F', and 'A'.

Set the flag(s) specified by `flag` without changing other flags. To add more than one flag at a time, `flag` may be a string of more than one character.

Unset the flag(s) specified by `flag` without changing other flags. To remove more than one flag at a time, `flag` may be a string of more than one character.

When an `MMDFMessage` instance is created based upon a `MaildirMessage` instance, a "From " line is generated based upon the `MaildirMessage` instance's delivery date, and the following conversions take place:

Resulting state	MaildirMessage	state	R flag	S flag	O flag	"cur" subdirectory	D flag	T flag	F flag	F flag	A flag	R flag

When an `MMDFMessage` instance is created based upon an `MHMessage` instance, the following conversions take place:

Resulting state	MHMessage	state	R flag and O flag	no "unseen" sequence	O flag	"unseen" sequence	F flag	"flagged" sequence	A flag	"replied" sequence

When an `MMDFMessage` instance is created based upon a `BabyMessage` instance, the following conversions take place:

```
Resulting state | BabylMessage | state | R flag and O flag | no "unseen" label | O flag |  
"unseen" label | D flag | "deleted" label | A flag | "answered" label
```

When an `MMDFMessage` instance is created based upon an `mboxMessage` instance, the "From " line is copied and all flags directly correspond:

```
Resulting state | mboxMessage | state | R flag | R flag | O flag | O flag | D flag | D flag | F  
flag | F flag | A flag | A flag
```

Exceptions

The following exception classes are defined in the `mailbox` module:

`exception mailbox.Error`

The base class for all other module-specific exceptions.

`exception mailbox.NoSuchMailboxError`

Raised when a mailbox is expected but is not found, such as when instantiating a `Mailbox` subclass with a path that does not exist (and with the `create` parameter set to `False`), or when opening a folder that does not exist.

`exception mailbox.NotEmptyError`

Raised when a mailbox is not empty but is expected to be, such as when deleting a folder that contains messages.

`exception mailbox.ExternalClashError`

Raised when some mailbox-related condition beyond the control of the program causes it to be unable to proceed, such as when failing to acquire a lock that another program already holds, or when a uniquely generated file name already exists.

`exception mailbox.FormatError`

Raised when the data in a file cannot be parsed, such as when an `MH` instance attempts to read a corrupted `.mh_sequences` file.

Examples

A simple example of printing the subjects of all messages in a mailbox that seem interesting:

```
import mailbox  
for message in mailbox.mbox('~/.mbox'):  
    subject = message['subject'] # Could possibly be None.  
    if subject and 'python' in subject.lower():  
        print(subject)
```

To copy all mail from a Babyl mailbox to an MH mailbox, converting all of the format-specific information that can be converted:

```
import mailbox  
destination = mailbox.MH('~/.Mail')  
destination.lock()  
for message in mailbox.Babyl('~/.RMAIL'):  
    destination.add(mailbox.MHMessage(message))  
destination.flush()  
destination.unlock()
```

This example sorts mail from several mailing lists into different mailboxes, being careful to avoid mail corruption due to concurrent modification by other programs, mail loss due to interruption of the program, or premature

termination due to malformed messages in the mailbox:

```
import mailbox
import email.errors

list_names = ('python-list', 'python-dev', 'python-bugs')

boxes = {name: mailbox.mbox('~/.email/%s' % name) for name in list_names}
inbox = mailbox.Maildir('~/.Maildir', factory=None)

for key in inbox.iterkeys():
    try:
        message = inbox[key]
    except email.errors.MessageParseError:
        continue          # The message is malformed. Just leave it.

    for name in list_names:
        list_id = message['list-id']
        if list_id and name in list_id:
            # Get mailbox to use
            box = boxes[name]

            # Write copy to disk before removing original.
            # If there's a crash, you might duplicate a message, but
            # that's better than losing a message completely.
            box.lock()
            box.add(message)
            box.flush()
            box.unlock()

            # Remove original message
            inbox.lock()
            inbox.discard(key)
            inbox.flush()
            inbox.unlock()
            break          # Found destination, so stop looking.

for box in boxes.itervalues():
    box.close()
```

mimetypes— Map filenames to MIME types

Source code: `Lib/mimetypes.py`

The `mimetypes` module converts between a filename or URL and the MIME type associated with the filename extension. Conversions are provided from filename to MIME type and from MIME type to filename extension; encodings are not supported for the latter conversion.

The module provides one class and a number of convenience functions. The functions are the normal interface to this module, but some applications may be interested in the class as well.

The functions described below provide the primary interface for this module. If the module has not been initialized, they will call `init()` if they rely on the information `init()` sets up.

`mimetypes.guess_type(url,strict=True)`

Guess the type of a file based on its filename, path or URL, given by `url`. URL can be a string or a path-like object. The return value is a tuple `(type,encoding)` where `type` is `None` if the type can't be guessed (missing or unknown suffix) or a string of the form `'type/subtype'`, usable for a `MIMEContent-type` header. `encoding` is `None` for no encoding or the name of the program used to encode (e.g. `compress` or `gzip`). The encoding is suitable for use as a `Content-Encoding` header, not as a `Content-Transfer-Encoding` header. The

`mimetypes.guess_file_type(path,*,strict=True)`

Guess the type of a file based on its path, given by `path`. Similar to the `guess_type()` function, but accepts a path instead of URL. Path can be a string, a bytes object or a path-like object. Added in version 3.13.

`mimetypes.guess_all_extensions(type,strict=True)`

Guess the extensions for a file based on its MIME type, given by `type`. The return value is a list of strings giving all possible filename extensions, including the leading dot (`'.'`). The extensions are not guaranteed to have been associated with any particular data stream, but would be mapped to the MIME type by `guess_type()` and `guess_file_type()`. The optional `strict` argument has the same meaning as with the `guess_type()` function.

`mimetypes.guess_extension(type,strict=True)`

Guess the extension for a file based on its MIME type, given by `type`. The return value is a string giving a filename extension, including the leading dot (`'.'`). The extension is not guaranteed to have been associated with any particular data stream, but would be mapped to the MIME type by `guess_type()` and `guess_file_type()`. If no extension can be guessed for `type`, `None` is returned. The optional `strict` argument has the same meaning as with the `guess_type()` function.

Some additional functions and data items are available for controlling the behavior of the module.

`mimetypes.init(files=None)`

Initialize the internal data structures. If given, `files` must be a sequence of file names which should be used to augment the default type map. If omitted, the file names to use are taken from `knownfiles`; on Windows, the current registry settings are loaded. Each file named in `files` or `knownfiles` takes precedence over those named before it. Calling `init()` repeatedly is allowed. Specifying an empty list for `files` will prevent the system defaults from being applied: only the well-known values will be present.

`mimetypes.read_mime_types(filename)`

Load the type map given in the file `filename`, if it exists. The type map is returned as a dictionary mapping filename extensions, including the leading dot (`'.'`), to strings of the form `'type/subtype'`. If the file `filename` does not exist or cannot be read, `None` is returned.

mimetypes.add_type(type,ext,strict=True)

Add a mapping from the MIME type `type` to the extension `ext`. When the extension is already known, the new type will replace the old one. When the type is already known the extension will be added to the list of known extensions. When `strict` is `True` (the default), the mapping will be added to the official MIME types, otherwise to the non-standard ones.

mimetypes.inited

Flag indicating whether or not the global data structures have been initialized. This is set to `True` by `init()`.

mimetypes.knownfiles

List of type map file names commonly installed. These files are typically named `mime.types` and are installed in different locations by different packages.

mimetypes.suffix_map

Dictionary mapping suffixes to suffixes. This is used to allow recognition of encoded files for which the encoding and the type are indicated by the same extension. For example, the `.tgz` extension is mapped to `.tar.gz` to allow the encoding and type to be recognized separately.

mimetypes.encodings_map

Dictionary mapping filename extensions to encoding types.

mimetypes.types_map

Dictionary mapping filename extensions to MIME types.

mimetypes.common_types

Dictionary mapping filename extensions to non-standard, but commonly found MIME types.

An example usage of the module:

```
>>> import mimetypes
>>> mimetypes.init()
>>> mimetypes.knownfiles
['/etc/mime.types', '/etc/httpd/mime.types', ... ]
>>> mimetypes.suffix_map['.tgz']
'.tar.gz'
>>> mimetypes.encodings_map['.gz']
'gzip'
>>> mimetypes.types_map['.tgz']
'application/x-tar-gz'
```

MimeTypes objects

The `MimeTypes` class may be useful for applications which may want more than one MIME-type database; it provides an interface similar to the one of the `mimetypes` module.

class mimetypes.MimeTypes(filenamees=(),strict=True)

suffix_map

encodings_map

types_map

types_map_inv

guess_extension(type,strict=True)

guess_type(url,strict=True)

guess_file_type(path,*,strict=True)
guess_all_extensions(type,strict=True)
read(filename,strict=True)
readfp(fp,strict=True)
read_windows_registry(strict=True)
add_type(type,ext,strict=True)

This class represents a MIME-types database. By default, it provides access to the same database as the rest of this module. The initial database is a copy of that provided by the module, and may be extended by loading additional mime.types-style files into the database using `read()` or `readfp()` methods. The mapping dictionaries may also be cleared before loading additional data if the default data is not desired. The optional `filename` parameter can be used to cause additional files to be loaded “

Dictionary mapping suffixes to suffixes. This is used to allow recognition of encoded files for which the encoding and the type are indicated by the same extension. For example, the `tgz` extension is mapped to `tar.gz` to allow the encoding and type to be recognized separately. This is initially a copy of the `global_suffix_map` defined in the module.

Dictionary mapping filename extensions to encoding types. This is initially a copy of the `global_encodings_map` defined in the module.

Tuple containing two dictionaries, mapping filename extensions to MIME types: the first dictionary is for the non-standards types and the second one is for the standard types. They are initialized by `common_types_and_types_map`.

Tuple containing two dictionaries, mapping MIME types to a list of filename extensions: the first dictionary is for the non-standards types and the second one is for the standard types. They are initialized by `common_types_and_types_map`.

Similar to the `guess_extension()` function, using the tables stored as part of the object.

Similar to the `guess_type()` function, using the tables stored as part of the object.

Similar to the `guess_file_type()` function, using the tables stored as part of the object. Added in version 3.13.

Similar to the `guess_all_extensions()` function, using the tables stored as part of the object.

Load MIME information from a file named `filename`. This uses `readfp()` to parse the file. If `strict` is `True`, information will be added to list of standard types, else to the list of non-standard types.

Load MIME type information from an open file `fp`. The file must have the format of the standard `mime.types` files. If `strict` is `True`, information will be added to the list of standard types, else to the list of non-standard types.

Load MIME type information from the Windows registry. Availability: Windows. If `strict` is `True`, information will be added to the list of standard types, else to the list of non-standard types. Added in version 3.2.

Add a mapping from the MIME type `typeto` to the extension `ext`. Valid extensions start with a `'.'` or are empty. When the extension is already known, the new type will replace the old one. When the type is already known the extension will be added to the list of known extensions. When `strict` is `True` (the default), the mapping will be added to the official MIME types, otherwise to the non-standard ones. Deprecated since version 3.14, will be removed in version 3.16: Invalid, undotted extensions will raise a `ValueError`.

Command-line usage

The `mimetypes` module can be executed as a script from the command line.

```
python -m mimetypes [-h] [-e] [-l] type [type ...]
```

The following options are accepted:

-h

--help

Show the help message and exit.

-e

--extension

Guess extension instead of type.

-l

--lenient

Additionally search for some common, but non-standard types.

By default the script converts MIME types to file extensions. However, if `--extension` is specified, it converts file extensions to MIME types.

For each type entry, the script writes a line into the standard output stream. If an unknown type occurs, it writes an error message into the standard error stream and exits with the return code 1.

Command-line example

Here are some examples of typical usage of the `mimetypes` command-line interface:

```
$ # get a MIME type by a file name
$ python -m mimetypes filename.png
type: image/png encoding: None

$ # get a MIME type by a URL
$ python -m mimetypes https://example.com/filename.txt
type: text/plain encoding: None

$ # get a complex MIME type
$ python -m mimetypes filename.tar.gz
type: application/x-tar encoding: gzip

$ # get a MIME type for a rare file extension
$ python -m mimetypes filename.pict
error: unknown extension of filename.pict

$ # now look in the extended database built into Python
$ python -m mimetypes --lenient filename.pict
type: image/pict encoding: None

$ # get a file extension by a MIME type
$ python -m mimetypes --extension text/javascript
.js

$ # get a file extension by a rare MIME type
$ python -m mimetypes --extension text/xul
error: unknown type text/xul

$ # now look in the extended database again
$ python -m mimetypes --extension --lenient text/xul
.xul
```

```
$ # try to feed an unknown file extension
$ python -m mimetypes filename.sh filename.nc filename.xxx filename.txt
type: application/x-sh encoding: None
type: application/x-netcdf encoding: None
error: unknown extension of filename.xxx

$ # try to feed an unknown MIME type
$ python -m mimetypes --extension audio/aac audio/opus audio/future audio/x-wav
.aac
.opus
error: unknown type audio/future
```

base64— Base16, Base32, Base64, Base85 Data Encodings

Source code: `Lib/base64.py`

This module provides functions for encoding binary data to printable ASCII characters and decoding such encodings back to binary data. This includes the encodings specified in RFC 4648 (Base64, Base32 and Base16) and the non-standard Base85 encodings.

There are two interfaces provided by this module. The modern interface supports encoding bytes-like objects to ASCII bytes, and decoding bytes-like objects or strings containing ASCII to bytes. Both base-64 alphabets defined in RFC 4648 (normal, and URL- and filesystem-safe) are supported.

The legacy interface does not support decoding from strings, but it does provide functions for encoding and decoding to and from file objects. It only supports the Base64 standard alphabet, and it adds newlines every 76 characters as per RFC 2045. Note that if you are looking for RFC 2045 support you probably want to be looking at the `email` package instead.

Changed in version 3.3: ASCII-only Unicode strings are now accepted by the decoding functions of the modern interface.

Changed in version 3.4: Any bytes-like objects are now accepted by all encoding and decoding functions in this module. ASCII85/Base85 support added.

RFC 4648 Encodings

The RFC 4648 encodings are suitable for encoding binary data so that it can be safely sent by email, used as parts of URLs, or included as part of an HTTP POST request.

base64.b64encode(s, altchars=None)

Encode the bytes-like object `s` using Base64 and return the encoded bytes. Optional `altchars` must be a bytes-like object of length 2 which specifies an alternative alphabet for the `+` and `/` characters. This allows an application to e.g. generate URL or filesystem safe Base64 strings. The default is `None`, for which the standard Base64 alphabet is used. May assert or raise a `ValueError` if the length of `altchars` is not 2. Raises a `TypeError` if `altchars` is not a bytes-like object.

base64.b64decode(s, altchars=None, validate=False)

Decode the Base64 encoded bytes-like object or ASCII string `s` and return the decoded bytes. Optional `altchars` must be a bytes-like object or ASCII string of length 2 which specifies the alternative alphabet used instead of the `+` and `/` characters. A `binascii.Error` exception is raised if `s` is incorrectly padded. If `validate` is `False` (the default), characters that are neither in the normal base-64 alphabet nor the alternative alphabet are discarded prior to the padding check. If `validate` is `True`, these non-alphabet characters

base64.standard_b64encode(s)

Encode bytes-like object `s` using the standard Base64 alphabet and return the encoded bytes.

base64.standard_b64decode(s)

Decode bytes-like object or ASCII string `s` using the standard Base64 alphabet and return the decoded bytes.

base64.urlsafe_b64encode(s)

Encode bytes-like object `s` using the URL- and filesystem-safe alphabet, which substitutes `-` instead of `+` and `_` instead of `/` in the standard Base64 alphabet, and return the encoded bytes. The result can still contain `=`.

base64.urlsafe_b64decode(s)

Decode bytes-like object or ASCII strings using the URL- and filesystem-safe alphabet, which substitutes `-` instead of `+` and `_` instead of `/` in the standard Base64 alphabet, and return the decoded bytes.

base64.b32encode(s)

Encode the bytes-like object using Base32 and return the encoded bytes.

base64.b32decode(s, casefold=False, map01=None)

Decode the Base32 encoded bytes-like object or ASCII strings and return the decoded bytes. Optional `casefold` is a flag specifying whether a lowercase alphabet is acceptable as input. For security purposes, the default is `False`. RFC 4648 allows for optional mapping of the digit 0 (zero) to the letter O (oh), and for optional mapping of the digit 1 (one) to either the letter I (eye) or letter L (el). The optional argument `map01` when not `None`, specifies which letter the digit 1 should be mapped to (when `map01` is

base64.b32hexencode(s)

Similar to `b32encode()` but uses the Extended Hex Alphabet, as defined in RFC 4648. Added in version 3.10.

base64.b32hexdecode(s, casefold=False)

Similar to `b32decode()` but uses the Extended Hex Alphabet, as defined in RFC 4648. This version does not allow the digit 0 (zero) to the letter O (oh) and digit 1 (one) to either the letter I (eye) or letter L (el) mappings, all these characters are included in the Extended Hex Alphabet and are not interchangeable. Added in version 3.10.

base64.b16encode(s)

Encode the bytes-like object using Base16 and return the encoded bytes.

base64.b16decode(s, casefold=False)

Decode the Base16 encoded bytes-like object or ASCII strings and return the decoded bytes. Optional `casefold` is a flag specifying whether a lowercase alphabet is acceptable as input. For security purposes, the default is `False`. A `binascii.Error` is raised if `s` is incorrectly padded or if there are non-alphabet characters present in the input.

Base85 Encodings

Base85 encoding is not formally specified but rather a de facto standard, thus different systems perform the encoding differently.

The `a85encode()` and `b85encode()` functions in this module are two implementations of the de facto standard. You should call the function with the Base85 implementation used by the software you intend to work with.

The two functions present in this module differ in how they handle the following:

- Whether to include enclosing `<~>` and `<~>` markers
- Whether to include newline characters
- The set of ASCII characters used for encoding
- Handling of null bytes

Refer to the documentation of the individual functions for more information.

base64.a85encode(b, *, foldspaces=False, wrapcol=0, pad=False, adobe=False)

Encode the bytes-like object using Ascii85 and return the encoded bytes. `foldspaces` is an optional flag that uses the special short sequence `'y'` instead of 4 consecutive spaces (ASCII 0x20) as supported by `'btoa'`. This feature is not supported by the “standard” Ascii85 encoding. `wrapcol` controls whether the output should have

newline (b'\n') characters added to it. If this is non-zero, each output line will be at most this many characters long, excluding the trailing newline. pad controls whether the in

base64.a85decode(b,*,foldspaces=False,adobe=False,ignorechars=b'\t\n\r\x0b')

Decode the Ascii85 encoded bytes-like object or ASCII string and return the decoded bytes. foldspaces is a flag that specifies whether the 'y' short sequence should be accepted as shorthand for 4 consecutive spaces (ASCII 0x20). This feature is not supported by the "standard" Ascii85 encoding. adobe controls whether the input sequence is in Adobe Ascii85 format (i.e. is framed with <~ and ~>). ignorechars should be a byte string containing characters to ignore from the input. This should only contain whi

base64.b85encode(b,pad=False)

Encode the bytes-like object using base85 (as used in e.g. git-style binary diffs) and return the encoded bytes. If pad is true, the input is padded with b'\0' so its length is a multiple of 4 bytes before encoding. Added in version 3.4.

base64.b85decode(b)

Decode the base85-encoded bytes-like object or ASCII string and return the decoded bytes. Padding is implicitly removed, if necessary. Added in version 3.4.

base64.z85encode(s)

Encode the bytes-like object using Z85 (as used in ZeroMQ) and return the encoded bytes. See Z85 specification for more information. Added in version 3.13.

base64.z85decode(s)

Decode the Z85-encoded bytes-like object or ASCII string and return the decoded bytes. See Z85 specification for more information. Added in version 3.13.

Legacy Interface

base64.decode(input,output)

Decode the contents of the binary input file and write the resulting binary data to the output file. input and output must be file objects. input will be read until input.readline() returns an empty bytes object.

base64.decodebytes(s)

Decode the bytes-like objects, which must contain one or more lines of base64 encoded data, and return the decoded bytes. Added in version 3.1.

base64.encode(input,output)

Encode the contents of the binary input file and write the resulting base64 encoded data to the output file. input and output must be file objects. input will be read until input.read() returns an empty bytes object. encode() inserts a newline character (b'\n') after every 76 bytes of the output, as well as ensuring that the output always ends with a newline, as per RFC 2045 (MIME).

base64.encodebytes(s)

Encode the bytes-like objects, which can contain arbitrary binary data, and return bytes containing the base64-encoded data, with newlines (b'\n') inserted after every 76 bytes of output, and ensuring that there is a trailing newline, as per RFC 2045 (MIME). Added in version 3.1.

An example usage of the module:

```
>>> import base64
```



```
>>> encoded = base64.b64encode(b'data to be encoded')
>>> encoded
b'ZGF0YSB0byBiZSB1bmNvZGVk'
>>> data = base64.b64decode(encoded)
>>> data
b'data to be encoded'
```

Security Considerations

A new security considerations section was added to RFC 4648 (section 12); it's recommended to review the security section for any code deployed to production.

See also

Module `binascii`

RFC 1521- MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies

Support module containing ASCII-to-binary and binary-to-ASCII conversions.

Section 5.2, "Base64 Content-Transfer-Encoding," provides the definition of the base64 encoding.

binascii— Convert between binary and ASCII

The `binascii` module contains a number of methods to convert between binary and various ASCII-encoded binary representations. Normally, you will not use these functions directly but use wrapper modules like `base64` instead. The `binascii` module contains low-level functions written in C for greater speed that are used by the higher-level modules.

■ Nota: Notea `a2b_*` functions accept Unicode strings containing only ASCII characters. Other functions only accept bytes-like objects (such as `bytes`, `bytearray` and other objects that support the buffer protocol). Changed in version 3.3: ASCII-only unicode strings are now accepted by the `a2b_*` functions.

The `binascii` module defines the following functions:

`binascii.a2b_uu(string)`

Convert a single line of uuencoded data back to binary and return the binary data. Lines normally contain 45 (binary) bytes, except for the last line. Line data may be followed by whitespace.

`binascii.b2a_uu(data, *, backtick=False)`

Convert binary data to a line of ASCII characters, the return value is the converted line, including a newline char. The length of `data` should be at most 45. If `backtick` is true, zeros are represented by ``` instead of spaces. Changed in version 3.7: Added the `backtick` parameter.

`binascii.a2b_base64(string, /, *, strict_mode=False)`

Convert a block of base64 data back to binary and return the binary data. More than one line may be passed at a time. If `strict_mode` is true, only valid base64 data will be converted. Invalid base64 data will raise `binascii.Error`. Valid base64: Conforms to RFC 3548. Contains only characters from the base64 alphabet. Contains no excess data after padding (including excess padding, newlines, etc.). Does not start with a padding. Changed in version 3.11: Added the `strict_mode` parameter.

`binascii.b2a_base64(data, *, newline=True)`

Convert binary data to a line of ASCII characters in base64 coding. The return value is the converted line, including a newline char if `newline` is true. The output of this function conforms to RFC 3548. Changed in version 3.6: Added the `newline` parameter.

`binascii.a2b_qp(data, header=False)`

Convert a block of quoted-printable data back to binary and return the binary data. More than one line may be passed at a time. If the optional argument `header` is present and true, underscores will be decoded as spaces.

`binascii.b2a_qp(data, quotetabs=False, istext=True, header=False)`

Convert binary data to a line(s) of ASCII characters in quoted-printable encoding. The return value is the converted line(s). If the optional argument `quotetabs` is present and true, all tabs and spaces will be encoded. If the optional argument `istext` is present and true, newlines are not encoded but trailing whitespace will be encoded. If the optional argument `header` is present and true, spaces will be encoded as underscores per RFC 1522. If the optional argument `header` is present and false, newline c

`binascii.crc_hqx(data, value)`

Compute a 16-bit CRC value of `data`, starting with `value` as the initial CRC, and return the result. This uses the CRC-CCITT polynomial $x^{16} + x^{12} + x^5 + 1$, often represented as 0x1021. This CRC is used in the binhex4 format.

`binascii.crc32(data[, value])`

Compute CRC-32, the unsigned 32-bit checksum of `data`, starting with an initial CRC of `value`. The default initial CRC is zero. The algorithm is consistent with the ZIP file checksum. Since the algorithm is designed for use as a

checksum algorithm, it is not suitable for use as a general hash algorithm. Use as follows:
`print(binascii.crc32(b"hello world"))` # Or, in two pieces:
`crc=binascii.crc32(b"hello")`
`crc=binascii.crc32(b"world",crc)`
`print('crc32 ={:010x}'.format(crc))`
Changed in version 3.0: The

`binascii.b2a_hex(data[,sep[,bytes_per_sep=1]])`

`binascii.hexlify(data[,sep[,bytes_per_sep=1]])`

Return the hexadecimal representation of the binary data. Every byte of data is converted into the corresponding 2-digit hex representation. The returned bytes object is therefore twice as long as the length of data. Similar functionality (but returning a text string) is also conveniently accessible using the `bytes.hex()` method. If `sep` is specified, it must be a single character str or bytes object. It will be inserted in the output after every `bytes_per_sep` input bytes. Separator placement is counted from

`binascii.a2b_hex(hexstr)`

`binascii.unhexlify(hexstr)`

Return the binary data represented by the hexadecimal string `hexstr`. This function is the inverse of `b2a_hex()`. `hexstr` must contain an even number of hexadecimal digits (which can be upper or lower case), otherwise an `Exception` is raised. Similar functionality (accepting only text string arguments, but more liberal towards whitespace) is also accessible using the `bytes.fromhex()` class method.

`exception binascii.Error`

Exception raised on errors. These are usually programming errors.

`exception binascii.Incomplete`

Exception raised on incomplete data. These are usually not programming errors, but may be handled by reading a little more data and trying again.

See also

`Module base64`

`Module quopri`

Support for RFC compliant base64-style encoding in base 16, 32, 64, and 85.

Support for quoted-printable encoding used in MIME email messages.

quopri— Encode and decode MIME quoted-printable data

Source code: `Lib/quopri.py`

This module performs quoted-printable transport encoding and decoding, as defined in RFC 1521: “MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies”. The quoted-printable encoding is designed for data where there are relatively few nonprintable characters; the base64 encoding scheme available via the `base64` module is more compact if there are many such characters, as when sending a graphics file.

`quopri.decode(input,output,header=False)`

Decode the contents of the `input` file and write the resulting decoded binary data to the `output` file. `input` and `output` must be binary file objects. If the optional argument `header` is present and true, underscore will be decoded as space. This is used to decode “Q”-encoded headers as described in RFC 1522: “MIME (Multipurpose Internet Mail Extensions) Part Two: Message Header Extensions for Non-ASCII Text”.

`quopri.encode(input,output,quotetabs,header=False)`

Encode the contents of the `input` file and write the resulting quoted-printable data to the `output` file. `input` and `output` must be binary file objects. `quotetabs`, a non-optional flag which controls whether to encode embedded spaces and tabs; when true it encodes such embedded whitespace, and when false it leaves them unencoded. Note that spaces and tabs appearing at the end of lines are always encoded, as per RFC 1521. `header` is a flag which controls if spaces are encoded as underscores as per RFC 1522.

`quopri.decodestring(s,header=False)`

Like `decode()`, except that it accepts a source bytes and returns the corresponding decoded bytes.

`quopri.encodestring(s,quotetabs=False,header=False)`

Like `encode()`, except that it accepts a source bytes and returns the corresponding encoded bytes. By default, it sends a `False` value to `quotetabs` parameter of the `encode()` function.

See also

Module `base64`

Encode and decode MIME base64 data

Structured Markup Processing Tools

Python supports a variety of modules to work with various forms of structured data markup. This includes modules to work with the Standard Generalized Markup Language (SGML) and the Hypertext Markup Language (HTML), and several interfaces for working with the Extensible Markup Language (XML).

- `html`— HyperText Markup Language support
- `html.parser`— Simple HTML and XHTML parserExample HTML Parser ApplicationHTMLParserMethodsExamples
- `html.entities`— Definitions of HTML general entities
- XML Processing ModulesXML security
- `xml.etree.ElementTree`— The ElementTree XML APITutorialXML tree and elementsParsing XMLPull API for non-blocking parsingFinding interesting elementsModifying an XML FileBuilding XML documentsParsing XML with NamespacesXPath supportExampleSupported XPath syntaxReferenceFunctionsXInclude supportExampleReferenceFunctionsElement ObjectsElementTree ObjectsQName ObjectsTreeBuilder ObjectsXMLParser Object
- `xml.dom`— The Document Object Model APIModule ContentsObjects in the DOMDOMImplementation ObjectsNode ObjectsNodeList ObjectsDocumentType ObjectsDocument ObjectsElement ObjectsAttr ObjectsNamedNodeMap ObjectsComment ObjectsText and CDATASection ObjectsProcessingInstruction ObjectsExceptionsConformanceType MappingAccessor Methods
- `xml.dom.minidom`— Minimal DOM implementationDOM ObjectsDOM Exampleminidom and the DOM standard
- `xml.dom.pulldom`— Support for building partial DOM treesDOMEventStream Objects
- `xml.sax`— Support for SAX2 parsersSAXException Objects
- `xml.sax.handler`— Base classes for SAX handlersContentHandler ObjectsDTDHandler ObjectsEntityResolver ObjectsErrorHandler ObjectsLexicalHandler Objects
- `xml.sax.saxutils`— SAX Utilities
- `xml.sax.xmlreader`— Interface for XML parsersXMLReader ObjectsIncrementalParser ObjectsLocator ObjectsInputSource ObjectsTheAttributesInterfaceTheAttributesNSInterface
- `xml.parsers.expat`— Fast XML parsing using ExpatXMLParser ObjectsExpatError ExceptionsExampleContent Model DescriptionsExpat error constants

html— HyperText Markup Language support

Source code: `Lib/html/__init__.py`

This module defines utilities to manipulate HTML.

html.escape(s,quote=True)

Convert the characters `&`, `<` and `>` in strings to HTML-safe sequences. Use this if you need to display text that might contain such characters in HTML. If the optional flag `quote` is true (the default), the characters `"` and `'` are also translated; this helps for inclusion in an HTML attribute value delimited by quotes, as in `<ahref="...">`. If `quote` is set to false, the characters `"` and `'` are not translated. Added in version 3.2.

html.unescape(s)

Convert all named and numeric character references (e.g. `>`, `>`, `>`) in the string to the corresponding Unicode characters. This function uses the rules defined by the HTML 5 standard for both valid and invalid character references, and the list of HTML5 named character references. Added in version 3.4.

Submodules in the `html` package are:

- `html.parser`— HTML/XHTML parser with lenient parsing mode
- `html.entities`— HTML entity definitions

html.parser— Simple HTML and XHTML parser

Source code: `Lib/html/parser.py`

This module defines a class `HTMLParser` which serves as the basis for parsing text files formatted in HTML (HyperText Mark-up Language) and XHTML.

`class html.parser.HTMLParser(*, convert_charrefs=True, scripting=False)`

Create a parser instance able to parse invalid markup. If `convert_charrefs` is true (the default), all character references (except the ones in elements like `script` and `style`) are automatically converted to the corresponding Unicode characters. If `scripting` is false (the default), the content of the `script` element is parsed normally; if it's true, it's returned as is without being parsed. An `HTMLParser` instance is fed HTML data and calls handler methods when start tags, end tags, text, comments, and other ma

Example HTML Parser Application

As a basic example, below is a simple HTML parser that uses the `HTMLParser` class to print out start tags, end tags, and data as they are encountered:

```
from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Encountered a start tag:", tag)

    def handle_endtag(self, tag):
        print("Encountered an end tag :", tag)

    def handle_data(self, data):
        print("Encountered some data :", data)

parser = MyHTMLParser()
parser.feed('<html><head><title>Test</title></head>'
          '<body><h1>Parse me!</h1></body></html>')
```

The output will then be:

```
Encountered a start tag: html
Encountered a start tag: head
Encountered a start tag: title
Encountered some data : Test
Encountered an end tag : title
Encountered an end tag : head
Encountered a start tag: body
Encountered a start tag: h1
Encountered some data : Parse me!
Encountered an end tag : h1
Encountered an end tag : body
Encountered an end tag : html
```

HTMLParserMethods

`HTMLParser` instances have the following methods:

`HTMLParser.feed(data)`

Feed some text to the parser. It is processed insofar as it consists of complete elements; incomplete data is buffered until more data is fed or `close()` is called. Data must be str.

HTMLParser.close()

Force processing of all buffered data as if it were followed by an end-of-file mark. This method may be redefined by a derived class to define additional processing at the end of the input, but the redefined version should always call the `HTMLParser.close()` method.

HTMLParser.reset()

Reset the instance. Loses all unprocessed data. This is called implicitly at instantiation time.

HTMLParser.getpos()

Return current line number and offset.

HTMLParser.get_starttag_text()

Return the text of the most recently opened start tag. This should not normally be needed for structured processing, but may be useful in dealing with HTML “as deployed” or for re-generating input with minimal changes (whitespace between attributes can be preserved, etc.).

The following methods are called when data or markup elements are encountered and they are meant to be overridden in a subclass. The base class implementations do nothing (except for `handle_startendtag()`):

HTMLParser.handle_starttag(tag, attrs)

This method is called to handle the start tag of an element (e.g. `<div="main">`). The `tag` argument is the name of the tag converted to lower case. The `attrs` argument is a list of (name, value) pairs containing the attributes found inside the tag's `<>` brackets. Then `name` will be translated to lower case, and quotes in the `value` have been removed, and character and entity references have been replaced. For instance, for the tag `<A HREF="https://www.cwi.nl/">`, this method would be called as `handle_starttag('a', [('hr`

HTMLParser.handle_endtag(tag)

This method is called to handle the end tag of an element (e.g. `</div>`). The `tag` argument is the name of the tag converted to lower case.

HTMLParser.handle_startendtag(tag, attrs)

Similar to `handle_starttag()`, but called when the parser encounters an XHTML-style empty tag (`<img.../>`). This method may be overridden by subclasses which require this particular lexical information; the default implementation simply calls `handle_starttag()` and `handle_endtag()`.

HTMLParser.handle_data(data)

This method is called to process arbitrary data (e.g. text nodes and the content of elements like `script` and `style`).

HTMLParser.handle_entityref(name)

This method is called to process a named character reference of the form `&name;` (e.g. `>`), where `name` is a general entity reference (e.g. `'gt'`). This method is only called if `convert_charrefs` is false.

HTMLParser.handle_charref(name)

This method is called to process decimal and hexadecimal numeric character references of the form `&#NNN;` and `&#xNNN;`. For example, the decimal equivalent for `>` is `>`, whereas the hexadecimal is `>`; in this case the method will receive `'62'` or `'x3E'`. This method is only called if `convert_charrefs` is false.

HTMLParser.handle_comment(data)

This method is called when a comment is encountered (e.g. `<!--comment-->`). For example, the comment `<!--comment-->` will cause this method to be called with the argument `'comment'`. The content of Internet

Explorer conditional comments (condcoms) will also be sent to this method, so, for<!--[if IE9]>IE9-specificcontent<![endif]-->, this method will receive'[if IE9]>IE9-specificcontent<![endif]'.

HTMLParser.handle_decl(decl)

This method is called to handle an HTML doctype declaration (e.g.<!DOCTYPEhtml>).Thedeclparameter will be the entire contents of the declaration inside the<!-->markup (e.g.'DOCTYPEhtml').

HTMLParser.handle_pi(data)

Method called when a processing instruction is encountered. Thedataparameter will contain the entire processing instruction. For example, for the processing instruction<?proccolor='red'>, this method would be called ashandle_pi("proccolor='red'"). It is intended to be overridden by a derived class; the base class implementation does nothing. NoteTheHTMLParserclass uses the SGML syntactic rules for processing instructions. An XHTML processing instruction using the trailing'? will cause the'? to

HTMLParser.unknown_decl(data)

This method is called when an unrecognized declaration is read by the parser. Thedataparameter will be the entire contents of the declaration inside the<![...]>markup. It is sometimes useful to be overridden by a derived class. The base class implementation does nothing.

Examples

The following class implements a parser that will be used to illustrate more examples:

```
from html.parser import HTMLParser
from html.entities import name2codepoint

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Start tag:", tag)
        for attr in attrs:
            print("    attr:", attr)

    def handle_endtag(self, tag):
        print("End tag  :", tag)

    def handle_data(self, data):
        print("Data      :", data)

    def handle_comment(self, data):
        print("Comment  :", data)

    def handle_entityref(self, name):
        c = chr(name2codepoint[name])
        print("Named ent:", c)

    def handle_charref(self, name):
        if name.startswith('x'):
            c = chr(int(name[1:], 16))
        else:
            c = chr(int(name))
        print("Num ent  :", c)

    def handle_decl(self, data):
        print("Decl      :", data)
```

```
parser = MyHTMLParser()
```

Parsing a doctype:

```
>>> parser.feed('<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" '
...             '"http://www.w3.org/TR/html4/strict.dtd">')
Decl      : DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd"
```

Parsing an element with a few attributes and a title:

```
>>> parser.feed('')
Start tag: img
  attr: ('src', 'python-logo.png')
  attr: ('alt', 'The Python logo')
>>>
>>> parser.feed('<h1>Python</h1>')
Start tag: h1
Data      : Python
End tag   : h1
```

The content of elements like `script` and `style` is returned as is, without further parsing:

```
>>> parser.feed('<style type="text/css">#python { color: green }</style>')
Start tag: style
  attr: ('type', 'text/css')
Data      : #python { color: green }
End tag   : style

>>> parser.feed('<script type="text/javascript">'
...             'alert("<strong>hello! &#9786;</strong>");</script>')
Start tag: script
  attr: ('type', 'text/javascript')
Data      : alert("<strong>hello! &#9786;</strong>");
End tag   : script
```

Parsing comments:

```
>>> parser.feed('<!--a comment-->'
...             '<!--[if IE 9]>IE-specific content<![endif]-->')
Comment   : a comment
Comment   : [if IE 9]>IE-specific content<![endif]
```

Parsing named and numeric character references and converting them to the correct char (note: these 3 references are all equivalent to '>'):

```
>>> parser = MyHTMLParser()
>>> parser.feed('&gt;&#62;&#x3E;')
Data      : >>>

>>> parser = MyHTMLParser(convert_charrefs=False)
>>> parser.feed('&gt;&#62;&#x3E;')
Named ent: >
Num ent   : >
Num ent   : >
```

Feeding incomplete chunks to `feed()` works, but `handle_data()` might be called more than once if `convert_charrefs` is false:

```
>>> for chunk in ['<sp', 'an>buff', 'ered', ' text</s', 'pan>']:
...     parser.feed(chunk)
...
Start tag: span
Data      : buff
Data      : ered
Data      : text
End tag   : span
```

Parsing invalid HTML (e.g. unquoted attributes) also works:

```
>>> parser.feed('<p><a class=link href=#main>tag soup</p ></a>')
Start tag: p
```

```
Start tag: a
  attr: ('class', 'link')
  attr: ('href', '#main')
Data      : tag soup
End tag   : p
End tag   : a
```

html.entities— Definitions of HTML general entities

Source code: `Lib/html/entities.py`

This module defines four dictionaries, `html5`, `name2codepoint`, `codepoint2name`, and `entitydefs`.

html.entities.html5

A dictionary that maps HTML5 named character references^[1] to the equivalent Unicode character(s), e.g. `html5['gt;'] == '>'`. Note that the trailing semicolon is included in the name (e.g. `'gt;'`), however some of the names are accepted by the standard even without the semicolon: in this case the name is present with and without the `'`. See also `html.unescape()`. Added in version 3.3.

html.entities.entitydefs

A dictionary mapping XHTML 1.0 entity definitions to their replacement text in ISO Latin-1.

html.entities.name2codepoint

A dictionary that maps HTML4 entity names to the Unicode code points.

html.entities.codepoint2name

A dictionary that maps Unicode code points to HTML4 entity names.

Footnotes

See <https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references>

XML Processing Modules

Source code: `Lib/xml/`

Python's interfaces for processing XML are grouped in the `xml` package.

■ **Nota:** Note if you need to parse untrusted or unauthenticated data, see [XML security](#).

It is important to note that modules in the `xml` package require that there be at least one SAX-compliant XML parser available. The Expat parser is included with Python, so `xml.parsers.expat` module will always be available.

The documentation for `xml.dom` and `xml.sax` packages are the definition of the Python bindings for the DOM and SAX interfaces.

The XML handling submodules are:

- `xml.etree.ElementTree`: the ElementTree API, a simple and lightweight XML processor
- `xml.dom`: the DOM API definition
- `xml.dom.minidom`: a minimal DOM implementation
- `xml.dom.pulldom`: support for building partial DOM trees
- `xml.sax`: SAX2 base classes and convenience functions
- `xml.parsers.expat`: the Expat parser binding

XML security

An attacker can abuse XML features to carry out denial of service attacks, access local files, generate network connections to other machines, or circumvent firewalls when attacker-controlled XML is being parsed, in Python or elsewhere.

The built-in XML parsers of Python rely on the library `libexpat`, commonly called Expat, for parsing XML.

By default, Expat itself does not access local files or create network connections.

Expat versions lower than 2.7.2 may be vulnerable to the “billion laughs”, “quadratic blowup” and “large tokens” vulnerabilities, or to disproportional use of dynamic memory. Python bundles a copy of Expat, and whether Python uses the bundled or a system-wide Expat, depends on how the Python interpreter has been configured in your environment. Python may be vulnerable if it uses such older versions of Expat. Check `pyexpat.EXPAT_VERSION`.

`xmlrpc` is vulnerable to the “decompression bomb” attack.

billion laughs / exponential entity expansion

quadratic blowup entity expansion

decompression bomb

large tokens

The Billion Laughs attack – also known as exponential entity expansion – uses multiple levels of nested entities. Each entity refers to another entity several times, and the final entity definition contains a small string. The exponential expansion results in several gigabytes of text and consumes lots of memory and CPU time.

A quadratic blowup attack is similar to a Billion Laughs attack; it abuses entity expansion, too. Instead of nested entities it repeats one large entity with a couple of thousand chars over and over again. The attack isn't as

efficient as the exponential case but it avoids triggering parser countermeasures that forbid deeply nested entities.

Decompression bombs (akaZIP bomb) apply to all XML libraries that can parse compressed XML streams such as gzipped HTTP streams or LZMA-compressed files. For an attacker it can reduce the amount of transmitted data by three magnitudes or more.

Expat needs to re-parse unfinished tokens; without the protection introduced in Expat 2.6.0, this can lead to quadratic runtime that can be used to cause denial of service in the application parsing XML. The issue is known asCVE 2023-52425.